UniSDK

Release main

Telink Semiconductor

Oct 14, 2026

GETTING STARTED

1 UniSDK Introduction **3**
- 1.1 Telink Semiconductor . . . 3
- 1.2 UniSDK Core Features and Advantages . . . 3
- 1.3 Architecture Overview . . . 4
- 1.4 Supported Chip Families . . . 4
- 1.5 License . . . 4
- 1.6 Community Resources . . . 4
- 1.7 Next Steps . . . 4

2 Overview **5**
- 2.1 About This Document . . . 5
- 2.2 Scope . . . 5
- 2.3 Hardware and Software Requirements . . . 5
- 2.4 Next Steps . . . 6

3 Development Environment Setup **7**
- 3.1 VS Code Extension (Recommended) . . . 7
- 3.2 CLI Setup (Alternative) . . . 12
- 3.3 Next Steps . . . 12

4 Get and Import the SDK **13**
- 4.1 Get the SDK . . . 13
- 4.2 SDK Directory Structure . . . 13
- 4.3 Next Steps . . . 13

5 Compile, Flash and Run Your First Example (VS Code) **15**
- 5.1 Select the First Example . . . 15
- 5.2 Build the Project . . . 15
- 5.3 Flash the Firmware . . . 18
- 5.4 Verify the Result . . . 19
- 5.5 Next Steps . . . 19

6 Troubleshooting **21**
- 6.1 Environment Issues . . . 21
- 6.2 Build Issues . . . 22
- 6.3 Menuconfig Issues . . . 22
- 6.4 Getting Help . . . 22

7 Development Guide **23**
- 7.1 Content Navigation . . . 23
- 7.2 Quick Start for a New Project . . . 23

7.3 Development Workflow Overview . . . 24
7.4 Next Steps . . . 24

8 Linux Environment Setup **25**
8.1 System Requirements . . . 25
8.2 Install System Dependencies . . . 25
8.3 Set Up Python Virtual Environment . . . 25
8.4 Initialize West . . . 26
8.5 Configure Toolchain . . . 26
8.6 Configure BDT Tool (Optional) . . . 26
8.7 Verify Environment . . . 27
8.8 Next Steps . . . 27

9 macOS Environment Setup **29**
9.1 System Requirements . . . 29
9.2 Install System Dependencies . . . 29
9.3 Set Up Python Virtual Environment . . . 29
9.4 Initialize West . . . 30
9.5 Configure Toolchain . . . 30
9.6 Verify Environment . . . 30
9.7 Next Steps . . . 30

10 Windows Environment Setup **31**
10.1 System Requirements . . . 31
10.2 Installation Methods . . . 31
10.3 Configure BDT Tool (Optional) . . . 33
10.4 Verify Environment . . . 34
10.5 Common Issues . . . 34
10.6 Notes . . . 34
10.7 Next Steps . . . 35

11 First Example: Blinky (CLI) **37**
11.1 Prerequisites . . . 37
11.2 Example Overview . . . 37
11.3 Step 1: Configure Chip Type . . . 37
11.4 Step 2: Compilation . . . 38
11.5 Step 3: Flashing . . . 38
11.6 Step 4: Verification . . . 38
11.7 Custom Modifications . . . 38
11.8 Next Steps . . . 39

12 SDK Directory Structure **41**
12.1 Overall Structure . . . 41
12.2 Key Directory Responsibilities . . . 42
12.3 Code Organization Principles . . . 43

13 Project Creation & Management **45**
13.1 Application Project Location . . . 45
13.2 Creating a New Project . . . 45
13.3 Building the Project . . . 46
13.4 Configuration Management . . . 46
13.5 Cleaning . . . 47
13.6 Development Workflow Recommendations . . . 47

14 Core Architecture **49**

14.1 System Startup Flow . . . 49
14.2 Initialization System . . . 49
14.3 Interrupt Management . . . 50
14.4 Low-Power Management . . . 50
14.5 Context Switch (tl_context.S) . . . 51
14.6 Clock System . . . 51

15 Init hooks documentation **53**
15.1 Overview . . . 53
15.2 Concepts . . . 53
15.3 Levels & Priorities . . . 53
15.4 API reference . . . 54
15.5 Additional Notes . . . 56

16 API Documentation **57**
16.1 Overview . . . 57
16.2 API Reference . . . 57

17 Debugging & Testing **61**
17.1 Hardware Debugging . . . 61
17.2 Software Debugging . . . 61
17.3 Performance Analysis . . . 62
17.4 Debugging with GDB . . . 62
17.5 Common Debugging Tips . . . 62
17.6 Next Steps . . . 62

18 IDE Integration **63**
18.1 VS Code Configuration (Recommended) . . . 63
18.2 Eclipse Configuration . . . 64
18.3 Segger Embedded Studio . . . 65
18.4 Next Steps . . . 65

19 clangd Setup Guide for VSCode **67**
19.1 What is clangd? . . . 67
19.2 Why use clangd? . . . 67
19.3 Key Files . . . 68
19.4 Installing and setup . . . 68
19.5 Troubleshooting . . . 69

20 Chips and Boards **71**
20.1 Chip Series Overview . . . 71
20.2 Selecting a Chip . . . 71
20.3 Board List . . . 71
20.4 Chip and Board Pair Validation . . . 72
20.5 Specifying at Build Time . . . 72

21 TL321X Chip Series **73**
21.1 Overview . . . 73
21.2 Core Parameters . . . 73
21.3 Memory . . . 73
21.4 Peripheral List . . . 73
21.5 Supported Boards . . . 74

22 TL721X Chip Series **75**
22.1 Overview . . . 75

22.2 Core Parameters . . . 75
22.3 Memory . . . 75
22.4 Peripheral List . . . 75
22.5 Supported Boards . . . 76

23 TLSR922X Chip Series **77**
23.1 Overview . . . 77
23.2 Core Parameters . . . 77
23.3 Memory . . . 77
23.4 Peripheral List . . . 77
23.5 Supported Boards . . . 78

24 TLSR952X Chip Series **79**
24.1 Overview . . . 79
24.2 Core Parameters . . . 79
24.3 Memory . . . 79
24.4 Peripheral List . . . 79
24.5 Supported Boards . . . 80

25 TL3218X_EVK Board **81**
25.1 Overview . . . 81
25.2 Building . . . 81

26 TL7218X_EVK Board **83**
26.1 Overview . . . 83
26.2 Building . . . 83

27 TLSR9228A_EVK Board **85**
27.1 Overview . . . 85
27.2 Building . . . 85

28 TLSR9528A_EVK Board **87**
28.1 Overview . . . 87
28.2 Building . . . 87

29 TLSR9528A_DONGLE **89**
29.1 Overview . . . 89
29.2 Building . . . 89

30 Peripherals and Drivers **91**
30.1 Driver Overview . . . 91
30.2 Driver Programming Model . . . 91
30.3 Kconfig Configuration . . . 92

31 GPIO Driver **93**
31.1 Overview . . . 93
31.2 Data Types and Enums . . . 93
31.3 Driver API . . . 95
31.4 Kconfig Configuration . . . 97
31.5 Related Resources . . . 98

32 UART Driver **99**
32.1 Overview . . . 99
32.2 Data Types and Enums . . . 99
32.3 Driver API . . . 102
32.4 Transfer Modes . . . 104

32.5 Kconfig Configuration . . . 104
32.6 Related Resources . . . 106

33 Power Management (PM) Driver **107**
33.1 Overview . . . 107
33.2 Sleep Modes . . . 107
33.3 Wakeup Sources . . . 107
33.4 GPIO Wakeup Level . . . 108
33.5 Driver API . . . 108
33.6 Typical Usage . . . 109
33.7 Kconfig Configuration . . . 110
33.8 Related Resources . . . 111

34 Analog Driver **113**
34.1 Overview . . . 113
34.2 API Reference . . . 113
34.3 Register Definitions . . . 114

35 PLIC Interrupt Controller **115**
35.1 Overview . . . 115
35.2 Data Types and Enums . . . 115
35.3 API Reference . . . 115
35.4 PLIC Configuration . . . 117
35.5 Interrupt Handling Modes . . . 118
35.6 Interrupt Nesting . . . 118
35.7 Integration with GPIO Interrupts . . . 118

36 I2C Driver **119**
36.1 Overview . . . 119
36.2 API Reference . . . 119
36.3 Samples . . . 119

37 SPI Driver **121**
37.1 Overview . . . 121
37.2 API Reference . . . 121
37.3 Samples . . . 121

38 ADC Driver **123**
38.1 Overview . . . 123
38.2 Samples . . . 123

39 DMA Driver **125**
39.1 Overview . . . 125
39.2 Samples . . . 125

40 Watchdog Driver **127**
40.1 Overview . . . 127
40.2 Samples . . . 127

41 Timer Driver **129**
41.1 Overview . . . 129
41.2 Samples . . . 129

42 USB Driver **131**
42.1 Overview . . . 131
42.2 Samples . . . 131

43 PMP Driver **133**
43.1 Overview . . . 133
43.2 Address Matching Modes . . . 133
43.3 Permissions . . . 133
43.4 Lock . . . 134
43.5 API . . . 134
43.6 Configuration . . . 134

44 Connectivity **135**
44.1 Overview . . . 135
44.2 Code Location . . . 135
44.3 Coding Standards . . . 135
44.4 Kconfig Configuration . . . 136
44.5 Examples . . . 136
44.6 Planned . . . 136

45 BLE Controller Coding Standards **137**
45.1 Overview . . . 137
45.2 File Naming . . . 137
45.3 Function Naming . . . 137
45.4 Variable Naming . . . 137
45.5 Interrupt Functions . . . 138

46 System Services **139**
46.1 Currently Available Components . . . 139

47 Build & Configuration **141**
47.1 Core Components . . . 141
47.2 Three Build Methods . . . 141
47.3 Documentation Navigation . . . 142
47.4 Build System Architecture . . . 142

48 CMake Build System **143**
48.1 Entry Point Modes . . . 143
48.2 Module Loading Order . . . 143
48.3 Core Module Details . . . 144
48.4 Configuration Flow . . . 145
48.5 Include Guard Mechanism . . . 145
48.6 Build Directory Structure . . . 145

49 Kconfig Configuration System **147**
49.1 Architecture . . . 147
49.2 Configuration Generation Flow . . . 147
49.3 Key Scripts . . . 148
49.4 Menuconfig Trigger Methods . . . 148
49.5 Configuration Option Examples . . . 148

50 West Commands **151**
50.1 Command Registration . . . 151
50.2 `west tl-build` —Build . . . 151
50.3 `west tl-config` —Configuration . . . 152
50.4 `west tl-bdt` —Burning & Debugging . . . 153
50.5 `west tl-boards` —Board Information . . . 154
50.6 `west tl-socs` —Chip Information . . . 154

51 Make Support **155**
51.1 Makefile Targets . . . 155
51.2 Makefile Variables . . . 155
51.3 Usage Examples . . . 155
51.4 Build Flow (make build) . . . 156
51.5 SOC/BOARD Parameter Forwarding . . . 156

52 Build Flow **157**
52.1 Overview . . . 157
52.2 Complete Build Flow . . . 157
52.3 Getting Started Guide . . . 157
52.4 Six Phases . . . 159
52.5 Configuration Commands . . . 160
52.6 Basic Build Commands . . . 161
52.7 Configuration File Generation Logic . . . 161
52.8 Build Results . . . 162
52.9 Building Without West . . . 162
52.10 Troubleshooting . . . 163

53 UniSDK Build System Architecture Documentation **165**
53.1 1. Overall Architecture Overview . . . 165
53.2 2. CMake Build System . . . 167
53.3 3. Kconfig Configuration System . . . 185
53.4 4. West Compilation Support . . . 187
53.5 5. Make Support . . . 198
53.6 6. Compilation Generation Flow . . . 202
53.7 7. Files and Scripts Composition . . . 203
53.8 8. Build System Working Principles . . . 207
53.9 9. Extension and Customization . . . 207

54 Extensions & Customization **211**
54.1 Adding New CMake Modules . . . 211
54.2 Extending West Commands . . . 211
54.3 Adding Kconfig Configuration . . . 212
54.4 Adding New Applications . . . 212
54.5 West Command Behavior Notes . . . 212

55 File & Script Reference **215**
55.1 Core Configuration Files . . . 215
55.2 Build Artifacts (build/ Directory) . . . 215
55.3 CMake Files . . . 215
55.4 Build Scripts (scripts/) . . . 216
55.5 Kconfig Files . . . 217

56 Development Tools **219**
56.1 Tool Overview . . . 219

57 BDT Burning and Debugging Tool **221**
57.1 Supported Operations . . . 221
57.2 Quick Start . . . 221
57.3 Flash Operations . . . 221
57.4 BDT Path Configuration . . . 222
57.5 Cross-Platform Notes . . . 223
57.6 FAQ . . . 223

58 Pinmux Pin Configuration Tool **225**
58.1 Overview . . . 225
58.2 Launch Methods . . . 225
58.3 Main View . . . 225
58.4 Selecting a Function . . . 226
58.5 Shortcuts . . . 226
58.6 Saving . . . 227
58.7 File Format . . . 227
58.8 Overriding Mechanism . . . 228
58.9 Internal Execution Flow . . . 228
58.10 Configuration File Locations . . . 229
58.11 Build Integration . . . 230
58.12 CI . . . 230

59 Menuconfig Configuration Interface **231**
59.1 Overview . . . 231
59.2 Launch Methods . . . 231
59.3 Navigation and Operations . . . 232
59.4 Dual-Phase Configuration . . . 232
59.5 Results . . . 233
59.6 Internal Execution Flow . . . 233
59.7 FAQ . . . 234

60 Security **235**

61 Production Guide **237**

62 Samples & Demos **239**
62.1 Sample List . . . 239
62.2 Building Samples . . . 240

63 GPIO Sample **241**
63.1 Overview . . . 241
63.2 Features Demonstrated . . . 241
63.3 How It Works . . . 241
63.4 Configuration Options . . . 242
63.5 Build & Run . . . 242

64 UART Sample **243**
64.1 Overview . . . 243
64.2 Features Demonstrated . . . 243
64.3 How It Works . . . 243
64.4 Configuration Options . . . 244
64.5 Build & Run . . . 244

65 PM Power Management Sample **245**
65.1 Overview . . . 245
65.2 Features Demonstrated . . . 245
65.3 How It Works . . . 245
65.4 Configuration Options . . . 246
65.5 Build & Run . . . 246

66 PMP Driver Sample **247**
66.1 Overview . . . 247
66.2 Features Demonstrated . . . 247

66.3 How It Works . . . 247
66.4 Configuration . . . 247
66.5 Build & Run . . . 248

67 U-mode Driver Sample 249
67.1 Overview . . . 249
67.2 Features Demonstrated . . . 249
67.3 How It Works . . . 249
67.4 Configuration . . . 250
67.5 Build & Run . . . 250

68 ADC Sample 251
68.1 Overview . . . 251
68.2 Features Demonstrated . . . 251
68.3 How It Works . . . 251
68.4 Configuration Options . . . 253
68.5 Build & Run . . . 253

69 DMA Sample 255
69.1 Overview . . . 255
69.2 Features Demonstrated . . . 255
69.3 How It Works . . . 255
69.4 Configuration Options . . . 257
69.5 Build & Run . . . 257

70 I2C Sample 259
70.1 Overview . . . 259
70.2 Features Demonstrated . . . 259
70.3 How It Works . . . 259
70.4 Configuration Options . . . 261
70.5 Build & Run . . . 261

71 SPI Sample 263
71.1 Overview . . . 263
71.2 Features Demonstrated . . . 263
71.3 How It Works . . . 263
71.4 Configuration Options . . . 265
71.5 Build & Run . . . 265

72 I2S Sample 267
72.1 Overview . . . 267
72.2 Features Demonstrated . . . 267
72.3 How It Works . . . 267
72.4 Configuration Options . . . 268
72.5 Build & Run . . . 268

73 BLE Sample 269
73.1 Overview . . . 269
73.2 Features Demonstrated . . . 269
73.3 How It Works . . . 270
73.4 Configuration Options . . . 270
73.5 Build & Run . . . 271
73.6 Related Resources . . . 271

74 BLE Host Sample 273

74.1 Overview . . . 273
74.2 Features Demonstrated . . . 273
74.3 How It Works . . . 273
74.4 Configuration Options . . . 275
74.5 Build & Run . . . 275

75 RF Sample **277**
75.1 Overview . . . 277
75.2 Features Demonstrated . . . 277
75.3 How It Works . . . 277
75.4 Configuration Options . . . 280
75.5 Build & Run . . . 280

76 IRQ Nesting Sample **281**
76.1 Overview . . . 281
76.2 Features Demonstrated . . . 281
76.3 How It Works . . . 281
76.4 Configuration Options . . . 282
76.5 Build & Run . . . 283

77 STIMER Sample **285**
77.1 Overview . . . 285
77.2 Features Demonstrated . . . 285
77.3 How It Works . . . 285
77.4 Configuration Options . . . 286
77.5 Build & Run . . . 287

78 Watchdog Sample **289**
78.1 Overview . . . 289
78.2 Features Demonstrated . . . 289
78.3 How It Works . . . 289
78.4 Configuration Options . . . 290
78.5 Build & Run . . . 290

79 USB CDC Sample **291**
79.1 Overview . . . 291
79.2 Features Demonstrated . . . 291
79.3 How It Works . . . 291
79.4 Configuration Options . . . 292
79.5 Build & Run . . . 293

80 Random Sample **295**
80.1 Overview . . . 295
80.2 Features Demonstrated . . . 295
80.3 How It Works . . . 295
80.4 Configuration Options . . . 296
80.5 Build & Run . . . 296

81 NV Storage Sample **297**
81.1 Overview . . . 297
81.2 Features Demonstrated . . . 297
81.3 How It Works . . . 297
81.4 Configuration Options . . . 298
81.5 Build & Run . . . 298

82 System Log Sample **299**
82.1 Overview . . . 299
82.2 Features Demonstrated . . . 299
82.3 How It Works . . . 299
82.4 Configuration Options . . . 301
82.5 Build & Run . . . 301

83 Task Planner Sample **303**
83.1 Overview . . . 303
83.2 Features Demonstrated . . . 303
83.3 How It Works . . . 303
83.4 Configuration Options . . . 305
83.5 Build & Run . . . 305

84 Audio Sample **307**
84.1 Overview . . . 307
84.2 Features Demonstrated . . . 307
84.3 How It Works . . . 307
84.4 Configuration Options . . . 309
84.5 Build & Run . . . 309

85 mbedTLS Sample **311**
85.1 Overview . . . 311
85.2 Features Demonstrated . . . 311
85.3 How It Works . . . 311
85.4 Configuration Options . . . 312
85.5 Build & Run . . . 313

86 MCUboot Bootloader Sample **315**
86.1 Overview . . . 315
86.2 Features Demonstrated . . . 315
86.3 How It Works . . . 315
86.4 Configuration Options . . . 317
86.5 Build & Run . . . 318

87 API Reference **319**
87.1 Auto-Generation Notes . . . 319

88 Kconfig Reference **321**
88.1 Planned Content . . . 321
88.2 Implementation Plan . . . 321

89 YAML Description Reference **323**
89.1 Planned Content . . . 323
89.2 Implementation Plan . . . 323

90 Releases **325**
90.1 Version Naming Convention . . . 325
90.2 Current Version . . . 325
90.3 Version Compatibility . . . 325
90.4 Version History . . . 326
90.5 Upgrade Guide . . . 326

91 V0.2.0(FR) **327**
91.1 Version . . . 327

91.2 CodeSize . . . 329

92 V0.3.0(FR) 331
92.1 Version . . . 331
92.2 Features . . . 331
92.3 Bug Fixes . . . 332
92.4 BREAKING CHANGES . . . 333
92.5 Note . . . 334
92.6 Royalty fee for certain Audio Codec: . . . 334
92.7 CodeSize . . . 334

93 V0.4.0(FR) 337
93.1 Version . . . 337
93.2 Features . . . 337
93.3 Bug Fixes . . . 339
93.4 BREAKING CHANGES . . . 340
93.5 Note . . . 341
93.6 CodeSize . . . 341

94 Contribution Guide 353
94.1 Documentation Repository: unisdk-doc . . . 353
94.2 Document Version vs SDK Version . . . 354
94.3 Release Process . . . 354
94.4 Multi-Version Maintenance . . . 355
94.5 CI/CD Behavior . . . 355
94.6 Document Structure Standards . . . 356
94.7 Submitting Issues . . . 358
94.8 Code Standards (SDK Contributions) . . . 358

95 Technical Support 359
95.1 Contact Us . . . 359
95.2 Submitting Issues . . . 359
95.3 Community Resources . . . 359
95.4 FAQ . . . 359
95.5 Resource Links . . . 360

96 UniSDK Glossary 361
96.1 Build System . . . 361
96.2 West Tool . . . 361
96.3 Hardware Abstraction . . . 361
96.4 Kconfig Configuration . . . 362
96.5 Peripherals & Drivers . . . 362
96.6 Document Status Markers . . . 362

97 About Telink Semiconductor 363
97.1 Our Mission . . . 363
97.2 Technology Leadership . . . 363
97.3 Global Presence . . . 363
97.4 Why Telink? . . . 364
97.5 Learn More . . . 364

Tip

Document Status: `STABLE` indicates content is complete and ready for use; `DRAFT` indicates content is under development. Issues and PRs are welcome to help improve documentation.

- **Multi-Chip Unified Platform** —One codebase, one build system, covering TL321X / TL721X / TLSR922X / TLSR952X full chip series
- **Modular Build System** —Flexible configuration based on CMake + Kconfig, supporting three build entry points: West / Make / CMake
- **Rich Peripheral Drivers** —Unified API layer providing full series peripheral drivers: GPIO, UART, I2C, SPI, ADC, DMA, USB, etc.
- **BLE Protocol Stack** —Integrated BLE Controller, supporting flexible wireless connectivity applications
- **Low Power Management** —Comprehensive power management framework, supporting multiple low power modes: Suspend / Retention, etc.
- **Pinmux Visualization Tool** —Graphical pin multiplexing configuration, simplifying hardware design
- **BDT Programming & Debugging** —One-click Flash programming, reading, erasing via `west tl-bdt`

</details>

CHAPTER

ONE

UNISDK INTRODUCTION

1.1 Telink Semiconductor

Telink Semiconductor is a company focused on designing low-power wireless connectivity chips, providing complete chip solutions covering protocols such as BLE, Zigbee, Thread, and Matter. Telink chips are widely used in smart homes, wearable devices, IoT sensors, and other scenarios.

1.2 UniSDK Core Features and Advantages

UniSDK (Unified Software Development Kit) provides a unified software development platform for the full range of Telink chips. The core advantages include:

1.2.1 Unified Development Experience

- **One Codebase, Multi-Chip Adaptation**: Based on a unified API layer and HAL abstraction, application code can seamlessly migrate between different Telink chips
- **Unified Build System**: All chip families share the same CMake + Kconfig build infrastructure

1.2.2 Modern Build System

UniSDK integrates industry-proven build toolchains:

Component	Purpose	Description
CMake	Build system core	Project configuration, dependency management, build rule generation
Ninja	Build execution engine	High-performance parallel compilation
West	Project management tool	Zephyr community-standard project management workflow
Kconfig	Configuration management system	Hierarchical management of compile-time configuration options
Make	Legacy interface	Frontend interface compatible with traditional Make usage habits

1.2.3 Rich Peripheral Drivers

Provides GPIO, UART, I2C, SPI, ADC, DMA, Watchdog, Timer, and other full-range peripheral drivers, encapsulating underlying hardware differences with a unified API.

1.2.4 Low-Power Design

A comprehensive power management framework supporting multiple low-power modes, delivering excellent energy efficiency for battery-powered devices.

1.3 Architecture Overview

1.4 Supported Chip Families

Chip Family	Core	Key Features	Typical Applications
TL321X	RISC-V	BLE, GPIO, UART, I2C, SPI, ADC, DMA	General IoT
TL721X	RISC-V	BLE, GPIO, UART, I2C, SPI, ADC, DMA, USB	General IoT
TLSR922X	RISC-V	BLE, GPIO, UART, I2C, SPI, ADC	General IoT
TLSR952X	RISC-V	BLE, GPIO, UART, I2C, SPI, ADC, DMA	General IoT

1.5 License

UniSDK is licensed under the Apache License 2.0.

1.6 Community Resources

- Official Website: https://www.telink-semi.com
- Technical Forum: Telink Developer Forum

1.7 Next Steps

- *Getting Started* —Set up the development environment
- *Development Guide* —Deep dive into the SDK development workflow
- *Samples & Demos* —Run code examples directly

CHAPTER

TWO

OVERVIEW

2.1 About This Document

This document guides you through the complete process of getting started with Telink UniSDK (Unified Software Development Kit) - from setting up the development environment, obtaining the SDK, and compiling and flashing your first project, to successfully running the first example. This guide is designed for embedded software developers who are new to the UniSDK and have a basic understanding of the C programming language and embedded development fundamentals.

2.2 Scope

Telink UniSDK supports the Bluetooth® Low Energy (BLE) protocol stack. It is applicable to the following chip series:

Chip Series	Core
TL321X	RISC-V
TL721X	RISC-V
TLSR922X	RISC-V
TLSR952X	RISC-V

See the *Release Notes* for details on supported chips and development boards.

2.3 Hardware and Software Requirements

2.3.1 Hardware Requirements

- A Telink official evaluation kit (choose one):
  - TL3218X_EVK
  - TL7218X_EVK
  - TLSR9528A_EVK
  - TLSR9228A_EVK
  - TLSR9528A_DONGLE
- A Telink programmer for flashing firmware (choose one):

- Programmer V1.0 ~ V3.0
- Programmer V5.0

2.3.2 Software Requirements

Component	Description
Operating System	Linux (Ubuntu 20.04+), Windows 10/11, macOS 12+
Git	Version control system
CMake	Build system generator (>= 3.20.0)
Ninja	Build system runner (>= 1.10)
Python	Scripting and west tool runtime (>= 3.10)
West	Zephyr project management tool (>= 0.14.0)
Telink RISC-V Toolchain	Cross-compilation toolchain (RISC-V GCC V5.4.1+)
BDT	Telink Burning and Debugging Tool (for flashing)

2.3.3 Quick Start Flow

Follow these steps in order:

1. ***Set Up the Development Environment*** —Install VS Code Extension and dependencies
2. ***Get and Import the SDK*** —Clone the repository and understand the directory layout
3. ***Compile, Flash and Run Your First Example*** —Build and run the gpio_demo

2.4 Next Steps

- *Development Guide* —Deep dive into the SDK development workflow
- *Build & Configuration* —Learn the CMake and Kconfig build system
- *Samples & Demos* —Explore more example projects
- *Peripherals & Drivers* —Learn to use GPIO, UART, I2C and other peripheral drivers

CHAPTER

THREE

DEVELOPMENT ENVIRONMENT SETUP

This chapter describes how to install and configure the UniSDK development environment. The recommended approach is to use the Telink VS Code Extension, which provides a graphical interface for managing dependencies. Alternatively, you can set up the environment from the command line.

3.1 VS Code Extension (Recommended)

The **Telink Development Tool** VS Code Extension provides a graphical interface for installing dependencies, managing the toolchain, and initializing the workspace with a single click.

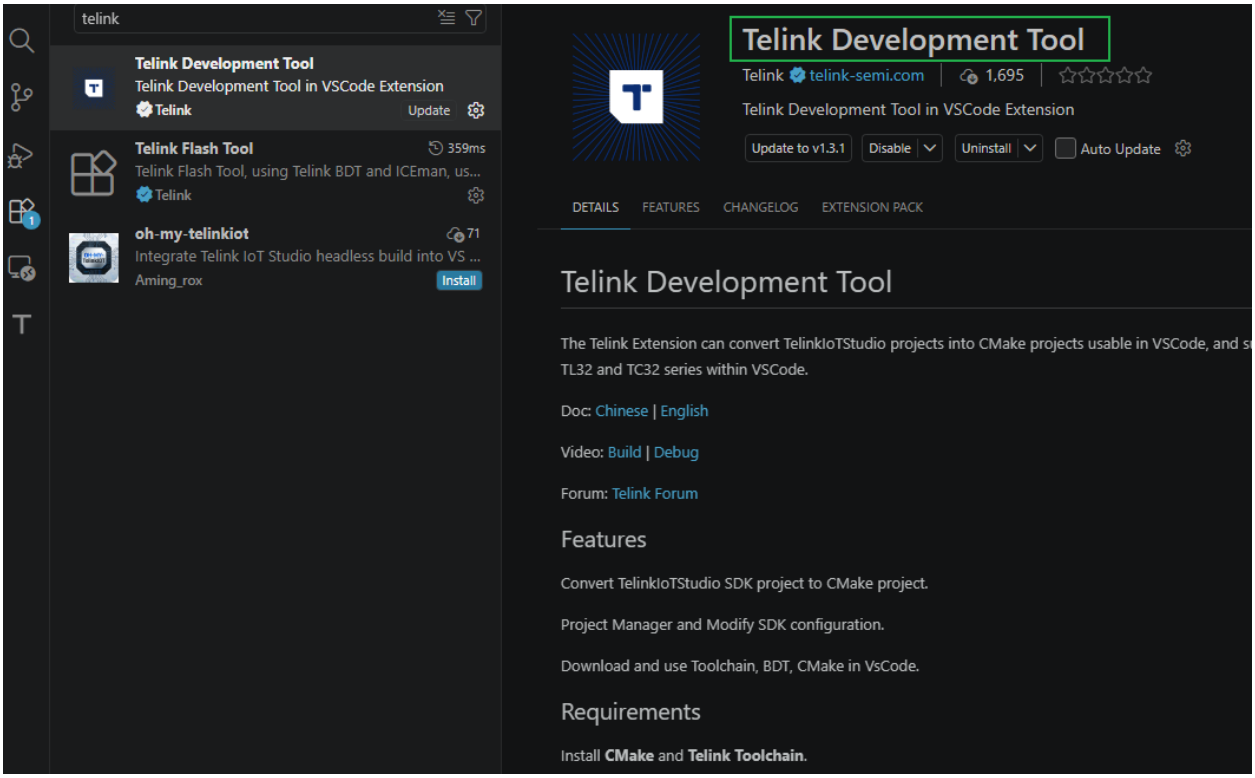
3.1.1 Prerequisites

- **VS Code**: Install the latest version of Visual Studio Code.

  If your VS Code version is not the latest, the extension may not work properly or some features may not function as expected.

3.1.2 Step 1: Install the Telink VS Code Extension

1. Launch VS Code.
2. Click the **Extensions** icon in the left activity bar (or press **Ctrl+Shift+X**).
3. Search for **Telink**.
4. Find **Telink Development Tool** in the results and click **Install**.

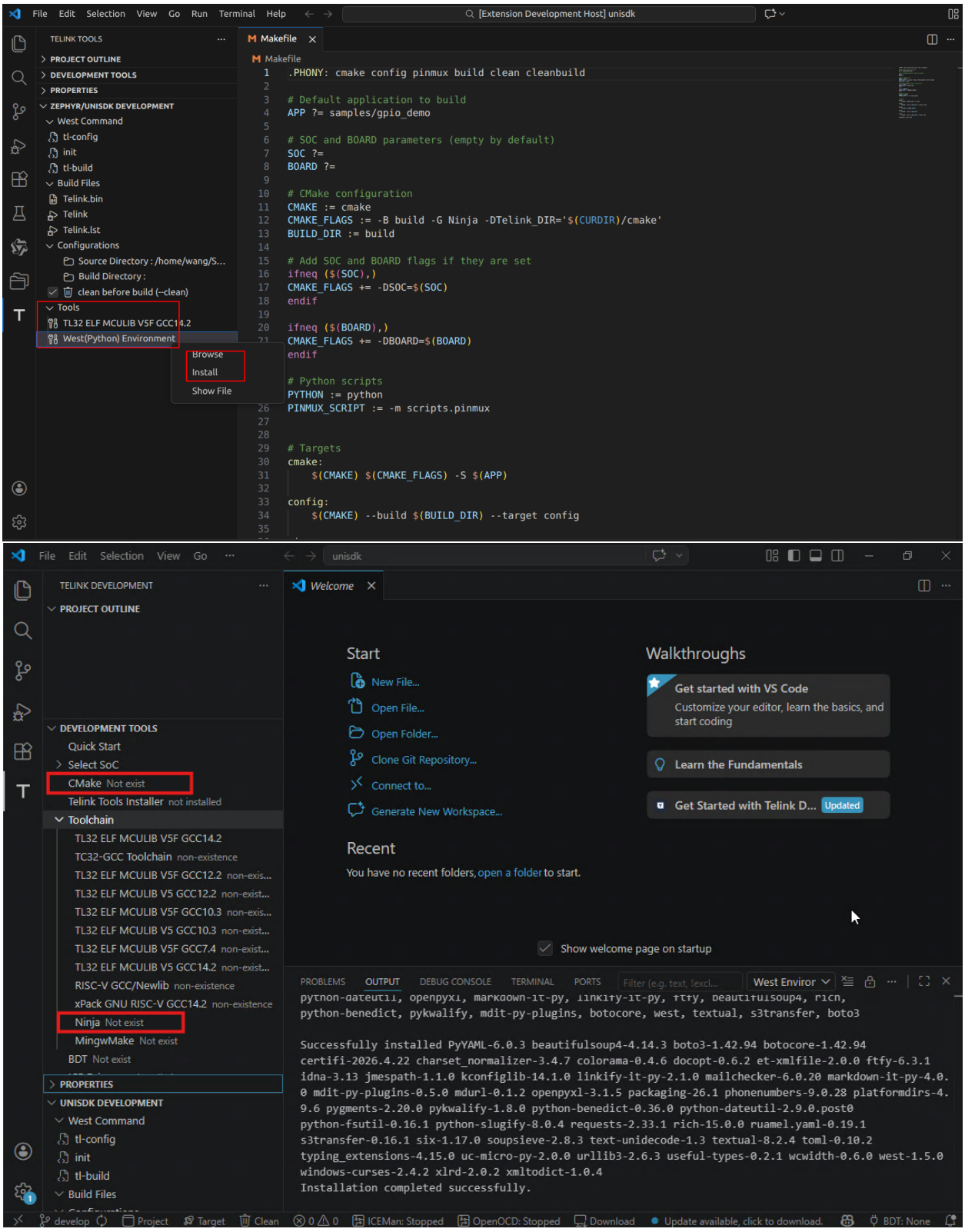


After installation, a **Telink DEVELOPMENT** icon will appear on the left sidebar.

> To facilitate automatic identification of the Unified SDK by subsequent tools, open the Unified SDK folder in VS Code after completing the installation.

3.1.3 Step 2: Install Dependencies via the Extension

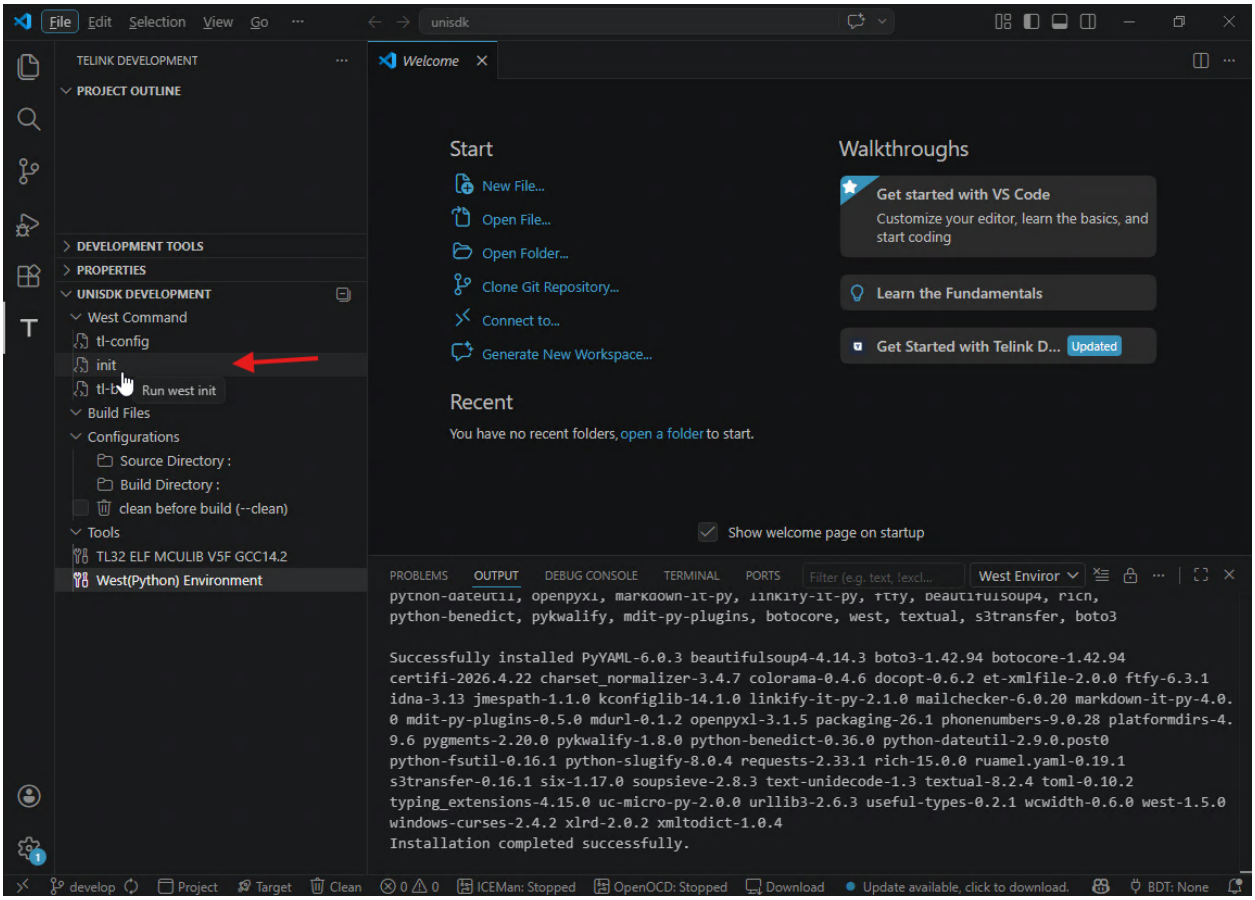
1. Click the **Telink DEVELOPMENT** button in the left sidebar.
2. In the **DEVELOPMENT TOOLS** tree, you will see the required components: **Toolchain**, **West**, **CMake**, and **Ninja**.
3. Install each component:
   - **Toolchain**: Click the toolchain entry and select **Install Toolchain**.
   - **West**: Right-click **West** and select **Install**. We recommend using a Python virtual environment for West.
   - **CMake**: Right-click **CMake** and select **Install**.
   - **Ninja**: Right-click **Ninja** and select **Install**.



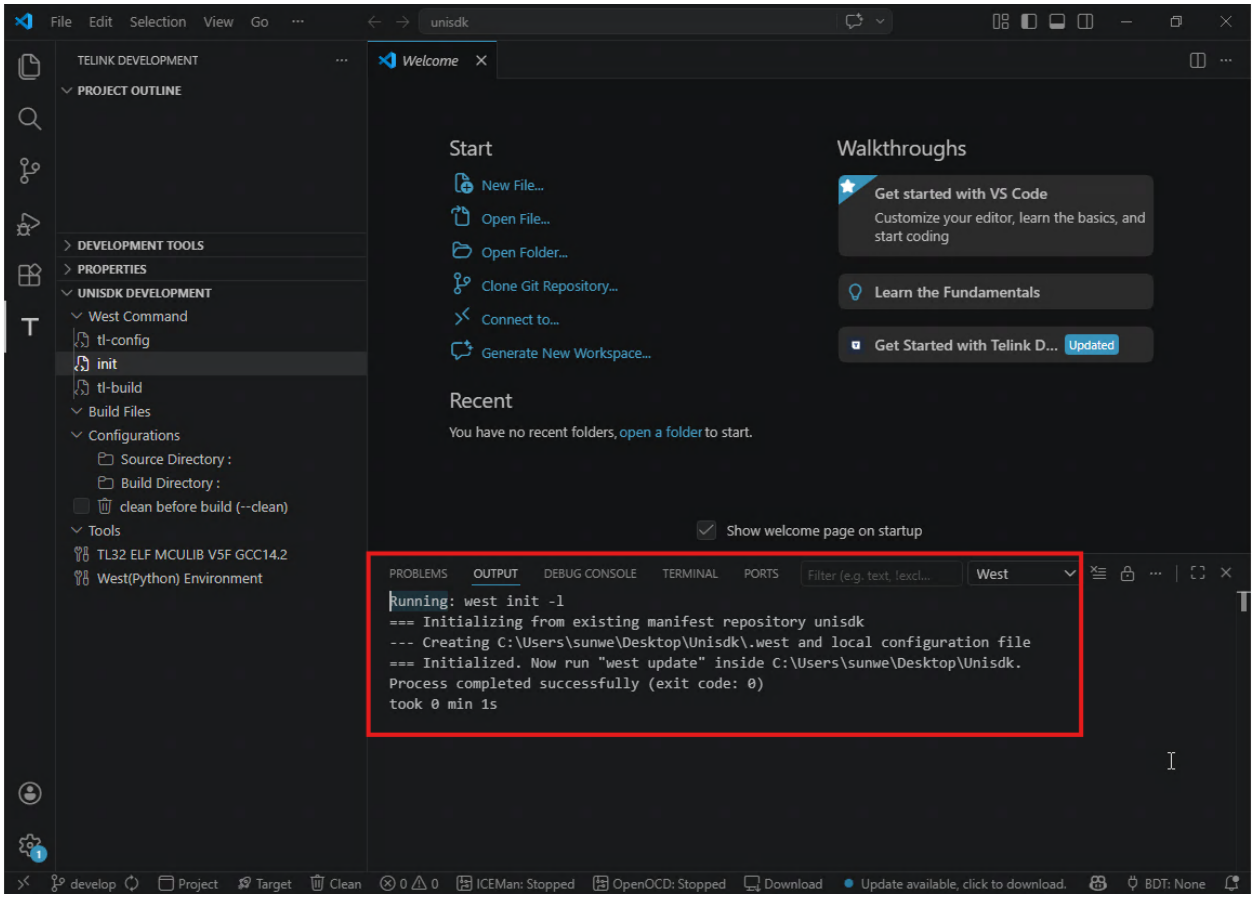
Wait for each component to finish installing. A notification appears in the bottom-right corner when installation is complete.

3.1.4 Step 3: Initialize the West Workspace

1. In the **Telink DEVELOPMENT** view, locate the **West Command** section.
2. Click the **init** button.

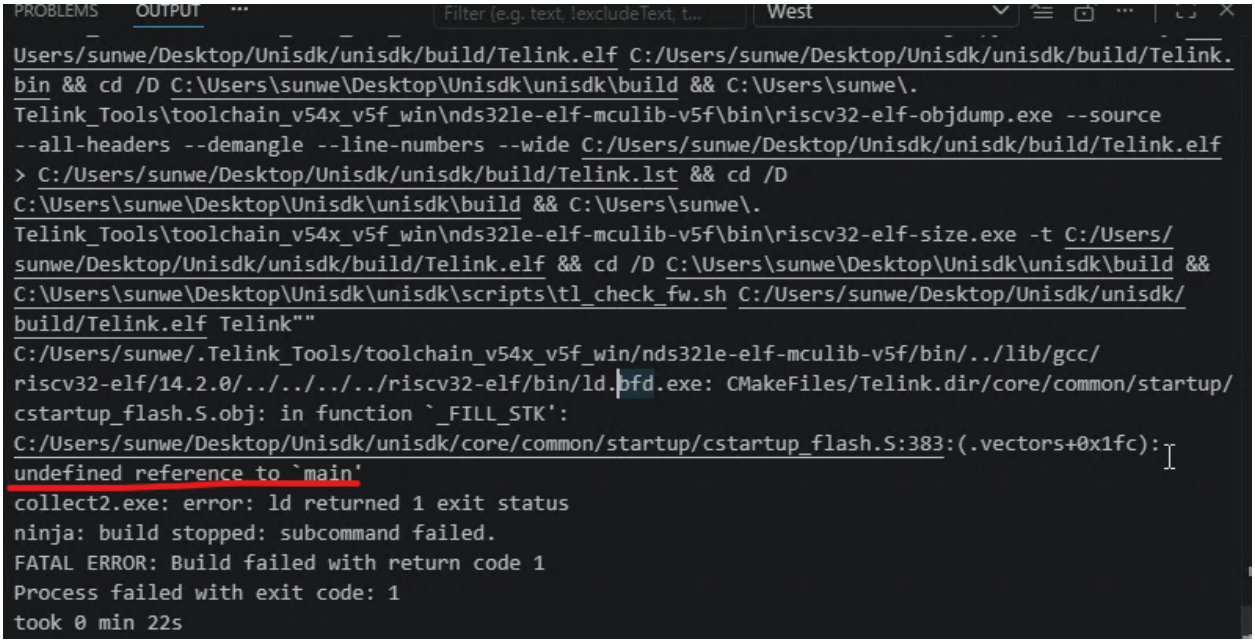


3. Check the **OUTPUT** terminal panel. A confirmation message appears when initialization completes successfully.



3.1.5 Step 4: Verify the Environment

1. In the **West Command** section, click the **tl-build** button.
2. A linker error is expected because no source directory has been configured yet. This indicates that the toolchain, build system, and West environment are configured correctly.



3.2 CLI Setup (Alternative)

If you prefer to set up the development environment from the command line, refer to the platform-specific guides:

- *Linux Environment Setup*
- *Windows Environment Setup*
- *macOS Environment Setup*

3.3 Next Steps

After completing the environment setup, proceed to *Get and Import the SDK*.

CHAPTER

FOUR

GET AND IMPORT THE SDK

4.1 Get the SDK

Use Git to obtain the UniSDK:

```
git clone https://github.com/telink-semi/unisdk.git
cd unisdk
```

4.2 SDK Directory Structure

After obtaining the SDK, the top-level directory structure is organized as follows:

SDK Top-Level Directory Structure

```
unisdk/
├── api/              # Public API layer
├── core/             # Core peripheral drivers
├── soc/              # SoC-specific features and drivers
├── boards/           # Board configurations (pinmux, etc.)
├── common/           # Common libraries
├── subsystem/        # Functional subsystems (e.g., BLE Controller)
├── modules/          # Third-party module integrations (e.g., TinyUSB)
├── samples/          # Example projects
├── cmake/            # CMake build system modules
└── scripts/          # Build scripts and tools
```

For a detailed explanation of each directory, see *SDK Directory Structure*.

4.3 Next Steps

After obtaining the SDK, proceed to *Compile, Flash and Run Your First Example*.

CHAPTER

FIVE

COMPILE, FLASH AND RUN YOUR FIRST EXAMPLE (VS CODE)

This guide walks you through compiling, flashing, and running the first UniSDK example program using the **VS Code Extension GUI**. If you prefer the command-line interface, see the *CLI-based Blinky Guide*.

5.1 Select the First Example

The recommended first example is **gpio_demo** (GPIO LED blinking). It has the simplest logic (only peripheral initialization and a basic loop), the fewest dependencies, and does not involve complex wireless protocol stacks. This provides the shortest path to verify that your toolchain, SDK configuration, and hardware connection (programmer to target board) are all functioning correctly.

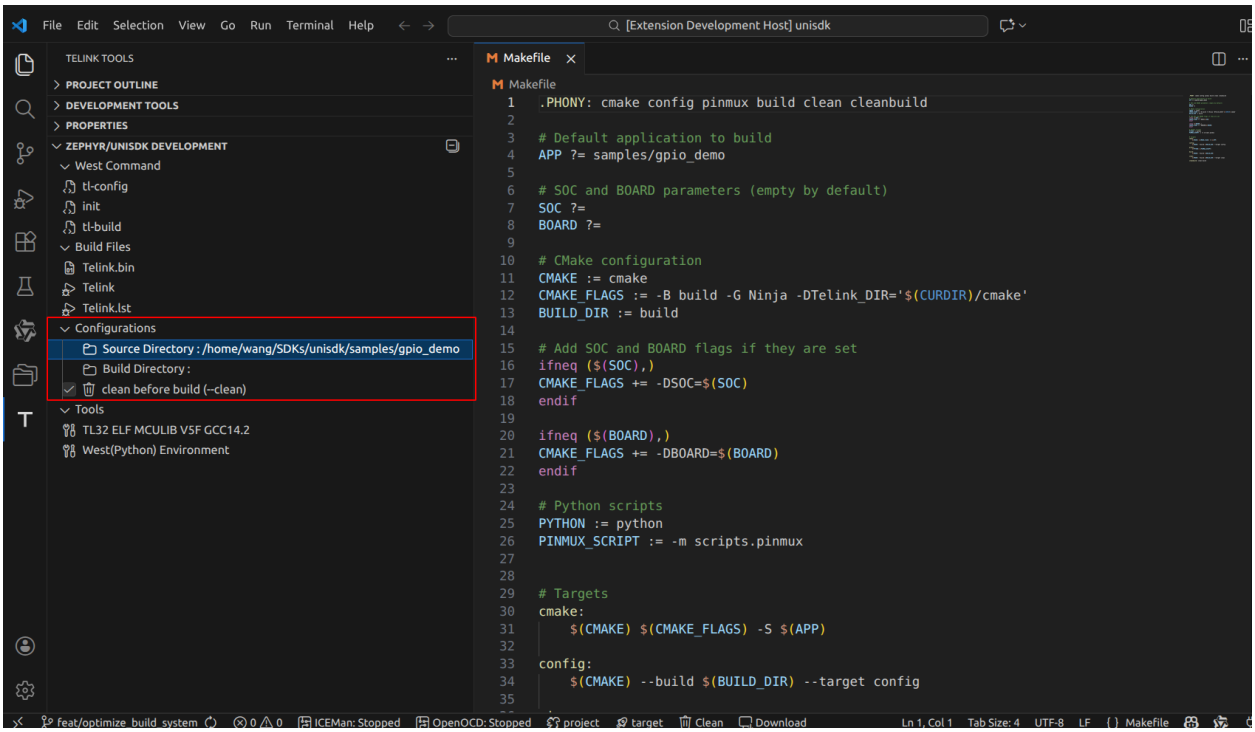
Source code location: `samples/gpio_demo/main.c`

5.2 Build the Project

5.2.1 VS Code Extension (Recommended)

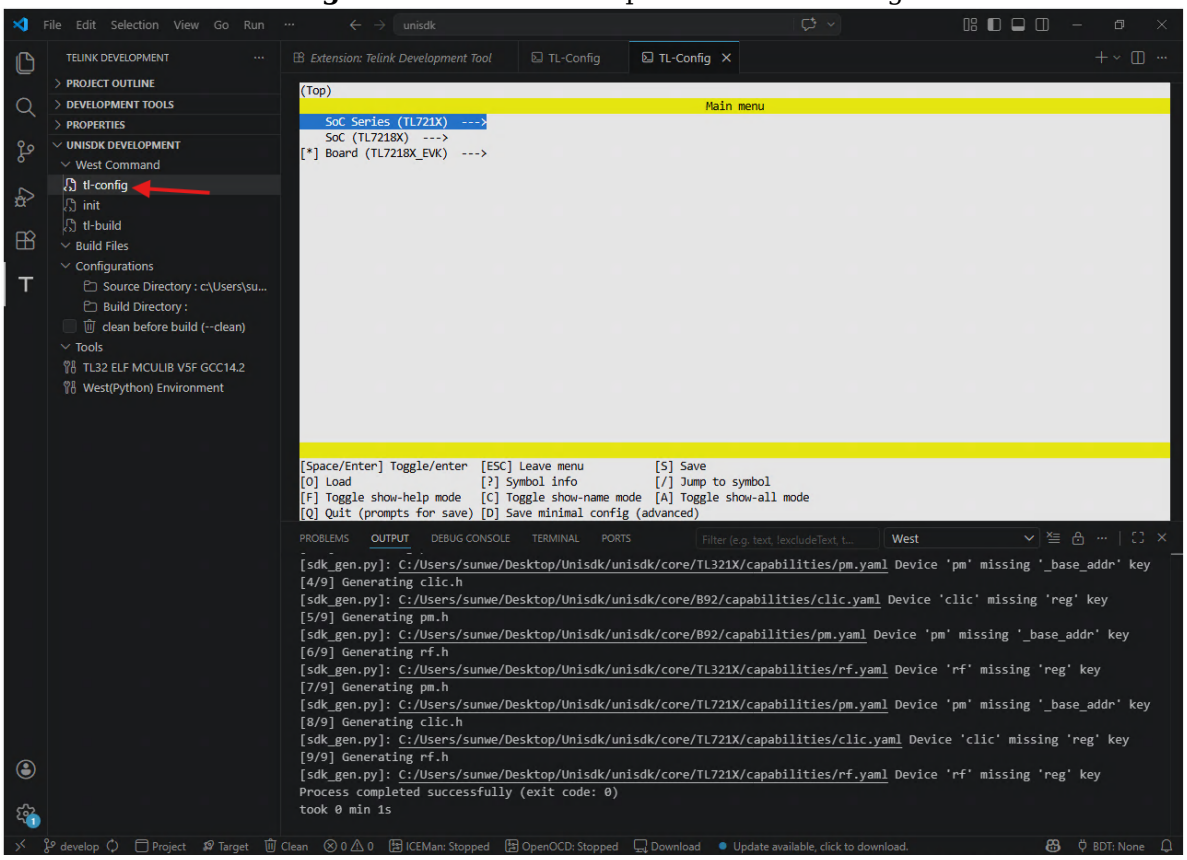
Step 1: Select the Source Directory

In the **Telink DEVELOPMENT** view, click the **Source Directory** button (or the folder icon) and choose the `samples/gpio_demo` directory.

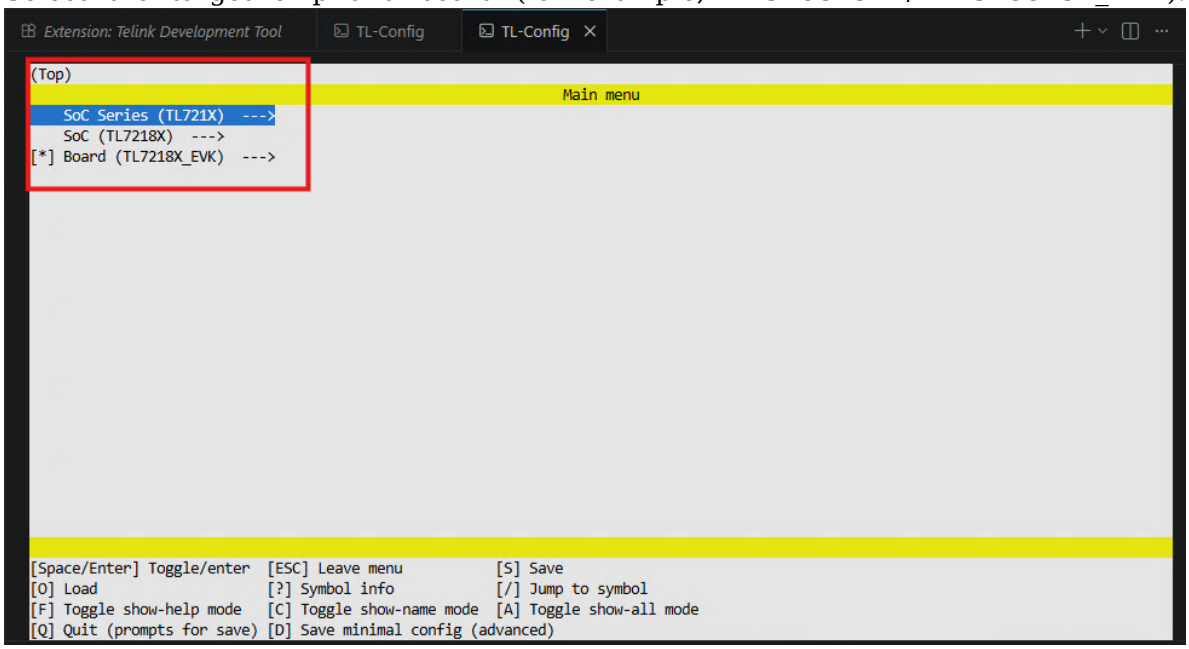


Step 2: Configure the Chip and Board

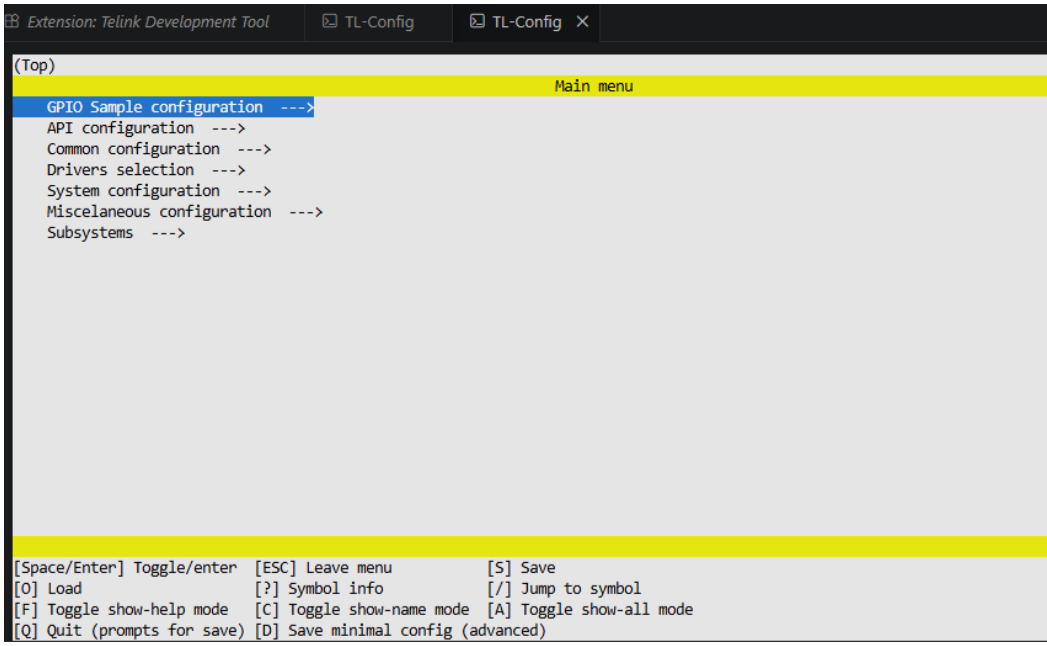
1. Click the **tl-config** button to open the configuration interface.



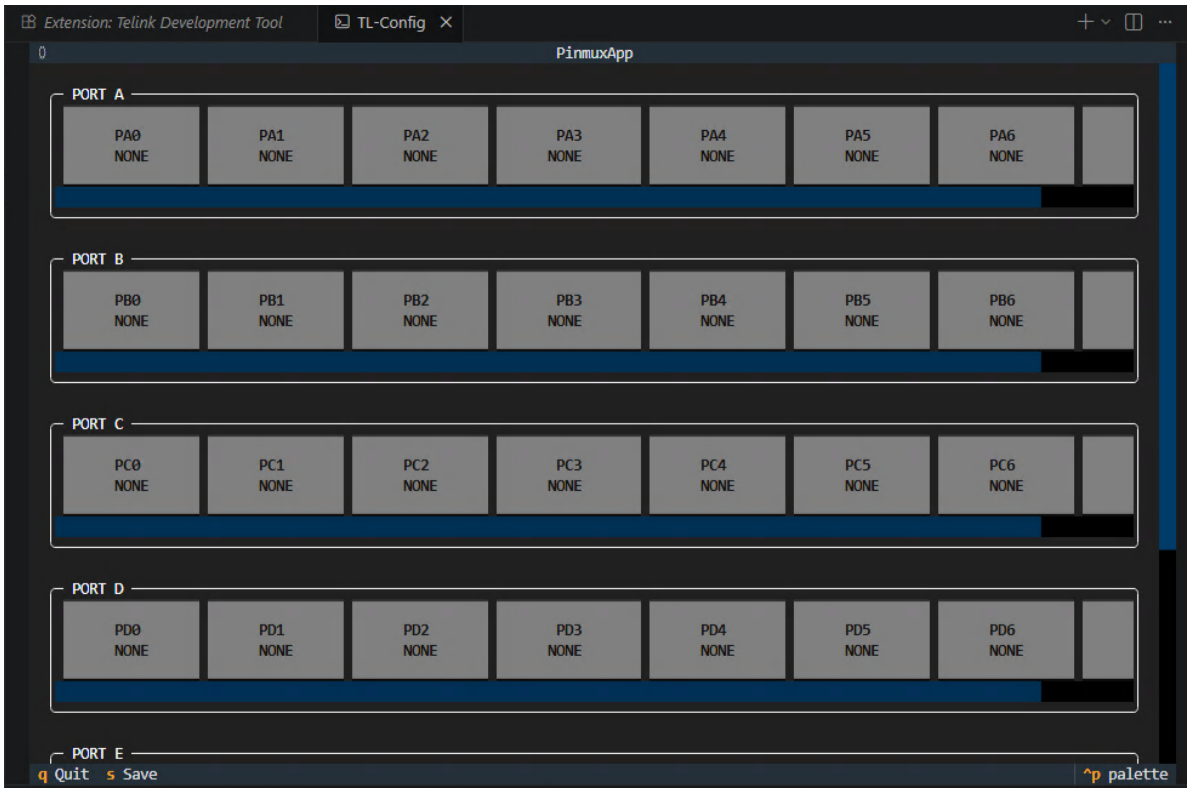
2. Select the target chip and board (for example, TLSR9528A / TLSR9528A_EVK).



3. Configure build options.



4. Configure the pinmux (pin functions). Press **Q** to save and exit at each step.



Step 3: Build

Click **tl-build** to start the build. The build output appears in the terminal panel.

After a successful build, the output files are generated in the `build/` directory:

Build Output Files

```
build/
├── Telink.bin       # Firmware binary file
├── Telink.elf       # ELF executable (contains debug information)
└── Telink.map       # Memory map file
```

5.2.2 CLI Approach (Alternative)

For command-line users, refer to the *CLI-based Blinky Guide* for detailed CLI build and flash instructions.

5.3 Flash the Firmware

5.3.1 Prerequisite: Connect the Hardware

1. Connect the PC to the Telink programmer using a USB cable.
2. Connect the programmer to the target board using jumper wires:
   - **Power**: VCC <-> VCC, GND <-> GND

- **Data**: SWM (Programmer) <-> SWS (Target board)

5.3.2 Flash via BDT (CLI)

```
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

To specify a flash address:

```
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x00
```

For GUI-based flashing instructions, refer to the *BDT Tool Guide*.

5.3.3 Flash via JTAG

JTAG flashing requires the ICEman backend service. For detailed instructions, refer to the *Development Guide*.

5.4 Verify the Result

After flashing is complete:

1. The development board resets automatically (or press the **Reset** button manually).
2. Observe the on-board LED —it should blink at a fixed interval.
3. (Optional) Connect a serial terminal to view debug log output.

Expected Result: The on-board LED blinks periodically, indicating that the application is running successfully on the target chip.

5.5 Next Steps

Congratulations on successfully running your first UniSDK program! Continue exploring:

- *Development Guide* —Deep dive into the SDK development workflow
- *Detailed Blinky Guide* —Step-by-step breakdown of the example
- *Samples & Demos* —Explore more example projects
- *Peripherals & Drivers* —Learn to use GPIO, UART, I2C and more

CHAPTER

SIX

TROUBLESHOOTING

6.1 Environment Issues

6.1.1 TELINK_TOOLCHAIN_PATH Not Found

Error: `Error: TELINK_TOOLCHAIN_PATH environment variable not set`

Solution:

```
# Linux/macOS
export TELINK_TOOLCHAIN_PATH=/path/to/toolchain

# Windows CMD
set TELINK_TOOLCHAIN_PATH=path\to\toolchain

# Windows PowerShell
$env:TELINK_TOOLCHAIN_PATH="path\to\toolchain"
```

6.1.2 west init -l Reports "already initialized"

Cause: The `ZEPHYR_BASE` environment variable points to a Zephyr workspace containing a `.west` directory.

Solution:

```
# Linux/macOS
unset ZEPHYR_BASE && west init -l

# Windows PowerShell
Remove-Item Env:ZEPHYR_BASE; west init -l
```

6.1.3 CMake Configuration Failed

Solution:

1. Verify the CMake version (requires >= 3.20.0).
2. Clear old build cache and retry:

   ```
   west tl-build --clean
   ```

3. Verify that all required dependencies are installed correctly.

6.1.4 Python Dependency Installation Failed

Solution:

- Check your network connection.
- Verify that the virtual environment is activated (the terminal prompt should have a `(.venv)` prefix).
- If required, configure a proxy:

```
pip install --proxy http://proxy:port -r requirements.txt
```

6.2 Build Issues

6.2.1 Compilation Error

Solution:

1. Check the source code for syntax errors.
2. Verify that the chip and board are configured correctly.
3. Perform a full clean and retry:

```
west tl-build --pristine always
```

6.2.2 Cannot Find pinmux.h

Solution:

```
# Manually generate pinmux configuration
make pinmux
# or
python -m scripts.pinmux
```

6.3 Menuconfig Issues

6.3.1 Arrow Keys Not Working in menuconfig Within VS Code (Windows)

Solution:

- Use the letter keys instead of the arrow keys.
- Use Windows 11.
- Use Command Prompt (CMD) instead of PowerShell for menuconfig operations.

6.4 Getting Help

- *Technical Support*
- *Submit an Issue*

CHAPTER

SEVEN

DEVELOPMENT GUIDE

This chapter is aimed at developers who have started using UniSDK and introduces the complete development workflow and best practices.

7.1 Content Navigation

Chapter	Description	Status
Environment Setup	Platform-specific setup guides (Linux / Windows / macOS)	STABLE
First Example: Blinky	Detailed step-by-step breakdown of the gpio_demo example	STABLE
SDK Directory Structure	Code organization conventions, responsibilities of each directory	STABLE
Project Creation & Management	Creating new projects, CMakeLists.txt configuration	STABLE
Core Architecture	Startup flow, exception handling, interrupt management, low-power mechanisms	STABLE
Debugging & Testing	Hardware debugging, logging system, shell interaction	STABLE
IDE Integration	VS Code clangd configuration, Eclipse, Segger Embedded Studio	STABLE

7.2 Quick Start for a New Project

```
# 1. Create project directory (can be anywhere)
mkdir my_app && cd my_app

# 2. Write CMakeLists.txt (see project_management.md)
cat > CMakeLists.txt << 'EOF'
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(MyApp LANGUAGES C)
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})
EOF
```

(continues on next page)

(continued from previous page)

```
# 3. Write main.c
cat > main.c << 'EOF'
#include <tlk_api.h>

int main(void) {
    while (1) {
        tlk_sleep_ms(1000);
    }
    return 0;
}
EOF

# 4. Build
west tl-build . --board TLSR9528A_EVK
```

7.3 Development Workflow Overview

7.4 Next Steps

- *Build & Configuration* —Deep dive into the CMake and Kconfig build system
- *Peripherals & Drivers* —Use GPIO, UART, I2C and other peripheral drivers
- *Samples & Demos* —Reference ready-to-run example code

CHAPTER

EIGHT

LINUX ENVIRONMENT SETUP

This document details the complete steps to set up the UniSDK development environment on a Linux system (using Ubuntu as an example).

8.1 System Requirements

- Ubuntu 20.04 LTS or higher
- At least 4 GB RAM, 8 GB+ recommended
- Sufficient disk space (SDK + toolchain requires approximately 2 GB)

8.2 Install System Dependencies

```
sudo apt update
sudo apt install --no-install-recommends git cmake ninja-build \
    python3-dev python3-venv python3-tk make
```

Verify installation:

```
cmake --version     # Should be >= 3.20.0
python3 --version   # Should be >= 3.10
git --version
ninja --version
```

8.3 Set Up Python Virtual Environment

```
# Enter the UniSDK directory
cd /path/to/unisdk

# Create virtual environment
python3 -m venv .venv

# Activate virtual environment
source .venv/bin/activate

# Upgrade pip and install dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

Tip

Recommendation Remember to activate the virtual environment before each development session:

```
source .venv/bin/activate
```

8.4 Initialize West

```
# Verify west installation
west --version

# Initialize workspace
west init -l
```

Warning

Common Issue: already initialized If you encounter the `already initialized in {path}, aborting.` error:

```
unset ZEPHYR_BASE
west init -l
```

8.5 Configure Toolchain

1. Download the Telink RISC-V toolchain (contact the Telink team for access)
2. Extract the toolchain to a designated directory:

   ```
   mkdir -p ~/telink/toolchain
   tar -xzf telink_toolchain_*.tar.gz -C ~/telink/toolchain
   ```

3. Set the environment variable (recommend adding to `~/.bashrc` or `~/.zshrc`):

   ```
   export TELINK_TOOLCHAIN_PATH=$HOME/telink/toolchain
   ```

8.6 Configure BDT Tool (Optional)

If you need to use the `west tl-bdt` command for flashing and debugging:

```
# Set BDT tool path
export BDT_PATH=/path/to/BDT

# Or specify temporarily at runtime
west tl-bdt download --chip TLSR9528A --bdt-path /path/to/BDT -i build/
↪firmware.bin
```

8.7 Verify Environment

```
# Test building an example
west tl-build samples/gpio_demo

# If the build succeeds, you will see output similar to:
# [100%] Built target Telink
```

8.8 Next Steps

- *First Example: Blinky*
- *Getting Started*

CHAPTER

NINE

MACOS ENVIRONMENT SETUP

This document details the complete steps to set up the UniSDK development environment on macOS.

9.1 System Requirements

- macOS 12 (Monterey) or higher
- Apple Silicon (M1/M2/M3) or Intel processor
- At least 4 GB RAM

9.2 Install System Dependencies

We recommend using the Homebrew package manager:

```
# Install Homebrew (if not already installed)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/
↪HEAD/install.sh)"

# Install UniSDK dependencies
brew install cmake ninja python3 python-tk git
```

Verify installation:

```
cmake --version     # Should be >= 3.20.0
python3 --version   # Should be >= 3.10
git --version
ninja --version
```

9.3 Set Up Python Virtual Environment

```
# Enter the UniSDK directory
cd /path/to/unisdk

# Create virtual environment
python3 -m venv .venv
```

(continues on next page)

(continued from previous page)

```
# Activate virtual environment
source .venv/bin/activate

# Install Python dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

9.4 Initialize West

```
west --version
west init -l
```

9.5 Configure Toolchain

Since UniSDK uses the RISC-V cross-compilation toolchain, you need to obtain the corresponding macOS version of the toolchain from Telink.

1. Contact Telink to obtain the macOS toolchain
2. Extract to a designated directory:

   ```
   mkdir -p ~/telink/toolchain
   tar -xzf telink_toolchain_macos_*.tar.gz -C ~/telink/toolchain
   ```

3. Set the environment variable:

   ```
   export TELINK_TOOLCHAIN_PATH=$HOME/telink/toolchain
   ```

   It is recommended to add the above command to `~/.zshrc` (the default shell on macOS).

9.6 Verify Environment

```
# Test build
west tl-build samples/gpio_demo
```

9.7 Next Steps

- *First Example: Blinky*
- *Linux Environment Setup*
- *Getting Started*

CHAPTER

TEN

WINDOWS ENVIRONMENT SETUP

This document details the complete steps to set up the UniSDK development environment on Windows 10/11.

10.1 System Requirements

- Windows 10 (1709+) or Windows 11
- At least 4 GB RAM, 8 GB+ recommended
- Administrator privileges (recommended)

10.2 Installation Methods

10.2.1 Method 1: One-Click Setup Script (Recommended)

UniSDK provides an automated deployment script (`windows_setup.ps1`) that simplifies the environment configuration process on Windows systems.

1. Open PowerShell as Administrator
2. Navigate to the scripts directory and run:

```
cd <path_to_unisdk_repository>\scripts
.\windows_setup.ps1
```

Script Features in Detail

The script automatically performs the following tasks:

1. **PowerShell Execution Policy Configuration**
   - Checks and sets the execution policy to `RemoteSigned` to allow running local scripts
2. **Tool Installation**
   - Detects if `winget` is installed
   - If winget is available: automatically downloads and installs CMake, Ninja, Python, and Git
   - If winget is not available: provides official download links to guide manual installation
3. **Python Environment Configuration**

- Creates `.venv` virtual environment in the UniSDK root directory
- Upgrades pip to the latest version
- Installs all Python dependencies from `requirements.txt`
- Ensures west tool is properly installed

4. **Environment Variable Settings**
   - Sets `TELINK_TOOLCHAIN_PATH` pointing to the toolchain directory
   - Adds CMake and Ninja paths to the system PATH
5. **Telink Toolchain Setup**
   - Creates the `toolchain` directory
   - Sets `TELINK_TOOLCHAIN_PATH` environment variable
   - Due to special permissions required for toolchain download, users need to manually download and extract the toolchain
6. **West Workspace Initialization**
   - Initializes the west workspace by executing `west init -l`

Logging

The script automatically generates a `setup_log.txt` file in the script directory, recording detailed logs of the entire configuration process for troubleshooting purposes.

Note

Toolchain Requires Manual Download Since toolchain download requires special permissions, the script only creates the `toolchain` directory and sets the environment variable. Please manually download the toolchain and extract it to the `<unisdk>\toolchain` directory. Download link: https://drive.weixin.qq.com/s?k=AKwA0AfNAA8Ox1r0ab

10.2.2 Method 2: Manual Installation

1. Install System Dependencies

Use winget for one-click installation:

```
winget install Kitware.CMake Ninja-build.Ninja Python Git.Git
```

Or manually download and install if winget is not supported:

- CMake —Check "Add CMake to PATH"
- Ninja —Manually add to PATH
- Python —Check "Add Python.exe to PATH"
- Git

Verify installation:

```
cmake --version
python --version
git --version
ninja --version
```

2. Set Up Python Virtual Environment

```
# Enter the UniSDK directory
cd <path_to_unisdk>

# Create virtual environment
python -m venv .venv

# Activate virtual environment
.venv\Scripts\activate.ps1

# If you encounter a script execution permission error, run this first:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Install dependencies
pip install -r requirements.txt
```

3. Initialize West

```
west init -l
```

If you encounter the error `already initialized in {where}, aborting.`, temporarily unset ZEPHYR_BASE:

```
Remove-Item Env:ZEPHYR_BASE; west init -l
```

4. Configure Toolchain

```
# PowerShell
$env:TELINK_TOOLCHAIN_PATH="path\to\toolchain"

# or CMD
set TELINK_TOOLCHAIN_PATH=path\to\toolchain
```

10.3 Configure BDT Tool (Optional)

```
# Set BDT path
$env:BDT_PATH="C:\BDT"

# Or specify temporarily at runtime
west tl-bdt download --chip TLSR9528A --bdt-path C:\BDT -i build\firmware.bin
```

10.4 Verify Environment

```
# Test build
west tl-build samples\gpio_demo
```

10.5 Common Issues

10.5.1 PowerShell Script Cannot Execute

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
```

10.5.2 Environment Variables Not Taking Effect

- Restart the terminal, VS Code, or the system
- Check system environment variable settings
- If the previous step doesn't solve the problem, manually check if the PATH contains the correct path and add it to the user's PATH

10.5.3 Arrow Keys Not Working in menuconfig Within VS Code

- Use letter keys instead of arrow keys
- Use Windows 11
- Use CMD instead of PowerShell for menuconfig operations

10.5.4 winget Installation Failure

- Check Windows version (requires Windows 10 1709 or higher)
- Try installing App Installer from Microsoft Store

10.5.5 Python Dependencies Installation Failure

- Ensure network connection is working
- Proxy server settings might be needed

10.5.6 CMake or Ninja Not Found

- Manually check installation paths
- The script will attempt to automatically find common installation locations, but may require manual addition to PATH

10.6 Notes

1. **Administrator Privileges**: While not strictly necessary, it's recommended to run the script as administrator to ensure proper environment variable settings

2. **Environment Variables Effectiveness**: After script completion, open a new PowerShell window to apply all environment variable changes
3. **Virtual Environment Activation**: Use the following command to activate the virtual environment:

```
<path_to_unisdk>\.venv\Scripts\Activate.ps1
```

10.7 Next Steps

- *First Example: Blinky*
- *Linux Environment Setup*
- *Getting Started*

CHAPTER

ELEVEN

FIRST EXAMPLE: BLINKY (CLI)

Tip: If you prefer using the VS Code Extension GUI, see the *VS Code quick-start guide* instead.

This document details how to compile, flash, and run the first UniSDK example program — the GPIO demo (Blinky LED blinking).

11.1 Prerequisites

- *Environment setup* is complete
- Toolchain `TELINK_TOOLCHAIN_PATH` is configured
- You have a Telink development board on hand (this example uses TLSR9528A_EVK)

11.2 Example Overview

The `gpio_demo` example demonstrates basic GPIO input/output functionality and interrupt handling:

- Configures a GPIO pin as output mode, periodically toggling to achieve LED blinking
- Configures interrupt handling to respond to button input
- Demonstrates the use of sleep APIs

Source code location: `samples/gpio_demo/main.c`

11.3 Step 1: Configure Chip Type

```
# Method 1: Interactive configuration (recommended for first use)
west tl-config

# Method 2: Using Make (with SOC/BOARD preset)
make config SOC=TLSR9528A BOARD=TLSR9528A_EVK
```

Tip

Available Chips and Boards List all supported options:

```
west tl-socs      # View all chips
west tl-boards    # View all boards
```

11.4 Step 2: Compilation

```
# West command (recommended)
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Or using Make
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK
```

After a successful build, the output files are located in the `build/` directory:

Build Output Files

```
build/
├── Telink.bin     # Firmware binary file
├── Telink.elf     # ELF executable (contains debug information)
└── Telink.map     # Memory map file
```

11.5 Step 3: Flashing

```
# Flash using BDT tool
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Specify flash address
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x00
```

11.6 Step 4: Verification

After flashing is complete, the LED on the development board will blink at a fixed frequency, indicating that the program is running normally.

11.7 Custom Modifications

Open `samples/gpio_demo/main.c` to modify parameters such as the LED blinking frequency:

```
// Modify the delay time to change the blinking frequency
tlk_sleep_ms(500);  // 500ms = blinks once per second
```

After making changes, recompile and flash:

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

11.8 Next Steps

- *GPIO Driver Documentation*
- *Build System Details*
- *More Examples*

CHAPTER

TWELVE

SDK DIRECTORY STRUCTURE

This document details the directory organization of the UniSDK code repository to help developers quickly locate various resources.

12.1 Overall Structure

UniSDK Directory Tree

```
unisdk/
├── CMakeLists.txt              # CMake main entry point (root directory mode)
├── Makefile                    # Make command interface
├── west.yml                    # West workspace configuration
├── Kconfig.chip               # Chip Kconfig entry point
├── Kconfig.build              # Build Kconfig entry point
├── readme.md                   # Project overview
│
├── api/                        # Public API layer
│   ├── include/                # API header files
│   ├── src/                    # API implementations
│   ├── CMakeLists.txt
│   └── Kconfig
│
├── core/                       # Chip core drivers
│   ├── B92/
│   ├── TL321X/
│   ├── TL721X/
│   ├── common/
│   ├── configs/
│   ├── cpp_wrappers/
│   ├── include/
│   └── CMakeLists.txt
│
├── soc/                        # SoC layer
│   ├── TL321X/
│   ├── TL721X/
│   ├── TLSR922X/
│   └── TLSR952X/
│
```

(continues on next page)

(continued from previous page)

```
├── boards/                     # Board configurations
│   ├── TL3218X_EVK/
│   ├── TL7218X_EVK/
│   ├── TLSR9228A_EVK/
│   ├── TLSR9528A_EVK/
│   └── TLSR9528A_DONGLE/
│
├── common/
│   ├── include/
│   └── src/
│
├── subsystem/
│   └── ble/
│       └── controller/
│
├── modules/
│   └── tinyUSB/
│
├── samples/
│   ├── gpio_demo/
│   ├── uart_demo/
│   ├── pm_demo/
│   ├── adc_demo/
│   ├── dma_demo/
│   ├── i2c_demo/
│   ├── spi_demo/
│   ├── adv_demo/
│   └── ...
│
├── cmake/
│   ├── modules/
│   └── TelinkConfig.cmake
│
└── scripts/
    ├── west_commands/
    ├── pinmux/
    ├── kconfig.py
    ├── menuconfig.py
    └── ...
```

12.2 Key Directory Responsibilities

12.2.1 `api/` —Public API Layer

Provides the most commonly used unified API interfaces for application development. All example applications should be developed using the APIs in `api/`. Main contents include:

- `tlk_sleep.h` / `tlk_time.h` —Sleep delay and time management

12.2.2 `core/` —Core Driver Layer

Organized by chip family, includes:

- **drivers/** —Peripheral driver implementations (GPIO, UART, I2C, SPI, ADC, DMA, etc.)
- **properties/** —YAML-format device property descriptions
- **registers/** —YAML-format register definitions
- **configs/** —Kconfig configuration options

12.2.3 `soc/` —SoC-Specific Layer

Contains chip-specific features:

- **drivers/** —Platform-specific drivers (MSPI, OTP, etc.)
- **pinmux/** —Pin multiplexing definitions (yaml files for each package variant)
- **registers/** —Chip-specific register definitions

12.2.4 `boards/` —Development Board Configurations

Contains only board-specific pin mapping configurations (`board_pinout.yaml`).

12.3 Code Organization Principles

1. **Public APIs go in `api/`** —Interfaces shared by all chips
2. **Core drivers go in `core/`** —Peripheral drivers for each chip family
3. **Chip-specific code goes in `soc/`** —Features unique to a particular chip
4. **Application code goes anywhere** —Not limited to the `samples/` directory
5. **Use Kconfig for configuration** —Manage compile-time options with Kconfig

CHAPTER

THIRTEEN

PROJECT CREATION & MANAGEMENT

13.1 Application Project Location

UniSDK application projects **can be located in any directory**, not limited to the `samples/` directory. Each application project needs to provide its own `CMakeLists.txt` and source code.

13.2 Creating a New Project

13.2.1 Minimum Project Structure

Minimum Project Structure

```
my_project/
├── CMakeLists.txt    # Build entry point
├── main.c            # Main program
└── Kconfig           # Optional: project-level Kconfig configuration
```

13.2.2 CMakeLists.txt Template

```
# SPDX-License-Identifier: Apache-2.0

cmake_minimum_required(VERSION 3.20.0)

# Load Telink SDK
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})

# Define project
project(MyProject LANGUAGES C CXX)

# Add C source files
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

# Optional: conditionally add C++ source files
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

13.2.3 Key Notes

1. **`find_package(Telink)`** —Locates the SDK path via the `TELINK_BASE` environment variable
2. **`project()`** —The project name will be used as the output firmware name
3. **`target_sources(Telink ...)`** —Application source files are added to the SDK's main target `Telink`
4. **`CONFIG_TLK_ALLOW_CPP_WRAPPERS`** —Checks whether C++ wrappers are enabled

13.3 Building the Project

13.3.1 West Command (Recommended)

```
# Run in the project directory (must be within a west workspace when using␣
↪west)
west tl-build . --board TLSR9528A_EVK

# Specify build directory
west tl-build -d build_custom . --board TLSR9528A_EVK
```

13.3.2 CMake Command

```
# Configure
cmake -B build -G Ninja -S . -DBOARD=TLSR9528A_EVK

# Build
cmake --build build
```

13.3.3 Make Command

```
make cmake build APP=. BOARD=TLSR9528A_EVK
```

13.4 Configuration Management

13.4.1 Using Project Kconfig

Create a `Kconfig` file in the project directory to define project-specific configuration options:

```
menu "My Project Configuration"

config MY_FEATURE_ENABLED
    bool "Enable my feature"
    default y
    help
        Enable custom project feature.

endmenu
```

The `Kconfig` file is automatically integrated into the build system via `KCONFIG_APPLICATION_DIR` in `kconfig.cmake`.

13.4.2 Interactive Configuration

```
west tl-config .
```

13.5 Cleaning

```
# Clean build artifacts
west tl-build . --clean

# Or manually delete
rm -rf build/
```

13.6 Development Workflow Recommendations

1. **Start from a sample** —Copy the most similar `samples/` example as a starting point
2. **Kconfig isolation** —Place project-specific configuration in the project-level `Kconfig`
3. **Version management** —Manage application projects and the SDK separately, linking them via `TELINK_BASE`

CHAPTER

FOURTEEN

CORE ARCHITECTURE

This document introduces the core operating mechanisms of UniSDK, including the startup flow, interrupt management, exception handling, and low-power framework.

14.1 System Startup Flow

The startup process of UniSDK begins from hardware reset and goes through the following stages:

14.1.1 Stage Descriptions

1. **`cstartup_flash.S`** —Assembly startup code, sets up the stack, copies the data segment, zeros BSS, jumps to the C entry point
2. **`tlk_init()`** —SDK initialization function, initializes each subsystem in sequence
3. **`tlk_loop()`** —Main event loop, handles BLE events, timers, etc.

14.2 Initialization System

14.2.1 init Framework

UniSDK uses a modular initialization framework, implemented through macros defined in `tlk_init.h`:

```
// Each subsystem registers an initialization callback via TLK_INIT_ENTRY
// Initialization is performed in priority order
```

Initialization Priority Hierarchy:

Priority	Typical Modules	Description
Earliest	Clock, Memory	System foundational capabilities
Early	GPIO, PLIC	Peripheral infrastructure
Middle	UART, I2C, SPI	Communication peripherals
Late	BLE	Protocol stack
Latest	Application Callback	User application initialization

14.3 Interrupt Management

14.3.1 PLIC (Platform-Level Interrupt Controller)

RISC-V platforms use PLIC to manage peripheral interrupts:

- **Interrupt Sources** —Each peripheral (GPIO, UART, I2C, etc.) has a dedicated PLIC interrupt number
- **Priority** —PLIC supports multiple interrupt priority levels
- **Interrupt Nesting** —Supports interrupt nesting through preemption

```
// Enable global interrupts
tlk_core_interrupt_enable();

// Disable global interrupts
tlk_core_interrupt_disable();
```

14.3.2 GPIO Interrupt Example

```
// Register GPIO interrupt callback
struct tlk_gpio_irq_callback cb = {
    .handler = my_gpio_handler,
    .pin = GPIO_PIN_0
};
tlk_gpio_irq_add_callback(GPIO_PORT_A, &cb);
tlk_gpio_irq_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_INTR_RISING_EDGE);
tlk_core_interrupt_enable();
```

14.4 Low-Power Management

UniSDK supports multiple low-power modes:

Mode	Power Draw	Wake-Up Speed	Retained Content	Use Case
Suspend	Medium	Fast (< 1ms)	Most context	Brief idle periods
Deep Retention	Low	Medium	Partial SRAM retention	Long idle, state must be retained
Deep Sleep	Very Low	Slow	SRAM not retained	Extended hibernation

14.4.1 Typical Usage

```
#include <tlk_api.h>

void deep_sleep_example(void) {
    // Set GPIO wake-up source
    tlk_pm_set_gpio_wakeup(GPIO_PORT_A, GPIO_PIN_0,
                           TLK_PM_GPIO_WAKEUP_LEVEL_HIGH);
```

(continues on next page)

(continued from previous page)

```
    // Enter Deep Sleep for 10 seconds
    tlk_pm_sleep(TLK_PM_SLEEP_MODE_DEEP_SLEEP, 10000000);

    // Check wake-up reason
    enum tlk_pm_wakeup_source reason = tlk_pm_get_wakeup_reason();
}
```

14.5 Context Switch (tl_context.S)

`tl_context.S` implements light-weight context saving and restoring, used for:

- Register saving on interrupt entry/exit
- State restoration after waking from low-power modes

14.6 Clock System

- **System Timer (STIMER)** —High-precision system timer
- **Machine Timer (MTIMER)** —RISC-V standard timer, used for scheduling and delays
- **Peripheral Clocks** —Independent clock source configuration for each peripheral

CHAPTER

FIFTEEN

INIT HOOKS DOCUMENTATION

15.1 Overview

This document describes the init and power-state hook mechanism provided by the `common/include/tlk_init.h` and `common/tlk_init.c` pair.

The system exposes several hook types that components can register. Hooks are collected into dedicated linker sections by registration macros and invoked by the runtime using the functions implemented in `tlk_init.c`.

Primary goals:

- Provide ordered initialization hooks (pre-init) for system, driver and application code.
- Provide lifecycle hooks for suspend/sleep transitions (before/after).
- Allow components to prevent suspend/sleep via boolean callbacks.

15.2 Concepts

- **Registration by linker sections:** Registration macros place function pointers into named linker sections (for example, `.pre_init_array_<level>_<priority>`). The build/linker is expected to place these sections in an order that respects level and priority so the runtime sees callbacks in the intended sequence.
- **Invocation:** Each public `tlk_*` function in `tlk_init.c` iterates from a linker-provided `__*_start` to `__*_end` range and calls every non-NULL function pointer it finds.
- **Ordering convention:** Lower numeric values indicate earlier/higher priority. The headers define named level and priority constants (e.g. `TLK_INIT_LEVEL_SYSTEM`/`TLK_INIT_PRIORITY_HIGH`). The final ordering depends on the linker script that arranges the sections, but the numeric convention should be used when registering callbacks.

15.3 Levels & Priorities

- **Levels (examples):**
  - `TLK_INIT_LEVEL_SYSTEM` = 2
  - `TLK_INIT_LEVEL_DRIVER` = 5
  - `TLK_INIT_LEVEL_APPLICATION` = 8
- **Priorities (examples):**

  - TLK_INIT_PRIORITY_HIGH = 2
  - TLK_INIT_PRIORITY_NORMAL = 5
  - TLK_INIT_PRIORITY_LOW = 8

Use the level to group callbacks by broad stage (system/driver/application) and use priority to fine-tune ordering inside that stage. Smaller numbers mean higher precedence. Additional levels or priorities are expected to be added in the future.

15.4 API reference

15.4.1 Initialization hooks (pre-init)

- **Type:** `typedef void (*tlk_pre_init_func_t)(void)`
- **Registration macro:** `TLK_REGISTER_PRE_INIT(fn, level, priority)` —places the function pointer in `.pre_init_array_<level>_<priority>` and marks it `used` so the linker doesn't discard it.
- **Invoker:** `void tlk_pre_init(void)` —iterates from `__pre_init_start` to `__pre_init_end` and calls each registered function.

Usage example:

```
void my_driver_init(void) {
    /* driver init work */
}

TLK_REGISTER_PRE_INIT(my_driver_init, TLK_INIT_LEVEL_DRIVER, TLK_INIT_
↪PRIORITY_HIGH);
```

15.4.2 Suspend hooks (before / after)

- **Types:**
  - `typedef void (*tlk_before_suspend_func_t)(void)`
  - `typedef void (*tlk_after_suspend_func_t)(void)`
- **Registration macros:**
  - `TLK_REGISTER_BEFORE_SUSPEND(fn, level, priority)` → section `.before_suspend_array_<level>_<priority>`
  - `TLK_REGISTER_AFTER_SUSPEND(fn, level, priority)` → section `.after_suspend_array_<level>_<priority>`
- **Invokers:**
  - `void tlk_before_suspend(void)` —calls all registered before-suspend callbacks.
  - `void tlk_after_suspend(void)` —calls all registered after-suspend callbacks.

Usage example:

```
void save_peripherals(void) { /* ... */ }

TLK_REGISTER_BEFORE_SUSPEND(save_peripherals, TLK_INIT_LEVEL_DRIVER, TLK_INIT_
↪PRIORITY_NORMAL);
```

15.4.3 Sleep hooks (before / after)

- **Types:**
  - `typedef void (*tlk_before_sleep_func_t)(void)`
  - `typedef void (*tlk_after_sleep_func_t)(void)`
- **Registration macros:**
  - `TLK_REGISTER_BEFORE_SLEEP(fn, level, priority)` → section `.before_sleep_array_<level>_<priority>`
  - `TLK_REGISTER_AFTER_SLEEP(fn, level, priority)` → section `.after_sleep_array_<level>_<priority>`
- **Invokers:**
  - `void tlk_before_sleep(void)`
  - `void tlk_after_sleep(void)`

Usage example:

```
void board_prepare_sleep(void) { /* disable sensors, clocks, ... */ }

TLK_REGISTER_BEFORE_SLEEP(board_prepare_sleep, TLK_INIT_LEVEL_SYSTEM, TLK_
↪INIT_PRIORITY_HIGH);
```

15.4.4 Prevent-suspend / prevent-sleep callbacks

- **Types:**
  - `typedef bool (*tlk_prevent_suspend_func_t)(void)`
  - `typedef bool (*tlk_prevent_sleep_func_t)(void)`
- **Registration macros:**
  - `TLK_REGISTER_PREVENT_SUSPEND(fn)` → section `.prevent_suspend_array`
  - `TLK_REGISTER_PREVENT_SLEEP(fn)` → section `.prevent_sleep_array`
- **Invokers and semantics:**
  - `bool tlk_prevent_suspend(void)` —returns `true` as soon as any registered `tlk_prevent_suspend_func_t` returns `true`; otherwise returns `false`.
  - `bool tlk_prevent_sleep(void)` —returns `true` as soon as any registered `tlk_prevent_sleep_func_t` returns `true`; otherwise returns `false`.

Usage example:

```
bool usb_active(void) {
    return tlk_usb_is_busy();
}

TLK_REGISTER_PREVENT_SLEEP(usb_active);
```

15.5 Additional Notes

The functions provided by this API **should not** be invoked by the user. They are called automatically on the corresponding lifecycle stages.

CHAPTER

SIXTEEN

API DOCUMENTATION

16.1 Overview

The API module provides a high-level abstraction layer for essential system utilities. It consolidates system timing, power management (sleep modes), and debug printing into a unified interface. The Print module acts as a lightweight, formatted output utility that directs debug messages to a configured UART instance.

Key Features:

- **System Timing:** Microsecond-precision timestamp generation.
- **Power Management:** Unified sleep function with configurable power modes (Suspend, Deep Sleep, Retention).
- **Debug Output:** `printf`-like formatting capability.
- **UART Integration:** Directs debug output to a specific, user-configurable UART hardware instance.

16.2 API Reference

This section describes the functions available in the API and Print modules.

16.2.1 1. `tlk_api_micros`

Returns the current system up-time in microseconds. This function utilizes the system timer (`stimer`) to calculate the tick count.

Prototype:

```
uint32_t tlk_api_micros(void);
```

Parameters: None

Return Value: `uint32_t`: The number of microseconds passed since the system started.

16.2.2 2. `tlk_api_sleep`

This API is available only if `CONFIG_TLK_API_SLEEP` is enabled.

Checks duration; if zero —returns. If `CONFIG_TLK_PM` is enabled, it calls `tlk_pm_sleep` with `TLK_PM_SLEEP_MAX_LEVEL-1` parameter. If `CONFIG_TLK_PM` is disabled, or `tlk_pm_sleep` returned err, it programs the timer for the requested delay, enables timer interrupt, and waits using WFI.

Prototype:

```
void tlk_api_sleep(uint32_t duration_ms);
```

Parameters:

Parameter	Type	Description
`duration_ms`	`uint32_t`	The duration to sleep in milliseconds.

Return Value: None

16.2.3 3. tlk_api_delay

Blocking delay function

Prototype:

```
void tlk_api_delay(uint32_t duration_us);
```

Parameters:

Parameter	Type	Description
`duration_us`	`uint32_t`	The duration to wait in microseconds.

Return Value: None

16.2.4 4. tlk_api_print

This API is available only if `CONFIG_TLK_DEBUG_PRINT` is enabled.

A formatted output function similar to the standard C `printf`. It formats the string and transmits it via the configured UART interface.

Prototype:

```
int tlk_api_print(const char *restrict format, ...);
```

Parameters:

Parameter	Type	Description
`format`	`const char *`	The format string (e.g., "Value: %d\n").
`...`	`varargs`	Variable arguments corresponding to the format specifiers.

Return Value: `int`: The number of characters written to the output buffer.

Note: This function uses an internal static buffer defined by `CONFIG_TLK_DEBUG_PRINT_BUFFER_SIZE`. Ensure your formatted message does not exceed

this limit. This function is used internally by the SDK and may not have a public declaration in API headers.

CHAPTER

SEVENTEEN

DEBUGGING & TESTING

17.1 Hardware Debugging

17.1.1 Debugger Support

UniSDK supports the following debug interfaces:

Debugger	Interface	Supported Platforms
J-Link	JTAG/SWD	All platforms
Telink Debugger	SWS (Single Wire)	All platforms
BDT	Flashing/Debugging	All platforms

17.1.2 Debugging Connection

1. Connect the debugger to the JTAG/SWS interface on the development board
2. Use GDB or an IDE for source-level debugging

17.2 Software Debugging

17.2.1 Logging System (Debug Print)

UniSDK has a built-in debug logging system that can be enabled via Kconfig:

```
# Enable debug logging
CONFIG_TLK_DEBUG_PRINT=y
CONFIG_TLK_DEBUG_PRINT_UART=0
CONFIG_TLK_DEBUG_PRINT_BUFFER_SIZE=256
```

Logs are output via UART and can be viewed in a serial terminal.

17.2.2 Firmware Verification Script

```
# Verify firmware integrity
bash scripts/tl_check_fw.sh build/Telink.elf
```

17.3 Performance Analysis

17.3.1 Memory Usage Report

A memory usage summary is automatically output after compilation:

```
Memory region         Used Size  Region Size  %age Used
          RAM:        1234 B        64 KB      1.88%
        FLASH:        5678 B       512 KB      1.08%
```

17.3.2 ELF Analysis

```
# Generate disassembly for analysis
riscv32-elf-objdump -d build/Telink.elf > disasm.txt

# View section sizes
riscv32-elf-size build/Telink.elf
```

17.4 Debugging with GDB

```
# Start GDB server (using J-Link or Telink Debugger)
# Then connect the GDB client

riscv32-elf-gdb build/Telink.elf
(gdb) target remote :2331
(gdb) break main
(gdb) continue
```

17.5 Common Debugging Tips

1. **LED Debugging** —Toggle LED state in critical code paths
2. **UART Output** —Output variable values via `TLK_DEBUG_PRINT`
3. **Assertion Checks** —Use assertions to catch abnormal states
4. **Memory Inspection** —Check the `.map` file to verify memory layout

17.6 Next Steps

- *IDE Integration* —VS Code development environment configuration
- *Build & Configuration* —Detailed build system documentation

CHAPTER

EIGHTEEN

IDE INTEGRATION

18.1 VS Code Configuration (Recommended)

18.1.1 clangd Language Server

clangd is an LLVM/Clang-based C/C++ language server that understands UniSDK's complex macro definitions and configuration system, providing:

- Intelligent code completion
- Go to definition
- Find references
- Semantic highlighting
- Code navigation
- Inline diagnostics
- Rename refactoring

Why Choose clangd?

UniSDK makes extensive use of Kconfig-generated configuration macros (`autoconf.h`, `capabilities.h`) and cross-compilation. Ordinary syntax highlighting cannot correctly parse these macros, leading to false error reports and navigation failures. clangd solves this problem by reading `compile_commands.json`.

Installation Steps

1. Install **clangd** (publisher: LLVM) from the VS Code extension marketplace
2. Disable the Microsoft C/C++ IntelliSense engine to avoid conflicts:
   - `Ctrl+Shift+P` → `Preferences: Open Settings (UI)` → Extensions → C/C++ → IntelliSense
   - Set `IntelliSenseEngine` to `disabled`

Key Files

`compile_commands.json`

Automatically generated in the `build/` directory by CMake during the build. clangd reads this file to understand the project's compilation options.

```
[
  {
    "file": "main.c",
    "command": "riscv32-elf-gcc -I/path/to/include -DCONFIG_DEBUG ...",
    "directory": "/project/build"
  }
]
```

.clangd (Optional configuration at project root)

```
CompileFlags:
  Add:
    - -Wall
    - -Wextra

Diagnostics:
  Suppress:
    - unused-parameter
```

Common Issues

Files Display Incorrectly After Switching to a Different Chip

clangd provides semantic analysis based on the current build's `compile_commands.json`. If the project was previously built for TL321X, files for TL721X will not be correctly parsed.

Solution: Rebuild the project with the target chip.

Warnings Still Present After a Recent Rebuild

File indexing happens lazily and needs a manual trigger to update.

Solution: Edit and save the target file to force an update.

18.1.2 Recommended Extensions

Extension	Purpose
clangd	C/C++ language server
CMake Tools	CMake project support
Cortex-Debug	ARM/RISC-V debugging
GitLens	Git enhancements
Error Lens	Inline error display

18.2 Eclipse Configuration

1. Install Eclipse CDT and the RISC-V GCC plugin
2. Import the project as "Existing CMake Project"
3. Configure the cross-compilation toolchain path

18.3 Segger Embedded Studio

1. Import the project via File → Open Solution
2. Configure the debugger as J-Link
3. Set the target device to Telink RISC-V chip

18.4 Next Steps

- *Project Creation & Management* —Create new projects
- *Debugging & Testing* —Debugging tips

CHAPTER

NINETEEN

CLANGD SETUP GUIDE FOR VSCODE

19.1 What is clangd?

clangd is a C/C++ language server based on the LLVM/Clang tooling ecosystem. It provides IDE-like features for editors such as:

- complex macro expansion
- intelligent autocompletion
- go to definition
- find references
- semantic highlighting
- inline diagnostics
- code navigation
- rename refactoring
- hover information

19.2 Why use clangd?

Unlike basic syntax highlighting, clangd actually understands:

- compiler defines
- include directories
- generated headers
- specific flags
- warning configuration

Which is quite important in projects using Kconfig, generated configuration headers, and cross-compilation toolchains. Without proper compile information, editors often show false errors and broken navigation. clangd solves this by reading the project compilation database.

19.3 Key Files

19.3.1 compile_commands.json

This is the most important file for clangd. It is generated by CMake and contains the exact compile commands used for every source file. clangd reads this file to understand your project correctly.

Example:

```
[
 {
    "file": "src/main.c",
    "command": "arm-none-eabi-gcc -Iinclude -DDEBUG ...",
    "directory": "/project/build"
 }
]
```

19.3.2 .clangd

Optional project configuration file for clangd.

Typical uses:

- add extra compiler flags
- suppress warnings
- configure indexing behavior
- fix generated include issues

Example:

```
CompileFlags:
  Add:
 - -Wall
 - -Wextra

Diagnostics:
  Suppress:
 - unused-parameter
```

19.4 Installing and setup

1. Open Extensions (Ctrl+Shift+X) and install clangd (publisher - LLVM).
2. Proceed with the download prompt.
3. It is recommended to disable the Microsoft C/C++ IntelliSense engine to avoid conflicts. `(Ctrl+Shift+P) -> Preferences: Open Settings (UI) -> Extension -> C/C++ -> IntelliSense`
4. Then set `IntelliSenseEngine` to `disabled`.

19.5 Troubleshooting

19.5.1 The extension works as expected until I look at the file for another core

Solution: build your project targeting the files displayed incorrectly.
Explanation: clangd provides the information based on the current build's `compile_commands.json`, so if you have built your project targeting the TL321X core, it cannot resolve the files that belong to the TL721X core correctly, since they are not included in the build.

19.5.2 I just rebuilt my project, but I still see the warning in the opened file

Solution: enforce the update by editing/saving the file.
Explanation: file indexing happens lazily, and since it is unknown what needs to be updated after a rebuild, you may need to enforce it.

CHAPTER

TWENTY

CHIPS AND BOARDS

20.1 Chip Series Overview

UniSDK supports the following Telink chip series:

Chip Series	Core	Flash	SRAM	Key Peripherals	Typical Application
TL321X	RISC-V	—	—	BLE, GPIO, UART, I2C, SPI, ADC, DMA	General IoT
TL721X	RISC-V	—	—	BLE, GPIO, UART, I2C, SPI, ADC, DMA, USB	General IoT
TLSR922X	RISC-V	—	—	BLE, GPIO, UART, I2C, SPI, ADC	General IoT
TLSR952X	RISC-V	—	—	BLE, GPIO, UART, I2C, SPI, ADC, DMA	General IoT

20.2 Selecting a Chip

```
# List all supported chips
west tl-socs

# List detailed chip information
west tl-socs -v
```

20.3 Board List

Board	Chip	Description
TL3218X_EVK	TL321X	TL321X Evaluation Kit
TL7218X_EVK	TL721X	TL721X Evaluation Kit
TLSR9228A_EVK	TLSR922X	TLSR922X Evaluation Kit
TLSR9528A_EVK	TLSR952X	TLSR952X Evaluation Kit
TLSR9528A_DONGLE	TLSR952X	TLSR952X USB Dongle

```
# List all supported boards
west tl-boards

# Filter boards by chip
west tl-boards --soc TLSR9528A

# List detailed board information
west tl-boards -v
```

20.4 Chip and Board Pair Validation

```
# Validate combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

20.5 Specifying at Build Time

```
# Specify chip via command line arguments
west tl-build samples/gpio_demo --soc TLSR9528A
# Specify board via command line arguments
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Specify chip via CMake arguments
cmake -B build -DSOC=TLSR9528A -S .
# Specify board via CMake arguments
cmake -B build -DBOARD=TLSR9528A_EVK -S .
```

CHAPTER

TWENTYONE

TL321X CHIP SERIES

Note

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

21.1 Overview

The TL321X is a RISC-V microcontroller series from Telink, designed for general IoT application scenarios.

21.2 Core Parameters

Parameter	Value
Core	RISC-V (32-bit)
Supported Protocols	BLE
Toolchain	`andes-riscv32-v5-gcc14.2` (TL32 ELF MCULIB V5 GCC14.2)

21.3 Memory

Region	Size	Start Address
ROM	2048 KB	0x20000000
IRAM	96 KB	0x00068000
DRAM	32 KB	0x00080000

21.4 Peripheral List

21.4.1 Standard Peripherals

- GPIO
- UART
- I2C

- SPI
- ADC
- DMA
- USB

21.4.2 System & Control

- PLIC (Platform-Level Interrupt Controller)
- PM (Power Management: Suspend, Deep Sleep, Deep Retention)
- PMP (Physical Memory Protection)
- STIMER (System Timer)
- MTIMER (Machine Timer)
- PWM
- QDEC (Quadrature Decoder)
- MSPI

21.4.3 Security & Crypto

- PKE (Public Key Engine)
- SKE (Symmetric Key Engine)
- HASH
- IR (Infrared)

21.4.4 Debug & Misc

- SWIRE (Single Wire Interface)
- JTAG
- TRAP

21.4.5 Supported Package Options

- QFN28, QFN32, QFN38, QFN48, QFN52, QFN64
- LGA42

21.5 Supported Boards

- *TL3218X_EVK*

CHAPTER

TWENTYTWO

TL721X CHIP SERIES

Note

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

22.1 Overview

The TL721X is a RISC-V microcontroller series from Telink, adding USB support on top of the TL321X.

22.2 Core Parameters

Parameter	Value
Core	RISC-V (32-bit)
Supported Protocols	BLE
Toolchain	`andes-riscv32-v5f-gcc14.2` (TL32 ELF MCULIB V5F GCC14.2)

22.3 Memory

Region	Size	Start Address
ROM	2048 KB	0x20000000
IRAM	256 KB	0x00040000
DRAM	256 KB	0x00080000

22.4 Peripheral List

22.4.1 Standard Peripherals

- GPIO
- UART
- I2C

- SPI
- ADC
- DMA
- USB

22.4.2 System & Control

- PLIC (Platform-Level Interrupt Controller)
- PM (Power Management: Suspend, Deep Sleep, Deep Retention)
- PMP (Physical Memory Protection)
- STIMER (System Timer)
- MTIMER (Machine Timer)
- PWM
- QDEC (Quadrature Decoder)
- MSPI

22.4.3 Security & Crypto

- PKE (Public Key Engine)
- SKE (Symmetric Key Engine)
- HASH
- TRNG (True Random Number Generator)
- ChaCha20/Poly1305

22.4.4 Debug & Misc

- SWIRE (Single Wire Interface)
- IR (Infrared)
- OTP (One-Time Programmable)
- TRAP

22.4.5 Supported Package Options

- QFN38, QFN68, QFN88
- BGA56, BGA94

22.5 Supported Boards

- *TL7218X_EVK*

CHAPTER

TWENTYTHREE

TLSR922X CHIP SERIES

Note

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

23.1 Overview

The TLSR922X is a RISC-V microcontroller series from Telink, designed for wireless audio application scenarios.

23.2 Core Parameters

Parameter	Value
Core	RISC-V (32-bit)
Supported Protocols	BLE
Toolchain	`andes-riscv32-v5f-gcc14.2` (TL32 ELF MCULIB V5F GCC14.2)

23.3 Memory

Region	Size	Start Address
ROM	—	—
IRAM	256 KB	0x00000000
DRAM	256 KB	0x00080000

23.4 Peripheral List

23.4.1 Standard Peripherals

- GPIO
- UART
- I2C

- SPI
- ADC
- DMA

23.4.2 Audio

- Audio Codec

23.4.3 System & Control

- PLIC (Platform-Level Interrupt Controller)
- PM (Power Management: Suspend, Deep Sleep, Deep Retention)
- PMP (Physical Memory Protection)
- STIMER (System Timer)
- MTIMER (Machine Timer)
- MSPI

23.4.4 Security & Crypto

- PKE (Public Key Engine)
- SKE (Symmetric Key Engine)
- HASH
- TRNG (True Random Number Generator)

23.4.5 Supported Package Options

- QFN48, QFN88

23.5 Supported Boards

- *TLSR9228A_EVK*

CHAPTER

TWENTYFOUR

TLSR952X CHIP SERIES

Note

Document Status: DRAFT —Detailed specifications to be added. The information below is based on SDK metadata and is subject to update.

24.1 Overview

The TLSR952X is a RISC-V microcontroller series from Telink, designed for general IoT application scenarios.

24.2 Core Parameters

Parameter	Value
Core	RISC-V (32-bit)
Supported Protocols	BLE
Toolchain	`andes-riscv32-v5f-gcc14.2` (TL32 ELF MCULIB V5F GCC14.2)

24.3 Memory

Region	Size	Start Address
ROM	2048 KB	0x20000000
IRAM	256 KB	0x00000000
DRAM	256 KB	0x00080000

24.4 Peripheral List

24.4.1 Standard Peripherals

- GPIO
- UART
- I2C

- SPI
- ADC
- DMA

24.4.2 System & Control

- PLIC (Platform-Level Interrupt Controller)
- PM (Power Management: Suspend, Deep Sleep, Deep Retention)
- PMP (Physical Memory Protection)
- STIMER (System Timer)
- MTIMER (Machine Timer)
- PWM
- QDEC (Quadrature Decoder)
- MSPI

24.4.3 Security & Crypto

- PKE (Public Key Engine)
- AES
- TRNG (True Random Number Generator)

24.4.4 Supported Package Options

- QFN40, QFN56, QFN88

24.5 Supported Boards

- *TLSR9528A_EVK*
- *TLSR9528A_DONGLE*

CHAPTER

TWENTYFIVE

TL3218X_EVK BOARD

Note

Document Status: DRAFT —Detailed specifications to be added

25.1 Overview

The TL3218X_EVK is the official evaluation kit for the TL321X chip series.

25.2 Building

```
west tl-build samples/gpio_demo --board TL3218X_EVK
```

CHAPTER

TWENTYSIX

TL7218X_EVK BOARD

Note

Document Status: DRAFT —Detailed specifications to be added

26.1 Overview

The TL7218X_EVK is the official evaluation kit for the TL721X chip series.

26.2 Building

```
west tl-build samples/gpio_demo --board TL7218X_EVK
```

CHAPTER

TWENTYSEVEN

TLSR9228A_EVK BOARD

Note

Document Status: DRAFT —Detailed specifications to be added

27.1 Overview

The TLSR9228A_EVK is the official evaluation kit for the TLSR922X chip series.

27.2 Building

```
west tl-build samples/gpio_demo --board TLSR9228A_EVK
```

CHAPTER

TWENTYEIGHT

TLSR9528A_EVK BOARD

Note

Document Status: DRAFT —Detailed specifications to be added

28.1 Overview

The TLSR9528A_EVK is the official evaluation kit for the TLSR952X chip series.

28.2 Building

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
```

CHAPTER

TWENTYNINE

TLSR9528A_DONGLE

Note

Document Status: DRAFT —Detailed specifications to be added

29.1 Overview

The TLSR9528A_DONGLE is the USB Dongle form factor for the TLSR952X chip series.

29.2 Building

```
west tl-build samples/gpio_demo --board TLSR9528A_DONGLE
```

CHAPTER

THIRTY

PERIPHERALS AND DRIVERS

UniSDK provides a unified peripheral driver API for the entire Telink chip family. All drivers follow a consistent programming model and naming convention.

30.1 Driver Overview

Driver	API Prefix	Transfer Mode	Description
GPIO	`tlk_gpio_*`	Direct Register	General-purpose input/output, interrupt handling
UART	`tlk_uart_*`	Blocking / PLIC / DMA	Universal asynchronous receiver/transmitter, flow control
Power Management PM	`tlk_pm_*`	—	Low-power mode management
Analog	`tlk_analog_*`	Direct Register	Analog register access
PLIC	`tlk_plic_*`	—	Platform-level interrupt controller
I2C	`tlk_i2c_*`	Blocking / DMA	I2C master/slave communication
SPI	`tlk_spi_*`	Blocking / DMA	SPI master/slave communication
ADC	`tlk_adc_*`	Blocking	Analog-to-digital conversion
DMA	`tlk_dma_*`	DMA	Direct memory access
Watchdog	`tlk_wdt_*`	—	Watchdog timer
Timer	`tlk_stimer_*` / `tlk_mtimer_*`	—	System/machine timer
USB	`tlk_usb_*` + TinyUSB	—	USB CDC, etc.

30.2 Driver Programming Model

All UniSDK drivers follow a unified programming model:

30.2.1 Configure → Use → Deinitialize

```
// 1. Configure (via menuconfig or code)
tlk_gpio_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_OUTPUT);
```

(continues on next page)

(continued from previous page)

```
// 2. Use
tlk_gpio_pin_write(GPIO_PORT_A, GPIO_PIN_0, true);

// 3. Optional: Disable
tlk_gpio_disable(GPIO_PORT_A, GPIO_PIN_0);
```

30.2.2 Transfer Modes

Some communication peripherals support multiple transfer modes:

- **Blocking** —Synchronously waits for transfer completion, suitable for simple scenarios
- **PLIC (Interrupt)** —Non-blocking transfer based on interrupts, no CPU polling required
- **DMA** —Most efficient bulk data transfer, zero CPU load

Configure transfer mode independently for each peripheral instance via Kconfig.

30.3 Kconfig Configuration

All peripheral drivers are configured at compile time via Kconfig. See *Kconfig Reference* for all configuration options.

Quick configuration entry:

```
# Interactive configuration
west tl-config

# or directly use
make config
```

CHAPTER

THIRTYONE

GPIO DRIVER

31.1 Overview

The GPIO (General Purpose Input/Output) driver provides an interface for configuring and using general-purpose microcontroller pins. It supports pin mode configuration (input, output, pull-up, pull-down), pin multiplexing (Pinmux), and direct pin state operations (write, read, toggle). It also supports interrupt handling with configurable trigger conditions and callback registration.

Key Features:

- GPIO mode configuration (input, output, pull-up, pull-down)
- Pin multiplexing (Mux)
- Direct pin control (write, read, toggle)
- Interrupt handling (edge/level trigger + callback)
- Ability to disable pin functionality

31.2 Data Types and Enums

31.2.1 Port Identifiers

`enum tlk_gpio_port` —Generated at compile time based on `UNISDK_GPIO_PORT_COUNT` and `CONFIG_TLK_GPIO_PORT_##num##_ENABLED`.

Value	Description
`GPIO_PORT_x`	Generated identifier for each enabled port

31.2.2 Pin Definitions

`enum tlk_gpio_pin` —Defines bitmasks for pins within a port.

Value	Description
`TLK_GPIO_PIN_NONE`	No pin selected (value is 0)
`GPIO_PIN_x`	Pin index bitmask (x is 0-7), defined as `TLK_BIT(x)`

31.2.3 Pin Modes

`enum tlk_gpio_mode`

Value	Description
`TLK_GPIO_OUTPUT`	Digital output (value is -1)
`TLK_GPIO_INPUT_NO_PU`	Floating digital input
`TLK_GPIO_INPUT_PULL_`	Digital input with pull-up resistor
`TLK_GPIO_INPUT_PULL_`	Digital input with pull-down resistor
`TLK_GPIO_INPUT_PULL_`	*Generated based on config `UNISDK_GPIO_PULL_DOWN_OPTIONS_COUNT`* Input with a specific pull-up value.
`TLK_GPIO_INPUT_PULL_`	*Generated based on config `UNISDK_GPIO_PULL_DOWN_OPTIONS_COUNT`* Input with a specific pull-down value.

31.2.4 Interrupt Trigger Types

`enum tlk_gpio_intr_trigger`

Value	Description
`TLK_GPIO_INTR_RISING_EDGE`	Rising edge trigger
`TLK_GPIO_INTR_FALLING_EDGE`	Falling edge trigger
`TLK_GPIO_INTR_HIGH_LEVEL`	High level trigger
`TLK_GPIO_INTR_LOW_LEVEL`	Low level trigger
`TLK_GPIO_INTR_BOTH_EDGE`	Both edges trigger

31.2.5 Interrupt Callback Types

Types used for handling GPIO interrupts via user-registered callbacks.

Callback Handler Type

Function pointer type for a GPIO interrupt callback.

```
typedef void (*tlk_gpio_irq_callback_handler)(enum tlk_gpio_port port,
                                              enum tlk_gpio_pin pin);
```

Parameter	Description
`port`	GPIO port on which the interrupt occurred.
`pin`	GPIO pin that triggered the interrupt.

Callback Registration Structure

Structure used to register a GPIO interrupt callback in the internal callback list.

```
struct tlk_gpio_irq_callback {
    struct tlk_list_node node;
    tlk_gpio_irq_callback_handler handler;
    enum tlk_gpio_pin pin;
};
```

Field	Description
node	Internal list node used to link callbacks.
handler	User-defined interrupt callback function.
pin	GPIO pin associated with this callback.

31.2.6 Port Pin Struct

```
struct tlk_gpio_port_pin {
    enum tlk_gpio_port port;
    enum tlk_gpio_pin pin;
};
```

31.3 Driver API

31.3.1 tlk_gpio_disable

Disables the pin function and resets configuration.

```
void tlk_gpio_disable(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.

Return value: None

31.3.2 tlk_gpio_configure

Sets the GPIO pin mode (input/output, optional pull-up/pull-down).

```
void tlk_gpio_configure(enum tlk_gpio_port port, enum tlk_gpio_pin pin,
                        enum tlk_gpio_mode mode);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.
mode	enum tlk_gpio_mode	GPIO mode (output / input pull-up / pull-down).

Return value: None

31.3.3 tlk_gpio_pin_write

Sets the pin output level.

```
void tlk_gpio_pin_write(enum tlk_gpio_port port, enum tlk_gpio_pin pin, bool␣
↪value);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.
value	bool	Logic level (true = high, false = low).

Return value: None

31.3.4 tlk_gpio_pin_toggle

Toggles the current output value of the pin.

```
void tlk_gpio_pin_toggle(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.

Return value: None

31.3.5 tlk_gpio_pin_read

Reads the current logic level of the pin.

```
bool tlk_gpio_pin_read(enum tlk_gpio_port port, enum tlk_gpio_pin pin);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.

Return value: Returns true if the pin level is high, and false if low.

31.3.6 tlk_gpio_set_mux

Configures the pin multiplexing function.

```
void tlk_gpio_set_mux(enum tlk_gpio_port port, enum tlk_gpio_pin pin, uint8_t␣
↪mux);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.
mux	uint8_t	Multiplexer index for the selected function.

Return value: None

31.3.7 tlk_gpio_irq_configure

Sets the interrupt trigger type.

```
void tlk_gpio_irq_configure(enum tlk_gpio_port port, enum tlk_gpio_pin pin,
                            enum tlk_gpio_intr_trigger trigger);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
pin	enum tlk_gpio_pin	GPIO pin.
trigger	enum tlk_gpio_intr_trigger	Interrupt trigger mode (edge or level).

Return value: None

31.3.8 tlk_gpio_irq_add_callback

Registers an interrupt callback function.

```
void tlk_gpio_irq_add_callback(enum tlk_gpio_port port,
                               struct tlk_gpio_irq_callback *callback);
```

Parameters:

Parameter	Type	Description
port	enum tlk_gpio_port	GPIO port.
callback	struct tlk_gpio_irq_callback *	Pointer to GPIO interrupt callback structure.

Return value: None

31.4 Kconfig Configuration

```
GPIO_PORT_{name}_ENABLED
|   Enables the specific GPIO port instance (where {name} is the port name, e.
↪g., A, B, etc.).
```

(continues on next page)

(continued from previous page)

```
|   Automatically selects the corresponding internal port index.
|   Default: `y`

TLK_GPIO_PREVENT_SLEEP
|   Indicates that the system should not enter sleep mode if any GPIO␣
↪configured as an output is enabled.
|   Default: `y` if `PM` is enabled, otherwise `n`.

TLK_GPIO_PM_DEVICE
|   Indicates that Power Management support is enabled for the GPIO driver.
|   Default: `y` if `PM` is enabled, otherwise `n`.
```

31.4.1 View in the menuconfig

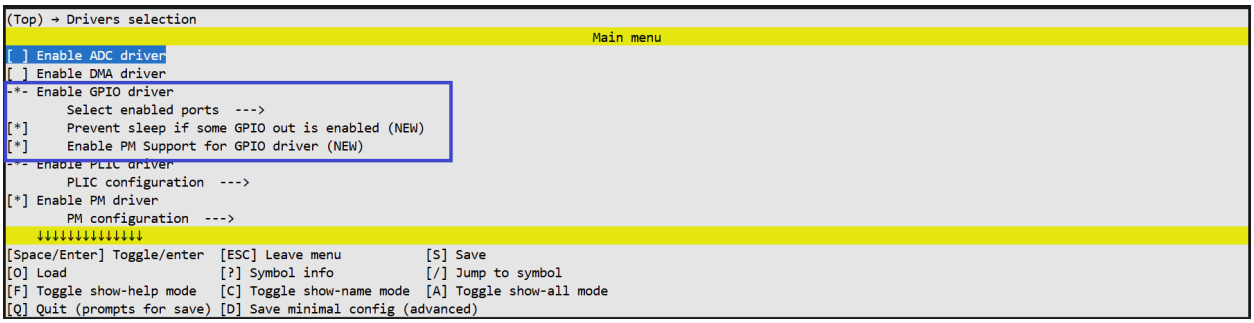


Figure 1. GPIO driver configurations

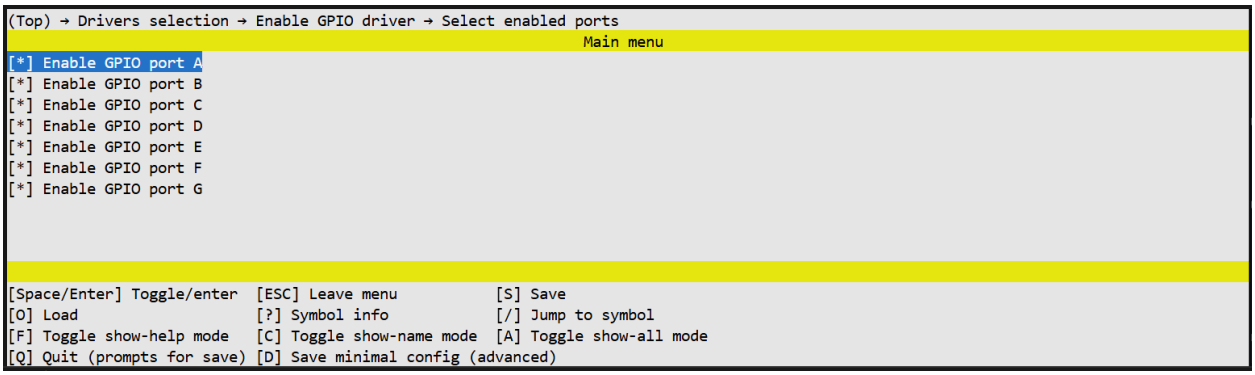


Figure 2. GPIO driver ports configurations

31.5 Related Resources

- *GPIO Samples* —Ready-to-run GPIO sample code

CHAPTER

THIRTYTWO

UART DRIVER

32.1 Overview

The UART driver provides an interface for configuring and using asynchronous serial ports. It supports communication parameter configuration (baud rate, parity, stop bits), pin multiplexing (Pinmux), data transmission (blocking mode), and data reception (blocking and non-blocking interrupt modes). It also supports hardware flow control (RTS/CTS).

Key Features:

- UART parameter configuration (baud rate, parity, stop bits)
- TX/RX and RTS/CTS pin multiplexing
- Data transmission (Blocking / PLIC / DMA modes)
- Data reception (Blocking / PLIC / DMA modes + callback)
- Hardware flow control (RTS/CTS)

32.2 Data Types and Enums

32.2.1 UART Module Identifiers

`enum tlk_uart_module` —Automatically generated based on `UNISDK_UART_COUNT` and enabled instances. Values include `UART0`, `UART1`, etc.

32.2.2 Parity

`enum tlk_uart_parity`

Value	Description
`TLK_UART_PARITY_NONE`	No parity
`TLK_UART_PARITY_EVEN`	Even parity
`TLK_UART_PARITY_ODD`	Odd parity

32.2.3 Stop Bits

`enum tlk_uart_stop_bit`

Value	Description
TLK_UART_STOP_BIT_ONE	1 stop bit
TLK_UART_STOP_BIT_ONE_DOT_FIVE	1.5 stop bits
TLK_UART_STOP_BIT_TWO	2 stop bits

32.2.4 RX Timeout Multiplier

`enum tlk_uart_rx_timeout_mul`

Multiplier for RX timeout duration. The total RX timeout = (one UART frame duration) × multiplier.

Value	Description
TLK_UART_RX_TIMEOUT_MUL_1	1× frame duration
TLK_UART_RX_TIMEOUT_MUL_2	2× frame duration
TLK_UART_RX_TIMEOUT_MUL_3	3× frame duration
TLK_UART_RX_TIMEOUT_MUL_4	4× frame duration

32.2.5 UART Configuration Structure

`struct tlk_uart_config`

A structure containing all configuration parameters for initializing a UART module. Used with `tlk_uart_configure` and `tlk_uart_configure_pinmux`.

Field	Type	Description
baudrate	uint32_t	Communication speed in baud
parity	enum tlk_uart_parity	Parity check mode
stop_bit	enum tlk_uart_stop_bit	Number of stop bits
rx_timeout_mul	enum tlk_uart_rx_timeout_mul	RX timeout multiplier
tx_port_pin	struct tlk_gpio_port_pin	TX port and pin
rx_port_pin	struct tlk_gpio_port_pin	RX port and pin

32.2.6 Operation Status

`enum tlk_uart_status`

Return codes used by driver functions to indicate the result of an operation.

Value	Description
TLK_UART_OK	Operation completed successfully.
TLK_UART_TIMEOUT	Operation timed out.
TLK_UART_ERROR	General execution error.

32.2.7 Flow Control Modes

`enum tlk_uart_flow_control`

Value	Description
`TLK_UART_FLOW_NONE`	Flow control disabled.
`TLK_UART_FLOW_RTS`	RTS signal only.
`TLK_UART_FLOW_CTS`	CTS signal only.
`TLK_UART_FLOW_RTS_CTS`	Full RTS/CTS flow control.

32.2.8 Flow Control Polarity

`enum tlk_uart_flow_polarity`

Defines the active level logic for RTS and CTS signals.

Value	Description
`TLK_UART_ACTIVE_L`	**Active Low.** UART stops sending when CTS is low, and sets RTS low to stop receiving.
`TLK_UART_ACTIVE_H`	**Active High.** UART stops sending when CTS is high, and sets RTS high to stop receiving.

32.2.9 Flow Control Configuration Structure

`struct tlk_uart_flow_control_config`

A structure containing all configuration parameters for UART hardware flow control. Used with `tlk_uart_configure_flow_control` and `tlk_uart_configure_flow_control_pinmux`.

Field	Type	Description
`flow_control`	`enum tlk_uart_flow_control`	Flow control mode (RTS/CTS)
`flow_polarity`	`enum tlk_uart_flow_polarity`	Active level for RTS/CTS signals
`rts_port_pin`	`struct tlk_gpio_port_pin`	RTS port and pin
`cts_port_pin`	`struct tlk_gpio_port_pin`	CTS port and pin

32.2.10 Transmit Handler Type

`tlk_uart_tx_handler_t`

A function pointer type for the callback invoked when data transmission is complete in interrupt and DMA modes.

```
typedef void (*tlk_uart_tx_handler_t)(void);
```

32.2.11 Receive Handler Type

`tlk_uart_rx_handler_t`

A function pointer type for the callback invoked when data reception is complete in interrupt and DMA modes.

```
typedef void (*tlk_uart_rx_handler_t)(uint32_t rx_count);
```

32.3 Driver API

32.3.1 tlk_uart_configure

Initializes the UART module with the provided configuration.

```
void tlk_uart_configure(enum tlk_uart_module uart_num,
                        struct tlk_uart_config *config);
```

Parameters:

Parameter	Type	Description
uart_nu	enum tlk_uart_module	UART module index.
config	struct tlk_uart_config *	Pointer to the configuration structure containing baudrate, parity, stop bits, and pin assignments.

Return value: None

32.3.2 tlk_uart_configure_pinmux

Configures UART TX/RX pin multiplexing.

```
void tlk_uart_configure_pinmux(enum tlk_uart_module uart_num,
                               struct tlk_uart_config *config);
```

Parameters:

Parameter	Type	Description
uart_nur	enum tlk_uart_module	UART module index.
config	struct tlk_uart_config *	Pointer to the configuration structure containing TX and RX pin assignments.

Return value: None

32.3.3 tlk_uart_send_bytes

Sends a data buffer.

```
enum tlk_uart_status tlk_uart_send_bytes(enum tlk_uart_module uart_num,
    const uint8_t *buff, uint32_t buff_size,
    tlk_uart_tx_handler_t tx_handler, uint32_t timeout_us);
```

32.3.4 tlk_uart_receive_bytes

Receives a data buffer. The provided handler is called when reception is complete.

```
enum tlk_uart_status tlk_uart_receive_bytes(enum tlk_uart_module uart_num,
    uint8_t *buff, uint32_t buff_size,
    tlk_uart_rx_handler_t rx_handler);
```

Parameters:

Parameter	Type	Description
uart_num	enum tlk_uart_module	UART module index.
buff	uint8_t *	Pointer to the receive buffer.
buff_size	uint32_t	Number of bytes to receive.
rx_handler	tlk_uart_rx_handler_t	Callback function invoked after reception completes.

Return value: enum tlk_uart_status —Returns TLK_UART_OK.

32.3.5 tlk_uart_configure_flow_control

Configure the flow control for the specified UART module. Disabled by default. Should be called after tlk_uart_configure.

```
void tlk_uart_configure_flow_control(enum tlk_uart_module uart_num,
                         struct tlk_uart_flow_control_config *config);
```

Parameters:

Parameter	Type	Description
uart_num	enum tlk_uart_module	UART module index.
config	struct tlk_uart_flow_control_cor *	Pointer to the flow control configuration structure.

Return value: None

32.3.6 tlk_uart_configure_flow_control_pinmux

Configure UART RTS and CTS pin multiplexing.

```
void tlk_uart_configure_flow_control_pinmux(enum tlk_uart_module uart_num,
                                struct tlk_uart_flow_control_config *config);
```

Parameters:

Parameter	Type	Description
uart_n	enum tlk_uart_module	UART module index.
config	struct tlk_uart_flow_control_c *	Pointer to the flow control configuration structure containing RTS and CTS pin assignments.

Return value: None

32.4 Transfer Modes

Each UART instance can independently configure its transfer mode:

Mode	Kconfig Value	Description
Blocking	TLK_UART_XFER_BLOCKING	Synchronous wait, with timeout
Interrupt	TLK_UART_XFER_PLIC	Interrupt-driven, auto-fill FIFO
DMA	TLK_UART_XFER_DMA	DMA driven, zero CPU load

32.5 Kconfig Configuration

32.5.1 Global options

TLK_UART_SELECTED Indicates that at least one UART instance is enabled in the system. This option is automatically selected when any UART{i}_ENABLED` option is enabled.

TLK_UART_USED_AUTO_CONFIG
Indicates that automatic UART configuration is enabled in the system. This option is automatically selected when UART{i}_AUTO_CONFIG is enabled for any UART instance.

TLK_UART_USED_IRQ
Indicates that at least one UART instance needs a PLIC functionality. This option is automatically selected when UART{i}_USED_IRQ or UART{i}_USED_DMA are enabled for any UART instance.

TLK_UART_USED_DMA
Indicates that at least one UART instance needs a DMA functionality. This option is automatically selected when UART{i}_USED_DMA is enabled for any UART instance.

TLK_UART_USED_PM_DEVICE
Indicates that UART power management support is enabled in the system. This option is automatically selected when UART{i}_PM_DEVICE is enabled for any UART instance.

TLK_UART_USED_FLOW_CONTROL
Indicates that hardware flow control (RTS/CTS) is enabled in the system. This option is automatically selected when UART{i}_FLOW_CONTROL is enabled for any UART instance.

32.5.2 Per-instance Configuration

Each UART instance (i = 0, 1, 2⋯) supports the following hierarchical configuration:

32.5.3 View in the menuconfig

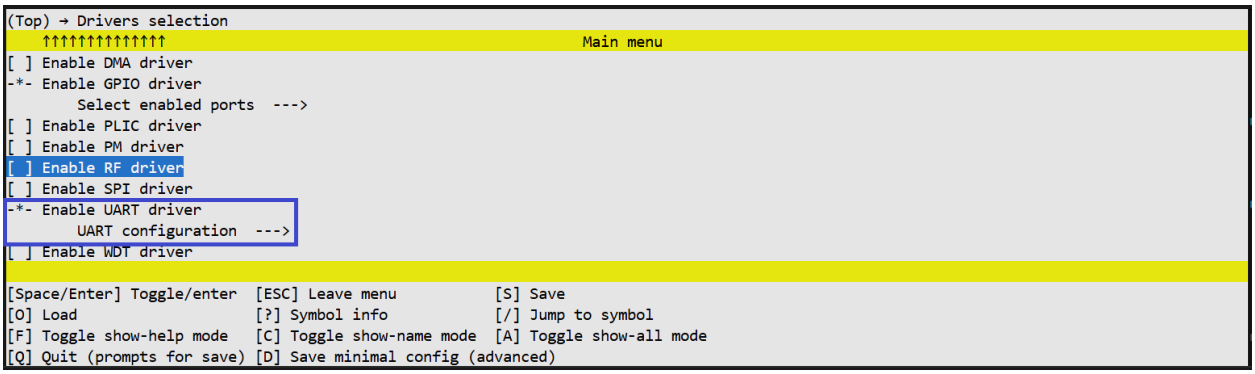


Figure 1. UART driver configurations

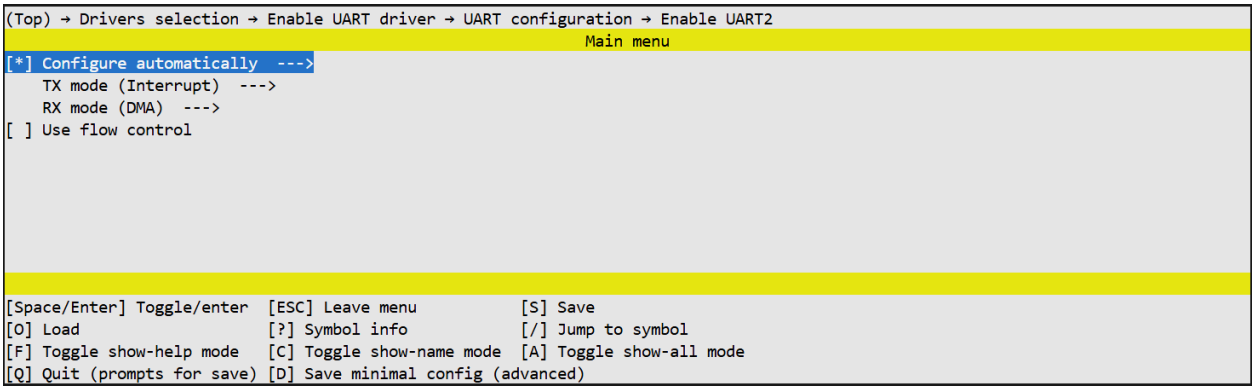


Figure 2. UART instance configurations

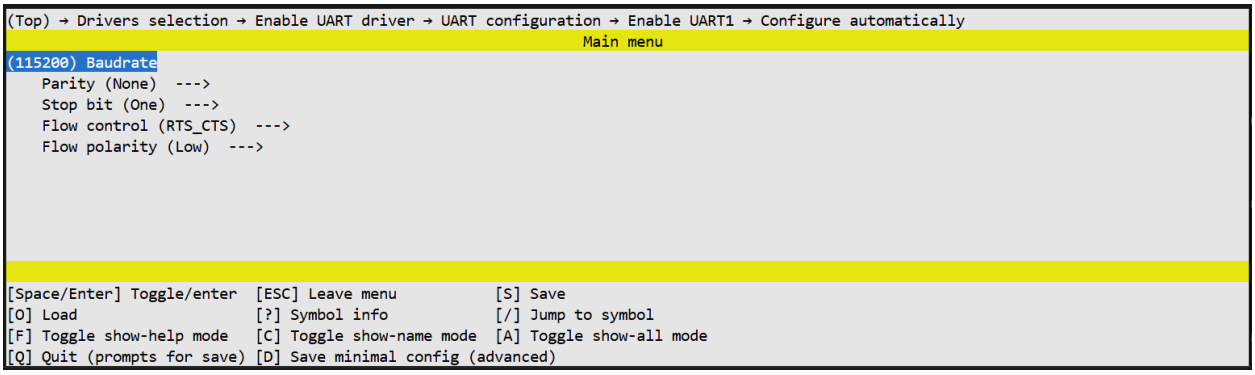


Figure 3. UART instance auto-config options

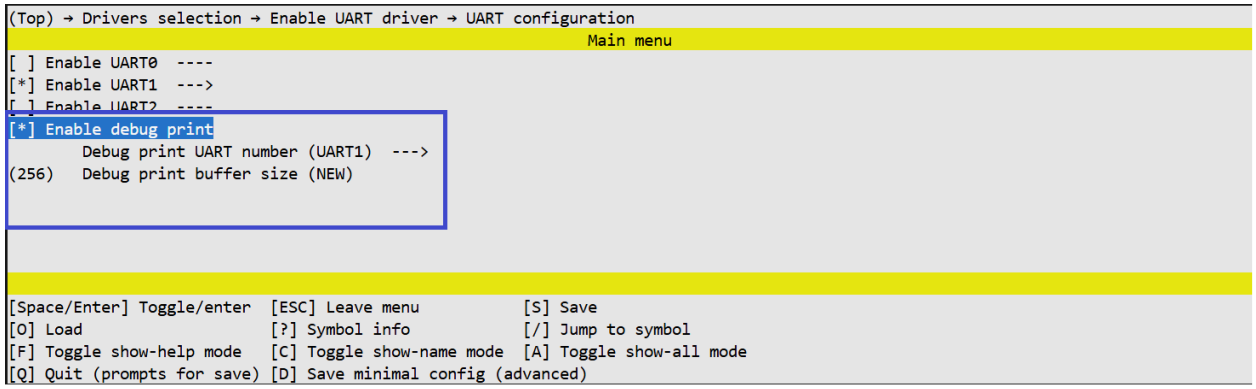


Figure 4. UART debug configurations

32.6 Related Resources

- *UART Samples*

CHAPTER

THIRTYTHREE

POWER MANAGEMENT (PM) DRIVER

33.1 Overview

The PM (Power Management) driver provides an interface for controlling microcontroller power states. It supports the system entering various low-power modes to save energy, and defines system wakeup mechanisms. The driver also manages retention memory and sleep duration validation.

Key Features:

- Multiple sleep modes (Suspend, Deep Sleep, Deep Retention)
- Wakeup source management (GPIO Pad, Timer, Core, Comparator)
- Configurable sleep duration limits (minimum time)
- Retention memory size configuration
- Error handling for invalid sleep requests

33.2 Sleep Modes

`enum tlk_pm_sleep_mode`

Mode	Power	Wakeup Speed	Description
`TLK_PM_SLEEP_MODE_SUSPEND`	Mediun	Fast	SRAM retained, peripherals partially powered
`TLK_PM_SLEEP_MODE_DEEP_SL`	Very low	Slow	SRAM not retained, most peripherals powered off
`TLK_PM_SLEEP_MODE_DEEP_RE`	Low	Medium	SRAM partially retained

33.3 Wakeup Sources

`enum tlk_pm_wakeup_source` (bitmask)

Value	Description
TLK_PM_WAKEUP_SOURCE_PAD	GPIO pin wakeup
TLK_PM_WAKEUP_SOURCE_TIMER	System timer wakeup
TLK_PM_WAKEUP_SOURCE_CORE	Core event wakeup
TLK_PM_WAKEUP_SOURCE_COMPARATOR	Comparator wakeup

33.4 GPIO Wakeup Level

`enum tlk_pm_gpio_wakeup_level`

Defines the logic level required on a GPIO pin to trigger a wakeup.

Value	Description
TLK_PM_GPIO_WAKEUP_LEVEL_LOW	Wake up when the pin is Low (0)
TLK_PM_GPIO_WAKEUP_LEVEL_HIGH	Wake up when the pin is High (1)

33.5 Driver API

33.5.1 tlk_pm_sleep

Requests the system to enter a low-power mode.

```
enum tlk_pm_sleep_status tlk_pm_sleep(enum tlk_pm_sleep_mode mode,
                                      uint32_t duration_us);
```

Return values:

Value	Description
TLK_PM_SLEEP_OK	Successfully entered and exited sleep
TLK_PM_SLEEP_UNSUPPORTED	The requested sleep mode is not supported by the hardware
TLK_PM_SLEEP_TOO_SHORT	Requested duration is less than the configured minimum
TLK_PM_SLEEP_TOO_LONG	The requested duration exceeded the hardware timer limits
TLK_PM_SLEEP_NO_WAKEUP_SOUR(	Sleep was aborted because no wakeup sources were configured
TLK_PM_SLEEP_DENIED	Sleep denied (e.g., blocked by a peripheral)

33.5.2 tlk_pm_set_gpio_wakeup

This API is available only if `CONFIG_TLK_GPIO` is enabled.

Configures a GPIO pin as a wakeup source. The system will exit sleep mode when the selected pin detects the configured wakeup polarity.

```
enum tlk_pm_sleep_status tlk_pm_set_gpio_wakeup(
    enum tlk_gpio_port port, enum tlk_gpio_pin pin,
    enum tlk_pm_gpio_wakeup_level polarity);
```

Parameters:

Parameter	Type	Description
`port`	`enum tlk_gpio_port`	GPIO port
`pin`	`enum tlk_gpio_pin`	GPIO pin
`polarity`	`enum tlk_pm_gpio_wakeup_level`	The logic level that triggers wakeup

Return Value:

`enum tlk_pm_sleep_status`: `TLK_PM_SLEEP_OK` if successful, or `TLK_PM_SLEEP_UNSUPPORTED` if the pin cannot be used as a wakeup source.

33.5.3 tlk_pm_get_wakeup_reason

Gets the last wakeup reason.

```
enum tlk_pm_wakeup_source tlk_pm_get_wakeup_reason(void);
```

33.5.4 tlk_pm_clear_wakeup_sources

Clears wakeup source flags in preparation for the next sleep.

```
void tlk_pm_clear_wakeup_sources(void);
```

33.6 Typical Usage

```
#include <tlk_api.h>

void sleep_example(void) {
    // Set GPIO A0 high level wakeup
    tlk_pm_set_gpio_wakeup(GPIO_PORT_A, GPIO_PIN_0,
                           TLK_PM_GPIO_WAKEUP_LEVEL_HIGH);

    // Enter Deep Sleep for 5 seconds
    enum tlk_pm_sleep_status status =
        tlk_pm_sleep(TLK_PM_SLEEP_MODE_DEEP_SLEEP, 5000000);

    if (status == TLK_PM_SLEEP_OK) {
        // Check wakeup reason
        enum tlk_pm_wakeup_source src = tlk_pm_get_wakeup_reason();
        if (src & TLK_PM_WAKEUP_SOURCE_PAD) {
            // GPIO wakeup
        }
        tlk_pm_clear_wakeup_sources();
    }
}
```

33.7 Kconfig Configuration

PM Kconfig Options

```
TLK_PM_SUSPEND_MIN_DURATION_MS
|   Defines the minimum allowed duration for Suspend mode (in milliseconds).
|   If `tlk_pm_sleep` is called with a time shorter than this, it returns␣
↪`TLK_PM_SLEEP_TOO_SHORT`.
|   Default: 2 ms
|
TLK_PM_DEEP_SLEEP_MIN_DURATION_MS
|   Defines the minimum allowed duration for Deep Sleep mode (in␣
↪milliseconds).
|   Default: 3 ms
|
TLK_PM_DEEP_RETENTION_MIN_DURATION_MS
|   Defines the minimum allowed duration for Deep Retention mode (in␣
↪milliseconds).
|   Default: 3 ms
|
TLK_PM_RETENTION_MEMORY_SIZE
|   Selects the size of the RAM to be retained during Deep Retention mode.
|   The available options (e.g., 16K, 32K) depend on the specific SoC␣
↪capabilities.
```

33.7.1 Menuconfig View

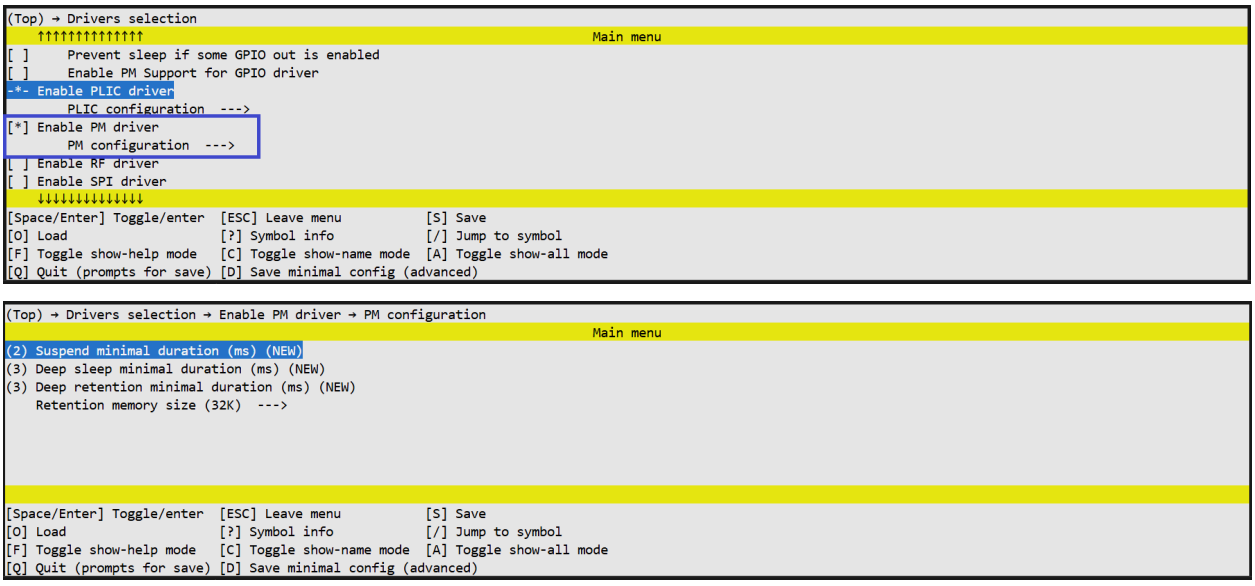


Figure 1-2. PM driver configurations

33.8 Related Resources

- *PM Samples*

CHAPTER

THIRTYFOUR

ANALOG DRIVER

34.1 Overview

The Analog driver provides raw access to chip analog registers and type-safe register field read/write macro definitions.

Key Features:

- Read/write analog registers by byte, half-word, and word
- YAML register descriptions converted to C type definitions
- Type-safe register read/write/modify macros

34.2 API Reference

34.2.1 C Functions

```c
// Analog register read/write
uint8_t  tlk_analog_read_reg8(uint8_t addr);
void     tlk_analog_write_reg8(uint8_t addr, uint8_t data);
uint16_t tlk_analog_read_reg16(uint8_t addr);
void     tlk_analog_write_reg16(uint8_t addr, uint16_t data);
uint32_t tlk_analog_read_reg32(uint8_t addr);
void     tlk_analog_write_reg32(uint8_t addr, uint32_t data);
```

These functions form the low-level transport layer, primarily used internally by the type-safe macros.

34.2.2 Type-Safe Macros

> **Important**: to take the most advantage of this feature, you need to have the language server capable of expanding deep macros, such as clangd. Then you will be able to fully use autocomplete, type hints, highlight, and so on.

To use these macros, include `registers/tlk_analog.h` with a complete analog register description. It contains named macros with corresponding addresses (e.g., `TLK_AREG_BG_CTRL0 -> 0x00`), which should be preferred over raw addresses.

If a register contains bitfields, a union type is generated:

```
typedef union tlk_areg_bg_ctrl0 {
    struct {
        <bitfields>
    } bit;
    uint8_t raw;
} tlk_areg_0x00;
```

Macro	Description
`TLK_AREG_TYPE(reg)`	Expands to the C type for the named analog register
`TLK_AREG_SIZE(reg)`	Expands to the generated size constant for the register (in bits)
`TLK_ANALOG_READ(reg)`	Reads the register, casting the returned value to the register value type
`TLK_ANALOG_WRITE(reg, var)`	Writes a typed register value back to the register
`TLK_ANALOG_MODIFY(reg, statement)`	Reads, modifies, and writes back a typed register value
`TLK_ANALOG_MODIFY_RAW8(reg, statement)`	Modifies raw 8-bit register data using `uint8_t`
`TLK_ANALOG_MODIFY_RAW16(reg, statement)`	Modifies raw 16-bit register data using `uint16_t`
`TLK_ANALOG_MODIFY_RAW32(reg, statement)`	Modifies raw 32-bit register data using `uint32_t`

Note: Internal helper macros in the header begin with underscores and are not intended for user code.

34.3 Register Definitions

Register definitions come from `core/*/registers/analog.yaml`, and type-safe C access macros are automatically generated via `registers_gen.py`.

CHAPTER

THIRTYFIVE

PLIC INTERRUPT CONTROLLER

35.1 Overview

The PLIC (Platform-Level Interrupt Controller) driver provides a system interrupt management interface. It allows developers to configure interrupt priorities, enable or disable specific interrupt sources, and manage interrupt preemption (nesting).

The interrupt handling mode (General vs. Vector) is determined at compile time via system configuration (`.config`).

Key Features:

- Compile-time selection of global interrupt mode (General / Vector)
- Interrupt priority and threshold management
- Interrupt source enable/disable
- Interrupt nesting support
- Optional software interrupt support (Trigger, Enable, Disable)

35.2 Data Types and Enums

35.2.1 Interrupt Priorities

`enum tlk_irq_priority`

Value	Description
`TLK_IRQ_PRI_LEV0`	Priority 0 —no interrupt generated (effectively disabled)
`TLK_IRQ_PRI_LEV1`	Priority Level 1 (Lowest)
`TLK_IRQ_PRI_LEV2`	Priority Level 2
`TLK_IRQ_PRI_LEV3`	Priority Level 3 (Highest)

35.3 API Reference

35.3.1 `TLK_PLIC_ISR_REGISTER` (Macro)

A helper macro to define and register an Interrupt Service Routine (ISR) for a specific interrupt number. It automatically generates the correct function attributes for Vector or General mode.

```
TLK_PLIC_ISR_REGISTER(my_uart_handler, IRQ_UART0);
```

Parameter	Description
`isr`	The C function name to be called as the handler
`irq_num`	The interrupt number (e.g., `IRQ_UART0`)

35.3.2 tlk_plic_interrupt_enable

Enables a specific interrupt source.

```
void tlk_plic_interrupt_enable(uint32_t src);
```

35.3.3 tlk_plic_interrupt_disable

Disables a specific interrupt source.

```
void tlk_plic_interrupt_disable(uint32_t src);
```

35.3.4 tlk_plic_set_priority

Sets the priority level for a specific interrupt source.

```
void tlk_plic_set_priority(uint32_t src, enum tlk_irq_priority priority);
```

35.3.5 tlk_plic_set_threshold

Sets the global interrupt threshold. Only interrupts with a priority strictly higher than this threshold will be serviced.

```
void tlk_plic_set_threshold(enum tlk_irq_priority threshold);
```

35.3.6 tlk_plic_set_pending

Manually sets an interrupt source as pending.

```
void tlk_plic_set_pending(uint32_t src);
```

35.3.7 tlk_plic_clr_all_request

Clears all pending interrupt requests.

```
int32_t tlk_plic_clr_all_request(void);
```

35.3.8 tlk_plic_preempt_enable

Enables interrupt preemption (nested interrupts) for priorities higher than the specified level.

```
void tlk_plic_preempt_enable(tlk_core_preempt_pri_e preempt_pri);
```

35.3.9 tlk_plic_preempt_disable

Disables interrupt preemption.

```
void tlk_plic_preempt_disable(void);
```

35.3.10 tlk_plic_irqs_preprocess_for_wfi

Prepares the PLIC controller before the system enters a "Wait For Interrupt" (WFI) or sleep state.

```
void tlk_plic_irqs_preprocess_for_wfi(uint8_t flag, tlk_mie_e mie);
```

35.3.11 tlk_plic_irqs_postprocess_for_wfi

Restores the PLIC state after the system wakes up from WFI or sleep.

```
void tlk_plic_irqs_postprocess_for_wfi(void);
```

35.3.12 Software Interrupt APIs

Function	Description
`tlk_plic_sw_set_pending()`	Manually triggers a software interrupt
`tlk_plic_sw_interrupt_enab`	Enables the software interrupt source
`tlk_plic_sw_interrupt_disa`	Disables the software interrupt source
`tlk_plic_sw_interrupt_clai`	Claims the software interrupt source (typically used inside the ISR)
`tlk_plic_sw_interrupt_comp`	Completes the software interrupt, allowing new requests to be processed

35.4 PLIC Configuration

This section describes the PLIC configuration options available in Kconfig.

Config	Default	Description
`TLK_PLIC_VE`	y	Selects the global interrupt handling mode. If enabled, Vector Mode is used (interrupts jump directly to their vector address). If disabled, General Mode is used (all interrupts share a common handler).
`PLIC_SW` (or `CONFIG_TLK_PLI`	n	Enables support for Software Interrupts. If enabled, APIs like `tlk_plic_sw_set_pending()` become available.

35.5 Interrupt Handling Modes

Mode	Description
General	All interrupts share a single entry point; query the interrupt source in the ISR
Vector	Each interrupt source has an independent vector address; direct jump

35.6 Interrupt Nesting

PLIC supports priority-based interrupt preemption:

- High-priority interrupts can preempt low-priority interrupts
- Control the minimum response priority via `tlk_plic_set_threshold`

35.7 Integration with GPIO Interrupts

GPIO interrupts are routed through PLIC. Configuration steps:

1. Configure PLIC interrupt priority
2. Configure GPIO interrupt trigger condition (`tlk_gpio_irq_configure`)
3. Register GPIO interrupt callback (`tlk_gpio_irq_add_callback`)
4. Enable global interrupts (`tlk_core_interrupt_enable`)

```
// Complete example
tlk_plic_set_priority(IRQ_GPIO, 3);
tlk_gpio_irq_configure(GPIO_PORT_A, GPIO_PIN_0, TLK_GPIO_INTR_RISING_EDGE);
tlk_gpio_irq_add_callback(GPIO_PORT_A, &my_callback);
tlk_core_interrupt_enable();
```

CHAPTER

THIRTYSIX

I2C DRIVER

Note

Document Status: DRAFT —Content is being finalized

36.1 Overview

The I2C driver supports master/slave mode communication, providing blocking and DMA transfer modes.

36.2 API Reference

```
// I2C initialization
void tlk_i2c_configure(enum tlk_i2c_module i2c_num, uint32_t speed);

// I2C read/write
enum tlk_i2c_status tlk_i2c_write(enum tlk_i2c_module i2c_num, ...);
enum tlk_i2c_status tlk_i2c_read(enum tlk_i2c_module i2c_num, ...);
```

36.3 Samples

See *I2C Samples*.

CHAPTER

THIRTYSEVEN

SPI DRIVER

Note

Document Status: DRAFT —Content is being finalized

37.1 Overview

The SPI driver supports master/slave mode communication, providing blocking and DMA transfer modes.

37.2 API Reference

```
// SPI initialization
void tlk_spi_configure(enum tlk_spi_module spi_num, uint32_t speed);

// SPI read/write
enum tlk_spi_status tlk_spi_write(enum tlk_spi_module spi_num, ...);
enum tlk_spi_status tlk_spi_read(enum tlk_spi_module spi_num, ...);
```

37.3 Samples

See *SPI Samples*.

CHAPTER

THIRTYEIGHT

ADC DRIVER

Note

Document Status: DRAFT —Content is being finalized

38.1 Overview

The ADC driver provides analog-to-digital conversion functionality.

38.2 Samples

See *ADC Samples*.

CHAPTER

THIRTYNINE

DMA DRIVER

Note

Document Status: DRAFT —Content is being finalized

39.1 Overview

The DMA driver provides direct memory access functionality, supporting efficient data transfer from peripheral to memory and memory to peripheral.

39.2 Samples

See *DMA Samples*.

CHAPTER

FORTY

WATCHDOG DRIVER

Note

Document Status: DRAFT —Content is being finalized

40.1 Overview

The Watchdog driver provides watchdog timer functionality for system error recovery.

40.2 Samples

See *Watchdog Samples*.

CHAPTER

FORTYONE

TIMER DRIVER

Note

Document Status: DRAFT —Content is being finalized

41.1 Overview

The Timer driver includes STIMER (System Timer) and MTIMER (Machine Timer), providing high-precision timing and scheduling functionality.

41.2 Samples

See *Timer Samples*.

CHAPTER

FORTYTWO

USB DRIVER

Note

Document Status: DRAFT —Content is being finalized

42.1 Overview

The USB driver is based on the TinyUSB framework, providing USB CDC and other device class support.

42.2 Samples

See *USB CDC Samples*.

CHAPTER

FORTYTHREE

PMP DRIVER

Provides an API for configuring the Physical Memory Protection (PMP) unit on RISC-V cores. PMP allows M-mode software to restrict physical memory access privileges (read, write, execute) for each defined region, enforced in hardware for U-mode and optionally for M-mode when locked.

43.1 Overview

The PMP unit supports up to **8 entries**. Each entry defines one protected memory region and a set of access permissions.

Any violation is trapped precisely at the processor. The trap cause can be inspected via `mcause`, `mdcause`, and `mtval` CSRs.

43.2 Address Matching Modes

Each entry operates in one of two address matching modes:

Mode	API	Description
`TOR`	`tlk_pmp_to`	Top Of Range. The protected region is `[pmpaddr[entry-1], address)`. For entry 0, the lower bound is implicitly `0x00000000`.
`NAPO`	`tlk_pmp_na`	Naturally Aligned Power Of Two. The region is `[address, address + size)`. Base must be aligned to `size`; `size` must be a power of two, at minimum 8 bytes.

To use TOR with an explicit lower bound, call `tlk_pmp_entry_disable` on `entry - 1` with the desired lower bound address. This writes the address into `pmpaddr[entry-1]` without activating protection on that entry.

43.3 Permissions

Permissions are defined per entry via `struct tlk_pmp_config`:

Field	Description
r	Allow read access.
w	Allow write access.
x	Allow instruction execution.
l	Lock the entry. See *Lock* below.

43.4 Lock

When `l = 1` is set on an entry:

- Writes to `PMPiCFG` and `PMPADDRi` are **ignored** by hardware until system reset.
- For TOR entries, `pmpaddr[entry-1]` is also frozen.
- Permissions apply to **all privilege modes**, including M-mode.

  A locked entry can only be cleared by a system reset. Set `l = 1` only when the protection must survive any software fault, including bugs in M-mode code.

43.5 API

```
void tlk_pmp_tor_config(enum tlk_pmp_entry entry, void *address,
                        struct tlk_pmp_config *config);
```

Configures `entry` in TOR mode. `address` is the exclusive upper bound of the protected region.

```
void tlk_pmp_napot_config(enum tlk_pmp_entry entry, void *address, uint32_t␣
↪size,
                          struct tlk_pmp_config *config);
```

Configures `entry` in NAPOT mode. `address` must be aligned to `size`. `size` must be a power of two and at least 8 bytes.

```
void tlk_pmp_entry_disable(enum tlk_pmp_entry entry, void *address);
```

Sets the entry to OFF mode, removing any active protection. The value of `address` is retained in `pmpaddr[entry]` and can serve as the lower bound for a subsequent TOR entry. Pass `NULL` if the address is not relevant. Has no effect if the entry is locked (`l = 1`).

43.6 Configuration

All options are available under `PMP` in Kconfig.

Symbol	Default	Description
TLK_PMP_PM_	n	Enable Power Management support. When enabled, all PMP entry registers are saved before sleep and restored after wakeup.

CHAPTER

FORTYFOUR

CONNECTIVITY

44.1 Overview

UniSDK integrates BLE (Bluetooth Low Energy) Controller (Link Layer), providing fundamental wireless connectivity for BLE application development.

44.2 Code Location

BLE Controller Directory

```
subsystem/ble/controller/
├── acl/                    # ACL Link Layer
├── CMakeLists.txt
├── Kconfig
├── README.md               # Coding standards
└── README_zh.md
```

44.3 Coding Standards

The BLE Controller follows strict coding standards to ensure performance and reliability:

44.3.1 File Naming

- Source files use lowercase letters and underscores, e.g., `tlk_ble_ll_acl.c`

44.3.2 Function Naming

- Public functions use the `tlk_ble_` prefix
- Internal functions use the _ prefix

44.3.3 Interrupt Functions

- Interrupt handler functions have specific naming and modifier requirements
- Interrupt functions should be as short as possible, avoiding complex operations

44.4 Kconfig Configuration

```
# BLE related configuration options (configured interactively via menuconfig)
```

44.5 Examples

- *BLE Example* —BLE advertising and connection demo

44.6 Planned

The following protocol stack features are under planning:

- BLE Host Layer (GAP, GATT, SM)
- Zigbee
- Thread
- Matter

  For more details, please contact the Telink team for the latest updates.

CHAPTER

FORTYFIVE

BLE CONTROLLER CODING STANDARDS

45.1 Overview

This document describes the coding standards for the BLE Controller ROM, applicable to all code under `subsystem/ble/controller/`.

45.2 File Naming

- Source files use lowercase letters separated by underscores
- Example: `tlk_ble_ll_acl.c`

45.3 Function Naming

45.3.1 Public Functions

Use the `tlk_ble_` prefix:

```
void tlk_ble_ll_init(void);
int  tlk_ble_ll_send_data(uint8_t *data, uint16_t len);
```

45.3.2 Internal Functions

Use the _ prefix to mark module-internal functions:

```
static void _internal_process(void);
```

45.4 Variable Naming

- Global variables use the `g_` prefix
- Static variables use the `s_` prefix
- Local variables use lowercase underscore style

45.5 Interrupt Functions

Interrupt handler functions should meet the following requirements:

- Names must include the `_irq_` or `_isr_` identifier
- Use specific modifiers to ensure correct calling conventions
- Avoid complex operations and blocking calls within interrupt functions
- Prefer the flag + main loop processing pattern whenever possible

CHAPTER

FORTYSIX

SYSTEM SERVICES

Note

Document Status: PLANNED —Content for this module has not yet been written

The System Services module will cover the high-level system service components provided by UniSDK, including:

- Task scheduling and event handling
- Memory management
- Logging system (Debug Print)
- System initialization framework

46.1 Currently Available Components

Component	API	Description
System Initialization	`tlk_init.h`	Modular initialization framework
Main Loop	`tlk_loop.h`	Event-driven main loop
Delay/Sleep	`tlk_sleep.h`	`tlk_sleep_ms()` / `tlk_sleep_us()`
Time Management	`tlk_time.h`	Timestamps and timers

Detailed documentation will be added later. PR contributions are welcome.

CHAPTER

FORTYSEVEN

BUILD & CONFIGURATION

UniSDK adopts a modular and extensible build system architecture, integrating tools such as CMake, Ninja, West, Kconfig, and Make to form a complete compilation and build workflow.

47.1 Core Components

Component	Purpose	Description
CMake	Build system core	Project configuration, dependency management, build rule generation
Ninja	Build execution engine	High-performance parallel compilation, CMake's default backend
West	Project management tool	Unified command-line entry point, simplified build workflow
Kconfig	Configuration management system	Hierarchical management of compile-time configuration options
Make	Traditional interface	Frontend interface compatible with traditional Make conventions

47.2 Three Build Methods

UniSDK supports three build entry points, adapting to different scenarios:

```
# Method 1: West command (recommended, simplest)
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Method 2: Make command (compatible with traditional conventions)
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK

# Method 3: Direct CMake invocation (most flexible)
cmake -B build -G Ninja -S samples/gpio_demo -DBOARD=TLSR9528A_EVK
cmake --build build
```

47.3 Documentation Navigation

Section	Description	Status
CMake Build System	CMake core flow, module loading, configuration phase details	STABLE
Kconfig Configuration System	Configuration hierarchy, processing flow, key scripts	STABLE
West Commands	tl-build / tl-config / tl-bdt and other command details	STABLE
Make Support	Makefile frontend interface, targets and variable descriptions	STABLE
Build Flow	Environment validation, config generation, compile & link full flow	STABLE
Extensions & Customization	Add new modules, extend West commands, Kconfig customization	STABLE
File & Script Reference	Config files, CMake modules, script inventory	STABLE

47.4 Build System Architecture

CHAPTER

FORTYEIGHT

CMAKE BUILD SYSTEM

CMake is the core of the UniSDK build system, responsible for project configuration, dependency resolution, and build rule generation.

48.1 Entry Point Modes

UniSDK supports two CMake entry point modes:

Mode	Entry	Purpose	Command Example
Root entry	`CMakeLists.txt` (repo root)	SDK development/testing	`cmake -B build -G Ninja .`
Application entry	App's `CMakeLists.txt`	Build specific application	`cmake -B build -G Ninja samples/gpio_demo`

48.1.1 Application-Level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(GPIO_Demo LANGUAGES C CXX)

file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

48.2 Module Loading Order

`telink_default.cmake` loads CMake modules in the following order:

1. **`python.cmake`** —Python environment detection (Python 3.10+)
2. **`extensions.cmake`** —CMake extension functions (conditional variables, JSON processing)
3. **`version.cmake`** —Version information processing

4. **west.cmake** —West tool integration (version >= 0.14.0)
5. **compiler.cmake** —RISC-V compiler configuration
6. **kconfig.cmake** —Kconfig configuration integration
7. **pinmux.cmake** —Pin multiplexing configuration generation
8. **linker.cmake** —Linker script processing

48.3 Core Module Details

48.3.1 compiler.cmake

- Detects the `TELINK_TOOLCHAIN_PATH` environment variable
- Configures the RISC-V GCC cross-compilation toolchain
- Provides the `telink_apply_compiler_options()` function
- Dynamically loads compilation options from `build_settings.json`

48.3.2 linker.cmake

- Sets the RISC-V link command template
- Reads linker options from `build_settings.json`
- Preprocesses linker scripts (`flash_boot.link` → `flash_boot.ld`)
- Post-processing: generates `.bin` + `.map` + firmware verification

48.3.3 kconfig.cmake

- Manages the generation flow of `chip.config` → `build.config` → `.config`
- Generates `capabilities.h` and `autoconf.h`
- Defines CMake targets such as `config` / `parse_config`
- SOC/BOARD parameter processing and combination validation

48.3.4 pinmux.cmake

- Generates `.pinmux` and `pinmux.h`
- Depends on the `.config` file
- Build directory is automatically added to the include path

48.3.5 extensions.cmake

```
# Conditional variable setting
set_ifndef(variable value [PARENT_SCOPE])

# Conditional source file addition
add_sources_ifdef(config src)
```

(continues on next page)

(continued from previous page)

```
# JSON array to CMake list
json_to_list(JSON_ARRAY_STRING OUTPUT_LIST)
```

48.4 Configuration Flow

48.5 Include Guard Mechanism

Guard	Set Location	Purpose
TELINK_ROOT_INCLUDED	Root CMakeLists.txt	Prevent duplicate root inclusion
TELINK_PACKAGE_INCLUDED	TelinkConfig.cmake	Prevent duplicate package loading

48.6 Build Directory Structure

Build Directory Structure

```
build/
├── .config              # Merged final configuration
├── chip.config          # Chip configuration
├── build.config         # Build configuration
├── autoconf.h           # C configuration macros
├── capabilities.h       # Chip capability declarations
├── pinmux.h             # Pin multiplexing configuration
├── flash_boot.ld        # Linker script
├── build.ninja          # Ninja build file
├── CMakeCache.txt       # CMake cache
└── Kconfig/             # Chip-specific Kconfig fragments
```

CHAPTER

FORTYNINE

KCONFIG CONFIGURATION SYSTEM

Kconfig is the configuration management core of UniSDK, using a hierarchical structure to manage compile-time configuration options.

49.1 Architecture

49.1.1 Configuration Entry Points

UniSDK uses a dual-entry Kconfig design:

```
Kconfig.chip     → Chip-related configuration (SoC selection, peripheral␣
↪enabling)
Kconfig.build    → Build-related configuration (feature options, parameter␣
↪settings)
```

49.1.2 Hierarchy

49.1.3 Application-Level Kconfig Integration

Through the `KCONFIG_APPLICATION_DIR` variable, the application layer can define its own Kconfig options, seamlessly integrating with the SDK configuration.

49.2 Configuration Generation Flow

49.2.1 Key Configuration Files

File	Source	Description
`chip.config`	Kconfig.chip	Chip-related configuration (SoC, peripheral enabling)
`build.config`	Kconfig.build	Build option configuration
`.config`	chip.config + build.config	Final merged configuration
`capabilities.h`	chip.config → preprocessing	Chip capability declaration header
`autoconf.h`	.config → parsing	C preprocessor macro definitions

49.2.2 SOC/BOARD Parameter Handling

```
# Specify SOC during build
cmake -B build -DSOC=TLSR9528A -S .
# Specify BOARD during build
cmake -B build -DBOARD=TLSR9528A_EVK -S .

# West command (specify SOC)
west tl-build --soc TLSR9528A

# West command (specify BOARD)
west tl-build --board TLSR9528A_EVK
```

SOC/BOARD passed this way only take effect the **first time** `chip.config` is generated for a build directory (as of UniSDK v0.3.0) —CMake seeds the choice into the non-interactive Kconfig pass so it comes out selected, instead of requiring the interactive `menuconfig` UI. If `chip.config` already exists (e.g. a build directory reused across configure runs), these flags are ignored; delete `chip.config` or edit it directly to change SOC/BOARD.

`kconfig_parser.py`/`soc_board_setter.py` provide standalone SOC/BOARD dependency resolution and combination validation, but are not currently invoked automatically as part of this flow.

49.3 Key Scripts

Script	Purpose
`kconfig.py`	Non-interactive Kconfig processing, configuration merging, validity verification
`menuconfig.py`	Interactive terminal configuration interface
`preprocess_kconfig.py`	Configuration preprocessing, YAML variable expansion
`kconfig_parser.py`	Kconfig parsing and SOC/BOARD validation
`soc_board_setter.py`	SOC/BOARD configuration and dependency resolution

49.4 Menuconfig Trigger Methods

1. **Auto-triggered during build** —Automatically opens the interactive interface when config files are missing
2. **Manually triggered** —`python scripts/menuconfig.py Kconfig.chip`
3. **Non-interactive generation** —CMake configuration phase uses `kconfig.py` to auto-generate defaults

49.5 Configuration Option Examples

```
# core/configs/Kconfig.gpio
```

(continues on next page)

(continued from previous page)

```
config TLK_GPIO_PORT_A_ENABLED
    bool "Enable GPIO Port A"
    default y
    help
        Enable GPIO Port A instance.

config TLK_GPIO_PREVENT_SLEEP
    bool "Prevent sleep when GPIO is active"
    depends on TLK_PM
    default y
```

CHAPTER

FIFTY

WEST COMMANDS

West is the unified command-line entry point for UniSDK. UniSDK registers multiple custom command extensions through `scripts/west-commands.yml`.

50.1 Command Registration

```yaml
west-commands:
  - file: scripts/west_commands/build.py
    commands:
      - name: tl-build
        class: Build
        help: compile a Telink application
  - file: scripts/west_commands/boards.py
    commands:
      - name: tl-boards
        class: Boards
        help: display information about supported Telink boards
  - file: scripts/west_commands/socs.py
    commands:
      - name: tl-socs
        class: Socs
        help: display information about supported Telink socs
  - file: scripts/west_commands/config.py
    commands:
      - name: tl-config
        class: Config
        help: configure a Telink application
  - file: scripts/west_commands/bdt.py
    commands:
      - name: tl-bdt
        class: Bdt
        help: use BDT tool to configure and flash a Telink chip
```

50.2 `west tl-build` —Build

Build a Telink application. The build directory defaults to `build/` in the current working directory.

```
# Basic usage
west tl-build [source_dir] [-d build_dir] [options]

# Specify source directory
west tl-build samples/gpio_demo

# Specify build directory
west tl-build -d build_tl3218

# Specify SOC during build
west tl-build --soc TLSR9528A

# Specify BOARD during build
west tl-build --board TLSR9528A_EVK

# Build after interactive configuration
west tl-build --kconfig

# Clean build
west tl-build --clean
west tl-build --pristine always

# Build and flash
west tl-build --flash

# Build specific target
west tl-build -t flash

# CMake configuration only, no compilation
west tl-build --cmake-only

# Pass additional CMake options
west tl-build -- -DCONFIG_DEBUG=y
```

50.2.1 Execution Flow

1. Check the `TELINK_TOOLCHAIN_PATH` environment variable
2. Validate the SOC/BOARD combination
3. Set source and build directories
4. Run Kconfig configuration if needed
5. Invoke CMake to generate the build system
6. Execute compilation and linking
7. Output build results

50.3 `west tl-config` —Configuration

Independently configure a Telink application.

```
# Basic usage
west tl-config [source_dir] [-d build_dir]

# CMake-only configuration (skip interactive GUI)
west tl-config -c

# List available SOCs
west tl-config --list-socs

# List available boards
west tl-config --list-boards

# Validate SOC/BOARD combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

50.4 `west tl-bdt` —Burning & Debugging

BDT (Burning and Debugging Tool) command interface.

50.4.1 Subcommands

Subcommand	Description
`download`	Download firmware to Flash
`read`	Read Flash data
`write`	Write Flash data
`erase`	Erase Flash
`lock`	Lock Flash region
`unlock`	Unlock Flash
`reset`	Reset device

50.4.2 Common Examples

```
# Download firmware
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Read 16 bytes from Flash
west tl-bdt read --chip TLSR9528A -a 0x00 -s 16

# Erase Flash
west tl-bdt erase --chip TLSR9528A -a 0x00 -s 4k

# Lock Flash
west tl-bdt lock --chip TLSR9528A -a 0x00 -s 512k

# Reset device
west tl-bdt reset --chip TLSR9528A

# Specify BDT tool path
```

(continues on next page)

(continued from previous page)

```
west tl-bdt download --bdt-path /path/to/BDT --chip TLSR9528A -i build/
↪firmware.bin
```

50.4.3 BDT Path Configuration (Priority from high to low)

1. Command line: `--bdt-path /path/to/bdt`
2. Environment variable: `BDT_PATH=/path/to/bdt`
3. System PATH
4. SDK directory: `${TELINK_BASE}/tools/`

50.4.4 Cross-Platform

Platform	Executable
Windows	`Cmd_download_tool.exe`
Linux	`bdt`

50.5 `west tl-boards` —Board Information

```
west tl-boards              # List all supported boards
west tl-boards -v           # Show detailed information
west tl-boards --soc TLSR9528A  # Filter by chip
```

50.6 `west tl-socs` —Chip Information

```
west tl-socs                # List all supported chips
west tl-socs -v             # Show detailed information
```

CHAPTER

FIFTYONE

MAKE SUPPORT

The UniSDK `Makefile` provides a traditional make command interface, serving as a CMake frontend to simplify build commands. **Can only be used at the project root directory.**

51.1 Makefile Targets

Target	Description	Command
`cmake`	Run CMake configuration	`make cmake`
`config`	Generate project configuration	`make config`
`pinmux`	Run Pinmux configuration tool	`make pinmux`
`build`	Full build (cmake + config + build)	`make build`
`clean`	Clean build artifacts	`make clean`
`cleanbuild`	Clean then rebuild	`make cleanbuild`

51.2 Makefile Variables

Variable	Default	Description
`APP`	`samples/gpio_demo`	Application path
`SOC`	(empty)	Target chip
`BOARD`	(empty)	Target development board
`BUILD_DIR`	`build`	Build directory

51.3 Usage Examples

```
# Default build
make build

# Specify application
make cmake build APP=samples/gpio_demo

# Specify chip and board
make cmake build SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Specify application, chip, and board simultaneously
```

(continues on next page)

(continued from previous page)

```
make cmake build APP=samples/gpio_demo SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Clean and rebuild
make cleanbuild

# Configure only
make config
make pinmux
```

51.4 Build Flow (make build)

1. **`cmake`** —Initialize the CMake build system for the specified application
2. **`config`** —Generate configuration files (`.config`, etc.)
3. **Pre-build checks** —4 automatic check and recovery steps:
   - Check if the build directory exists
   - Check if the `.config` file exists
   - Check if `pinmux.h` exists
   - Check if `pinmux.h` is up to date
4. **`cmake --build`** —Execute Ninja compilation

51.5 SOC/BOARD Parameter Forwarding

The Makefile automatically forwards SOC and BOARD variables to CMake:

```
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif
ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif
```

CHAPTER

FIFTYTWO

BUILD FLOW

52.1 Overview

This document details the complete build process flow from configuration to firmware generation.

52.2 Complete Build Flow

52.3 Getting Started Guide

52.3.1 Install Dependencies

You’ll install some host dependencies using your package manager.

Ubuntu

Use apt to install the required dependencies:

```
sudo apt install --no-install-recommends git cmake ninja-build python3-dev␣
↪python3-venv python3-tk make
```

Verify the versions:

```
cmake --version
python3 --version
git --version
ninja --version
```

If “command not found” is displayed, the corresponding tool is not installed.

MacOS

Use Homebrew to install the required dependencies:

```
brew install cmake ninja python3 python-tk
```

Verify the versions:

```
cmake --version
python3 --version
git --version
ninja --version
```

Windows

You can install using the following two methods:

1. For systems that support winget:

```
winget install Kitware.CMake Ninja-build.Ninja Python Git.Git
```

2. For systems that don't support winget, download and install from official websites:
   - CMake (check "Add CMake to the PATH")
   - Ninja (manually add to PATH)
   - Python (check "Add Python.exe to PATH")
   - Git

After installation, verify:

```
cmake --version
python --version
git --version
ninja --version
```

52.3.2 Install Python Dependencies

Python Virtual Environment

```
# Create virtual environment
python -m venv .venv          # Windows
python3 -m venv .venv         # Linux/MacOS
```

```
# Activate virtual environment
source .venv/bin/activate           # Linux/Mac
.venv\Scripts\activate.bat          # Windows Batchfile
.venv\Scripts\activate.ps1          # Windows PowerShell
```

When using PowerShell, you might encounter:

```
Activate.ps1 cannot be loaded because running scripts is disabled on this␣
↪system.
```

Resolve by:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Then install Python dependencies:

```
pip install -r requirements.txt
```

West Tool Setup

```
# Verify west installation
west --version

# Initialize west workspace
west init -l
```

Potential Issue with west init -l:

If you encounter the error `already initialized in {where}, aborting.`, this is likely due to the `ZEPHYR_BASE` environment variable being set to a Zephyr workspace with an existing `.west` directory.

Solution: Temporarily unset `ZEPHYR_BASE`:

- **Windows (PowerShell):**

```
Remove-Item Env:ZEPHYR_BASE; west init -l
```

- **Linux/macOS:**

```
unset ZEPHYR_BASE; west init -l
```

Toolchain Configuration

Download and extract the toolchain (only available for Telink): Toolchain_V5.4.1

Set the `TELINK_TOOLCHAIN_PATH` environment variable and ensure toolchain binaries are accessible in PATH.

52.4 Six Phases

52.4.1 1. Environment Validation

- Verify that the Python virtual environment is activated
- Verify that West is available and version >= 0.14.0
- Verify that `TELINK_TOOLCHAIN_PATH` is set
- Verify that the toolchain executables are accessible

52.4.2 2. CMake Configuration

```
cmake -B build -G Ninja -S samples/gpio_demo
```

Load and execute all CMake modules (python -> extensions -> west -> compiler -> kconfig -> pinmux -> linker).

52.4.3 3. Configuration Generation (Config Target)

```
cmake --build build --target config
```

Executed in order:

1. Clean old config -> 2. Generate `chip.config` -> 3. Generate `capabilities.h` -> 4. Generate `build.config` -> 5. Merge `.config` -> 6. Generate pinmux configuration

52.4.4 4. Source File Compilation

Ninja compiles all source files concurrently according to `build.ninja` rules.

52.4.5 5. Linking

Link object files into an ELF executable, outputting `.elf`, `.bin`, `.map`.

52.4.6 6. Firmware Generation

- Use `objcopy` to convert ELF to binary `.bin` file
- Use `objdump` to generate disassembly for debugging
- Run `tl_check_fw.sh` to verify firmware

52.5 Configuration Commands

52.5.1 Basic Configuration

```
west tl-config
```

This command configures the application in the current directory using default settings.

Process:

1. Sets up environment variables (TELINK_BASE)
2. Creates build directory if needed
3. Runs initial CMake configuration if required
4. Executes the unified `config` target which:
    - Generates `chip.config` from `Kconfig.chip`
    - Generates `capabilities.h` from chip configuration
    - Generates `build.config` from `Kconfig.build`
    - Merges configurations into final `.config`
    - Generates pinmux configuration files

52.5.2 Common Configuration Examples

```
# Specify build directory
west tl-config -d build_custom

# Using different source directory
west tl-config path/to/application -d build_custom

# Configure and build in one command
west tl-build -k
```

52.6 Basic Build Commands

```
# Simple build (current directory)
west tl-build
```

This command builds the application in the current directory. If it's the first build, it will automatically configure and build.

Pre-build Checks:

- Checks for existing `.config`, `build.config`, `chip.config`, `pinmux.h`, and `capabilities.h` files
- Invokes CMake to generate the build system
- Compiles source code using the selected toolchain
- Generates firmware binaries in the `build/` directory

52.6.1 Common Build Command Examples

```
# Specify build directory
west tl-build -d build_tl3218

# Specify source directory
west tl-build path/to/application

# Run CMake after configuration
west tl-build -c

# Clean build directory and rebuild
west tl-build --clean

# Pristine build
west tl-build --pristine=always
```

52.7 Configuration File Generation Logic

52.7.1 File Hierarchy

Configuration files are loaded in order of increasing priority:

1. **chip.config** –Chip default configuration (lowest priority)
2. **build.config** –User-defined build options
3. **.config** –Final merged configuration (highest priority)

52.7.2 Configuration Generation Flow

When executing `west tl-config`, the flow is:

1. **Parameter Processing Phase**: Parses source directory and build directory options
2. **CMake Configuration Phase**: Checks for existing `CMakeCache.txt`, runs CMake configuration with Ninja generator

3. **Configuration Target Execution**: Executes `cmake --build <build_dir> --target config`

When executing `west tl-build` (with `-k/--kconfig`), the flow is:

1. **Parameter Processing**: Checks for `--board` or `--soc` parameters
2. **Default Configuration Generation**: Runs `scripts/kconfig.py` to generate `chip.config` and `build.config`, merges into `.config`
3. **CMake Configuration**: Passes `BOARD` and `SOC` parameters to CMake

52.7.3 Dependencies

- `.config` depends on `chip.config` and `build.config` (when automatically generated)
- The CMake build system depends on `.config` to determine compilation options
- CMake monitors changes to `.config` and reconfigures the project when the file is modified

52.8 Build Results

Build Results

```
build/
├── Telink.bin          # Firmware binary (for flashing)
├── Telink.elf          # ELF executable (for debugging)
├── Telink.map          # Memory map file
└── Telink.lst          # Disassembly file
```

52.9 Building Without West

This section explains how to build without using the west tool.

52.9.1 CMake Configuration

```
cmake -B path/to/build -G Ninja -DTelink_DIR='path/to/telink-package' -S path/
↪to/source
```

Parameters:

- **`-B path/to/build`**: Specifies the build directory
- **`-G Ninja`**: Specifies Ninja as the generator
- **`-DTelink_DIR='path/to/telink-package'`**: Points to the TelinkConfig.cmake file (not needed if `TELINK_BASE` is set)
- **`-S path/to/source`**: Specifies the source code directory

52.9.2 CMake Build

```
# Build the project
cmake --build path/to/build

# Or use Ninja directly
ninja -C path/to/build

# Clean the build directory
cmake --build path/to/build --target clean
```

52.10 Troubleshooting

52.10.1 Environment Issues

Problem: TELINK_TOOLCHAIN_PATH not found

```
# Windows
set TELINK_TOOLCHAIN_PATH=path/to/toolchain           # CMD
$env:TELINK_TOOLCHAIN_PATH=path/to/toolchain           # PowerShell

# Linux/macOS
export TELINK_TOOLCHAIN_PATH=path/to/toolchain
```

Problem: CMake configuration fails

- Ensure all dependencies are installed
- Check CMake version compatibility
- Clear old build cache with `--clean`

52.10.2 Build Issues

Problem: Compilation errors

- Check application code for syntax errors
- Ensure correct board and chip configurations are used
- Try a clean build with `--pristine=always`

52.10.3 menuconfig Issues

Problem: Arrow icons not working in VSCode PowerShell

- Roll back to an earlier VSCode version or use the test ConPty library
- Use Windows 11
- Use letter keys instead of arrow icons
- Use Command Prompt instead of PowerShell

52.10.4 Environment Variable Issues

Problem: Environment variables not taking effect

- Restart the terminal, VS Code, or the system
- Manually check if PATH contains the correct path and add to user PATH

52.10.5 Other Issues

1. Ensure you’re using the latest version of UniSDK
2. Clear the build directory and rebuild:

```
west tl-build --clean
```

3. Check system logs for more detailed error information

CHAPTER

FIFTYTHREE

UNISDK BUILD SYSTEM ARCHITECTURE DOCUMENTATION

53.1 1. Overall Architecture Overview

The UniSDK build system adopts a modular, extensible architecture design that integrates multiple tools such as Make, CMake, Ninja, West, and Kconfig to form a complete compilation and build workflow. The system supports multiple compilation methods, including West commands, direct CMake calls, and Make commands, providing developers with flexible build options.

53.1.1 1.1 Core Components

- **CMake**: The main build system responsible for project configuration, dependency management, and build process definition
- **Ninja**: A high-performance build tool that executes actual compilation tasks as CMake's backend
- **West**: Zephyr project management tool extended in UniSDK to manage project and simplify build workflows
- **Kconfig**: Configuration management system for handling compile-time configuration options
- **Make**: Provides a traditional make command interface as a frontend to CMake

53.1.2 1.2 Directory Structure

Build system-related files are mainly distributed in the following directories:

SDK Directory Structure (Build System)

```
unisdk/
├── CMakeLists.txt         # Main CMake configuration file
├── Makefile               # Make command interface
├── west.yml               # West workspace configuration
├── Kconfig.chip           # Chip Kconfig entry
├── Kconfig.build          # Build Kconfig entry
├── cmake/                 # CMake modules
│   ├── modules/
│   └── TelinkConfig.cmake
├── scripts/               # Build scripts
```

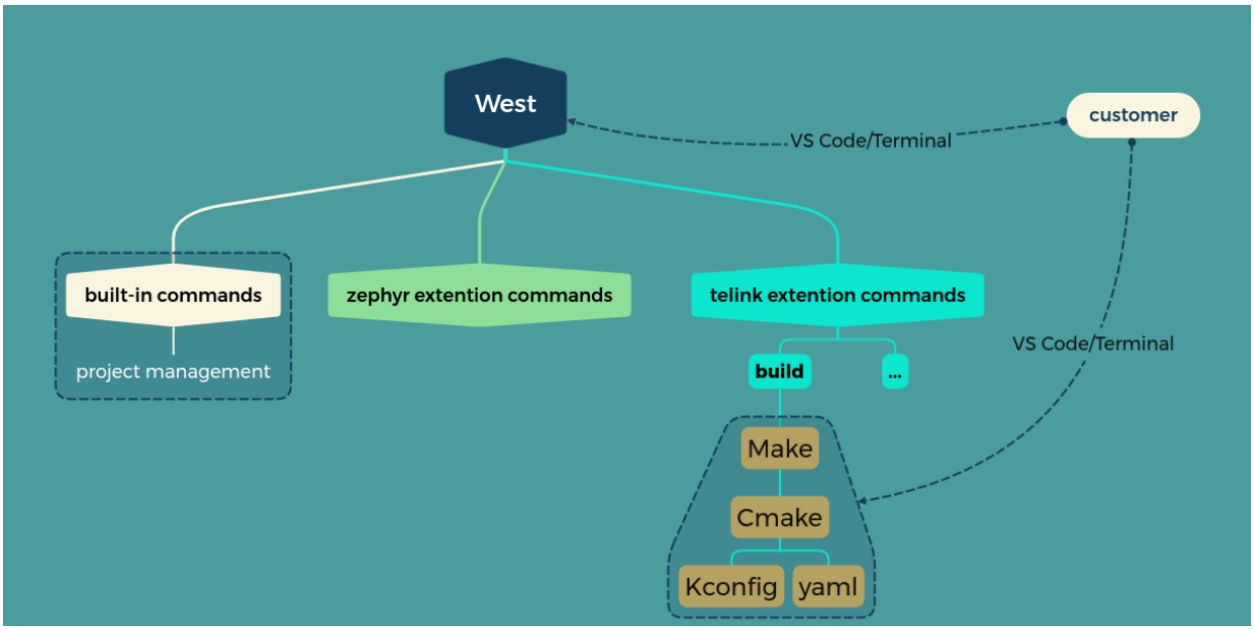
(continues on next page)

(continued from previous page)

```
│   ├── ci/
│   ├── pinmux/
│   ├── pre_build/
│   ├── west_commands/
│   ├── kconfig.py
│   ├── menuconfig.py
│   ├── preprocess_kconfig.py
│   ├── set_telink_base.sh
│   ├── tlk_check_fw.py
│   ├── west-commands.yml
│   └── windows_setup.ps1
├── core/
├── soc/
├── boards/
└── docs/
```

53.1.3 1.3 Build System

The UniSDK build system is a comprehensive framework that combines multiple tools to provide a seamless development experience. The following diagram illustrates the overall architecture and toolchain relationships:



The build system is designed around the following key components:

- **West**: The command-line interface for developers, providing a unified entry point for various build operations
- **Make**: Handles high-level build orchestration and pre-build checks
- **CMake**: Core build system responsible for project configuration, dependency management, and build rule generation
- **Kconfig**: Configuration management system for handling compile-time options

- **YAML**: Configuration format used for various system configurations, including pin multiplexing

This architecture enables a flexible and modular build process, allowing developers to easily configure, build, and manage their projects while maintaining consistency across different target platforms.

53.2 2. CMake Build System

53.2.1 2.1 CMake Core Process

The CMake build system is the core of the UniSDK compilation process, responsible for project configuration, dependency resolution, and build rule generation.

2.1.1 Initialization Phase

1. **Entry Point**: The build system supports **two entry point modes**:
   - **Root Directory Entry**: Uses the root directory's `CMakeLists.txt` as the entry point to build the entire SDK
   - **Application Entry**: Uses an application-level `CMakeLists.txt` (typically located in the `samples/` directory) as the entry point to build a specific application
2. **Version Requirements**: Sets the minimum CMake version (3.20.0)
3. **SDK Integration**: Locates and loads the Telink package configuration through `find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})`
4. **Project Setup**: Configures project-specific settings, including project name, supported languages, and target creation

Key Include Guards: The build system uses two important include guards to manage inclusion and prevent redundant processing:

1. **TELINK_ROOT_INCLUDED**:
   - Set to `TRUE` in the root directory's `CMakeLists.txt`
   - Used by `telink_default.cmake` to determine if the root directory has already been included
   - If not defined, `telink_default.cmake` will include the root directory's `CMakeLists.txt`
   - It will be removed in future versions when the application-level CMakeLists.txt is supported as the entry point.
2. **TELINK_PACKAGE_INCLUDED**:
   - Set to `TRUE` in `TelinkConfig.cmake` when the Telink package is loaded
   - Prevents multiple loading of the Telink package configuration
   - Checked in the root directory's `CMakeLists.txt` to avoid redundant package loading

Entry Point Modes Comparison:

Mode	Entry Point	Usage	Output
Root Directory Entry	`CMakeLists.txt` (root)	`cmake -B build -G Ninja .`	Builds the entire SDK as a library
Application Entry	`CMakeLists.txt` (samples/)	`cmake -B build -G Ninja samples/gpio_demo`	Builds a specific application executable

Application-Level Entry Point Implementation: All applications in the `samples/` directory must provide their own `CMakeLists.txt` file, which serves as the entry point for the build process when building a specific application. This file should:

1. Set the minimum CMake version
2. Define the project name and default language (C)
3. Add C sources using wildcards
4. Create an executable target
5. Conditionally add C++ support if CONFIG_TLK_ALLOW_CPP_WRAPPERS is enabled

The application-level CMakeLists.txt is automatically processed by the build system (via Make or West commands), which handles the integration with the core SDK components.

2.1.2 Module Loading Phase

`telink_default.cmake` loads all necessary CMake modules in a specific order:

```
list(APPEND telink_cmake_modules python)
list(APPEND telink_cmake_modules extensions)
list(APPEND telink_cmake_modules west)
list(APPEND telink_cmake_modules compiler)
list(APPEND telink_cmake_modules kconfig)
list(APPEND telink_cmake_modules pinmux)
list(APPEND telink_cmake_modules linker)
```

Each module is responsible for the configuration and implementation of specific functionality, together forming a complete build system.

53.2.2 2.2 Core CMake Modules

2.2.1 python.cmake

This module is from the Zephyr project.

The `python.cmake` module is responsible for configuring and validating the Python environment required by the UniSDK build system. As the first module loaded in the build process, it ensures that all Python-dependent components have a consistent and compatible Python environment.

Key Features and Functionality:

1. **UTF-8 Encoding Configuration**
   - Sets `PYTHONIOENCODING` to "utf-8" on Windows systems
   - Ensures consistent output encoding when Python scripts are executed by CMake
   - Prevents encoding issues that could occur when Python isn't connected to a terminal

2. **Python Version Management**
   - Defines a minimum required Python version (3.10)
   - Implements a robust Python executable discovery mechanism
   - Prioritizes using the same Python interpreter as West when available
3. **Python Discovery Process**
   - Checks for an existing `Python3_EXECUTABLE` variable first
   - Falls back to using `WEST_PYTHON` if available
   - Searches for "python" and "python3" executables in the system path and virtual environments
   - Verifies discovered Python versions meet the minimum requirements
   - Rejects Python 2.x installations explicitly
4. **Integration with CMake**
   - Uses `find_package(Python3)` to finalize Python configuration
   - Sets the legacy `PYTHON_EXECUTABLE` variable for compatibility with scripts expecting this variable
   - Uses `include_guard(GLOBAL)` to prevent multiple inclusions

This module ensures that all Python scripts used throughout the build process (such as Kconfig processing, pinmux generation, and build utilities) have access to a compatible Python interpreter, providing a consistent build environment across different platforms.

2.2.2 compiler.cmake

The `compiler.cmake` module is one of the core modules of the UniSDK build system, responsible for configuring the compiler toolchain, compiler options, and compilation flags. This module provides a complete RISC-V compiler configuration and management mechanism.

Key features include:

1. **Compiler Environment Detection and Configuration**
   - Automatically detects the TELINK_TOOLCHAIN_PATH environment variable setting
   - Supports cross-platform path handling
   - Verifies the existence and validity of the RISC-V GCC compiler
   - Provides clear error messages and configuration guidance when the compiler is not found
2. **Toolchain Component Setup**
   - Configures primary compilation tools: C/C++ compiler, assembler
   - Sets up auxiliary tools: objdump, objcopy, etc.
   - Uniformly configures RISC-V target architecture and processor type
   - Sets the system name to Generic (for bare-metal development)
3. **Compiler Options Application Function**
   - Provides the `telink_apply_compiler_options()` function to apply compilation options to targets

- Automatically applies preprocessor macro definitions (autoconf.h, capabilities.h, pinmux.h)
- Supports dynamically loading compilation options from JSON configuration files
- Applies specific compilation flags based on different languages (ASM, C, C++)

This module ensures compilation consistency across different platforms for the entire project through a unified compiler configuration mechanism, while providing a flexible option configuration method to support project customization requirements.

2.2.3 linker.cmake

The `linker.cmake` module is responsible for configuring the linking process in the UniSDK build system. It handles linker scripts, link options, and post-build processing commands. The following are its key functionalities:

2.2.3.1 Basic Linker Configuration

- Sets CMAKE_C_LINK_EXECUTABLE command template for RISC-V cross-compilation
- Configures verbose makefile output for better build visibility
- Disables RPATH to ensure no runtime path dependencies are embedded
- Removes default library suffix to match embedded system requirements

2.2.3.2 Linker Options Management

- Reads linker options from the `build_settings.json` configuration file
- Converts JSON-defined linker options into CMake-compatible list format
- Adds essential linker flags

The `build_settings.json` file is a central configuration file used by the UniSDK build system to store various build-related settings. It follows a JSON format and contains platform-specific configuration options, including:

```
{
    "linker_options": [
      "--gc-sections",
      "--print-memory-usage"
    ]
    "c_compile_options": [
        "-O2",
        "-Wall"
    ],
}
```

This file provides a flexible way to configure build settings across different platforms and projects, allowing for easy customization without modifying CMake scripts directly.

2.2.3.3 Linker Script Preprocessing

- Implements preprocessing of linker scripts using the C compiler
- Supports conditional compilation in linker scripts using capabilities.h and autoconf.h macros
- Creates a custom target `preprocess_linker` that ensures the linker script is always up-to-date
- Automatically processes the base flash_boot.link script and outputs the final linker script to the build directory

2.2.3.4 Post-Build Processing

- Provides the `telink_apply_linker_options(target)` function to apply linker settings to a specific target
- Configures objcopy to generate binary firmware files from ELF executables
- Sets up objdump to create assembly listings for debugging and analysis
- Adds size reporting to monitor memory usage of the compiled firmware
- Integrates firmware validation through the tlk_check_fw.py script

2.2.4 kconfig.cmake

Kconfig integration module is responsible for managing the entire configuration system within the CMake build process:

- **Configuration File Management**:
  - Defines key configuration file paths (`.config`, `chip.config`, `build.config`, `autoconf.h`, `capabilities.h`)
  - Sets up build directory properties to ensure proper cleanup of configuration files
  - Manages configuration file regeneration and dependencies
- **Configuration Generation Flow**:
  - Handles automatic generation of `chip.config` from `Kconfig.chip` when SOC and BOARD are not defined
  - Generates `capabilities.h` using preprocessing script based on chip configuration
  - Creates `build.config` from `Kconfig.build` with dependencies on capabilities
  - Merges `chip.config` and `build.config` into the final `.config` file
- **Preprocessor Integration**:
  - Integrates Python scripts (`menuconfig.py`, `preprocess_kconfig.py`, `kconfig.py`)
  - Parses `.config` file to generate `autoconf.h` with appropriate C defines
  - Handles different value types (boolean, string, numeric) in configuration
- **Custom Targets**:
  - **`parse_config`**: Ensures configuration parsing during build
  - **`config`**: Unified target executing the entire configuration flow in sequence
  - **`generate_chip_config`**: Generates `chip.config` using menuconfig or kconfig.py

  - **generate_capabilities**: Generates `capabilities.h` from chip configuration
  - **generate_build_config**: Generates `build.config` using menuconfig or kconfig.py
  - **update_dotconfig**: Merges `chip.config` and `build.config` into `.config`
- **Dependency Tracking**:
  - Sets up explicit target dependencies: `generate_chip_config` → `generate_capabilities` → `generate_build_config` → `update_dotconfig`
  - Creates cross-platform CMake commands for configuration file operations
  - Manages the configuration generation order with proper dependencies

2.2.4.1 SOC and BOARD Parameter Processing

The `kconfig.cmake` module includes comprehensive SOC and BOARD parameter handling:

Parameter Detection and Processing:

```
# Process SOC and BOARD parameters if provided
if(DEFINED SOC)
  message(STATUS "SOC parameter detected: ${SOC}")
  set(SOC_CONFIG_ARG "SOC=${SOC}")
else()
  set(SOC_CONFIG_ARG "")
endif()

if(DEFINED BOARD)
  message(STATUS "BOARD parameter detected: ${BOARD}")
  set(BOARD_CONFIG_ARG "BOARD=${BOARD}")
else()
  set(BOARD_CONFIG_ARG "")
endif()
```

SOC/BOARD Combination Validation:

- Validates SOC and BOARD combinations using dynamic Kconfig parsing
- Extracts SOC and BOARD names from command-line arguments
- Calls `kconfig_parser.py` with `--validate` option to verify compatibility
- Provides clear error messages for invalid combinations

Automatic Configuration Application:

- Applies SOC and BOARD configurations using `soc_board_setter.py` script
- Handles dependency resolution automatically (BOARD dependencies trigger SOC configuration)
- Supports both individual SOC/BOARD setting and combined setting
- Integrates with the existing configuration generation flow

Integration Points:

- **Command Line**: Parameters passed via `-DSOC=` and `-DBOARD=` CMake flags
- **Make Integration**: Makefile automatically forwards SOC and BOARD variables to CMake

- **West Integration**: West commands can specify SOC/BOARD through command-line options
- **Validation**: Real-time validation prevents incompatible hardware configurations

2.2.5 west.cmake

This module is from the Zephyr project.

The `west.cmake` module is responsible for integrating the West build system tool with the UniSDK build infrastructure. West is the official meta-tool for Zephyr-based projects, and this module ensures proper configuration and version compatibility within the UniSDK environment.

2.2.5.1 West Installation Detection

- Checks for West installation by attempting to import the west.version module
- Verifies installation status through the presence of WEST_PYTHON environment variable
- Handles both installed and non-installed West scenarios gracefully
- Provides clear error messages with installation instructions when West is required but not found

2.2.5.2 Python Environment Consistency

- Ensures the Python interpreter used by West matches the one used by CMake
- Resolves symbolic links to identify the actual Python executable paths
- Detects and reports Python interpreter mismatches between West and CMake
- Adds detailed diagnostic information when Python environments are out of sync

2.2.5.3 Version Compatibility Validation

- Enforces minimum required West version (0.14.0)
- Compares detected version against minimum requirements
- Provides upgrade instructions when version requirements are not met
- Maintains version consistency with requirements specified in scripts/requirements-base.txt

2.2.5.4 West Command Configuration

- Sets up the WEST variable as a command prefix for invoking West
- Configures the WEST variable to be cached internally for IDE integration
- Formats the command as `${PYTHON_EXECUTABLE} -m west` to ensure proper module execution
- Outputs status information about the detected West version

2.2.5.5 Workspace Detection

- Determines the West workspace top-level directory using `west topdir` command
- Configures the WEST_TOPDIR variable for use in other build scripts
- Handles non-West projects gracefully by setting WEST to WEST-NOTFOUND
- Maintains compatibility with both West-managed and custom project structures

2.2.6 pinmux.cmake

Pin multiplexing configuration module responsible for generating and managing pin mapping configuration files for the project:

Pinmux Configuration Files

- Defines the `PINMUX_H` variable for the generated pinmux header (${CMAKE_BINARY_DIR}/pinmux.h)
- Defines the `DOTPINMUX` variable for the intermediate configuration file (${CMAKE_BINARY_DIR}/.pinmux)

Custom Targets

- **`generate_dotpinmux`**: Generates `.pinmux` configuration file using `scripts.pinmux` Python module with optional `--skip-gui` parameter
- **`generate_pinmux_h`**: Converts `.pinmux` to `pinmux.h` using the `pinmux_gen.py` preprocessing script
- **`generate_pinmux`**: Main target that depends on `generate_pinmux_h` to ensure proper execution order

Build Target and Dependency Management

- `generate_dotpinmux` depends on `.config` file
- `generate_pinmux_h` depends on `.pinmux` file
- `generate_pinmux` depends only on `generate_pinmux_h` to avoid duplicate execution
- Uses `USES_TERMINAL` option for real-time output during generation

Build Integration

- Adds the build directory to the include path to ensure `pinmux.h` can be properly included in source files
- The main build target depends on `generate_pinmux` to ensure pin configuration is ready before compilation

Python Script Enhancements

- The `scripts.pinmux` module now supports a `--skip-gui` parameter to generate default configuration without launching the interactive GUI
- The `pinmux_gen.py` preprocessing script converts `.pinmux` to `pinmux.h` during the build process
- Both scripts depend on the Python environment provided by the `python.cmake` module

Clean Target Integration

- Adds `pinmux.h` and `.pinmux` to the list of files cleaned by the project's clean target

2.2.7 extensions.cmake

The `extensions.cmake` module provides essential CMake utility functions that extend the core CMake functionality to support the UniSDK build system with conditional logic, source file management, and data processing capabilities.

Key Functions and Features:

1. **Conditional Variable Management**
   - `set_ifndef(variable value [PARENT_SCOPE])`: Sets a variable to the specified value only if it is not already defined. This function helps establish default values while allowing them to be overridden. The optional `PARENT_SCOPE` parameter makes the variable available in the parent scope.
2. **Conditional Source File Management**
   - `add_sources_ifdef(config src)`: Adds source files to the target only when a specific configuration option is enabled. This enables feature-specific code inclusion based on build configurations.
   - `add_sources(src)`: A simplified wrapper for `target_sources()` that adds source files to the current target (`${TARGET_NAME}`) without conditions.
3. **JSON Array Processing**
   - `json_to_list(JSON_ARRAY_STRING OUTPUT_LIST [OUTPUT_STRING])`: Converts a JSON array string into a CMake list. This function performs type validation to ensure the input is a valid JSON array, processes each element, and handles space-separated values within elements. It can output both a CMake list and an optional space-joined string representation.
4. **Implementation Details**
   - The module uses `include_guard(GLOBAL)` to prevent multiple inclusions
   - Functions are designed with parent scope propagation where appropriate
   - Error handling is implemented for invalid JSON input

This module serves as a foundational component of the build system, providing reusable functionality that simplifies conditional builds, feature toggling, and data processing within CMake scripts.

53.2.3 2.3 CMake Configuration System Flow

The UniSDK CMake configuration system follows a structured flow that orchestrates the initialization, module loading, and integration of various build components. This section provides a detailed walkthrough of the configuration process from start to finish.

2.3.1 Initialization Phase

The configuration process begins when CMake is invoked on the main `CMakeLists.txt` file. This phase sets up the fundamental environment for the build system by integrating with the UniSDK infrastructure.

2.3.1.1 CMakeLists.txt Entry Point

The UniSDK build system supports two distinct entry point modes, each serving different purposes:

1. Root Directory Entry Point

The root directory's `CMakeLists.txt` file is primarily used for building the core SDK functionality or testing SDK components. It provides the foundation for the build system but **does not automatically include application code from the `samples` directory**. Key characteristics:

1. **Build System Initialization**:
   - Sets up essential variables like `TELINK_BASE`, `AUTOCONF_H`, `CAPABILITIES_H`, and `PINMUX_H`
   - Adds `cmake/modules` to the CMake module path for custom module discovery
   - Sets `TELINK_ROOT_INCLUDED` to `TRUE` to prevent redundant inclusion
   - Loads the Telink package configuration if `TELINK_PACKAGE_INCLUDED` is not already defined
2. **Project Configuration**:
   - Defines the main project with the name "Telink"
   - Configures supported languages (ASM, C, CXX)
   - Creates the main executable target with core startup files
3. **Component Integration**:
   - Adds key SDK subdirectories (api, common, core, soc) to the build
   - **Does NOT include the `samples` directory**
   - Sets up dependencies between targets (parse_config, generate_pinmux, preprocess_linker)
   - Applies compiler and linker options to the main target
4. **Limitations**:
   - Lacks a `main` function - will produce a linker error `undefined reference to 'main'`
   - Cannot compile complete applications from the `samples` directory
   - Intended for SDK development/testing rather than application building

2. Application Entry Point

The recommended way to build applications is to use application-level `CMakeLists.txt` files. Applications **can be located anywhere** on the filesystem, not just in the `samples/` directory. The build system uses the `TELINK_BASE` environment variable or path to locate the Telink SDK package. A typical application-level CMakeLists.txt includes:

```
# SPDX-License-Identifier: Apache-2.0

cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(GPIO_Demo LANGUAGES C CXX)

# Add C source files
```

(continues on next page)

(continued from previous page)

```
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

# Add C++ files only if CONFIG_TLK_ALLOW_CPP_WRAPPERS is enabled
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

Key Differences Between Entry Points:

Feature	Root Directory Entry Point	Application Entry Point
Primary Use	SDK development/testing	Building complete applications
Application Location	N/A	Anywhere on filesystem (not just `samples/` directory). When using west, must be in the same directory as `.west` file or its subdirectories
Contains `main` function	❌ No	✅ Yes (in application code)
Build Command	`cmake -B build -G Ninja .`	`cmake -B build -G Ninja /path/to/application`
Output	Core SDK components (will fail without `main`)	Complete executable firmware

Correct Usage Examples:

```
# To build an application (recommended)
cmake -B build -G Ninja samples/gpio_demo
cmake --build build

# To build SDK core (for SDK development/testing)
cmake -B build -G Ninja .
cmake --build build  # Will fail without manual main function addition
```

Current Implementation Status and Limitations: While the application-level CMakeLists.txt is designed as the entry point, there are currently some limitations in the implementation due to tight coupling between demo names, application layers, and the overall SDK Kconfig system:

1. **Kconfig System Coupling**: The demo applications' configuration options are deeply integrated with the SDK's global Kconfig system, making it challenging to completely isolate application-specific configurations.
2. **Demo Name Dependencies**: Some build system components rely on specific demo naming conventions, which creates dependencies between application names and internal SDK build logic.
3. **Application Layer Integration**: The application layer's interaction with core SDK components is not yet fully modularized, requiring careful management of include paths

and configuration settings.

These limitations are being addressed in ongoing development efforts to create a more modular and flexible build system that fully realizes the application-level entry point design.

2.3.1.2 Package Loading: find_package(Telink)

If `TELINK_PACKAGE_INCLUDED` is not defined, the system loads the Telink package:

```
if(NOT DEFINED TELINK_PACKAGE_INCLUDED)
    # Load Telink default modules
    find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
endif()
```

This triggers the loading of `TelinkConfig.cmake` from the `cmake` directory.

2.3.1.3 TelinkConfig.cmake Processing

The `TelinkConfig.cmake` file performs the following tasks:

1. Sets `TELINK_PACKAGE_INCLUDED` to `TRUE` to prevent reloading
2. Defines the `include_boilerplate()` macro which:
   - Sets `TELINK_BASE` if not already set (using environment variable or relative path)
   - Configures `Telink_DIR` cache variable
   - Adds the CMake modules directory to the module path
   - Sets application source and binary directories if not defined
   - Includes `telink_default.cmake`
3. Calls `include_boilerplate()` to execute the boilerplate code

2.3.2 Module Loading Phase

After the initial environment is set up, the configuration system loads the core CMake modules that implement the build system functionality.

2.3.2.1 telink_default.cmake Processing

The `telink_default.cmake` file is responsible for loading all necessary CMake modules in a specific order:

1. Sets minimum CMake version again (for validation)
2. Outputs application directory and CMake version information
3. Performs a check for a known CMake bug in versions 3.22.1/3.22.2
4. Defines the list of required modules in order:
   - `python` - Python interpreter detection and configuration
   - `extensions` - Custom CMake functions and macros
   - `version` - Version information processing
   - `west` - West tool integration

- `compiler` - Compiler configuration
- `generated_file_directories` - Directories for generated files
- `kconfig` - Kconfig integration (critical for configuration)
- `pinmux` - Pin multiplexing configuration
- `linker` - Linker script processing

5. Validates any requested sub-components against the available modules
6. Includes each module in sequence
7. Removes loaded modules from the `SUB_COMPONENTS` list
8. If `TELINK_ROOT_INCLUDED` is not defined, it adds the root directory as a subdirectory

```
# Load core CMake modules in sequence
include(python)
include(extensions)
include(version)
include(west)
include(compiler)
include(generated_file_directories)
include(kconfig)
include(pinmux)
include(linker)
```

This sequential loading ensures that each module has access to the necessary infrastructure provided by previously loaded modules. The order is carefully maintained to manage dependencies between modules.

2.3.3 Kconfig Integration Phase

The Kconfig system integration is a critical phase that manages the configuration options for the build system and target firmware.

2.3.3.1 kconfig.cmake Processing

The `kconfig.cmake` module is responsible for configuration file generation and processing:

1. Defines key file paths:
   - `CHIP_CONFIG_FILE` - Chip-specific configuration
   - `BUILD_CONFIG_FILE` - Build-specific configuration
   - `DOTCONFIG` - Merged configuration file (.config)
   - `CAPABILITIES_H` - Generated capabilities header
   - `AUTOCONF_H` - Auto-generated configuration header
   - `MENUCONFIG_SCRIPT` - Path to the interactive menuconfig script
   - `PARSE_SCRIPT` - Path to the configuration parsing script
   - `PREPROCESS_SCRIPT` - Path to the Kconfig preprocessing script
   - `KCONFIG_BINARY_DIR` - Directory for Kconfig source includes
2. Configures clean targets to remove generated files:

- Sets ADDITIONAL_CLEAN_FILES properties for all generated configuration files including:
  - CHIP_CONFIG_FILE
  - BUILD_CONFIG_FILE
  - DOTCONFIG
  - AUTOCONF_H
  - CAPABILITIES_H
  - PINMUX_H
  - KCONFIG_BINARY_DIR

3. Chip Configuration Generation:
   - Creates a custom command to generate chip.config using menuconfig.py and Kconfig.chip
   - **When chip.config is missing**: The custom command triggers menuconfig.py with the Kconfig.chip file, which will:
     - Automatically launch the interactive menuconfig interface if no existing config exists
     - Allow the user to select SoC and related configuration options
     - Save the selections to chip.config after the user exits the menu
4. Capabilities Header Generation:
   - Creates a custom command to generate capabilities.h using preprocess_kconfig.py
   - Depends on chip.config
   - **When capabilities.h is missing**: The custom command will automatically execute when its dependencies (chip.config) are available, generating the header based on the selected chip's capabilities
5. Build Configuration Generation:
   - Creates a custom command to generate build.config using menuconfig.py and Kconfig.build
   - Depends on capabilities.h
   - **When build.config is missing**: The custom command triggers menuconfig.py with the Kconfig.build file, which will:
     - Automatically launch the interactive menuconfig interface if no existing config exists
     - Allow the user to select build-time options and features
     - Take into account the capabilities defined in capabilities.h to filter available options
     - Save the selections to build.config after the user exits the menu
6. .config File Creation (During Configuration Phase):
   - If .config doesn't exist during the CMake configuration phase, the system performs the following steps:

1. Checks if `chip.config` exists; if not, executes `kconfig.py` to generate it non-interactively
2. Executes `preprocess_kconfig.py` to generate `capabilities.h`
3. Checks if `build.config` exists; if not, executes `kconfig.py` to generate it non-interactively
4. Creates a new `.config` file with an auto-generated header comment
5. Merges content from `chip.config` and `build.config` into the new `.config` file

7. Configuration Dependency Tracking:
   - Sets `CMAKE_CONFIGURE_DEPENDS` to `.config` to trigger reconfiguration when it changes
8. Merge Script Creation:
   - Writes a CMake script (`merge_config_files.cmake`) to merge configuration files
   - This script reads content from `build.config` and `chip.config` and appends it to `.config`
9. .config Update During Build:
   - Creates a custom command to update `.config` when dependencies (`build.config` or `chip.config`) change
   - Removes the old `.config` file, touches a new one, and then runs the merge script
10. Parse Configuration Target:
    - Creates a custom target `parse_config` that depends on `.config`
    - This target ensures that the configuration parsing happens before the main build
11. Configuration Parsing (During Build):
    - Reads lines from `.config`
    - Generates `autoconf.h` with `#define` statements for each configuration option
    - Handles different value types (boolean, string, number)
    - **When `autoconf.h` is missing**: The file is automatically generated during the build process based on the contents of `.config`

2.3.3.2 Menuconfig Triggering Mechanism

The menuconfig interface can be triggered in several ways:

1. **Automatic Trigger During Build (Interactive)**: When a configuration file (`chip.config` or `build.config`) is missing and a build is initiated, the system will automatically launch the interactive menuconfig interface through `menuconfig.py`
2. **Manual Execution**: Users can manually invoke menuconfig for specific configuration types:
   - For chip configuration: `python scripts/menuconfig.py Kconfig.chip`
   - For build configuration: `python scripts/menuconfig.py Kconfig.build`
3. **Non-Interactive Generation**: During the CMake configuration phase, if `.config` is missing, the system uses `kconfig.py` to non-interactively generate `chip.config` and `build.config` with default values

4. **Custom Command Execution**: The custom commands defined in `kconfig.cmake` will trigger menuconfig when their respective output files (e.g., `chip.config`, `build.config`) are required but missing

2.3.4 Pinmux Configuration Generation

The `pinmux.cmake` module is responsible for generating the `pinmux.h` header file that contains pin multiplexing configuration:

1. Sets the path for the generated `pinmux.h` file to `${CMAKE_BINARY_DIR}/pinmux.h`
2. Creates a custom command to generate `pinmux.h`:
   - Uses Python script `scripts.pinmux` module
   - Takes `CMAKE_BINARY_DIR` as an argument
   - Sets working directory to `TELINK_BASE`
   - Adds a descriptive comment for build output
   - Uses terminal for command execution
   - Depends on `.config` file
3. Creates a custom target `generate_pinmux` that:
   - Is always built (ALL)
   - Depends on both `pinmux.h` and `.config`
4. The binary directory is added to the include path so that `pinmux.h` can be included in source files

```
# Example of pinmux configuration generation
add_custom_command(
  OUTPUT "${PINMUX_H}"
  COMMAND ${PYTHON_EXECUTABLE} -m scripts.pinmux "${CMAKE_BINARY_DIR}"
  WORKING_DIRECTORY "${TELINK_BASE}"
  COMMENT "Generating pinmux.h..."
  USES_TERMINAL
  DEPENDS "${DOTCONFIG}"
)

add_custom_target(generate_pinmux ALL
  DEPENDS "${PINMUX_H}" "${DOTCONFIG}"
)

# Make main target depend on pinmux configuration
add_dependencies(${TARGET_NAME} generate_pinmux)
```

2.3.5 Build Configuration Phase

After all modules are loaded and the Kconfig system is integrated, the configuration system proceeds to set up the build environment for the specific target.

2.3.5.1 Target Configuration

The configuration system configures the main target with the appropriate settings:

```
# Define the main target
add_executable(${TARGET_NAME} ${SOURCES})

# Configure target properties
target_include_directories(${TARGET_NAME} PUBLIC
  "${CMAKE_CURRENT_BINARY_DIR}"  # For autoconf.h
  "${PROJECT_INCLUDE_DIRS}"
  "${SDK_INCLUDE_DIRS}"
)

# Set target linker script
target_link_options(${TARGET_NAME} PRIVATE
  -T${LINKER_SCRIPT_PATH}
  --gc-sections
)

# Add libraries to link
target_link_libraries(${TARGET_NAME} PRIVATE
  ${SDK_LIBS}
  ${PROJECT_LIBS}
)
```

2.3.5.2 Build Directory Structure

The configuration system sets up a structured build directory to organize build outputs:

Build Directory Structure

```
build/
├── .config              # Current configuration
├── ${APP_NAME}          # Final executable output
├── autoconf.h           # Generated C header
├── build.config         # Build configuration
├── build.config.old     # Previous build configuration
├── build.ninja          # Ninja build file
├── capabilities.h       # SoC capabilities header
├── chip.config          # Chip-specific configuration
├── chip.config.old      # Previous chip configuration
├── CMakeCache.txt       # CMake cache
├── CMakeFiles/          # CMake build files
├── cmake_install.cmake  # CMake install script
├── flash_boot.ld        # Linker script
├── Kconfig/             # Chip-specific Kconfig files
│   ├── Kconfig.dma
│   ├── Kconfig.gpio
│   ├── Kconfig.pm
│   ├── Kconfig.rf
```

(continues on next page)

(continued from previous page)

```
│   └── ...
├── merge_config_files.cmake
├── pinmux.h              # Generated pinmux configuration
└── telink/               # SDK build files
    ├── api/
    ├── common/
    ├── core/
    ├── samples/
    └── soc/
```

2.3.6 Complete Configuration Flow Diagram

The following diagram illustrates the complete configuration flow from start to finish:

2.3.7 Configuration System Integration

2.3.7.1 Python Integration

The configuration system heavily relies on Python scripts for Kconfig operations:

- `menuconfig.py` - Interactive menuconfig interface
- `kconfig.py` - Non-interactive Kconfig processing
- `preprocess_kconfig.py` - Preprocessing Kconfig files
- `kconfig_parser.py` - Kconfig parsing and validation utilities
- `soc_board_setter.py` - SOC/BOARD configuration and validation

The `python.cmake` module ensures that a compatible Python interpreter is available.

2.3.8 CMake Config Target Execution Flow

The `config` target executes a sequential flow of configuration tasks:

1. **Chip Configuration**: `generate_chip_config` generates `chip.config` using Kconfig
2. **Capabilities Generation**: `generate_capabilities` creates `capabilities.h` based on the selected chip
3. **Build Configuration**: `generate_build_config` generates `build.config` using Kconfig
4. **Configuration Merge**: `update_dotconfig` merges `chip.config` and `build.config` into `.config`
5. **Pinmux Configuration**: `generate_pinmux` handles pin multiplexing configuration:
   - First `generate_dotpinmux` creates `.pinmux` from Kconfig using `scripts.pinmux` with `--skip-gui`
   - Then `generate_pinmux_h` converts `.pinmux` to `pinmux.h` using `pinmux_gen.py`

This target ensures that all configuration files are generated in the correct order with proper dependencies between tasks.

2.3.9 Key Configuration Files and Their Purpose

- **Kconfig.chip**: Defines available SoC options and their configurations
- **Kconfig.build**: Defines build-time options and features
- **chip.config**: Generated SoC-specific configuration
- **build.config**: Generated build-specific configuration
- **.config**: Merged configuration file used for the build
- **capabilities.h**: Generated header with SoC capabilities based on selected chip
- **autoconf.h**: Generated header with all configuration options as C defines
- **pinmux.h**: Generated header with pin multiplexing configuration

2.3.10 Troubleshooting Configuration Issues

Common issues and their solutions:

- **Configuration not applying**: Ensure the `parse_config` target is properly built before the main target
- **Missing options in menuconfig**: Check if the Kconfig files are properly included and dependencies are met
- **Python script errors**: Verify that Python 3.10+ is installed and all dependencies are available
- **CMake module not found**: Ensure `TELINK_BASE` is correctly set and points to the SDK root
- **Generated files not cleaned**: The system is configured to clean generated files with `make clean`, but you can also manually remove the build directory

53.3 3. Kconfig Configuration System

53.3.1 3.1 Kconfig Architecture

The Kconfig system adopts a hierarchical structure, organizing configuration files through the `rsource` directive:

3.1.1 Configuration File Hierarchy

- **Top-level Configuration**: `Kconfig.chip` and `Kconfig.build` as configuration entry points
- **Feature Configuration**: `Kconfig` files in subdirectories define specific feature configurations
- **Configuration Tools**: `kconfig.py`, `menuconfig.py`, and `preprocess_kconfig.py` provide configuration processing

3.1.2 Kconfig File Organization

The Kconfig files form a hierarchical structure where higher-level Kconfig files include lower-level ones using the `rsource` directive. The organization is as follows:

3.1.3 Application Layer Kconfig Integration

In addition to the SDK's built-in Kconfig files, the application layer can also provide its own Kconfig configuration. This is achieved through the `KCONFIG_APPLICATION_DIR` variable, which is defined in `kconfig.cmake`:

```
set(KCONFIG_APPLICATION_DIR ${CMAKE_CURRENT_SOURCE_DIR})
```

This variable is set to the current source directory where CMake is invoked. When running Kconfig commands (such as `menuconfig`), the system automatically looks for Kconfig files in this directory, allowing application-specific configurations to be seamlessly integrated with the SDK's built-in configurations.

The application layer Kconfig integration process:

1. `kconfig.cmake` sets `KCONFIG_APPLICATION_DIR` to the current CMake source directory
2. This variable is passed to the Python scripts (menuconfig.py, kconfig.py, etc.) as an environment variable
3. The scripts use this variable to find and include application-specific Kconfig files
4. Application configurations are merged with SDK configurations during the configuration generation process

This design allows applications to define their own configuration options that are specific to their functionality, while still leveraging the SDK's core configuration system.

53.3.2 3.2 Configuration Processing Flow

1. **Configuration Generation**: Merging configuration files through `kconfig.py`
2. **Interactive Configuration**: Using `menuconfig.py` to provide a menu interface
3. **Configuration Preprocessing**: `preprocess_kconfig.py` processes configuration macros and variables
4. **Configuration Application**: The CMake system uses generated configuration files for build configuration

53.3.3 3.3 Key Configuration Scripts

3.3.1 kconfig.py

Main functions:

- Merges multiple configuration files
- Validates configuration validity
- Generates the final `.config` file
- Executes configuration checks and warnings

3.3.2 menuconfig.py

Interactive configuration interface implementation:

- Curses-based text interface
- Supports menu navigation and option setting
- Provides help information display
- Supports configuration saving and loading

3.3.3 preprocess_kconfig.py

Configuration preprocessing tool:

- Processes YAML configuration variables
- Supports configuration macro expansion
- Handles nested configuration structures
- Supports arrays and special format values

53.4 4. West Compilation Support

53.4.1 4.1 West Command Extensions

UniSDK extends the West command set, registering custom commands through `west-commands.yml`:

```
west-commands:
  - file: scripts/west_commands/build.py
    commands:
      - name: tl-build
        class: Build
        help: compile a Telink application
  - file: scripts/west_commands/boards.py
    commands:
      - name: tl-boards
        class: Boards
        help: display information about supported Telink boards
  - file: scripts/west_commands/socs.py
    commands:
      - name: tl-socs
        class: Socs
        help: display information about supported Telink socs
  - file: scripts/west_commands/config.py
    commands:
      - name: tl-config
        class: Config
        help: configure a Telink application
```

53.4.2 4.2 west tl-build Command Details

`build.py` implements the main build functionality:

4.2.1 Command Line Options

Positional Arguments:

- `source_dir`: Application source directory (default: current directory)

Optional Arguments:

- `-d, --build-dir`: Specify build directory
- `-c, --cmake-only`: Run only CMake configuration, don't build
- `-k, --kconfig`: Run interactive configuration before building
- `-t, --target`: Build target (e.g., flash, debug, etc.)
- `-p, --pristine`: Control pristine build directory usage (choices: auto, always, never)
- `--clean`: Clean build directory before building
- `--flash`: Flash firmware after building
- `-o, --build-option`: Extra options to pass to the build system (can be used multiple times)
- `--soc SOC`: Specify SOC for configuration
- `--board BOARD`: Specify BOARD for configuration
- `cmake_opt`: Extra options to pass to CMake (must come after "–" separator)

Usage Examples:

```
# Basic build
west tl-build

# Build with specific directory
west tl-build samples/gpio_demo

# Build with BOARD
west tl-build --board TLSR9528A_EVK

# Configuration only
west tl-build --cmake-only

# Interactive configuration followed by build
west tl-build --kconfig

# Clean build
west tl-build --clean

# Pristine build
west tl-build --pristine always

# Pass extra CMake options
west tl-build -- -DCONFIG_DEBUG=y
```

4.2.2 Execution Flow

1. **Environment Check**: Verify TELINK_TOOLCHAIN_PATH environment variable is set
2. **SOC/BOARD Validation**: Validate SOC and BOARD combination if both are provided
3. **Path Setting**: Configure source directory and build directory
4. **TELINK_BASE Setup**: Set TELINK_BASE environment variable to SDK root directory
5. **Cleanup Handling**: Clean build directory if –clean or –pristine options are used
6. **Build Directory Creation**: Create build directory if it doesn't exist
7. **Interactive Configuration**: Run menuconfig if –kconfig option is specified
8. **CMake Configuration**: Call CMake with SOC/BOARD parameters and generate build system
9. **Project Building**: Execute compilation and linking (skipped if –cmake-only)
10. **Result Output**: Display build status and results

Key Features:

- **SOC/BOARD Support**: Directly specify SOC and BOARD via command-line options
- **Dependency Resolution**: Automatic SOC/BOARD dependency handling during CMake configuration
- **Validation**: Real-time validation of SOC/BOARD combinations before build
- **Flexible Configuration**: Support for both interactive and non-interactive configuration
- **Pristine Builds**: Options for clean and pristine build environments

53.4.3 4.3 west tl-config Command Details

`config.py` implements the Telink application configuration functionality, providing a unified configuration interface:

4.3.1 Command Line Options

Positional Arguments:

- `source_dir`: Application source directory (optional, defaults to current directory)

Optional Arguments:

- `-d, --build-dir`: Specify the build directory (optional, defaults to `source_dir/build`)
- `--soc SOC`: Specify SOC for configuration
- `--board BOARD`: Specify BOARD for configuration
- `--list-socs`: List all available SOCs with their dependencies
- `--list-boards`: List all available boards with their dependencies
- `--validate SOC BOARD`: Validate SOC and BOARD combination

Usage Examples:

```
# Interactive configuration for current directory
west tl-config

# Configure specific project
west tl-config samples/gpio_demo

# Configure with specific build directory
west tl-config -d build_debug

# List available SOCs
west tl-config --list-socs

# List available boards
west tl-config --list-boards

# Validate SOC/BOARD combination
west tl-config --validate TLSR9528A TLSR9528A_EVK
```

4.3.2 Configuration Process

The `tl-config` command supports multiple operation modes:

List Operations:

- `--list-socs`: Display all available SOCs with their dependencies
- `--list-boards`: Display all available boards with their dependencies
- `--validate`: Validate SOC/BOARD combination without configuring

Configuration Flow:

1. **SOC/BOARD Validation**:
   - Validate SOC and BOARD combination if both are provided
   - Use KconfigParser to check dependency compatibility
   - Provide clear error messages for invalid combinations
2. **Environment Setup**:
   - Set `TELINK_BASE` environment variable to SDK root directory
   - Resolve source and build directory paths
   - Create build directory if it doesn't exist
3. **CMake Configuration** (if needed):
   - Check for existing `CMakeCache.txt` to determine if configuration is needed
   - Run initial CMake configuration with Ninja generator and Telink package path
   - Apply SOC and BOARD settings if specified via command line
4. **Configuration Target Execution**:
   - Execute `cmake --build <build_dir> --target config`
   - This triggers the complete configuration sequence:
     - Clean existing configuration files

  - Generate `chip.config` from `Kconfig.chip`
  - Generate `capabilities.h` from chip configuration
  - Generate `build.config` from `Kconfig.build`
  - Merge configurations into final `.config`
  - Generate pinmux configuration files
5. **Result Handling**:
   - Return appropriate exit codes (0 for success, non-zero for errors)
   - Provide informative status messages throughout the process

Key Features:

- **SOC/BOARD Discovery**: List available SOCs and boards with dependency information
- **Validation**: Pre-build validation of SOC/BOARD combinations
- **Dependency Resolution**: Automatic handling of SOC/BOARD dependencies
- **Interactive Configuration**: Menuconfig-based configuration interface

4.3.3 Code Reuse Integration

The `tl-config` command shares its core configuration logic with the `tl-build` command through the `run_config()` function. This function is extracted into a reusable module that:

- Provides consistent configuration behavior across both commands
- Handles all configuration file generation and CMake target execution
- Ensures proper error handling and status reporting
- Maintains compatibility with existing build workflows

The `tl-build` command's `-k/--kconfig` option internally calls the same configuration logic, ensuring consistency between standalone configuration and build-time configuration.

53.4.4 4.4 west tl-boards Command Details

`boards.py` implements the board information display functionality:

4.4.1 Command Line Options

- `-v, --verbose`: Show detailed information about each board
- `--soc SOC`: Filter boards by supported SOC (e.g., TL3218X)

4.4.2 Command Features

1. **Board Enumeration**: Lists all supported development boards from Kconfig.chip
2. **SOC Filtering**: Allows filtering boards by specific SOC compatibility
3. **Dependency Display**: Shows board dependencies and requirements
4. **Verbose Information**: Provides detailed board specifications including:
   - Full board name and prompt
   - Type and dependencies

- Supported SOCs for each board
- Help documentation

4.4.3 Usage Examples

```
# List all supported boards
west tl-boards

# Show detailed board information
west tl-boards -v

# Filter boards for specific SOC
west tl-boards --soc TLSR9528A

# Detailed information for specific SOC boards
west tl-boards -v --soc TLSR9528A
```

53.4.5 4.5 west tl-socs Command Details

`socs.py` implements the SOC information display functionality:

4.5.1 Command Line Options

- `-v, --verbose`: Show detailed information about each SOC

4.5.2 Command Features

1. **SOC Enumeration**: Lists all supported System-on-Chip variants from Kconfig.chip
2. **Dependency Display**: Shows SOC dependencies and requirements
3. **Verbose Information**: Provides detailed SOC specifications including:
   - Full SOC name and prompt
   - Type and dependencies
   - Help documentation

4.5.3 Usage Examples

```
# List all supported SOCs
west tl-socs

# Show detailed SOC information
west tl-socs -v
```

53.4.6 4.6 BDT Command Extension

The `west tl-bdt` command is a West extension that provides an interface to the Telink Burning and Debugging Tool (BDT) for performing flash operations on Telink devices. This command simplifies the process of interacting with BDT by providing a unified, cross-platform interface.

4.6.1 Command Syntax

```
west tl-bdt [-h] [-d BUILD_DIR] [-c] [--usb] [--bdt-path BDT_PATH] [--chip␣
↪CHIP]
           [--sws EVK_CLK EVK_RATE TARGET_CLK TARGET_RATE] [--bus BUS] [--
↪devid DEVID]
           {download,read,write,erase,lock,unlock,reset} ...
```

4.6.2 Supported Operations

Operation	Description
download	Download firmware to flash
read	Read data from flash
write	Write data to flash
erase	Erase flash
lock	Lock flash region
unlock	Unlock flash region
reset	Reset device

4.6.3 BDT Tool Path Configuration

4.6.3.1 Setting BDT Path

The BDT tool path can be set through the following methods (in order of priority):

1. **Command line option**: `--bdt-path /path/to/bdt_executable`
2. **Environment variable**: `BDT_PATH=/path/to/bdt_executable`
3. **System PATH**: The command will search for BDT in the system PATH
4. **Telink base directory**: If `TELINK_BASE` is set, it will look in `${TELINK_BASE}/tools/`

4.6.3.2 Important Notes

- **Parameter Order**: The `--bdt-path` parameter can be specified either before or after the subcommand (e.g., download, read, write) starting from version 5.9.0.

```
# Both are now acceptable
west tl-bdt --bdt-path C:\path\to\bdt --chip B92 download -i firmware.bin
west tl-bdt download --chip B92 -i firmware.bin --bdt-path C:\path\to\bdt
```

- **Correct Path Format**: The `--bdt-path` parameter can accept either:
  1. **Path to executable file**:
     - **Windows**: `Cmd_download_tool.exe`
     - **Linux**: `bdt`
  2. **Path to directory containing the executable**:
     - The tool will automatically search for the appropriate executable in the directory

  Example:

```
# Path to executable file
west tl-bdt --bdt-path C:\BDT\Cmd_download_tool.exe

# Path to directory containing executable
west tl-bdt --bdt-path C:\BDT
```

- **Path Format**: On Windows, you can use either forward slashes or backslashes in paths:

```
# Both are acceptable
west tl-bdt --bdt-path C:\BDT\Cmd_download_tool.exe
west tl-bdt --bdt-path C:/BDT/Cmd_download_tool.exe
```

4.6.4 Global Parameters

Parameter	Description	Default
`-h, --help`	Show help message and exit	-
`-d, --build-dir`	Build directory	`build` in current directory
`-c, --core`	Operate on core instead of flash	False
`--usb`	Use USB mode (default: EVK mode)	False
`--bdt-path`	Path to BDT tool executable	-
`--chip`	Chip type (e.g., B92, TL721X, TL321X)	Detected from build config
`--sws EVK_CLK EVK_RATE TARGET_CLK TARGET_RATE`	Set SWire clock values	-
`--bus`	USB bus ID (required when multiple USB devices exist)	-
`--devid`	USB device ID (required when multiple USB devices exist)	-

4.6.5 Subcommands

4.6.5.1 download

Download firmware to flash.

```
west tl-bdt download [-h] [-i INPUT] [-a ADDR]
```

Parameter	Description	Default
`-i, --input`	Path to firmware file	`build/${TLK_ARTIFACT_NAME}.bin`
`-a, --addr`	Flash address to download to	`0x00`

4.6.5.2 read

Read data from flash.

```
west tl-bdt read [-h] [-a ADDR] -s SIZE [-o OUTPUT]
```

Parameter	Description	Default
-a, --addr	Flash address to read from	0x00
-s, --size	Number of bytes to read (e.g., 16, 1k)	Required
-o, --output	Output file path	save<timestamp>.bin

4.6.5.3 write

Write data to flash.

```
west tl-bdt write [-h] [-a ADDR] [-s SIZE] [-i INPUT] [-e] [data ...]
```

Parameter	Description	Default
-a, --addr	Flash address to write to	0x00
-s, --size	Number of bytes to write (required if writing raw data)	-
-i, --input	Path to input file (required if not writing raw data)	-
data	Raw data bytes to write (e.g., 01 02 03 04)	-
-e, --erase	Erase before writing	False

4.6.5.4 erase

Erase flash.

```
west tl-bdt erase [-h] [-a ADDR] -s SIZE
```

Parameter	Description	Default
-a, --addr	Flash address to erase from	0x00
-s, --size	Number of bytes to erase (e.g., 1k, 4k)	Required

4.6.5.5 lock

Lock flash region.

```
west tl-bdt lock [-h] [-a ADDR] [-s SIZE]
```

Parameter	Description	Default
-a, --addr	Flash address to lock	0x00
-s, --size	Size of region to lock (e.g., 512k)	Required

4.6.5.6 unlock

Unlock flash region.

```
west tl-bdt unlock [-h]
```

4.6.5.7 reset

Reset device.

```
west tl-bdt reset [-h] [-c]
```

Parameter	Description	Default
-c, --core	Reset from core instead of flash	False

4.6.6 Examples

4.6.6.1 Download Firmware

```
# Basic download using default chip and firmware path
west tl-bdt download

# Download with specific chip and firmware path
west tl-bdt download --chip B92 -i build/${TLK_ARTIFACT_NAME}.bin

# Download to specific address
west tl-bdt download --chip B92 -i build/${TLK_ARTIFACT_NAME}.bin -a 0x1000

# Download using USB mode
west tl-bdt download --chip B92 --usb -i build/${TLK_ARTIFACT_NAME}.bin
```

4.6.6.2 Read Flash

```
# Read 16 bytes from address 0x00
west tl-bdt read --chip B92 -a 0x00 -s 16

# Read 1KB to output file
west tl-bdt read --chip B92 -a 0x00 -s 1k -o flash_dump.bin
```

4.6.6.3 Write Flash

```
# Write raw data
west tl-bdt write --chip B92 -a 0x1000 -s 4 01 02 03 04

# Write from file
west tl-bdt write --chip B92 -a 0x1000 -i data.bin

# Write from file with erase
west tl-bdt write --chip B92 -a 0x1000 -i data.bin -e
```

4.6.6.4 Erase Flash

```
# Erase 4KB at address 0x00
west tl-bdt erase --chip B92 -a 0x00 -s 4k
```

(continues on next page)

(continued from previous page)

```
# Erase 1 sector
west tl-bdt erase --chip B92 -a 0x4000 -s 4k
```

4.6.6.5 Lock/Unlock Flash

```
# Lock region
west tl-bdt lock --chip B92 -a 0x00 -s 512k

# Unlock flash (Windows)
west tl-bdt unlock --chip B92
```

4.6.6.6 Reset Device

```
# Reset device
west tl-bdt reset --chip B92

# Reset from core
west tl-bdt reset --chip B92 -c
```

4.6.7 Advanced Usage

4.6.7.1 Using Specific BDT Path

```
west tl-bdt download --chip B92 --bdt-path /path/to/bdt -i build/${TLK_
↪ARTIFACT_NAME}.bin
```

4.6.7.2 Using SWire Clock Settings

```
west tl-bdt download --chip B92 --sws 8000000 1 4000000 1 -i build/${TLK_
↪ARTIFACT_NAME}.bin
```

4.6.7.3 Multiple USB Devices

```
west tl-bdt download --chip B92 --usb --bus 1 --devid 5 -i build/${TLK_
↪ARTIFACT_NAME}.bin
```

4.6.8 Cross-Platform Considerations

The `west tl-bdt` command automatically handles differences between Windows and Linux environments:

- **Windows**: Uses `Cmd_download_tool.exe` and automatically adds device ID and voltage information if needed
- **Linux**: Uses `bdt` with standard Linux command syntax
- **Path handling**: Automatically converts path separators (e.g., `/` → `\` on Windows)

4.6.9 Troubleshooting

4.6.9.1 BDT Tool Not Found

```
Error: BDT tool not found. Please set BDT_PATH environment variable or use --
↪bdt-path option.
```

Solution: Set the BDT path using one of the methods described in section 4.

4.6.9.2 Chip Type Not Specified

```
Error: Chip type not specified. Please use --chip option.
```

Solution: Specify the chip type using `--chip` option, or ensure that the build configuration contains the chip type information.

4.6.9.3 BDT Command Failed

```
Error: BDT command failed with exit code X
```

Solution: Check the BDT output for detailed error information. Common issues include:

- Incorrect chip type
- Invalid address or size
- Device not connected

53.4.7 4.7 West Workspace Configuration

`west.yml` defines the West workspace configuration:

- Remote repository settings
- Project path configuration
- Command extension definitions

53.5 5. Make Support

The `Makefile` provides a traditional make command interface as a frontend to CMake, offering a simplified workflow for developers. This only can be used in root directory of the project.

53.5.1 5.1 Makefile Structure

The `Makefile` in the project root directory provides a traditional make command interface as a frontend to CMake, offering a simplified workflow for developers. It has been recently refactored to improve maintainability and flexibility:

```
.PHONY: cmake config pinmux build clean cleanbuild

# Default application to build
APP ?= samples/gpio_demo

```

(continues on next page)

(continued from previous page)

```
# SOC and BOARD parameters (empty by default)
SOC ?=
BOARD ?=

# CMake configuration
CMAKE := cmake
CMAKE_FLAGS := -B build -G Ninja -DTelink_DIR='./cmake'
BUILD_DIR := build

# Python scripts
PYTHON := python
PINMUX_SCRIPT := -m scripts.pinmux

# Add SOC and BOARD flags if they are set
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif

ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif

# Pre-build checks
PRE_BUILD_CHECKS := \
        @$(PYTHON) scripts/pre_build/build_folder_check.py || (echo "Build␣
↪folder not found — running config..." && "$(MAKE)" config) \
        @$(PYTHON) scripts/pre_build/config_check.py || (echo ".config not␣
↪found — running config..." && "$(MAKE)" config) \
        @$(PYTHON) scripts/pre_build/pinmux_check.py || (echo "pinmux.h not␣
↪found — running pinmux..." && $(PYTHON) $(PINMUX_SCRIPT)) \
        @$(PYTHON) scripts/pre_build/pre_build_check.py || (echo "pinmux.h␣
↪needs update — running pinmux..." && $(PYTHON) $(PINMUX_SCRIPT))

# Targets
cmake:
        $(CMAKE) $(CMAKE_FLAGS) -S $(APP)

config:
        $(CMAKE) --build $(BUILD_DIR) --target config

pinmux:
        $(PYTHON) $(PINMUX_SCRIPT)

build: cmake config
        $(PRE_BUILD_CHECKS)
        $(CMAKE) --build $(BUILD_DIR)

clean:
        $(CMAKE) --build $(BUILD_DIR) --target clean

cleanbuild: clean build
```

The refactored Makefile defines the following key components:

- **Phony Targets**: All targets (`cmake`, `config`, `pinmux`, `build`, `clean`, `cleanbuild`) are marked as phony to avoid conflicts with actual files
- **Configuration Variables**: Centralized configuration using variables for easier maintenance
- **Default Application**: Set default application to build (`gpio_demo`) with override capability
- **CMake Integration**: Uses CMake commands (`cmake -B`, `cmake --build`) for all build operations
- **Pre-build Checks**: Implements automatic validation and recovery mechanisms
- **Automatic Dependency Resolution**: Ensures all required components are present before building
- **Pinmux Target**: Added explicit `pinmux` target for easier pin multiplexing configuration

5.1.1 SOC and BOARD Parameters

The Makefile provides SOC and BOARD parameters to allow targeting specific hardware configurations:

Parameter Definition:

```
# SOC and BOARD parameters (empty by default)
SOC ?=
BOARD ?=
```

Parameter Integration:

```
# Add SOC and BOARD flags if they are set
ifneq ($(SOC),)
CMAKE_FLAGS += -DSOC=$(SOC)
endif

ifneq ($(BOARD),)
CMAKE_FLAGS += -DBOARD=$(BOARD)
endif
```

Usage Examples:

```
# Build with specific SOC and BOARD
make build SOC=TLSR9528A BOARD=TLSR9528A_EVK

# Build with only SOC (board will be auto-selected or default)
make build SOC=TLSR9528A

# Build with only BOARD (SOC dependencies will be resolved automatically)
make build BOARD=TLSR9528A_EVK
```

Key Features:

- **Optional Parameters**: Both SOC and BOARD parameters are optional and can be used independently
- **CMake Integration**: Parameters are automatically passed to CMake as `-DSOC` and `-DBOARD` flags

- **Dependency Resolution**: When BOARD is specified, required SOC dependencies are automatically resolved
- **Validation**: The build system validates SOC/BOARD combinations during configuration

53.5.2 5.2 Make Command Flow

5.2.1 cmake Target

```
cmake:
	$(CMAKE) $(CMAKE_FLAGS) -S $(APP)
```

- **Purpose**: Initializes the CMake build system for the specified application
- **Parameters**:
  - `$(CMAKE)`: Path to the CMake executable
  - `$(CMAKE_FLAGS)`: Common CMake flags including build directory and generator
  - `-S $(APP)`: Specifies the source directory as the selected application (default: `samples/gpio_demo`)

5.2.2 config Target

```
config:
	$(CMAKE) --build $(BUILD_DIR) --target config
```

- **Purpose**: Generates the project configuration files
- **Operation**: Calls CMake's config target to generate configuration files like `.config`
- **Parameters**:
  - `$(CMAKE)`: Path to the CMake executable
  - `--build $(BUILD_DIR)`: Specifies the build directory
  - `--target config`: CMake target to generate configuration files

5.2.3 build Target

```
build: cmake config
	$(PRE_BUILD_CHECKS)
	$(CMAKE) --build $(BUILD_DIR)
```

- **Dependencies**: Automatically runs `cmake` and `config` targets if they haven't been executed
- **Pre-build Checks** (via `$(PRE_BUILD_CHECKS)`):
  1. **Build Folder Check**: Ensures the build directory exists, runs `config` if not
  2. **Config File Check**: Verifies `.config` file exists, runs `config` if not
  3. **Pinmux Check**: Confirms `pinmux.h` exists, runs pinmux tool if not
  4. **Pinmux Update Check**: Verifies `pinmux.h` is up-to-date, runs pinmux tool if not
- **Build Execution**: Calls `cmake --build $(BUILD_DIR)` to execute the actual build using Ninja

5.2.4 clean Target

```
clean:
	$(CMAKE) --build $(BUILD_DIR) --target clean
```

- **Purpose**: Removes build artifacts while preserving configuration files
- **Operation**: Calls CMake's clean target to remove generated binary files
- **Parameters**:
  - `$(CMAKE)`: Path to the CMake executable
  - `--build $(BUILD_DIR)`: Specifies the build directory
  - `--target clean`: CMake target to clean build artifacts

5.2.5 cleanbuild Target

```
cleanbuild: clean build
```

- **Purpose**: Cleans existing build artifacts and immediately rebuilds the project
- **Operation**: Combines the `clean` and `build` operations in sequence
- **Parameters**: Inherits all parameters from the `clean` and `build` targets

5.2.6 pinmux Target

```
pinmux:
	$(PYTHON) $(PINMUX_SCRIPT)
```

- **Purpose**: Configures pin multiplexing
- **Operation**: Runs the pinmux configuration tool directly
- **Parameters**:
  - `$(PYTHON)`: Path to the Python executable
  - `$(PINMUX_SCRIPT)`: Pinmux script module

53.6 6. Compilation Generation Flow

53.6.1 6.1 Complete Build Flow

1. **Environment Validation**: Check toolchain, Python environment, and dependencies
2. **CMake Configuration**: Generate build system and Makefiles, load CMake modules
3. **Configuration Generation**: Generate configuration files through Kconfig as CMake targets
4. **Source Compilation**: Compile C/C++ source code
5. **Linking**: Link object files into executable files
6. **Firmware Generation**: Generate final firmware binary files

6.1.1 Complete Build Flow Diagram

The following diagram illustrates the complete build flow from start to finish:

53.6.2 6.2 Configuration File Generation Flow

1. **Default Configuration**: Load chip default configuration
2. **User Configuration**: Apply user-defined configuration
3. **Configuration Merging**: Merge configuration files through `kconfig.py`
4. **Header File Generation**: Generate `autoconf.h` and `capabilities.h`

53.6.3 6.3 Build File Generation

- **Compilation Commands**: Generate compilation commands for each source file
- **Dependency Information**: Generate source file dependency relationships
- **Linker Scripts**: Process and generate linker scripts
- **Firmware Files**: Generate final binary and flashing files

53.7 7. Files and Scripts Composition

53.7.1 7.1 Core Configuration Files

File	Location	Function
CMakeLists.txt	Root directory	Main CMake configuration entry
Makefile	Root directory	Make command interface
west.yml	Root directory	West workspace configuration
Kconfig.chip	Root directory	Defines available SoC options and their configurations
Kconfig.build	Root directory	Defines build-time options and features
chip.config	Build directory	Generated SoC-specific configuration
build.config	Build directory	Generated build-specific configuration
.config	Build directory	Merged configuration file used for the build
capabilities.h	Build directory	Generated header with SoC capabilities based on selected chip
autoconf.h	Build directory	Generated header with all configuration options as C defines
pinmux.h	Build directory	Generated header with pin multiplexing configuration

53.7.2 7.2 CMake Files and Modules

7.2.1 Core CMake Files

File	Location	Main Function
CMakeLists.txt	Root directory	Main CMake entry point, sets up project structure and includes subdirectories
TelinkConfig.cmake	cmake/	Telink package configuration, provides include_boilerplate macro and loads default modules
CMakeLists.txt	Application directories	Application-specific CMake configuration, creates executable targets and adds sources

7.2.2 CMake Module Files

Module	Location	Main Function
compiler.cmake	cmake/modules/	Compiler configuration and compilation options
linker.cmake	cmake/modules/	Linker configuration and linker scripts
kconfig.cmake	cmake/modules/	Kconfig integration and configuration processing
west.cmake	cmake/modules/	West tool integration
pinmux.cmake	cmake/modules/	Pin multiplexing configuration
extensions.cmake	cmake/modules/	CMake extension functions
python.cmake	cmake/modules/	Python environment configuration
telink_default.cmake	cmake/modules/	Module loading and default configuration

53.7.3 7.3 Build Scripts

Script	Location	Function
build.py	scripts/west_commands	Implement west tl-build command
config.py	scripts/west_commands	Implement west tl-config command
boards.py	scripts/west_commands	Display supported development boards
socs.py	scripts/west_commands	Display supported chips
kconfig.py	scripts/	Process Kconfig configuration files
menuconfig.py	scripts/	Interactive configuration interface
preprocess_kconfig.py	scripts/	Kconfig preprocessing
kconfig_parser.py	scripts/	Kconfig parsing and validation utilities
soc_board_setter.py	scripts/	SOC/BOARD configuration and validation
pinmux_gen.py	scripts/preprocess/	Generate pinmux header from .pinmux file
config_parser.py	scripts/parsers/	Configuration file parsing utilities
yaml_parser.py	scripts/parsers/	YAML file parsing utilities
pinmux_update_check.py	scripts/checks/	Check if pinmux needs updating
tlk_check_fw.py	scripts/	Firmware validation script

7.3.1 SOC/BOARD Related Scripts

soc_board_setter.py

- **Purpose**: Handles SOC and BOARD configuration with proper dependency resolution
- **Key Features**:
  - Validates SOC/BOARD combinations using Kconfig dependencies
  - Automatically resolves and sets required dependencies
  - Supports individual SOC or BOARD setting, or combined setting
  - Provides both programmatic API and command-line interface
- **Usage Examples**:

```
# Set SOC and BOARD together
python scripts/soc_board_setter.py --kconfig Kconfig.chip --soc␣
↪TLSR9528A --board TLSR9528A_EVK

# Validate combination without setting
python scripts/soc_board_setter.py --kconfig Kconfig.chip --validate-
↪only --soc TLSR9528A --board TLSR9528A_EVK

# Set only BOARD (SOC dependencies resolved automatically)
python scripts/soc_board_setter.py --kconfig Kconfig.chip --board␣
↪TLSR9528A_EVK
```

kconfig_parser.py

- **Purpose**: Provides Kconfig parsing and validation utilities for SOC/BOARD management
- **Key Features**:
  - Parses Kconfig.chip to extract available SOCs and boards
  - Validates SOC/BOARD combinations based on dependencies
  - Provides methods to query supported boards for each SOC and vice versa
  - Used by both West commands and CMake configuration system
- **Integration Points**:
  - Used by `boards.py` and `socs.py` West commands
  - Called by `kconfig.cmake` for validation during configuration
  - Supports dependency resolution and compatibility checking

7.3.2 Utility Scripts

pinmux_gen.py

- **Purpose**: Generate pinmux header file from .pinmux configuration
- **Location**: `scripts/preprocess/`
- **Key Features**:
  - Reads .pinmux file from build directory
  - Converts CONFIG_ format definitions to C preprocessor macros

  - Generates pinmux.h header for inclusion in source code
- **Integration**: Used by CMake build system during pinmux configuration generation

config_parser.py

- **Purpose**: Configuration file parsing utilities
- **Location**: `scripts/parsers/`
- **Key Features**:
  - Parse .config files for specific configuration values
  - Extract pinmux configurations from config files
  - Provides consistent interface for config file operations
- **Usage**: Used by various scripts for config file access and validation

yaml_parser.py

- **Purpose**: YAML file parsing utilities
- **Location**: `scripts/parsers/`
- **Key Features**:
  - Load and parse YAML configuration files
  - Handle errors gracefully with informative messages
  - Return dictionary structure for programmatic access
- **Usage**: Used for parsing pinmux YAML files and other configuration data

7.3.3 Validation Scripts

pinmux_update_check.py

- **Purpose**: Check if pinmux configuration needs updating
- **Location**: `scripts/checks/`
- **Key Features**:
  - Validate existence of .config file
  - Check GPIO driver enable status
  - Compare TLK_SOC_SERIES between .config and .pinmux
  - Validate required functions are present in .pinmux
  - Check enabled ports consistency
- **Integration**: Used by pre-build checks to ensure pinmux is up-to-date

| pre_build_check.py | scripts/ | Pre-build environment checks |

53.7.4 7.4 Kconfig Files

File	Location	Function
core/Kconfig	core/	Core feature configuration
soc/Kconfig	soc/	Chip series selection
boards/Kconfig	boards/	Development board selection
api/Kconfig	api/	API feature configuration

53.8 8. Build System Working Principles

53.8.1 8.1 Configuration and Build Separation

The UniSDK build system clearly separates the configuration process from the build process:

1. **Configuration Phase**: Generate configuration files through Kconfig
2. **Generation Phase**: CMake uses configuration files to generate the build system
3. **Build Phase**: Ninja executes actual compilation and linking tasks

53.8.2 8.2 Modular Design

The system adopts a highly modular design, with each module responsible for specific functionality:

- Modules interact through clear interfaces
- Configuration information is passed through variables and properties
- Supports conditional loading and replacement of modules

53.8.3 8.3 Multiple Build Method Support

The system simultaneously supports multiple build methods to meet different usage scenarios:

- **West Commands**: Provide simplified command-line interfaces
- **Direct CMake Calls**: Provide complete configuration control
- **Make Commands**: Compatible with traditional build habits

53.9 9. Extension and Customization

53.9.1 9.1 West Extension Commands Behavior

West Command Behavior in Different Directories

The west tool's output varies depending on the current working directory due to its extension command mechanism:

1. **Extension Command Discovery**: West searches for `.west/config` configuration files in the current directory and its parent directories to locate the workspace.
2. **Project Manifest Loading**: When a workspace is found, west loads the `west.yml` manifest file which defines project-specific extension commands through the `west-commands` field.

3. **Command Availability**:
    - Inside a configured workspace, west loads unisdk project extensions and displays custom commands like `tl-build`, `tl-boards`, and `tl-socs`
    - Outside any workspace, west only shows built-in commands since no project-specific extensions are available
4. **UniSDK Extension Commands**: The unisdk project defines three custom extensions in `scripts/west-commands.yml`:
    - `tl-build`: Compile a Telink application
    - `tl-boards`: Display information about supported Telink boards
    - `tl-socs`: Display information about supported Telink SOCs

This behavior allows west to provide context-aware tooling based on the current project workspace while maintaining consistency across different development environments.

53.9.2 9.2 Adding New Build Modules

To add a new build module:

1. Create a new `.cmake` file in the `cmake/modules/` directory
2. Add the module to the loading list in `telink_default.cmake`
3. Implement the module's configuration and functionality

53.9.3 9.3 Extending West Commands

To add a new West command:

1. Create a command implementation file in the `scripts/west_commands/` directory
2. Register the command in `scripts/west-commands.yml`
3. Implement the command's parameter parsing and execution logic

53.9.4 9.4 Adding Kconfig Configuration

To add new Kconfig configuration:

1. Create or modify Kconfig files in appropriate locations
2. Use the `rsource` directive to integrate new configurations into the configuration tree
3. Define configuration options, dependencies, and default values

53.9.5 9.5 Adding new application

To add a new application:

1. Create a new directory **anywhere** on your filesystem
2. Add the application source files to the new directory (must include a `main` function)
3. Create a `CMakeLists.txt` file in the new directory with the following structure:

```
cmake_minimum_required(VERSION 3.20.0)

# Load the Telink SDK package
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})

# Set project name and supported languages
project(YourAppName LANGUAGES C CXX)

# Add C source files to the existing Telink target
file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

# Add C++ support conditionally if CONFIG_TLK_ALLOW_CPP_WRAPPERS is enabled
if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    # Add C++ sources using wildcard
    file(GLOB CPP_SOURCES "*.cpp")

    # Update the Telink target with C++ sources
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

This implementation fully realizes the application-level entry point design, providing a modular and flexible build system for Telink chip development.

This document provides a detailed introduction to the architecture, components, and working principles of the UniSDK build system, helping developers understand and use the build system. With modular design and support for multiple build methods, the UniSDK build system provides a flexible, powerful build toolchain for Telink chip development.

CHAPTER

FIFTYFOUR

EXTENSIONS & CUSTOMIZATION

54.1 Adding New CMake Modules

1. Create a `.cmake` file in `cmake/modules/`
2. Add it to the module loading list in `telink_default.cmake`

```
# telink_default.cmake
list(APPEND telink_cmake_modules my_module)
```

54.2 Extending West Commands

1. Create a command implementation file in `scripts/west_commands/`
2. Register the command in `scripts/west-commands.yml`

```
# west-commands.yml
west-commands:
  - file: scripts/west_commands/my_command.py
    commands:
      - name: tl-mycmd
        class: MyCommand
        help: my custom command
```

3. Implement the command class:

```
from west.commands import WestCommand

class MyCommand(WestCommand):
    def __init__(self):
        super().__init__('tl-mycmd', 'my custom command', '...')

    def do_add_parser(self, parser_adder):
        parser = parser_adder.add_parser(self.name, help=self.help)
        return parser

    def do_run(self, args, unknown_args):
        print("Hello from my custom command!")
```

54.3 Adding Kconfig Configuration

1. Create or modify a Kconfig file in the appropriate location
2. Integrate into the configuration tree using the `rsource` directive

```
# Add my_feature/Kconfig
menu "My Feature"

config MY_FEATURE_ENABLED
    bool "Enable my feature"
    default n
    help
        Enable custom feature.

endmenu
```

Then include it in the parent Kconfig:

```
# Kconfig.build
rsource "my_feature/Kconfig"
```

54.4 Adding New Applications

1. Create an application directory at any location
2. Add `CMakeLists.txt` and source files
3. Build:

```
west tl-build /path/to/my_app --board TLSR9528A_EVK
```

54.4.1 CMakeLists.txt Template

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Telink REQUIRED HINTS $ENV{TELINK_BASE})
project(MyApp LANGUAGES C CXX)

file(GLOB C_SOURCES "*.c")
target_sources(Telink PRIVATE ${C_SOURCES})

if(CONFIG_TLK_ALLOW_CPP_WRAPPERS)
    file(GLOB CPP_SOURCES "*.cpp")
    target_sources(Telink PRIVATE ${CPP_SOURCES})
endif()
```

54.5 West Command Behavior Notes

West command availability depends on the current working directory:

- **Inside a West workspace**: Loads UniSDK extension commands (`tl-build`, `tl-boards`, `tl-socs`, etc.)

- **Outside a workspace**: Only shows West built-in commands

West discovers extensions through the following mechanism:

1. Searches upward from the current directory for `.west/config`
2. Loads `west-commands` declarations from `west.yml`

CHAPTER

FIFTYFIVE

FILE & SCRIPT REFERENCE

55.1 Core Configuration Files

File	Location	Function
`CMakeLists.txt`	Repository root	CMake main entry point
`Makefile`	Repository root	Make command interface
`west.yml`	Repository root	West workspace configuration
`Kconfig.chip`	Repository root	Chip Kconfig entry point
`Kconfig.build`	Repository root	Build Kconfig entry point

55.2 Build Artifacts (build/ Directory)

File	Description
`.config`	Final merged configuration
`chip.config`	Chip configuration
`build.config`	Build configuration
`autoconf.h`	C configuration macro definitions
`capabilities.h`	Chip capability declarations
`pinmux.h`	Pin multiplexing configuration

55.3 CMake Files

55.3.1 Core Files

File	Location	Function
`CMakeLists.txt`	Repository root	CMake main entry point
`TelinkConfig.cmake`	`cmake/`	Telink package configuration
`telink_default.cmake`	`cmake/modules/`	Module loading and default configuration

55.3.2 CMake Modules (cmake/modules/)

Module	Function
`compiler.cmake`	RISC-V compiler configuration
`extensions.cmake`	CMake extension functions
`kconfig.cmake`	Kconfig integration
`linker.cmake`	Linker configuration
`pinmux.cmake`	Pin multiplexing configuration
`python.cmake`	Python environment configuration
`west.cmake`	West tool integration
`library.cmake`	Library build module
`headers_gen.cmake`	Header file generation

55.4 Build Scripts (scripts/)

Script	Function
`kconfig.py`	Non-interactive Kconfig processing
`menuconfig.py`	Interactive configuration interface
`preprocess_kconfig.py`	Kconfig preprocessing
`kconfig_parser.py`	Kconfig parsing and validation
`soc_board_setter.py`	SOC/BOARD configuration
`tl_check_fw.sh`	Firmware verification
`set_telink_base.sh`	Set TELINK_BASE (Linux/Mac)
`set_telink_base.cmd`	Set TELINK_BASE (Windows CMD)
`set_telink_base.ps1`	Set TELINK_BASE (Windows PowerShell)
`windows_setup.ps1`	Windows automated environment setup
`sdk_exporter.sh`	SDK export script

55.4.1 West Command Implementations (scripts/west_commands/)

Script	Corresponding Command
`build.py`	`west tl-build`
`config.py`	`west tl-config`
`bdt.py`	`west tl-bdt`
`boards.py`	`west tl-boards`
`socs.py`	`west tl-socs`
`zcmake.py`	CMake helper

55.4.2 Pinmux Tools (scripts/pinmux/)

File	Function
`app.py`	Pinmux application core
`__main__.py`	CLI entry point
`data/`	Data processing modules
`ui/`	Terminal UI components
`styles.tcss`	UI stylesheet

55.4.3 CI Scripts (scripts/ci/)

Script	Function
`ci_build.sh`	Single project CI build
`ci_build_all.sh`	All projects CI build
`ci_prepare.sh`	CI preparation
`defconfig_to_dotconfig.py`	defconfig to .config conversion
`dotconfig_to_defconfig.py`	.config to defconfig conversion

55.5 Kconfig Files

File	Location	Function
`Kconfig.chip`	Repository root	Chip Kconfig entry point
`Kconfig.build`	Repository root	Build Kconfig entry point
`core/Kconfig`	`core/`	Core configuration
`core/configs/Kconfig`	`core/configs/`	Core feature configuration
`soc/Kconfig`	`soc/`	Chip series configuration
`boards/Kconfig`	`boards/`	Development board configuration
`api/Kconfig`	`api/`	API configuration
`samples/Kconfig`	`samples/`	Sample configuration

CHAPTER

FIFTYSIX

DEVELOPMENT TOOLS

UniSDK provides a series of development tools to simplify chip configuration, flashing/debugging, and code editing workflows.

Tool	Description	Documentation
BDT	Telink Burning and Debugging Tool	*BDT Tool*
Pinmux	Graphical pin multiplexing configuration tool	*Pinmux Tool*
Menuconfig	Terminal interactive system configuration interface	*Menuconfig*
West	Unified command-line entry point	*West Commands*

56.1 Tool Overview

CHAPTER

FIFTYSEVEN

BDT BURNING AND DEBUGGING TOOL

BDT (Burning and Debugging Tool) is the flashing and debugging tool for Telink chips, providing a unified cross-platform interface via the `west tl-bdt` command.

57.1 Supported Operations

Operation	Description
`download`	Download firmware to Flash
`read`	Read data from Flash
`write`	Write data to Flash
`erase`	Erase Flash
`lock`	Lock Flash region
`unlock`	Unlock Flash region
`reset`	Reset device

57.2 Quick Start

```
# Download firmware (default from build/ directory)
west tl-bdt download --chip TLSR9528A

# Specify firmware file
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# Specify flash address
west tl-bdt download --chip TLSR9528A -i build/Telink.bin -a 0x1000

# Use USB mode
west tl-bdt download --chip TLSR9528A --usb -i build/Telink.bin
```

57.3 Flash Operations

57.3.1 Read Flash

```
# Read 16 bytes from address 0x00
west tl-bdt read --chip TLSR9528A -a 0x00 -s 16
```

(continues on next page)

(continued from previous page)

```
# Read 1KB to a file
west tl-bdt read --chip TLSR9528A -a 0x00 -s 1k -o flash_dump.bin
```

57.3.2 Write Flash

```
# Write raw data
west tl-bdt write --chip TLSR9528A -a 0x1000 -s 4 01 02 03 04

# Write from a file
west tl-bdt write --chip TLSR9528A -a 0x1000 -i data.bin

# Erase then write
west tl-bdt write --chip TLSR9528A -a 0x1000 -i data.bin -e
```

57.3.3 Erase Flash

```
# Erase 4KB
west tl-bdt erase --chip TLSR9528A -a 0x00 -s 4k
```

57.3.4 Flash Lock Operations

```
# Lock 512KB
west tl-bdt lock --chip TLSR9528A -a 0x00 -s 512k

# Unlock
west tl-bdt unlock --chip TLSR9528A
```

57.3.5 Reset

```
# Reset device
west tl-bdt reset --chip TLSR9528A

# Reset from Core
west tl-bdt reset --chip TLSR9528A -c
```

57.4 BDT Path Configuration

Priority from high to low:

1. Command line: `--bdt-path /path/to/bdt`
2. Environment variable: `export BDT_PATH=/path/to/BDT`
3. System PATH
4. SDK directory: `${TELINK_BASE}/tools/`

```
# Method 1: Specify via command line
west tl-bdt download --bdt-path /path/to/BDT --chip TLSR9528A -i build/
↪firmware.bin

# Method 2: Environment variable
export BDT_PATH=/path/to/BDT  # Linux/macOS
set BDT_PATH=C:\BDT           # Windows CMD
$env:BDT_PATH="C:\BDT"        # Windows PowerShell
```

57.5 Cross-Platform Notes

Platform	Executable	Notes
Windows	`Cmd_download_tool.exe`	Automatically handles path separator conversion
Linux	`bdt`	Standard Linux command

57.6 FAQ

57.6.1 BDT Tool Not Found

```
Error: BDT tool not found. Please set BDT_PATH or use --bdt-path.
```

Solution: Configure the BDT path using the methods described above.

57.6.2 Chip Type Not Specified

```
Error: Chip type not specified. Please use --chip option.
```

Solution: Use the `--chip` argument to specify the chip, or ensure the build configuration includes chip information.

57.6.3 BDT Command Failed

Check common causes:

- Incorrect chip type
- Invalid address or size
- Device not connected

CHAPTER

FIFTYEIGHT

PINMUX PIN CONFIGURATION TOOL

58.1 Overview

The Pinmux (Pin Multiplexing) tool is a graphical pin configuration tool provided by UniSDK to help developers visually configure chip pin multiplexing functions in the terminal.

Key Features:

- Graphical terminal interface (based on the Textual framework)
- Visual selection of pin functions with interactive popup menus
- Pin configuration conflict detection
- Requirement validation (ensures all required functions are assigned)
- Support for multiple chip package variants
- Integration with the Kconfig configuration system
- Automatic generation of the `pinmux.h` header file

58.2 Launch Methods

```
# Method 1: Via CMake target
make pinmux

# Method 2: Direct Python module invocation
python -m scripts.pinmux

# Method 3: Skip GUI, generate default configuration
python -m scripts.pinmux --skip-gui
```

58.3 Main View

Once opened, you will see the main dashboard:

- **Ports:** A scrollable vertical list of all GPIO ports (e.g., PORT A, PORT B)
- **Pins:** Inside each port section, individual pins are displayed as interactive buttons showing the pin's name and its currently assigned function

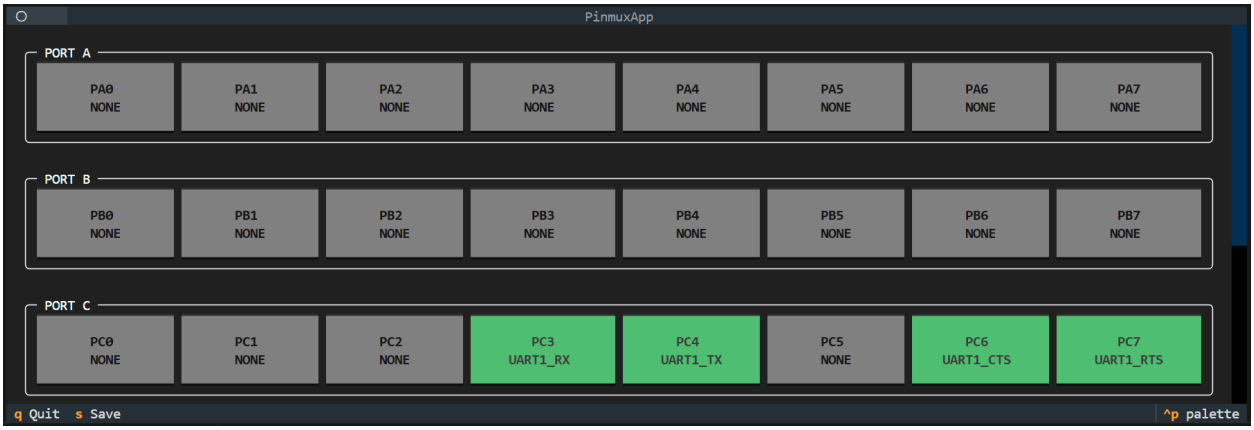


Figure 1. Pinmux Interface overview showing GPIO ports, pins, and assigned functions.

58.4 Selecting a Function

1. Click on a pin (or navigate to it and press **Enter**)
2. A popup window titled **Function Select** will appear
3. The list shows all supported hardware functions for that specific pin
4. Select the desired function (e.g., `UART0_RX`) or select **NONE** to disable the pin
5. Press **Enter** to confirm

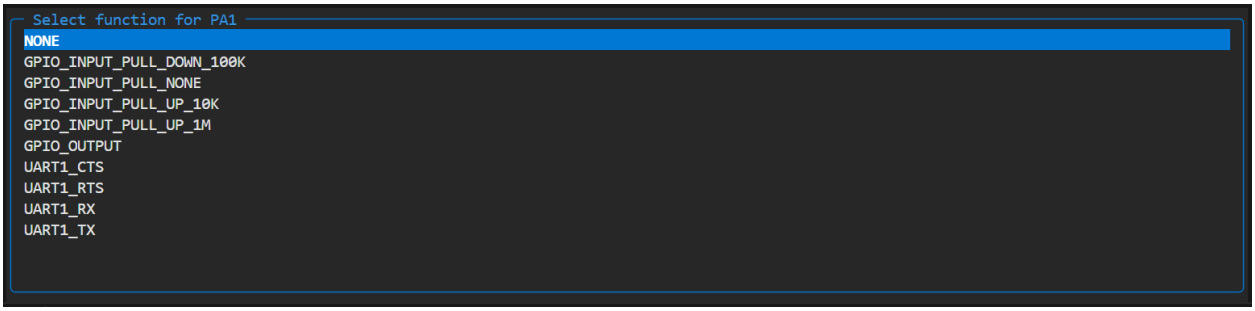


Figure 2. Function select screen for a GPIO pin.

58.5 Shortcuts

Key	Action
s (Save)	Validates configuration —checks for conflicts and missing functions. If valid, saves and exits. If invalid, shows an error notification
q (Quit)	Exits the application. A confirmation dialog appears if there are unsaved modifications

Validation errors occur when:

- Necessary functions are not assigned to pins (e.g., UART0 is enabled but UART0_TX is not assigned)
- Conflicting pins exist (e.g., UART0_TX assigned to both PC3 and PC4)

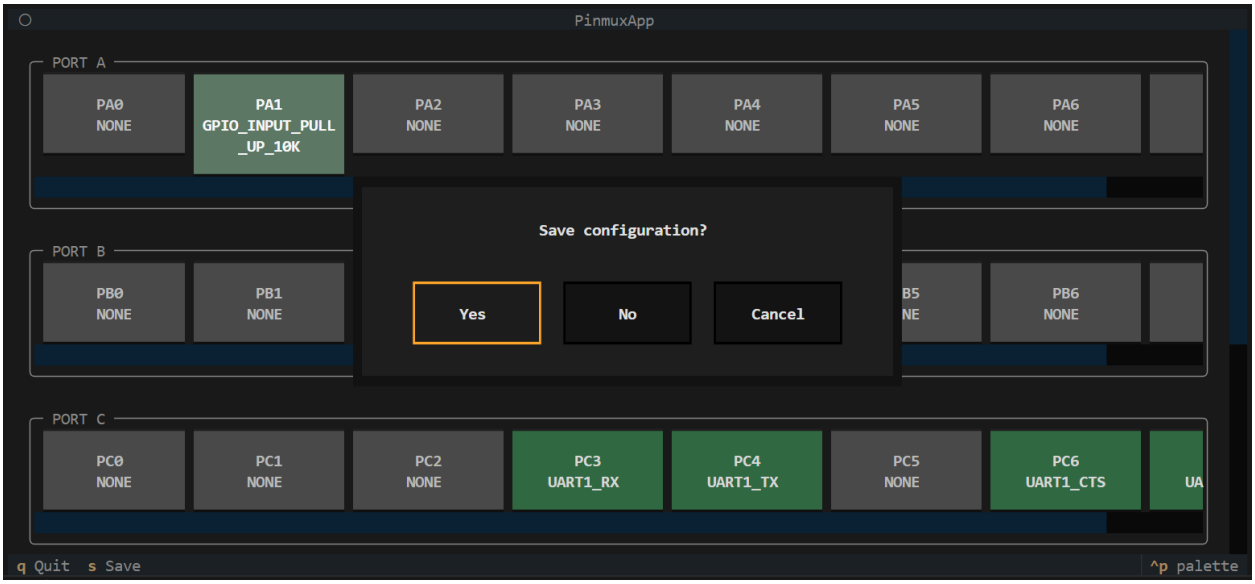


Figure 3. Exit Screen.

58.6 Saving

The configuration is saved to a file named `<SoC or board name>.<soc or board>.pinmux`. For example, building for TL3218X_EVK generates `TL3218X_EVK.board.pinmux`.

This `.pinmux` file contains pin-to-function mappings in a human-readable format and is used to restore previous pin settings when the tool is invoked again.

Additionally, `pinmux.h` is generated in the build folder based on the saved settings, containing C preprocessor definitions usable by firmware code.

The `.pinmux` files persist until the project switches, allowing previous pin settings to be restored when switching between chips.

58.7 File Format

The `.pinmux` files use a simple key-value format:

```
PC4=UART0_TX
PB2=UART0_RX
PA0=GPIO_OUTPUT
```

- Each line represents a single pin assignment
- All letters are upper-case
- Format: `<pin name>=<function name>`
- Pin name consists of letter **P**, port letter, and pin number

During parsing, the following lines are ignored:

- Invalid data that doesn’t match the pattern
- Valid data referencing non-existent pins or functions
- Assignments to pins on ports disabled in the current configuration

- Assignments to functions not enabled in the current configuration

58.8 Overriding Mechanism

Each time pinmux is invoked, data is loaded in the following sequence (each overrides the previous):

1. **SoC defaults** —from `soc/<soc>/pinmux/function.yaml` (used for development)
2. **Project `.pinmux`** —in the project directory (sets custom defaults for any chip)
3. **`<identifier>.pinmux`** —in the project directory (chip-specific overrides)
4. **`<identifier>.pinmux`** —in the build directory (last actual user pin settings)

This allows projects to have out-of-the-box pin settings while still allowing customization.

58.9 Internal Execution Flow

58.9.1 Data Loading & Synchronization

When the application starts, it combines multiple data sources:

- `soc/<soc>/pinmux/function.yaml` —The "Function Definitions": describes all available functions for the chip
- `soc/<soc>/pinmux/pinmux.yaml` —The "Dictionary": lists every physical pin and its supported functions. Each port can have an `exclude` property, which describes what functions aren't supported by its pins. Individual pins can also use `exclude` and `include` properties. The restrictions are applied in the following order: port's exclude → pin's exclude → pin's include. For simplicity, the `all` keyword represents all functions. For groups of functions, you may use a group name (e.g., `uart0` represents `uart0_tx`, `uart0_rx`, `uart0_cts`, and `uart0_rts`; `uart` covers `uart0`, `uart1`, etc.). If a pin supports only one exact function, use `exclude: all` together with `include: your_function`.
- `soc/<soc>/pinmux/pinmux_<case>.yaml` —The "Restriction": overrides `pinmux.yaml` to remove pins or ports for specific chip packages by assigning the `delete` keyword
- `.config` —The "Requirements": tells the app which drivers are enabled (e.g., "UART0 is enabled, so we need pins for TX and RX")
- `.pinmux` files —Previous user configurations (see overriding mechanism)

The Loading Sequence:

1. The `.config` file, which contains the current configuration and identifies the current chip.
2. The function description from `function.yaml`, skipping functions disabled by configuration.
3. The port and pin description from `pinmux.yaml` and `pinmux_<case>.yaml` for the selected chip.
4. The `.pinmux` files in the order described in the *Overriding Mechanism* section.

58.9.2 Changing a Pin Function

When a new function is selected for a pin, the following happens instantly:

- **Collision Check:** The system scans all other pins. If you try to assign a unique function (like `UART0_TX`) to PC0, but PB2 already has it, the system flags a **Conflict**.
- **Observer Update:** The app uses an "Observer Pattern."
  - The Pin data changes.
  - The UI widget receives a notification and redraws itself (updating the text and color).
  - The next `is_modified` property access in `PinmuxManager` will return `True`, until the save action is triggered.

58.9.3 The Saving Process

When you press Save (`s`), the app performs strict validation before writing to disk.

Step 1: Validation

- **Check Conflicts:** Are two pins assigned to the same non-repeatable function? If yes → Error.
- **Check Requirements:** Are all functions required by the configuration assigned to valid pins? If no → Error.

Step 2: Writing

- If there are no errors, the pin mapping is saved to the `<identifier>.pinmux` file using the simple key-value format.

58.9.4 Generating pinmux.h

After saving, `pinmux.h` is automatically generated with C preprocessor definitions:

```
#define PINMUX_UART0_TX_PORT TLK_GPIO_PORT_C
#define PINMUX_UART0_TX_PIN TLK_GPIO_PIN_4
#define PINMUX_UART0_RX_PORT TLK_GPIO_PORT_B
#define PINMUX_UART0_RX_PIN TLK_GPIO_PIN_2
#define PINMUX_GPIO_OUTPUT_0_PORT TLK_GPIO_PORT_A
#define PINMUX_GPIO_OUTPUT_0_PIN TLK_GPIO_PIN_0
#define PINMUX_GPIO_OUTPUT_COUNT 1
```

58.10 Configuration File Locations

Pinmux Configuration Files

```
soc/<chip>/pinmux/
├── function.yaml          # Pin function definitions
├── pinmux.yaml            # Pin multiplexing configuration
├── pinmux_qfn48.yaml      # QFN48 package
├── pinmux_qfn88.yaml      # QFN88 package
└── ...
```

58.11 Build Integration

Pinmux configuration is deeply integrated into the build system:

- The `generate_pinmux` CMake target automatically executes before compilation
- Pre-build checks automatically detect whether `pinmux.h` needs to be updated
- The generated `pinmux.h` is placed in the `build/` directory and automatically added to the include path

58.12 CI

The CI uses a single configuration across all samples. This is possible because unused assignments are skipped, so you just need to consider all pin usages in the samples within this file. Before the build, the CI copies this file to the build directory. Since the build' s settings are applied last during overriding, the resulting configuration corresponds to what is described in the file —all previous settings are overridden.

CHAPTER

FIFTYNINE

MENUCONFIG CONFIGURATION INTERFACE

59.1 Overview

Menuconfig is UniSDK's terminal-based interactive system configuration tool, providing visual menu navigation to set all Kconfig configuration options.

Key Features:

- Dual-phase configuration: Chip configuration and Build configuration are separated
- Terminal graphical interface with arrow key navigation
- Search function (press `/` key)
- Help system (press `?` key to view option descriptions)
- Automatic dependency management (unavailable options are automatically hidden)
- Merged output: Automatically merges hardware and software configurations into `.config`

59.2 Launch Methods

```
# Via Make
make config

# Via West (interactive)
west tl-config

# Via West (CMake only, skip interactive GUI)
west tl-config -c

# Direct invocation
python scripts/menuconfig.py Kconfig.chip     # Chip configuration
python scripts/menuconfig.py Kconfig.build    # Build configuration
```

Note

First-time Use `make config` will open the interface twice in sequence: first the chip configuration (Kconfig.chip), then the peripheral build configuration (Kconfig.build).

59.3 Navigation and Operations

Operation	Key
Move up/down	↑ ↓ or j k
Enter submenu	Enter or →
Return to parent	ESC or ←
Toggle option	Space
Search	/
View help	?
Save and exit	Select < Save > then < Exit >

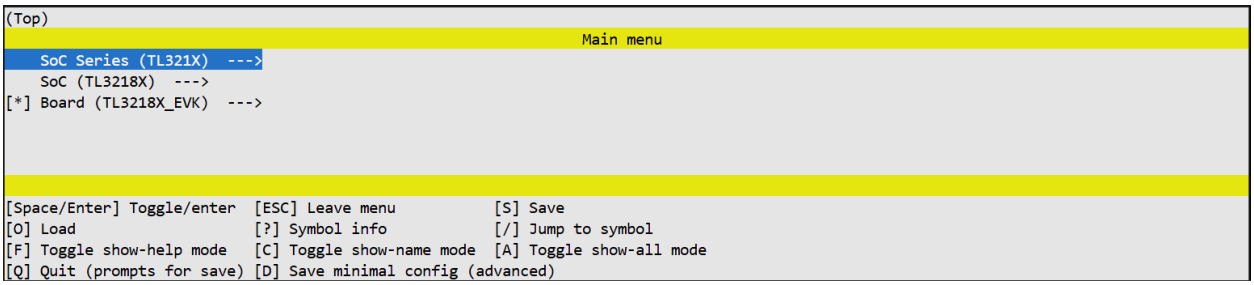


Figure 1. Chip configuration screen —select target SoC and board

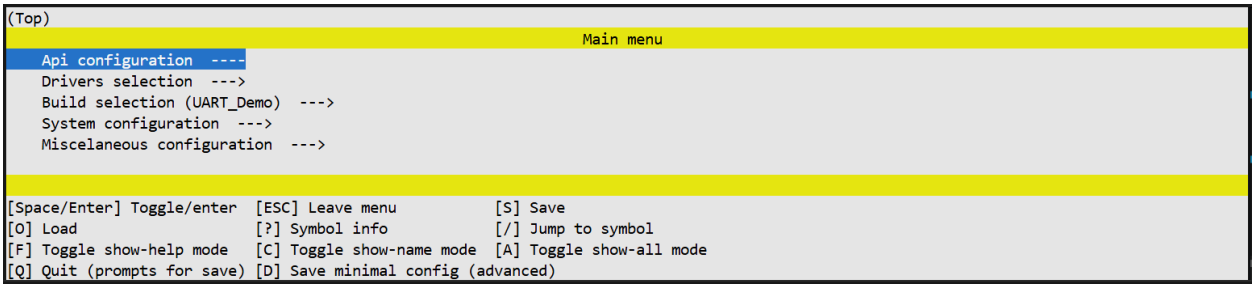


Figure 2. Build configuration screen —configure software drivers and settings

59.4 Dual-Phase Configuration

59.4.1 Phase 1: Chip Configuration (chip.config)

When the tool opens for the first time, you are configuring the hardware.

- **What you see:** Options related to SoC Series, SoC, and Board
- **Action:** Select your target SoC and enable required chip peripherals (GPIO ports, UART instances, etc.)
- **Exit:** Select < Save > and < Exit > to close the window

59.4.2 Phase 2: Build Configuration (build.config)

Immediately after the first window closes, the tool opens again for software configuration.

- **What you see:** Options filtered based on chip capabilities (`capabilities.h`), including API configurations, driver selections, and feature switches

- **Action:** Configure feature switches, parameters, examples, and debugging options
- **Exit:** Select < Save > and < Exit > to finish the process

59.5 Results

After configuration completes, the following files are generated:

Generated Configuration Files

```
build/
├── chip.config       # Chip configuration
├── build.config      # Build configuration
├── .config           # Final merged configuration
├── capabilities.h    # Chip capability macros
├── autoconf.h        # Configuration macro definitions
└── pinmux.h          # Pin multiplexing configuration
```

59.6 Internal Execution Flow

59.6.1 Stage 1: Chip Configuration

1. The system reads `Kconfig.chip` —defines all hardware-related options
2. The user selects settings in the UI
3. Output is saved to a temporary `chip.config` file

59.6.2 Stage 2: Pre-processing

Between the two windows, `preprocess_kconfig.py` runs automatically to prepare the build configuration stage.

59.6.3 Stage 3: Build Configuration

1. The system reads `Kconfig.build` —defines software and project settings
2. The user selects settings in the UI
3. Output is saved to a temporary `build.config` file

59.6.4 Stage 4: Merging

Both stages are combined into the final `.config`:

```
cat chip.config build.config > .config
```

This merged file contains the complete system configuration (both hardware and software) and is used by the compiler and the Pinmux tool.

59.7 FAQ

59.7.1 Arrow Keys Not Working in Windows PowerShell / VS Code

- Use letter keys instead (j/k for up/down movement)
- Use Windows 11
- Use CMD instead of PowerShell

CHAPTER

SIXTY

SECURITY

Note

Document Status: PLANNED —Content for this module has not yet been written

The Security module will cover security-related features in UniSDK, including:

- Secure Boot
- Firmware signing and verification
- Flash read/write protection
- Debug interface security
- Hardware cryptographic engine (AES / PKE / HASH / TRNG)

Detailed documentation will be added later.

CHAPTER

SIXTYONE

PRODUCTION GUIDE

Note

Document Status: PLANNED —Content for this module has not yet been written

The Production Guide will help developers transition prototypes developed with UniSDK into mass production products, covering:

- Mass production flashing solutions
- OTA (Over-The-Air) firmware upgrades
- Production line testing and calibration
- Hardware design reference
- Certification and compliance

Detailed documentation will be added later.

CHAPTER

SIXTYTWO

SAMPLES & DEMOS

UniSDK provides a rich set of ready-to-compile-and-run sample programs in the `samples/` directory. Each sample demonstrates the usage of one or more SDK features.

62.1 Sample List

62.1.1 Basic Peripherals

Sample	Description	Status
GPIO Sample	GPIO input/output, interrupt handling, sleep API	STABLE
UART Sample	UART echo, sleep test, TX/RX protection	STABLE
PM Power Management	Low-power modes, GPIO wake-up, retention variables	STABLE
PMP Sample	Physical Memory Protection driver demo	DRAFT
User Mode Sample	RISC-V User Mode / Machine Mode switching	DRAFT

62.1.2 Communication Interfaces

Sample	Description	Status
ADC Sample	ADC analog-to-digital conversion	DRAFT
DMA Sample	DMA direct memory access	DRAFT
I2C Sample	I2C master-slave communication (DS1307 / Ping-Pong)	DRAFT
SPI Sample	SPI master-slave communication	DRAFT
I2S Sample	I2S master/slave audio streaming	DRAFT

62.1.3 Wireless Connectivity

Sample	Description	Status
BLE Sample	Raw RF advertising and BLE Controller HCI demos	DRAFT
BLE Host Sample	BLE Host v2 peripheral with GATT services	DRAFT
RF Sample	RF driver ping-pong demo (BLE/Zigbee/Private/Hybee)	DRAFT

62.1.4 System Features

Sample	Description	Status
IRQ Nesting	Interrupt nesting demo	DRAFT
Timer Sample	STIMER system timer demo	DRAFT
Watchdog Sample	Watchdog timer demo	DRAFT
USB CDC Sample	TinyUSB CDC serial port demo	DRAFT
Random Sample	Hardware random number generation	DRAFT
NV Storage Sample	Non-volatile storage device read/write/erase	DRAFT
Log Sample	Debug logging over the console/USB	DRAFT
Task Planner Sample	Cooperative loop, scheduler, and software timer execution models	DRAFT

62.1.5 Audio & Security

Sample	Description	Status
Audio Sample	Audio input/output capture and playback via I2S/DMA	DRAFT
mbedTLS Sample	PSA Crypto random, ECDSA key generation, sign/verify	DRAFT
MCUboot Bootloader Sample	MCUboot bootloader with optional UART DFU recovery	DRAFT

62.2 Building Samples

All samples use a unified build approach:

```
# West command (recommended)
west tl-build samples/gpio_demo --board TLSR9528A_EVK

# Make command
make build APP=samples/gpio_demo BOARD=TLSR9528A_EVK
```

Each sample directory typically contains:

Sample Directory Structure

```
samples/gpio_demo/
├── CMakeLists.txt    # Build entry
├── Kconfig           # Sample-specific configuration options
├── main.c            # Main program
├── main.cpp          # C++ version (optional)
└── README.md         # Sample description
```

CHAPTER

SIXTYTHREE

GPIO SAMPLE

63.1 Overview

The `gpio_demo` sample demonstrates the basic usage of the GPIO driver, including input/output operations, interrupt handling, and sleep APIs.

63.2 Features Demonstrated

- Basic GPIO configuration and pin control (`tlk_gpio_configure`, `tlk_gpio_pin_write`, `tlk_gpio_pin_toggle`)
- GPIO interrupt usage and callback registration (requires `CONFIG_TLK_GPIO_DEMO_INTERRUPT` enabled)

63.3 How It Works

63.3.1 Initialization

1. **LED0 Configuration** —Configured as output mode, toggled every 2 seconds in the main loop
2. **Interrupt Configuration (optional)** —When `CONFIG_TLK_GPIO_DEMO_INTERRUPT` is enabled:
   - LED1 is configured as output (toggled in the interrupt callback)
   - KEY2 is configured as output and driven high, so it can supply a logic ‘1’ to KEY0 when KEY0 is pressed
   - KEY0 is configured as input with pull-down
   - Registers a GPIO interrupt callback (`gpio_cb`)
   - Sets rising-edge trigger
   - Enables global interrupts

63.3.2 Main Loop

```
while (1)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
```

(continues on next page)

(continued from previous page)

```
    tlk_api_time_delay(TLK_SEC_TO_US(2));
    tlk_api_sleep(TLK_SEC_TO_MS(2));
}
```

Note

Board-Specific Macros The `PINMUX_LED_0_PORT` and `PINMUX_LED_0_PIN` macros (and their `LED_1`/`KEY_0`/`KEY_2` counterparts) are defined in the board-specific `tlk_board_pinout.h` header. These resolve to the correct port and pin for the target board's LEDs and keys.

63.4 Configuration Options

Option	Description
`CONFIG_TLK_GPIO_DEMO_INTERRU`	Enable interrupt demo: KEY0 rising-edge trigger, LED1 toggled in callback

63.5 Build & Run

```
west tl-build samples/gpio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: `samples/gpio_demo/main.c`

CHAPTER

SIXTYFOUR

UART SAMPLE

64.1 Overview

The `uart_demo` sample demonstrates the UART driver's echo functionality: receiving data and sending it back as-is.

64.2 Features Demonstrated

- Manual UART configuration using `tlk_uart_configure` (when `CONFIG_TLK_UART_DEMO_AUTO_CONFIG` is disabled) —TX/RX pins are supplied directly in the `struct tlk_uart_config` passed to `tlk_uart_configure`
  - Manual UART flow control configuration using `tlk_uart_flow_control_configure` if `CONFIG_TLK_UART0_FLOW_CONTROL` is enabled
- Callback-based data reception (`tlk_uart_receive_bytes`)
- Data transmission (`tlk_uart_send_bytes`)
- Power management and sleep prevention (optional)

64.3 How It Works

64.3.1 Initialization

1. **Interrupt Enable** —Call `tlk_core_interrupt_enable()` to enable asynchronous callbacks
2. **Manual Configuration** —If `CONFIG_TLK_UART_DEMO_AUTO_CONFIG` is disabled, manually configure the selected UART (`CONFIG_TLK_UART_DEMO_UART`) using `tlk_uart_configure`, with TX on port C pin 4 and RX on port C pin 5. If `CONFIG_TLK_UART0_FLOW_CONTROL` is enabled, also configure flow control using `tlk_uart_flow_control_configure` (RTS on port C pin 6, CTS on port C pin 7)

64.3.2 Main Loop (Echo Flow)

1. **Power Management Test** —If `CONFIG_TLK_UART_DEMO_SLEEP` is enabled, first print "Before sleep", sleep for 2 seconds, then print "After sleep" upon wake-up
2. Call `tlk_uart_receive_bytes` to listen for data, using the `rx_handler` callback
3. Wait for `rx_handler` to set the `rx_count` variable to the number of bytes received

4. Send the received data back to the serial port (`tlk_uart_send_bytes`)
5. Reset `rx_count` and repeat

64.3.3 Sleep Prevention Test

When `CONFIG_TLK_UART_DEMO_PREVENT` is enabled, the sample attempts to sleep for 10 ms during reception (`TLK_PM_SLEEP_MS(TLK_PM_SLEEP_MODE_SUSPEND, 10)`) to verify that `tlk_pm_sleep` returns `TLK_PM_SLEEP_DENIED`.

64.4 Configuration Options

Option	Description
`CONFIG_TLK_UART_DEMO_UART{i}`	Selects which UART instance (`UART0`, `UART1`, ⋯) the sample uses
`CONFIG_TLK_UART_DEMO_BUFFER_S`	Size of the receive buffer (default 15)
`CONFIG_TLK_UART_DEMO_SLEEP`	Test whether UART works correctly after wake-up
`CONFIG_TLK_UART_DEMO_PREVENT`	Test whether sleep is correctly prevented during reception

Note

Auto-Configuration `CONFIG_TLK_UART_DEMO_AUTO_CONFIG` is not a user-selectable option in this sample's Kconfig —it is automatically set based on whether the selected UART's own `TLK_UART{i}_AUTO_CONFIG` option is enabled.

64.5 Build & Run

```
west tl-build samples/uart_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

CHAPTER

SIXTYFIVE

PM POWER MANAGEMENT SAMPLE

65.1 Overview

The `pm_demo` sample demonstrates the usage of the Power Management (PM) subsystem. It shows how to enter low-power modes, configure GPIO wake-up, and preserve application state across low-power transitions.

65.2 Features Demonstrated

- Entering Deep/Suspend sleep using `tlk_api_sleep`
- GPIO wake-up configuration (optional)
- Before/After sleep hook functions
- Variable retention across Deep Sleep Retention mode

65.3 How It Works

65.3.1 Initialization

1. **LED State Initialization** —When `CONFIG_TLK_PM_DEMO_ENABLE_LED` is enabled:
   - LED state variables are stored in the **retention section**
   - Initial state: LED0 ON, LED1 OFF
2. **Hook Registration** —Registers pre-sleep and post-sleep callbacks:
   - `flip_led()` → Flips LED state before sleep
   - `init_led()` → Re-initializes GPIO after wake-up

65.3.2 Main Loop

1. Delay 1 second
2. Optionally configure GPIO wake-up source
3. Enter low-power mode for 1 second

65.3.3 GPIO Wake-up (Optional)

When `CONFIG_TLK_PM_DEMO_ENABLE_GPIO_INPUT` is enabled:

- GPIO C4 is configured as a wake-up source (high-level wake-up)
- Suspend mode is used instead of Deep Sleep when GPIO wake-up is enabled

65.4 Configuration Options

Option	Description
`CONFIG_TLK_PM_DEMO_ENABLE_LED`	LED demo: state preserved in retention area
`CONFIG_TLK_PM_DEMO_ENABLE_GPIO_INPUT`	GPIO wake-up demo
`CONFIG_TLK_PM_DEMO_DISABLE_LED_BEFORE`	Turn off LED before sleep (visual sleep entry indicator)

65.5 Build & Run

```
west tl-build samples/pm_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

CHAPTER

SIXTYSIX

PMP DRIVER SAMPLE

66.1 Overview

A sample application demonstrating the usage of the TLK PMP API. It protects the upper half of a 16-element array against all access, then attempts to read or write the full array, triggering a PMP violation trap on the protected region.

66.2 Features Demonstrated

- PMP region protection using TOR (Top-Of-Range) address matching (`tlk_pmp_configure_tor`)
- PMP region protection using NAPOT (Naturally Aligned Power-Of-Two) address matching (`tlk_pmp_configure_napot`)
- Locked PMP entries (`l = 1`) that deny all read/write/execute access
- PMP state retention across a sleep/wake cycle (optional)

66.3 How It Works

On startup the sample:

1. *(Optional)* Configures PMP protection on `demo_array[8..15]` using the selected address matching scheme. All permissions are disabled and the entry is locked (`l = 1`).
2. *(Optional)* If `CONFIG_TLK_PMP_DEMO_SLEEP` is set, enters sleep for 500 ms and wakes up, demonstrating that PMP state is correctly saved and restored across the sleep.
3. Iterates over all 16 elements of `demo_array`:
   - With `CONFIG_TLK_PMP_DEMO_READ` —reads and logs each value.
   - With `CONFIG_TLK_PMP_DEMO_WRITE` —writes and logs each value.

Elements `[0..7]` complete successfully. On the first access to `demo_array[8]` a PMP violation trap is raised.

66.4 Configuration

All options are exposed via **Kconfig** under *PMP Sample configuration*. The PMP mode and the access action are each modeled as a single-choice group (`choice`), so exactly one option in each group is active at a time; `TLK_PMP_DEMO_TOR` and `TLK_PMP_DEMO_READ` are the defaults.

Sym-bol	Type	De-fault	Description
TLK_PMP	bool (choi	—	PMP mode: no protection is configured; all 16 elements are accessible without a fault.
TLK_PMP	bool (choi	y	PMP mode: protect demo_array[8..15] using TOR address matching. Entry 0 holds the lower bound; Entry 1 holds the upper bound.
TLK_PMP	bool (choi	n	PMP mode: protect demo_array[8..15] using NAPOT address matching.
TLK_PMP	bool (choi	y	Action mode: access the array by reading.
TLK_PMP	bool (choi	n	Action mode: access the array by writing.
TLK_PMP	bool	n	Enter sleep between PMP configuration and array access (only selectable when the PMP mode is not OFF). Demonstrates PMP state retention across sleep; selects TLK_API_SLEEP and implies TLK_PMP_PM_DEVICE.

Note: With TLK_PMP_DEMO_OFF selected, no protection is configured and all 16 elements are accessible without a fault.

66.5 Build & Run

```
west tl-build samples/pmp_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/pmp_demo.bin
```

Source code: samples/pmp_demo/main.c

CHAPTER

SIXTYSEVEN

U-MODE DRIVER SAMPLE

67.1 Overview

A sample application demonstrating the TLK U-mode API. It switches the core from Machine mode (M-mode) to User mode (U-mode), executes a task under reduced privilege, optionally dispatches a service request back to M-mode via `ecall`, and optionally returns to M-mode permanently through `tlk_core_enable_mmode`.

67.2 Features Demonstrated

- Switching from M-mode to U-mode and back (`tlk_core_enable_umode`, `tlk_core_enable_mmode`)
- PMP-based privilege isolation: a NAPOT region granting full access to the 4 GB address space, and (optionally) a higher-priority region denying UART access from U-mode
- Environment calls (`ecall`) from U-mode to request a service performed in M-mode
- Illegal-instruction trap on a privileged CSR read attempted from U-mode

67.3 How It Works

On startup the sample:

1. Registers `ecall_handler` as the ecall exception callback via `tlk_trap_register_ecall_callback`.
2. Configures PMP entry 7 to grant full RWX access to the entire 4 GB address space —this is the baseline permission for U-mode.
3. *(Optional)* If `CONFIG_TLK_UMODE_DEMO_ENABLE_UART` is **not** set, configures PMP entry 0 to deny all access to the UART peripheral region. Because PMP entries are checked from lowest index first, entry 0 overrides entry 7 for that range, blocking any UART access from U-mode.
4. Calls `tlk_core_enable_umode(user_task, task_handler)` to switch to U-mode:
   - **`user_task`** executes in U-mode and attempts to send `"User task.\n"` over UART0. Succeeds only if `CONFIG_TLK_UMODE_DEMO_ENABLE_UART` is set; otherwise a PMP violation trap fires on the first UART register access. If `CONFIG_TLK_UMODE_DEMO_ENABLE_SLEEP` is set, `user_task` also sets `USER_REQUEST_SLEEP` before returning.

- After `user_task` returns, **`task_handler`** executes in U-mode. This function calls the `ecall`.
- The `ecall` causes an environment-call trap which transfers control to M-mode where **`ecall_handler`** runs:
  - If `USER_REQUEST_SLEEP` is set, it logs `"Ecall. Sleep request.\n"`, sleeps for 1 second via `tlk_api_sleep`, then clears the request.
  - If `CONFIG_TLK_UMODE_DEMO_ENABLE_MMODE` is set, it calls `tlk_core_enable_mmode()` to permanently restore M-mode privilege before returning.

5. After `tlk_core_enable_umode` returns, `main` performs a privilege check: it attempts to read the `mstatus` CSR. This read succeeds in M-mode and raises an illegal-instruction exception in U-mode. If `CONFIG_TLK_UMODE_DEMO_ENABLE_MMODE` is not set, `ecall_handler` never calls `tlk_core_enable_mmode()`, so execution returns to `main` still in U-mode and the read traps.

67.4 Configuration

All options are exposed via **Kconfig** under *UMODE Sample configuration*.

Symbol	Type	Default	Description
CONFIG_TLK_	bool	n	Allow U-mode access to the UART peripheral. When disabled, PMP entry 0 is configured to deny all access to the UART region, causing a PMP violation trap on any UART access attempted from U-mode.
CONFIG_TLK_	bool	n	Enable the sleep service request. When set, `user_task` issues a `USER_REQUEST_SLEEP` request before returning, causing `ecall_handler` to perform a 1-second sleep in M-mode.
CONFIG_TLK_	bool	n	Return to M-mode after the ecall. When set, `ecall_handler` calls `tlk_core_enable_mmode()` so that execution continues in `main` with full M-mode privilege. When disabled, `main` remains in U-mode after `tlk_core_enable_umode` returns, and the subsequent `mstatus` CSR read raises an illegal-instruction exception.

Note: With all options disabled the sample demonstrates two protections in sequence: a PMP violation trap when `user_task` touches the blocked UART region, and an illegal-instruction trap when `main` attempts to read a privileged CSR from U-mode.

67.5 Build & Run

```
west tl-build samples/umode_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/umode_demo.bin
```

Source code: `samples/umode_demo/main.c`

CHAPTER

SIXTYEIGHT

ADC SAMPLE

Note

Document Status: DRAFT —Content is being finalized

68.1 Overview

The `adc_demo` sample demonstrates the basic usage of the ADC driver: configuring a single-ended channel, sampling into a buffer, and logging the results in a loop. An optional configuration shows the ADC read loop running alongside the power-management sleep API.

68.2 Features Demonstrated

- ADC channel configuration (`tlk_adc_configure`) —channel, prescale, resolution, sample frequency, voltage reference, and sampling pin
- Buffered sampling (`tlk_adc_read`) with timeout/error handling via `enum tlk_adc_status`
- Optional DMA channel reservation for the ADC (`tlk_dma_chn_request`, guarded by `CONFIG_TLK_DMA`)
- Optional low-power sampling loop using `tlk_api_sleep` (requires `CONFIG_TLK_ADC_DEMO_PM`)

68.3 How It Works

68.3.1 Initialization

The sample builds a `struct tlk_adc_config` and passes it to `tlk_adc_configure`:

```
struct tlk_adc_config cfg = {
    .channel      = TLK_ADC_CHANNEL_0,
    .prescale     = TLK_ADC_PRESCALE_1F4,
    .resolution   = TLK_ADC_RESOLUTION_12,
    .sample_freq = TLK_ADC_SAMPLE_FREQ_96K,
    .vref         = TLK_ADC_VREF_1P2V,
    .sampling_port_pin =
```

(continues on next page)

(continued from previous page)

```
        {
            .port = TLK_GPIO_PORT_B,
            .pin  = TLK_GPIO_PIN_0,
        },
    .is_differential = false,
    .differential_port_pin =
        {
            .port = TLK_GPIO_PORT_D,
            .pin  = TLK_GPIO_PIN_0,
        },
    .timeout_us = 10000,
#if TLK_IS_ENABLED(CONFIG_TLK_DMA)
    .dma_en      = false,
    .dma_channel = tlk_dma_chn_request(),
#endif
};

tlk_adc_configure(&cfg);
```

The channel samples on GPIO port B pin 0 in single-ended mode (`is_differential = false`); the differential pin is set but unused in that mode. When `CONFIG_TLK_DMA` is enabled, a DMA channel is reserved for the ADC configuration up front (though `dma_en` is left `false` in this sample).

68.3.2 Main Loop

```
while (1)
{
    enum tlk_adc_status status = tlk_adc_read(adc_values, ADC_BUFFER_SIZE);

    if (status == TLK_ADC_OK)
    {
        for (uint32_t i = 0; i < ADC_BUFFER_SIZE; i++)
        {
            TLK_LOG_INFO(adc_demo, "%u", adc_values[i]);
        }
    }
    else
    {
        TLK_LOG_ERROR(adc_demo, "Timeout");
    }

#if TLK_IS_ENABLED(CONFIG_TLK_ADC_DEMO_PM)
    tlk_api_sleep(TLK_SEC_TO_MS(1));
#else
    tlk_api_time_delay(TLK_SEC_TO_US(1));
#endif
}
```

Each iteration reads `ADC_BUFFER_SIZE` (8) samples into `adc_values` and logs every value on success, or logs a timeout error otherwise. The loop then either sleeps for 1 second (PM build) or busy-delays for 1 second (default build).

68.4 Configuration Options

Option	Description
CONFIG_TLK_ADC	Build the ADC demo with power management: uses tlk_api_sleep instead of a busy delay between reads, selects TLK_API_SLEEP and implies TLK_ADC_PM_DEVICE

The sample's base TLK_ADC_DEMO config selects TLK_ADC, TLK_DEBUG_LOG_CONSOLE, and TLK_API_TIME unconditionally.

68.5 Build & Run

```
west tl-build samples/adc_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: samples/adc_demo/main.c

CHAPTER

SIXTYNINE

DMA SAMPLE

Note

Document Status: DRAFT —Content is being finalized

69.1 Overview

The `dma_demo` sample demonstrates memory-to-memory transfers using the DMA driver: requesting and configuring a DMA channel, starting a transfer, aborting a transfer mid-flight, and completing a transfer —optionally driven by interrupt callbacks and/or combined with sleep/PM-prevent logic.

69.2 Features Demonstrated

- DMA channel request and configuration (`tlk_dma_chn_request`, `tlk_dma_chn_configure`)
- Memory-to-memory transfer setup, start, and abort (`tlk_dma_chn_transfer_configure`, `tlk_dma_chn_transfer_start`, `tlk_dma_chn_transfer_abort`)
- Optional DMA interrupt callback registration (`tlk_dma_chn_add_callback`, requires `CONFIG_TLK_DMA_DEMO_INTERRUPT`)
- Optional sleep between transfer cycles (requires `CONFIG_TLK_DMA_DEMO_SLEEP`)
- Optional PM-suspend-with-prevent-sleep pattern around an active transfer (requires `CONFIG_TLK_DMA_DEMO_PREVENT`)

69.3 How It Works

69.3.1 Initialization

```
demo_chn = tlk_dma_chn_request();
tlk_dma_chn_configure(demo_chn, &chn_config);

#if TLK_IS_ENABLED(CONFIG_TLK_DMA_DEMO_INTERRUPT)
tlk_dma_chn_add_callback(demo_chn, chn_handler);
#endif
```

chn_config is a struct tlk_dma_config configured for a normal-mode, byte-width, incrementing-address transfer on both source and destination, with a single-transfer burst size:

```
struct tlk_dma_config chn_config = {
    .dst_req_sel    = 0,
    .src_req_sel    = 0,
    .dst_addr_ctrl  = TLK_DMA_ADDR_INCREMENT,
    .src_addr_ctrl  = TLK_DMA_ADDR_INCREMENT,
    .dstmode        = TLK_DMA_NORMAL_MODE,
    .srcmode        = TLK_DMA_NORMAL_MODE,
    .dstwidth       = TLK_DMA_BYTE_WIDTH,
    .srcwidth       = TLK_DMA_BYTE_WIDTH,
    .src_burst_size = TLK_DMA_BURST_1_TRANSFER,
    .read_num_en    = 0,
    .priority       = 0,
    .write_num_en   = 0,
    .auto_en        = 0,
};
```

When CONFIG_TLK_DMA_DEMO_INTERRUPT is enabled, chn_handler sets a dma_complete or dma_abort flag depending on the IRQ type (TLK_DMA_IRQ_TC / TLK_DMA_IRQ_ABT), which the main loop then polls.

Note

Minimum Stack Size The sample enforces a minimum 2KB stack at compile time via #if CONFIG_TLK_MEMORY_STACK_SIZE < 2 / #error.

69.3.2 Main Loop

Each iteration of the loop:

1. (Optional, CONFIG_TLK_DMA_DEMO_PREVENT) Starts a transfer of the full src/dst arrays (ARRAY_LEN = 300 bytes), then calls TLK_PM_SLEEP_MS(TLK_PM_SLEEP_MODE_SUSPEND, 10) —if the DMA transfer is still active, sleep entry is denied and the demo logs "The sleep was prevented due to active receiving".
2. Fills src with incrementing values and clears dst for TRANSFER_SIZE (32) bytes, then logs both buffers.
3. Starts a TRANSFER_SIZE-byte transfer and immediately calls tlk_dma_chn_transfer_abort, demonstrating mid-transfer abort. With CONFIG_TLK_DMA_DEMO_INTERRUPT enabled, the loop spins on dma_abort before logging "DMA IRQ: abort". Buffers are logged again afterward.
4. Clears dst, starts a fresh TRANSFER_SIZE-byte transfer and lets it run to completion. With interrupts enabled, the loop spins on dma_complete before logging "DMA IRQ: complete". Buffers are logged again to show the completed copy.
5. Delays 5 seconds —via tlk_api_sleep (if CONFIG_TLK_DMA_DEMO_SLEEP) or tlk_api_time_delay otherwise —before repeating.

69.4 Configuration Options

Option	Description
CONFIG_TLK_DMA_DEMO_	Use DMA interrupts (TLK_PLIC, TLK_DMA_IRQ_HANDLER) to detect transfer completion/abort instead of polling registers directly
CONFIG_TLK_DMA_DEMO_	Use tlk_api_sleep for the end-of-loop delay; selects TLK_API_SLEEP and implies TLK_DMA_PM_DEVICE
CONFIG_TLK_DMA_DEMO_	Demonstrate PM sleep prevention while a DMA transfer is active; selects TLK_PM, TLK_PM_SUSPEND, and TLK_DMA_PREVENT_SLEEP

The sample's base TLK_DMA_DEMO config selects TLK_DMA, TLK_API_PRINT, and TLK_API_TIME unconditionally.

69.5 Build & Run

```
west tl-build samples/dma_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: samples/dma_demo/main.c

CHAPTER

SEVENTY

I2C SAMPLE

Note

Document Status: DRAFT —Content is being finalized

70.1 Overview

The `i2c_demo` sample includes two independent sub-samples that both configure `tlk_i2c0` on GPIO port C pins 6/7 (SCL/SDA):

- **`ds1307`** —I2C master driving a DS1307 real-time-clock chip: periodically reads the current time and, on a button press, writes a new time.
- **`ping_pong`** —a minimal I2C master/slave loopback demo (role selected at build time) that exchanges a fixed 8-byte buffer and toggles LEDs on activity.

70.2 Features Demonstrated

- I2C pinmux and master configuration (`tlk_i2c_pinmux_configure`, `tlk_i2c_master_configure`)
- Single-buffer master writes/reads (`tlk_i2c_master_write`, `tlk_i2c_master_read`)
- Combined write+read transactions using `tlk_i2c_master_xfer` with a `struct tlk_i2c_message` array (DS1307 register-pointer + burst read pattern)
- I2C slave mode configuration and event-driven handler (`tlk_i2c_slave_configure`, `tlk_i2c_slave_set_handler`, `TLK_I2C_EVENT_WRITE_REQUEST` / `TLK_I2C_EVENT_READ_REQUEST` / `TLK_I2C_EVENT_STOP`)
- GPIO interrupt integration: the DS1307 sample sets the time from a button-press callback (`tlk_gpio_irq_add_callback`)

70.3 How It Works

70.3.1 ds1307 sub-sample

`main.c` configures the I2C pinmux and master mode, then arms a GPIO interrupt on `KEY_0` (rising edge) whose callback writes a new fixed time to the RTC via `ds1307_set_time`:

```
struct tlk_i2c_pinmux_config pinmux = {
    .scl_port_pin = {.port = TLK_GPIO_PORT_C, .pin = TLK_GPIO_PIN_6},
    .sda_port_pin = {.port = TLK_GPIO_PORT_C, .pin = TLK_GPIO_PIN_7},
};

tlk_i2c_pinmux_configure(dev, &pinmux);

struct tlk_i2c_master_config config = {
    .clock_speed = 100000,
    .stretch_en  = 1,
};

tlk_i2c_master_configure(dev, &config);
```

The main loop reads the current time every second via `ds1307_read_time` and logs it. The DS1307 driver (`ds1307.c`) implements the register protocol on top of the I2C API:

- `ds1307_set_time` writes an 8-byte buffer (start register `0x00` followed by BCD-encoded sec/min/hour/day/date/month/year) with a single `tlk_i2c_master_write`.
- `ds1307_read_time` issues a combined transaction with `tlk_i2c_master_xfer`: a 1-byte write of the register pointer followed by a 7-byte read, using two `struct tlk_i2c_message` entries (`is_read = 0` then `is_read = 1`).

70.3.2 ping_pong sub-sample

Depending on `CONFIG_TLK_I2C_DEMO_ROLE_MASTER` / `CONFIG_TLK_I2C_DEMO_ROLE_SLAVE`, `main.c` builds one of two roles against slave address `0x5a`:

- **Master**: repeatedly writes an 8-byte buffer (`"i2c_demo"`) with `tlk_i2c_master_write`, then reads it back with `tlk_i2c_master_read`, and logs whether the round-tripped data matches. Delays between operations via `tlk_api_time_delay`, or `tlk_api_sleep` if `CONFIG_TLK_I2C_DEMO_PM` is enabled.
- **Slave**: configures slave mode with `tlk_i2c_slave_configure`, registers an event handler with `tlk_i2c_slave_set_handler`, and sets its TX/RX buffers with `tlk_i2c_slave_set_tx` / `tlk_i2c_slave_set_rx`. The handler toggles LED0 on a write request, toggles LED1 and echoes the received buffer back into the TX buffer on a read request, and logs on a stop condition:

```
void handler(struct tlk_i2c_device* dev, enum tlk_i2c_event event)
{
    switch (event)
    {
    case TLK_I2C_EVENT_WRITE_REQUEST:
        tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
        break;
    case TLK_I2C_EVENT_READ_REQUEST:
        tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
        memcpy(tx_buffer, rx_buffer, BUFFER_SIZE);
        break;
    case TLK_I2C_EVENT_STOP:
        TLK_LOG_INFO(i2c_demo, "Stop received.");
        break;
    default:
```

(continues on next page)

(continued from previous page)

```
        break;
    }
}
```

70.4 Configuration Options

70.4.1 ds1307

The TLK_I2C_DEMO config for this sub-sample selects TLK_I2C, TLK_I2C0, TLK_I2C0_MASTER, TLK_GPIO, and TLK_PLIC unconditionally (no sub-options exposed).

70.4.2 ping_pong

Option	Description
TLK_I2C_DEMO_RO	Build as I2C master (selects TLK_I2C0_MASTER) —choice option, default role
TLK_I2C_DEMO_RO	Build as I2C slave (selects TLK_I2C0_SLAVE) —choice option
CONFIG_TLK_I2C_DEM	Master role only: use tlk_api_sleep for the end-of-loop delay; selects TLK_API_SLEEP and implies TLK_I2C0_PM_DEVICE

The base TLK_I2C_DEMO config selects TLK_I2C and TLK_I2C0 unconditionally; exactly one of the two role options must be selected (choice block).

70.5 Build & Run

```
# ds1307
west tl-build samples/i2c_demo/ds1307 --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin

# ping_pong
west tl-build samples/i2c_demo/ping_pong --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: samples/i2c_demo/ds1307/main.c, samples/i2c_demo/ds1307/ds1307.c, samples/i2c_demo/ping_pong/main.c

CHAPTER

SEVENTYONE

SPI SAMPLE

Note

Document Status: DRAFT —Content is being finalized

71.1 Overview

The `spi_demo` sample demonstrates SPI master/slave communication on `tlk_spi0`, using a command byte to select between a write and a read phase. The role (master or slave) is selected at build time.

71.2 Features Demonstrated

- SPI pinmux configuration (`tlk_spi_pinmux_configure`) for CS/SCK/MOSI/MISO
- SPI master configuration and command-based transfers (`tlk_spi_master_configure`, `tlk_spi_master_xfer` with `struct tlk_spi_xfer`)
- SPI slave configuration with a command handler (`tlk_spi_slave_configure`, `tlk_spi_slave_set_handler`, `tlk_spi_slave_read`/`tlk_spi_slave_write`)
- LED toggling to visualize transfer activity and success

71.3 How It Works

71.3.1 Initialization

Both roles share pin and mode setup:

```
struct tlk_spi_config config = {
    .mode    = TLK_SPI_MODE0,
    .io_mode = TLK_SPI_SINGLE_MODE,
    /* lower the frequency in case of errors on the slave side, if the master
 ↪'s div allows it */
    .frequency = 2 * 1000 * 1000,
};

struct tlk_spi_pinmux pinmux = {
```

(continues on next page)

(continued from previous page)

```
    .cs   = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_0},
    .sck  = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_1},
    .mosi = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_2},
    .miso = {.port = TLK_GPIO_PORT_E, .pin = TLK_GPIO_PIN_3},
};

tlk_gpio_configure(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, TLK_GPIO_OUTPUT);
tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);

tlk_core_interrupt_enable();

tlk_spi_pinmux_configure(dev, &pinmux);
```

SPI runs in mode 0, single-IO mode, at 2 MHz.

71.3.2 Master Role (CONFIG_TLK_SPI_DEMO_ROLE_MASTER)

The master issues two command-tagged transfers per loop using `struct tlk_spi_xfer` with `use_cmd = 1`:

```
xfer.type = TLK_SPI_DUMMY_TX;
xfer.cmd  = COMMAND_WRITE;
status = tlk_spi_master_xfer(dev, &xfer);
...
xfer.type = TLK_SPI_DUMMY_RX;
xfer.cmd  = COMMAND_READ;
status = tlk_spi_master_xfer(dev, &xfer);
```

`COMMAND_WRITE` (1) sends `tx_data` to the slave; `COMMAND_READ` (2) reads back into `rx_data`. LED0 toggles every cycle; LED1 toggles when the first byte of `tx_data` matches the first byte of `rx_data`, indicating a successful round trip. A 10 ms delay separates the write and read phases, and a 1 second delay separates loop iterations.

71.3.3 Slave Role (CONFIG_TLK_SPI_DEMO_ROLE_SLAVE)

The slave registers a command handler and then idles in an empty `while (1)` loop, relying entirely on the interrupt-driven handler:

```
void handler(struct tlk_spi_device* dev, tlk_spi_cmd_t cmd)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);

    switch (cmd)
    {
    case COMMAND_WRITE:
        tlk_spi_slave_read(dev, rx_data, BUFFER_SIZE);
        break;
    case COMMAND_READ:
        tlk_spi_slave_write(dev, tx_data, BUFFER_SIZE);
        break;
    default:
        break;
```

(continues on next page)

(continued from previous page)

```
    }
}
```

On `COMMAND_WRITE` the slave reads the incoming bytes into `rx_data`; on `COMMAND_READ` it writes `tx_data` back to the master. LED0 toggles on every command received.

71.4 Configuration Options

Option	Description
`TLK_SPI_DEMO_ROLE_MASTER`	Build as SPI master —choice option
`TLK_SPI_DEMO_ROLE_SLAVE`	Build as SPI slave —choice option

The base `TLK_SPI_DEMO` config selects `TLK_SPI`, `TLK_SPI0`, and `TLK_API_TIME` unconditionally; exactly one role must be selected (`choice` block). `TLK_BLE_CONTROLLER` is explicitly disabled (`default n`) for this sample.

71.5 Build & Run

```
west tl-build samples/spi_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/Telink.bin
```

Source code: `samples/spi_demo/main.c`

CHAPTER

SEVENTYTWO

I2S SAMPLE

72.1 Overview

The `i2s_demo` sample demonstrates the I2S driver by continuously transmitting a fixed test pattern out over I2S while simultaneously receiving on the same bus, and using an LED to indicate whether the transmitted and received data match.

72.2 Features Demonstrated

- I2S pinmux configuration (`tlk_i2s_pinmux_configure`) for BCLK, ADC (RX) LR clock/data, and DAC (TX) LR clock/data pins
- I2S device configuration (`tlk_i2s_configure`) with 16-bit data width and I2S mode
- Master/Slave role selection (`CONFIG_TLK_I2S_DEMO_ROLE_MASTER` / `CONFIG_TLK_I2S_DEMO_ROLE_SLAVE`)
- Simultaneous DMA-driven output and input paths (`tlk_i2s_configure_output`, `tlk_i2s_input_configure`) using circular DMA linked-list nodes (`struct tlk_dma_ll_node`, self-referencing `next` for continuous looping)
- Starting the I2S device with `tlk_i2s_start`

72.3 How It Works

72.3.1 Initialization

1. **Pinmux** —BCLK, ADC LR clock/data, and DAC LR clock/data pins are assigned (`struct tlk_i2s_pinmux`).
2. **Role Selection** —At compile time, if `CONFIG_TLK_I2S_DEMO_ROLE_MASTER` is enabled, the device is configured as `TLK_I2S_MASTER` with an explicit sample-rate divider (`{8, 625, 0, 64, 64}`); if `CONFIG_TLK_I2S_DEMO_ROLE_SLAVE` is enabled, it is configured as `TLK_I2S_SLAVE` (sample rate is driven externally).
3. **Circular DMA Buffers** —`rx_buff0` and `tx_buff0` are each set up as a single-node circular linked list (`next` points back to itself), so RX and TX run continuously without needing new nodes queued.
4. **I2S Configuration** —`tlk_i2s_configure`, `tlk_i2s_pinmux_configure`, `tlk_i2s_configure_output`, and `tlk_i2s_input_configure` are called on the `tlk_i2s0` device, wiring DMA channels from `tlk_dma_chn_request()` to FIFO0 on the left I2S channel for both TX and RX.

5. **Start** —`tlk_i2s_start(dev)` begins continuous transmit/receive.

72.3.2 Main Loop

```
while (1)
{
    // Mark the successful transmission with the enabled LED
    // I2S has a known issue that the rx buffer has 3 dummy elements at the␣
 ↪beginning
    tlk_gpio_pin_write(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, tx_buffer[0] ==␣
 ↪rx_buffer[3]);
}
```

The loop continuously compares `tx_buffer[0]` (the first transmitted sample) against `rx_buffer[3]` (offset by 3 to account for a known driver quirk where the RX buffer starts with 3 dummy elements) and drives the board LED accordingly —the LED lights up once transmitted data is correctly looped back into the receive buffer.

Note

Loopback Wiring This sample transmits and receives at the same time on the same I2S device. To see the LED light up, the DAC output pins must be physically wired to the ADC input pins on the board (or an external I2S codec must loop the signal back).

72.4 Configuration Options

Option	Description
`CONFIG_TLK_I2S_DEMO`	Enables the sample; selects `TLK_I2S` and `TLK_I2S0` (default y, not user-facing)
`CONFIG_TLK_I2S_DEMO_ROLE_`	Configure the I2S device as bus master, driving BCLK/LRCLK with an explicit sample-rate divider
`CONFIG_TLK_I2S_DEMO_ROLE_`	Configure the I2S device as bus slave, following an externally supplied clock

72.5 Build & Run

```
west tl-build samples/i2s_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/i2s_demo.bin
```

Source code: `samples/i2s_demo/main.c`

CHAPTER

SEVENTYTHREE

BLE SAMPLE

Note

Document Status: DRAFT —Content is being finalized

73.1 Overview

This page covers two low-level BLE radio/controller samples:

- **`samples/adv_demo`** —a minimal example that drives the RF driver directly to transmit raw BLE advertising packets, without using the BLE Controller stack.
- **`samples/ble_controller_demo`** —a full BLE Controller demo that exposes the standard HCI interface over a UART transport, for use with an external Host.

For the newer SAL-based BLE Host stack sample, see the *BLE Host Sample*.

73.2 Features Demonstrated

73.2.1 adv_demo

- Direct `tlk_rf_*` API usage to configure the radio for BLE (`tlk_rf_configure`)
- Manual construction of a raw BLE advertising PDU (flags, complete local name, appearance)
- Packet transmission via `tlk_rf_tx_prepare_packet` / `tlk_rf_start_stx`
- GPIO toggling to visually indicate each transmitted packet

73.2.2 ble_controller_demo

- BLE Controller stack initialization (`tlk_ble_controller_initial`)
- Standard HCI transport carried over UART1 (`tlk_uart_send_bytes` / `tlk_uart_receive_bytes`)
- HCI command/event dispatch via `tlk_ble_controller_hci_handler` and `tlk_ble_controller_hci_registerEventHandler`
- Kconfig-selectable BLE Controller capabilities: accept list, resolving list, legacy advertising/scanning, and ACL central/peripheral roles

73.3 How It Works

73.3.1 adv_demo

1. **RF Configuration** —`rf_configure()` sets up the radio in `TLK_RF_MODE_BLE` mode, 1M bitrate, channel 37, TX power 4, with the standard BLE advertising access code `0xd6be898e`.
2. **PDU Setup** —A fixed-content advertising PDU is prepared in `tx_buffer`: advertiser address `3c:cf:b4:03:e3:23`, AD flags, complete local name `"adv_demo"`, and an appearance field.
3. **Main Loop** —Repeatedly prepares and transmits the packet, toggles LED2 on each iteration, waits for the transmission to complete, then delays 100 ms before repeating:

```
while (1)
{
    tlk_rf_tx_prepare_packet(tx_packet, TX_PDU_SIZE);

    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);

    tlk_rf_start_stx(tlk_rf_get_tick());

    while (!tlk_rf_instance.tx_end)
    {
    }

    tlk_sys_delay(TLK_MS_TO_US(TX_DELAY_MS));
}
```

73.3.2 ble_controller_demo

1. **UART Configuration** —Configures UART1 at 115200-8N1, with TX on GPIO port B pin 3 and RX on GPIO port B pin 2, used as the HCI transport.
2. **Controller Initialization** —Calls `tlk_ble_controller_initial()` and registers `app_ble_controller_hci_send_data` (via `tlk_ble_controller_hci_registerEventHandler`) so that outgoing HCI events are written back to the UART.
3. **Main Loop** —Receives HCI command bytes from the UART into `hci_rx_buffer` and, once a full receive completes, forwards them to `tlk_ble_controller_hci_handler`. The controller stack processes the command and emits HCI events through the registered callback, which sends them back out over the same UART.

This models a standard UART HCI transport for driving the Telink BLE Controller from an external Host implementation (e.g. a PC-based Host stack or a Host running on another MCU).

73.4 Configuration Options

`ble_controller_demo` exposes the BLE Controller stack sizing/feature options directly (there is no dedicated demo submenu):

Option	Description
CONFIG_TLK_BLE_CONTROLLER_HCI_ACL_DATA	Maximum HCI ACL data payload size (default 50)
CONFIG_TLK_BLE_CONTROLLER_HCI_ACL_BUFF	Number of HCI ACL buffers (default 10)
CONFIG_TLK_BLE_CONTROLLER_ACCEPT_LIST_	Enable the controller accept (white) list (default y)
CONFIG_TLK_BLE_CONTROLLER_RESOLVING_LI	Enable the resolving list for address resolution (default y)
CONFIG_TLK_BLE_CONTROLLER_LEGACY_ADV_E	Enable legacy advertising (default y)
CONFIG_TLK_BLE_CONTROLLER_LEGACY_SCAN_	Enable legacy scanning (default y)
CONFIG_TLK_BLE_CONTROLLER_ACL_PERIPHER	Enable the ACL peripheral (slave) role (default y)
CONFIG_TLK_BLE_CONTROLLER_ACL_PERIPHER	Maximum concurrent peripheral connections (default 1)
CONFIG_TLK_BLE_CONTROLLER_ACL_CENTRAL_	Enable the ACL central (master) role (default y)
CONFIG_TLK_BLE_CONTROLLER_ACL_CENTRAL_	Maximum concurrent central connections (default 1)
CONFIG_TLK_BLE_CONTROLLER_ACL_RX_MAX_L	Maximum ACL RX payload length (default 27)

adv_demo has no demo-specific Kconfig options; it only selects TLK_RF and TLK_GPIO.

73.5 Build & Run

```
# adv_demo — raw advertising over the RF driver
west tl-build samples/adv_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/adv_demo.bin
```

```
# ble_controller_demo — full BLE Controller with UART HCI transport
west tl-build samples/ble_controller_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/ble_controller_demo.bin
```

73.6 Related Resources

- *BLE Controller Documentation*
- *BLE Host Sample* —newer SAL-based BLE Host stack sample (samples/ble_host_demo)

Source code: samples/adv_demo/main.c, samples/ble_controller_demo/main.c

CHAPTER

SEVENTYFOUR

BLE HOST SAMPLE

74.1 Overview

The `ble_host_demo` sample demonstrates the **BLE Host v2** stack: a newer host implementation built on a SAL (Service Abstraction Layer, `tlk_ble_host_sal.h`), distinct from the lower-level advertising and controller-only samples. It brings up a BLE peripheral that advertises, accepts multiple simultaneous ACL connections, registers the standard Core and Device Information Service (DIS) GATT groups, and re-starts advertising automatically as connections come and go.

See also: *BLE Sample* for the lower-level advertising/controller demos.

74.2 Features Demonstrated

- BLE Host v2 stack bring-up (`ble_host_v1_init`, `tlk_ble_host_loop`)
- Peripheral advertising setup with custom advertising and scan-response data (`ble_host_gap_adv_set_adv_ind_param`, `ble_host_gap_adv_start`)
- GATT service groups: Core group and Device Information Service (`blc_svc_addCoreGroup`, `blc_svc_addDisGroup`, `blc_svc_calculateDatabaseHash`, `blc_svc_setDeviceName`)
- ACL connection lifecycle callbacks (`ble_host_acl_conn_register_user_data` with `connected`/`disconnected` callbacks)
- Tracking up to 4 simultaneous peripheral-role ACL connections
- SMP (Security Manager Protocol) initialization for pairing (`ble_host_smp_initial` with `BLE_HOST_SMP_LEGACY_JUST_WORKS_INIT_PARAMS`, `ble_host_smp_store_init`)

74.3 How It Works

74.3.1 Advertising Data

Advertising and scan-response payloads are built from static LTV (length-type-value) structures:

```
static const struct ad_data_flags s_adv_flags = {
    .header.length                      = 0x02,
    .header.type                        = DT_FLAGS,
    .flags.le_limited_discoverable_mode = 1,
```

(continues on next page)

(continued from previous page)

```
    .flags.br_edr_not_supported         = 1,
};

static const struct ad_data_complete_local_name_complete s_adv_complete_name␣
↪= {
    .header.length = sizeof(BLE_PERIPHERAL_DEVICE_NAME),
    .header.type   = DT_COMPLETE_LOCAL_NAME,
    .name          = BLE_PERIPHERAL_DEVICE_NAME,
};
```

The device advertises as `"tlk_bluetooth_ble_per"`, including the flags and complete local name AD types in both the advertising data and the scan response.

74.3.2 Initialization (`ble_host_v2_peripheral_init`)

1. Sets the advertising interval to 150 (units per `ble_host_gap_adv_set_adv_ind_param`) along with the advertising/scan-response data.
2. Registers the Core GATT group and the Device Information Service group (`blc_svc_addCoreGroup`, `blc_svc_addDisGroup`), computes the GATT database hash, and sets the device name.
3. Resets the local peripheral-connection tracking table (`s_app_acl_peripheral_info`), up to `APP_BLE_ACL_PERIPHERAL_MAX_COUNT` (4) entries, all initialized to `BLE_CONN_HANDLE_INVALID`.
4. Registers `s_app_acl_callbacks` (connected/disconnected handlers) via `ble_host_acl_conn_register_user_data`.
5. Initializes SMP pairing with Legacy Just Works parameters and sets up SMP bond storage for up to 4 devices (`ble_host_smp_store_init(4, 0)`).

74.3.3 Connection Callbacks

- **`app_connected_callback`** —If the new connection is peripheral-role, records its connection handle in the first free slot and increments the connection count. As long as fewer than 4 peripheral connections are active, advertising is restarted (`ble_host_gap_adv_start`) so additional centrals can connect.
- **`app_disconnected_callback`** —If the disconnected connection was peripheral-role, frees its slot and decrements the count, then unconditionally restarts advertising.

74.3.4 Main Loop

```
int main(void)
{
    TLK_LOG_INFO(ble_host_v2_demo, "BLE HOST v2 starting...");

    ble_host_v1_init(NULL, NULL);
    tlk_core_interrupt_enable();

    ble_host_v2_peripheral_init();
    ble_host_v2_peripheral_start();
```

(continues on next page)

(continued from previous page)

```
    // Main loop
    while (1)
    {
        tlk_ble_host_loop();
    }

    return 0;
}
```

`ble_host_v1_init` brings up the underlying host, interrupts are enabled, the peripheral role is initialized and advertising is started (`ble_host_v2_peripheral_start` calls `ble_host_gap_adv_start`), and the main loop repeatedly pumps the host's event loop via `tlk_ble_host_loop()`.

Note

SAL-Based Host Stack This sample's naming (`ble_host_v1_init`, log tag `ble_host_v2_demo`) reflects that it exercises the newer BLE Host v2 architecture through the SAL header `tlk_ble_host_sal.h`, layered on top of the v1 host init call. This is architecturally different from `samples/adv_demo` and `samples/ble_controller_demo`, which work directly against the advertising/controller APIs —see *BLE Sample*.

74.4 Configuration Options

Option	Description
`CONFIG_TLK_BLE_HOST_V2`	Enables the sample; selects `TLK_BLE_HOST_V2` and `TLK_BLE_CONTROLLER` (default y, not user-facing)

Note

This sample has no user-selectable Kconfig choices beyond the auto-selected stack dependencies above —the peripheral behavior (advertising params, service groups, max connections) is fixed in `main.c`.

74.5 Build & Run

```
west tl-build samples/ble_host_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/ble_host_demo.bin
```

Source code: `samples/ble_host_demo/main.c`

CHAPTER

SEVENTYFIVE

RF SAMPLE

75.1 Overview

The `rf_demo` sample demonstrates the RF driver capabilities by implementing a ping-pong application. It requires two devices for communication:

- **Transmitter** —sends a **ping** packet to the **Receiver**, then listens for the **pong** response. To avoid deadlocking, it has a timeout to resend the **ping** if no **pong** is received.
- **Receiver** —waits indefinitely for a **ping** packet, then sends a **pong** response once received.

LED1 indicates whether the chip is on or off (mainly useful with PM enabled). LED2 is toggled each time the Receiver sends a **pong**, or once the Transmitter's receive attempt ends (regardless of the result).

75.2 Features Demonstrated

- RF configuration for BLE, Zigbee, Private, and Hybee modes (`tlk_rf_configure`)
- Multiple exchange APIs: Manual, STX/SRX, STR/SRT, BTX/BRX, PTX/PRX
- Transmitter and receiver roles (`CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER` / `_RECEIVER`)
- IRQ-driven and polling-mode packet handling
- Optional pipe switching and SB packet format (Private mode, PTX/PRX only)
- Optional power management integration (`CONFIG_TLK_RF_DEMO_PM`)

75.3 How It Works

75.3.1 Initialization

```
int main(void)
{
    tlk_core_interrupt_enable();

    tlk_gpio_configure(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);

    tlk_gpio_pin_write(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, 1);
```

(continues on next page)

(continued from previous page)

```
    rf_configure(CONFIG_TLK_RF_DEMO_MODE, CONFIG_TLK_RF_DEMO_BITRATE);

#if TLK_IS_ENABLED(CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER)
    transmitter_loop();
#elif TLK_IS_ENABLED(CONFIG_TLK_RF_DEMO_ROLE_RECEIVER)
    receiver_loop();
#endif

    return 0;
}
```

`rf_configure()` builds a `struct tlk_rf_config` based on the selected mode (BLE, Zigbee, Private, Hybee), setting the RF channel, handling mode (IRQ or polling, per `CONFIG_TLK_RF_DEMO_POLLING_MODE`), and mode-specific parameters such as the BLE/Private access code and, for Private mode, the preamble length (derived from the selected bitrate) and optional SB packet format/length.

75.3.2 Exchange APIs

`main.h/main.c` are shared across five `demo_*.c` files, each compiled only when its matching `CONFIG_TLK_RF_DEMO_EXCHANGE_*` option is selected, and each implementing `transmitter_loop()/receiver_loop()`:

- **Manual** (`demo_manual.c`) —Drives the RF state machine directly with `tlk_rf_set_tx_state/tlk_rf_start_tx_state`, polling `tlk_rf_instance.tx_end/rx_end`. Also supports optional mode switching between two configured modes each iteration, and BLE channel switching with fast-settle calibration.
- **STX & SRX** (`demo_stx_srx.c`) —Uses `tlk_rf_start_stx()` to transmit and `tlk_rf_start_srx()` to receive, as two separate blocking calls.
- **STR & SRT** (`demo_str_srt.c`) —Uses `tlk_rf_start_stx2rx()` on the transmitter (send then auto-switch to receive) and `tlk_rf_start_srx2tx()` on the receiver (receive then auto-switch to send).
- **BTX & BRX** (`demo_btx_brx.c`) —Uses `tlk_rf_start_btx()` / `tlk_rf_start_brx()`, the "block" variant of send-then-receive / receive-then-send.
- **PTX & PRX** (`demo_ptx_prx.c`) —Uses `tlk_rf_start_ptx()` / (implicit PRX receive), and is the only mode that supports pipe switching (`CONFIG_TLK_RF_DEMO_PIPE_SWITCHING`) across `UNISDK_RF_PIPE_COUNT` logical pipes, each with its own access code, and the SB packet format (`CONFIG_TLK_RF_DEMO_SB_FORMAT`).

Each variant times the round trip via `tlk_api_time_get_micros()` and reports it, together with RSSI, through `on_pong_received()`.

75.3.3 Callbacks

```
void on_pong_received(void)
{
    ...
    if (ping >= TLK_MS_TO_US(PING_TIMEOUT_MS) || tlk_rf_instance.rx_timeout_
↪error)
```

(continues on next page)

(continued from previous page)

```
    {
        TLK_LOG_INFO(rf_demo, "Error: Ping timeout.");
    }
    else if (tx_payload[0] == rx_payload[0])
    {
        TLK_LOG_INFO(rf_demo, "[%u] Received!", rx_payload[0]);
        TLK_LOG_INFO(rf_demo, "Time: %-4u us | RSSI: %i", ping, tlk_rf_
↪instance.rx_rssi);
        success_packets++;
    }
    else
    {
        TLK_LOG_INFO(rf_demo, "Error: Diff in packets. Expected [%u], but got␣
↪[%u].", tx_payload[0], rx_payload[0]);
    }

    TLK_LOG_INFO(rf_demo, "Count: %-6u | Success count: %u", sent_packets,␣
↪success_packets);
    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);
}
```

`on_ping_sent()`, `on_ping_received()`, `on_pong_sent()`, and `on_pong_received()` log each stage of the exchange; `on_pong_sent()` and `on_pong_received()` also toggle LED2.

Note

Power management When `CONFIG_TLK_RF_DEMO_PM` is enabled, most exchange variants replace the fixed post-exchange delay with `tlk_api_sleep(1000 + 500)` (a 1.5 s low-power sleep, preceded by a short visible delay), and the sample implies `TLK_RF_PM_DEVICE`.

75.4 Configuration Options

Option	Description
CONFIG_TLK_RF_DEMO_PM	Enable PM integration: sleeps between exchange iterations instead of delaying; selects TLK_API_SLEEP and implies TLK_RF_PM_DEVICE.
CONFIG_TLK_RF_DEMO_MODE_{BLE,ZIGBEE,PRIVATE,HYBEE}	Selects the RF protocol mode (choice).
CONFIG_TLK_RF_DEMO_BITRATE_{1Z 250K,500K,1M,2M}	Selects the bitrate (choice); available options depend on the selected mode.
CONFIG_TLK_RF_DEMO_ROLE_{TRANS RECEIVER}	Selects the device role (choice).
CONFIG_TLK_RF_DEMO_EXCHANGE_{M STX_SRX,STR_SRT,BTX_BRX,PTX_PRX}	Selects the exchange API used for the ping-pong loop (choice).
CONFIG_TLK_RF_DEMO_POLLING_MOI	(Manual exchange only) Use polling mode instead of IRQ-driven handling.
CONFIG_TLK_RF_DEMO_MODE_SWITCH	(Manual exchange only) Switch between the primary mode and a second configured mode each iteration.
CONFIG_TLK_RF_DEMO_SECOND_MODE / CONFIG_TLK_RF_DEMO_SECOND_BITR	(Manual + mode switching only) Mode/bitrate of the secondary configuration used when switching.
CONFIG_TLK_RF_DEMO_CHANNEL_SWI	(Manual exchange, BLE mode, not combined with mode switching) Alternate between BLE channels 17 and 25 each iteration, using fast-settle calibration.
CONFIG_TLK_RF_DEMO_SB_FORMAT	(Private mode, PTX/PRX exchange) Use the SB packet format.
CONFIG_TLK_RF_DEMO_PIPE_SWITCH	(Private mode, PTX/PRX exchange) Cycle transmissions across the device's logical pipes, each with a distinct access code.

75.5 Build & Run

```
west tl-build samples/rf_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/rf_demo.bin
```

Note

Two boards are required to observe the ping-pong exchange: flash one with CONFIG_TLK_RF_DEMO_ROLE_TRANSMITTER and the other with CONFIG_TLK_RF_DEMO_ROLE_RECEIVER, using matching mode/bitrate/exchange-API configuration.

Source code: samples/rf_demo/main.c

CHAPTER

SEVENTYSIX

IRQ NESTING SAMPLE

Note

Document Status: DRAFT —Content is being finalized

76.1 Overview

The `irq_demo` sample (source directory `samples/irq_demo`) demonstrates interrupt management, prioritization, and nesting using the PLIC (Platform-Level Interrupt Controller), Machine Software Interrupts (PLIC SW / MSI), and Machine Timer Interrupts (MTIMER / MTI).

Note

Source directory name The sample' s source directory is `samples/irq_demo` (project name `IRQ_Demo`), not `irq_nesting_demo`. Build and download paths below use the actual directory name.

76.2 Features Demonstrated

- PLIC software interrupt (MSI) setup and callback registration (`tlk_plic_sw_register_callback`)
- Machine timer interrupt (MTI) setup and callback registration (`tlk_mtimer_irq_register_callback`)
- PLIC external interrupt (MEI) priority configuration across three tiers (`tlk_plic_set_priority`)
- Optional nested/preemptive interrupt handling via `tlk_plic_enable_preempt()`
- Cross-triggering of interrupts from within handlers to visualize nesting depth

76.3 How It Works

76.3.1 Initialization

1. **MSI Configuration** —Enables the PLIC_SW subsystem and registers `tlk_sample_msi_handler` as its callback.

2. **MTI Configuration** —Registers `tlk_sample_mti_handler` as the machine timer interrupt callback.
3. **MEI Configuration** —Sets PLIC priorities for three sources:
   - `TLK_SAMPLE_PLIC_SRC_LOW` (vector 12) → `TLK_IRQ_PRI_LEV1`
   - `TLK_SAMPLE_PLIC_SRC_HIGH` (vector 13) → `TLK_IRQ_PRI_LEV2`
   - `UNISDK_PLIC_IRQ_NUM_UART0` → `TLK_IRQ_PRI_LEV3` (used for debug logging output)
4. **Preemption** —If both `CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC` and `CONFIG_TLK_IRQ_DEMO_PLIC_PREEMPT` are enabled, `tlk_plic_enable_preempt()` is called to allow nested PLIC interrupt execution.
5. **Global Enable** —Enables the MSIE, MTIE and MEIE bits in `mie`, then calls `tlk_core_interrupt_enable()`.

76.3.2 Cascading Nesting Cycle

Based on the `TLK_IRQ_DEMO_IRQ_BASE` choice, `main()` triggers one of the primary interrupt sources to kick off the cascade:

```
#if TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC_SW)
    tlk_plic_sw_set_pending();
#elif TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_MTIMER)
    tlk_mtimer_set_mtime_compare(tlk_mtimer_get_mtime());
#elif TLK_IS_ENABLED(CONFIG_TLK_IRQ_DEMO_IRQ_BASE_PLIC)
    tlk_plic_set_pending(TLK_SAMPLE_PLIC_SRC_LOW);
#endif
```

Once inside a handler, it purposely cross-triggers the other interrupt sources by setting their pending registers (`tlk_plic_set_pending`, `tlk_plic_sw_set_pending`, or writing `mtimecmp`). To make the nesting visible over UART, each handler:

- increments a shared `tlk_log_depth` counter and indents its log output accordingly,
- logs the current `mie` bits via `tlk_sample_log_mie()`,
- uses a local `static volatile uint8_t processed` guard (in the PLIC handlers) to avoid re-triggering itself indefinitely, and
- decrements `tlk_log_depth` before returning.

The PLIC low/high handlers additionally call `tlk_api_time_delay(100)` before returning, giving the newly-pended higher/lower-priority interrupt time to become pending before the current handler exits.

76.4 Configuration Options

All options are exposed via **Kconfig** under *IRQ Sample configuration*.

Option	Description
CONFIG_TLK_IRQ_DEMO_IR(	Use the PLIC software interrupt (MSI) as the cascade's starting trigger (default choice)
CONFIG_TLK_IRQ_DEMO_IR(	Use the machine timer interrupt (MTI) as the cascade's starting trigger
CONFIG_TLK_IRQ_DEMO_IR(	Use the PLIC external low-priority interrupt (MEI) as the cascade's starting trigger
CONFIG_TLK_IRQ_DEMO_PL	Enable PLIC preemption (nested execution) of external interrupts; only selectable when TLK_IRQ_DEMO_IRQ_BASE_PLIC is chosen

TLK_IRQ_DEMO_IRQ_BASE_PLIC_SW, _MTIMER, and _PLIC form a single-choice group (exactly one is active at a time); PLIC_SW is the default.

76.5 Build & Run

```
west tl-build samples/irq_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/irq_demo.bin
```

Source code: samples/irq_demo/main.c

CHAPTER

SEVENTYSEVEN

STIMER SAMPLE

Note

Document Status: DRAFT —Content is being finalized

77.1 Overview

The `stimer_demo` sample demonstrates the usage of the System Timer (STIMER) driver on Telink RISC-V MCUs: timer initialization, interrupt configuration, periodic callbacks, and integration with GPIO and the sleep API.

77.2 Features Demonstrated

- System timer configuration and start (`tlk_stimer_enable`, `tlk_stimer_set_tick`, `tlk_stimer_set_auto_mode`)
- Timer interrupt setup and callback registration (`tlk_stimer_register_callback`)
- Periodic interrupt generation using capture-compare (`tlk_stimer_irq_set_capture`)
- GPIO control from both the main loop and interrupt context
- Low-power sleep usage in the main loop while the timer interrupt continues in the background

77.3 How It Works

77.3.1 Initialization

1. **GPIO Configuration** —LED0 and LED1 are both configured as outputs. LED0 is toggled in the main loop; LED1 is toggled inside the timer interrupt callback.
2. **System Timer Setup** —The timer is enabled (`tlk_stimer_enable()`), its tick counter reset to 0 (`tlk_stimer_set_tick(0)`), and auto mode enabled (`tlk_stimer_set_auto_mode(true)`) for continuous free-running operation.
3. **Interrupt Configuration** —A callback (`timer_cb`) is registered via `tlk_stimer_register_callback()`. The first capture-compare event is scheduled 500 ms out, the STIMER IRQ mask is enabled, the STIMER PLIC interrupt is enabled, and global interrupts are enabled with `tlk_core_interrupt_enable()`.

```
tlk_stimer_enable();
tlk_stimer_set_tick(0);
tlk_stimer_set_auto_mode(true);
tlk_stimer_register_callback(timer_cb);

tlk_stimer_irq_set_capture(tlk_stimer_get_tick() + TLK_SYSTEM_TIMER_TICK_1MS␣
↪* 500);
tlk_stimer_irq_set_mask(TLK_STIMER_IRQ_MASK_TIMER_IRQ_EN);
tlk_plic_interrupt_enable(UNISDK_PLIC_IRQ_NUM_STIMER);

tlk_core_interrupt_enable();
```

77.3.2 Main Loop

The main loop toggles LED0 and sleeps for 1 second between toggles:

```
while (1)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
    tlk_api_sleep(TLK_SEC_TO_MS(1));
}
```

77.3.3 Timer Callback

`timer_cb()` runs on every STIMER interrupt (every 500 ms): it reschedules the next capture compare event, clears the interrupt status, and toggles LED1.

```
void timer_cb(void)
{
    tlk_stimer_irq_set_capture(tlk_stimer_get_tick() + TLK_SYSTEM_TIMER_TICK_
↪1MS * 500);
    tlk_stimer_irq_clr_status(TLK_STIMER_IRQ_STATUS_TIMER_IRQ);
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
}
```

Because the main loop toggles LED0 once per second while the timer interrupt toggles LED1 twice per second (500 ms period), LED0 and LED1 blink at visibly different rates, demonstrating that the STIMER interrupt keeps firing independently of the main loop's sleep/wake cycle.

77.4 Configuration Options

Option	Description
`CONFIG_TLK_ST`	Enable the STIMER interrupt demo: registers `timer_cb`, schedules periodic capture-compare events, and toggles LED1 from interrupt context. Enabled by default (`def_bool y`) and selects `TLK_SYSTEM_TIMER_IRQ_ENABLE`.

77.5 Build & Run

```
west tl-build samples/stimer_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/stimer_demo.bin
```

Source code: `samples/stimer_demo/main.c`

CHAPTER

SEVENTYEIGHT

WATCHDOG SAMPLE

Note

Document Status: DRAFT —Content is being finalized

78.1 Overview

The `wdt_demo` sample demonstrates the usage of the Watchdog Timer (WDT) driver, including detecting a watchdog-triggered reboot, feeding the watchdog to pass its window, and letting it expire to force a reset.

78.2 Features Demonstrated

- Detecting the previous reboot's cause via `tlk_sys_get_reboot_reason` (`TLK_SYS_REBOOT_REASON_WDT`)
- Starting the watchdog with a timeout (`tlk_wdt_start`) and feeding it (`tlk_wdt_feed`)
- Optional GPIO indication of demo progress (LED0/LED1)
- Optional low-power sleep while the watchdog is running (`CONFIG_TLK_WDT_DEMO_PM`)
- Optional watchdog auto-start from startup code (`CONFIG_TLK_WDT_DEMO_STARTUP`)

78.3 How It Works

On each boot:

1. **Reboot Reason Check** —Reads `tlk_sys_get_reboot_reason()` and logs whether the reset was caused by the watchdog or by another source.
2. **GPIO Indication (optional)** —If `CONFIG_TLK_WDT_DEMO_GPIO_INDICATION` is enabled, LED0 is configured as output and driven high, and LED1 is configured as output and driven low.
3. **Initial Delay** —Blocks for 2 seconds (`tlk_api_time_delay`).
4. **Watchdog Start** —If `CONFIG_TLK_WDT_DEMO_STARTUP` is enabled, the watchdog is assumed to already be running (started from startup code via `TLK_WDT_STARTUP_ENABLE`), so the sample just feeds it (`tlk_wdt_feed()`). Otherwise it starts the watchdog itself with a 5-second timeout (`tlk_wdt_start(TLK_SEC_TO_MS(5))`).

5. **Pass the WDT Window** —Waits 2 seconds, either via blocking delay (`tlk_api_time_delay`) or, if `CONFIG_TLK_WDT_DEMO_PM` is enabled, via low-power sleep (`tlk_api_sleep`) —demonstrating that the watchdog configuration survives a sleep cycle.
6. **LED1 On (optional)** —If GPIO indication is enabled, LED1 is driven high to show that the WDT window was passed successfully without a reset.
7. **Final Feed & Expire** —Feeds the watchdog one last time, then enters an infinite empty loop. Because nothing feeds the watchdog again, it eventually expires and resets the chip, and on the next boot `tlk_sys_get_reboot_reason()` reports `TLK_SYS_REBOOT_REASON_WDT`.

```
// Feed the WDT and wait until it fires.
tlk_wdt_feed();

while (1)
{
}
```

78.4 Configuration Options

All options are exposed via **Kconfig** under *WDT Sample configuration*.

Option	Description
`CONFIG_TLK_WDT_DEMO_GPI`	Enable LED0/LED1 GPIO indication of demo progress (selects `TLK_GPIO`)
`CONFIG_TLK_WDT_DEMO_PM`	Use `tlk_api_sleep` instead of a blocking delay to pass the WDT window, exercising PM (selects `TLK_API_SLEEP`, `TLK_PM`)
`CONFIG_TLK_WDT_DEMO_STA`	Start the watchdog from startup code instead of from `main()` (selects `TLK_WDT_STARTUP_ENABLE`)

78.5 Build & Run

```
west tl-build samples/wdt_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/wdt_demo.bin
```

Source code: `samples/wdt_demo/main.c`

CHAPTER

SEVENTYNINE

USB CDC SAMPLE

Note

Document Status: DRAFT —Content is being finalized

79.1 Overview

The USB CDC demo, built on the TinyUSB framework, demonstrates a USB CDC (virtual serial port) device that echoes back any data received from the host.

Note

Source directory name The sample currently lives at `samples/tinyusb_demos/usb_device/cdc_demo` (project name `USB_Demo`), not `samples/tinyusb_demos/usb_cdc_demo`. Build and download paths below use the actual directory.

79.2 Features Demonstrated

- TinyUSB device stack initialization (`tud_init`) and task servicing (`tud_task`)
- USB device/configuration/string descriptor callbacks for a single CDC interface (`tud_descriptor_device_cb`, `tud_descriptor_configuration_cb`, `tud_descriptor_string_cb`)
- CDC data echo using `tud_cdc_available`, `tud_cdc_read`, `tud_cdc_write`, `tud_cdc_write_flush`
- Device lifecycle callbacks: `tud_mount_cb`, `tud_umount_cb`, `tud_suspend_cb`, `tud_resume_cb`
- CDC line coding/state callbacks: `tud_cdc_line_coding_cb`, `tud_cdc_line_state_cb`

79.3 How It Works

79.3.1 Descriptors

The device descriptor uses the Miscellaneous class (`TUSB_CLASS_MISC`) with the Interface Association Descriptor subclass/protocol, which is the recommended setup for

CDC so host drivers (Windows/Linux) enumerate it correctly. Vendor ID, product ID, device version, manufacturer, product, serial number, and interface name strings are all pulled from Kconfig (`CONFIG_TLK_USB_VENDOR_ID`, `CONFIG_TLK_USB_PRODUCT_ID`, `CONFIG_TLK_USB_DEVICE_VERSION`, `CONFIG_TLK_USB_DEVICE_MANUFACTURER`, `CONFIG_TLK_USB_DEVICE_PRODUCT_NAME`, `CONFIG_TLK_USB_DEVICE_SERIAL_NUMBER`, `CONFIG_TLK_USB_DEVICE_INTERFACE_NAME`). The single CDC interface's notification/OUT/IN endpoint numbers come from `CONFIG_TLK_USB_CDC_NOTIF_EP`, `CONFIG_TLK_USB_CDC_OUT_EP`, and `CONFIG_TLK_USB_CDC_IN_EP`.

79.3.2 Main Loop

```
int main(void)
{
    tud_init(BOARD_TUD_RHPORT); // Initialize USB Device Stack

    while (1)
    {
        tud_task(); // Device task: Handle USB events (MUST be called␣
↪frequently)
        cdc_task(); // Application task: Handle Serial data
    }

    return 0;
}
```

79.3.3 Echo Task

`cdc_task()` checks that a host is connected and that RX data is available, reads up to 64 bytes from the CDC RX FIFO, and immediately writes the same bytes back out, flushing to force transmission without waiting for the buffer to fill:

```
void cdc_task(void)
{
    if (tud_cdc_connected())
    {
        if (tud_cdc_available())
        {
            uint8_t buf[64];
            uint32_t count = tud_cdc_read(buf, sizeof(buf));

            tud_cdc_write(buf, count);
            tud_cdc_write_flush();
        }
    }
}
```

79.4 Configuration Options

Option	Description
`CONFIG_TLK_USB_CDC_DEMO`	Enables the sample (selects `TLK_USB`); enabled by default

The device/interface identity (vendor/product IDs, strings) and endpoint numbers are configured via the shared `TLK_USB` Kconfig options referenced above rather than demo-specific symbols.

79.5 Build & Run

```
west tl-build samples/tinyusb_demos/usb_device/cdc_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/cdc_demo.bin
```

Source code: `samples/tinyusb_demos/usb_device/cdc_demo/main.c`

CHAPTER

EIGHTY

RANDOM SAMPLE

80.1 Overview

The `random_demo` sample demonstrates the usage of the TLK Random API. It generates a configurable sequence of random numbers and logs them to the console in a loop.

80.2 Features Demonstrated

- Software random number generation (`tlk_random()`)
- Optional hardware TERO-based random source (`tlk_random_tero()`, TL721X only)
- Delay vs. sleep between iterations (`tlk_api_delay` / `tlk_api_sleep`)

80.3 How It Works

80.3.1 Main Loop

On every loop iteration the sample generates `CONFIG_TLK_RANDOM_DEMO_NUMBERS_COUNT` random values in the range `[0, CONFIG_TLK_RANDOM_DEMO_MAX_NUMBER]` using `tlk_random()` and logs each one:

```
int main(void)
{
    while (1)
    {
        TLK_LOG_INFO(random_demo, "Random numbers:\n");

        for (uint8_t i = 0; i < CONFIG_TLK_RANDOM_DEMO_NUMBERS_COUNT; i++)
        {
            TLK_LOG_INFO(random_demo,
                         "\t%u: %u\n",
                         (i + 1),
                         tlk_random() % (CONFIG_TLK_RANDOM_DEMO_MAX_NUMBER +␣
↪1));
        }

        TLK_LOG_INFO(random_demo, "\n");

#ifndef CONFIG_TLK_RANDOM_DEMO_SLEEP
```

(continues on next page)

(continued from previous page)

```
        tlk_api_delay(TLK_SEC_TO_US(4));
#else
        tlk_api_sleep(TLK_SEC_TO_MS(4));
#endif
    }
}
```

Between iterations, the sample either busy-delays for 4 seconds via `tlk_api_delay`, or — if `CONFIG_TLK_RANDOM_DEMO_SLEEP` is enabled —enters low-power sleep for 4 seconds via `tlk_api_sleep`.

Note

TERO Module (TL721X only) The sample can optionally repeat the same sequence using `tlk_random_tero()`, a hardware TERO-based random source available only on **TL721X**. It is disabled by default; to enable it, edit `main.c` and set `#define USE_TERO_MODULE 1`. This is a source-level toggle, not a Kconfig option.

80.4 Configuration Options

Option	Type	Default	Description
CONFIG_TLK_RANDOM_DEMO	int	10	How many numbers to generate per iteration. Range: 1–100.
CONFIG_TLK_RANDOM_DEMO	int	10	Upper bound (inclusive) of the generated values.
CONFIG_TLK_RANDOM_DEMO	bool	n	Use sleep instead of delay between iterations. Automatically selects TLK_API_SLEEP and implies TLK_RANDOM_PM.

`TLK_RANDOM_DEMO` itself auto-selects `TLK_RANDOM` and `TLK_API_TIME`, so no manual selection of those is required.

80.5 Build & Run

```
west tl-build samples/random_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/random_demo.bin
```

Source code: `samples/random_demo/main.c`

CHAPTER

EIGHTYONE

NV STORAGE SAMPLE

81.1 Overview

The `nv_storage_demo` sample demonstrates the non-volatile storage abstraction layer: enumerating the available storage devices and performing a read/erase/write/read round-trip against flash.

81.2 Features Demonstrated

- Storage device enumeration (`tlk_storage_get_next_device`)
- Locating the storage device backing a given address range (`tlk_storage_find_device`)
- Reading and writing raw flash data (`tlk_storage_read`, `tlk_storage_write`)
- Log output via `TLK_LOG_INFO` / `TLK_LOG_HEXDUMP_INFO`

81.3 How It Works

81.3.1 Device Enumeration

At startup, the sample walks the list of registered storage devices and logs each one's name, base address, size, page size, clean (erased) byte pattern, and erase/read/write capability flags:

```
for (struct tlk_storage_device* dev = tlk_storage_get_next_device(NULL); dev !
↪= NULL;
     dev                            = tlk_storage_get_next_device(dev))
{
    TLK_LOG_INFO(nv_storage_demo, "device: %s", dev->name);
    TLK_LOG_INFO(nv_storage_demo, "\tbase address: 0x%zx", dev->base);
    TLK_LOG_INFO(nv_storage_demo, "\tsize: %zu (0x%zx)", dev->size, dev->
↪size);
    TLK_LOG_INFO(nv_storage_demo, "\tpage: %zu", dev->page);
    TLK_LOG_INFO(nv_storage_demo, "\tclean pattern: 0x%02x", dev->clean_byte);
    TLK_LOG_INFO(nv_storage_demo,
                 "\tcapable: %c%c%c",
                 dev->erase ? 'E' : '_',
                 dev->read ? 'R' : '_',
```

(continues on next page)

(continued from previous page)

```
                dev->write ? 'W' : '_');
}
```

81.3.2 Read / Write Round-Trip

The sample picks a 1024-byte test region near the top of chip ROM (`UNISDK_CHIP_MEMORY_ROM_STARTADDR + UNISDK_CHIP_MEMORY_ROM_SIZE * 1024 - 5000`) and:

1. Locates the storage device covering that region with `tlk_storage_find_device`.
2. Reads the current contents with `tlk_storage_read` and hex-dumps them.
3. Checks whether the region is in its "clean" (erased) state by comparing every byte to `s_dev->clean_byte`.
4. Builds new test data —an incrementing byte pattern if the region was clean, or each byte incremented by one otherwise.
5. Writes the new data with `tlk_storage_write`.
6. Reads the region back and hex-dumps it again to confirm the write.

Any failure at each step (`tlk_storage_find_device` returning `NULL`, or a non-zero result from `tlk_storage_read`/`tlk_storage_write`) is logged and the sample stops early.

Note

Device-agnostic API The sample never assumes a specific flash chip —it discovers devices via `tlk_storage_get_next_device`/`tlk_storage_find_device`, so the same code works across boards with different storage layouts.

81.4 Configuration Options

The sample itself exposes no user-facing Kconfig options —its `Kconfig` only auto-selects the storage and logging subsystems:

Option	Description
`TLK_NV_STORAGE`	Always enabled (`def_bool y`) when this sample is built; selects `TLK_NV_STORAGE` and `TLK_DEBUG_LOG_CONSOLE`.

81.5 Build & Run

```
west tl-build samples/nv_storage_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/nv_storage_demo.bin
```

Source code: `samples/nv_storage_demo/main.c`

CHAPTER

EIGHTYTWO

SYSTEM LOG SAMPLE

82.1 Overview

The `log_demo` sample (under `samples/system_demo/`) demonstrates the debug logging subsystem, emitting a log message roughly once per second. Its behavior adapts to whether the Task Planner is enabled and, if so, which execution mode it runs in.

Note

README/sample mismatch The `README.md` shipped alongside this sample currently describes the **Task Planner** demo (GPIO/LED toggling, software timers), not this log sample. This page is based directly on `main.c` and `Kconfig` instead —see *Task Planner Sample* for the sample that README actually documents.

82.2 Features Demonstrated

- Logging via `TLK_LOG_CREATE` / `TLK_LOG_INFO` / `tlk_log`
- Log rotation and the log interface loop (`tlk_logrotate`, `tlk_log_iface_loop`) when running without the Task Planner
- Integration with the Task Planner's Loop mode (`user_setup` / `user_loop`) and Scheduler mode

82.3 How It Works

82.3.1 Without Task Planner

If `CONFIG_TLK_TASK_PLANNER` is disabled, the sample owns its own `main()` and drives the logging pump directly:

```
#if TLK_IS_DISABLED(CONFIG_TLK_TASK_PLANNER)

static uint32_t last = 0;
int             main(void)
{
    while (1)
    {
```

(continues on next page)

(continued from previous page)

```
        tlk_log_iface_loop();
        tlk_logrotate();
        uint32_t now = tlk_api_time_get_micros();
        if (now - last >= 1000000)
        {
            last = now;
            tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB");
        }
    }
}

#endif
```

`tlk_log_iface_loop()` and `tlk_logrotate()` are called on every iteration to service the log transport and rotate log storage; the actual message is only emitted once a second.

82.3.2 With Task Planner —Loop Mode

When `CONFIG_TLK_TASK_PLANNER_MODE_LOOP` is enabled, the sample instead defines `user_setup()`/`user_loop()`, which the platform dispatcher calls repeatedly:

```
#elif TLK_IS_ENABLED(CONFIG_TLK_TASK_PLANNER_MODE_LOOP)
void user_setup(void) {}

void user_loop(void)
{
    static uint32_t last = 0;
    uint32_t        now  = tlk_api_time_get_micros();
    if (now - last >= 1000000)
    {
        last = now;
        tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB from Telink Loop
 ↪");
    }
}
```

82.3.3 With Task Planner —Scheduler Mode

When `CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER` is enabled, `user_loop()` is invoked periodically by the default task and simply logs on every call (the periodic interval is controlled by the Task Planner's own configuration, not by the sample):

```
#elif TLK_IS_ENABLED(CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER)
void user_setup(void) {}

void user_loop(void)
{
    tlk_log(demo_log, TLK_LOG_LEVEL_ERROR, "Test log USB with Telink Scheduler
 ↪");
}
#endif
```

A C++ variant of the same logic (`main.cpp`) is compiled instead of `main.c` when `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled.

82.4 Configuration Options

The sample's own `Kconfig` only pulls in the modules it needs; the execution-mode behavior above is controlled by the Task Planner subsystem's own options.

Option	Description
`CONFIG_TLK_GPIO`	Always enabled (`def_bool y`) —required by the shared board pinout headers.
`CONFIG_TLK_DEBUG_LOG_(`	Always enabled (`def_bool y`) —routes log output to the console/log interface.
`CONFIG_TLK_BLE_CONTROI`	Defaults to `n` for this sample.
`CONFIG_TLK_TASK_PLANNI`	(Task Planner subsystem) When disabled, the sample runs its own `main()` loop; when enabled, behavior depends on the mode below.
`CONFIG_TLK_TASK_PLANNI`	(Task Planner subsystem) Cooperative loop execution —logs once per second, gated in `user_loop()`.
`CONFIG_TLK_TASK_PLANNI`	(Task Planner subsystem) Task scheduler execution —logs unconditionally on each `user_loop()` invocation, at the scheduler's configured period.

82.5 Build & Run

```
west tl-build samples/system_demo/log_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/log_demo.bin
```

Source code: `samples/system_demo/log_demo/main.c`

CHAPTER

EIGHTYTHREE

TASK PLANNER SAMPLE

83.1 Overview

The `task_planner_demo` sample (under `samples/system_demo/`) demonstrates the Task Planner subsystem on the Telink RISC-V MCU platform: basic GPIO output driven through cooperative loop execution, task-scheduler execution, and an optional software timer, depending on which Task Planner mode is enabled.

83.2 Features Demonstrated

- GPIO output configuration and control (`tlk_gpio_configure`, `tlk_gpio_pin_toggle`)
- Cooperative loop execution model (`user_loop()`, `TLK_REGISTER_LOOP`)
- Scheduler-based task execution (`TASK_SPAWN`, `TASK_DELAY`)
- Periodic execution using software timers (`TLK_SW_TIMER_REGISTER`, `tlk_task_sw_timer_start`), when enabled

83.3 How It Works

83.3.1 Initialization (`user_setup()`)

Called once during system startup. LED0, LED1, and LED2 are configured as GPIO outputs; if `CONFIG_TLK_SOFTWARE_TIMERS` is enabled, the periodic software timer `test_timer1` is also started:

```
void user_setup(void)
{
    tlk_gpio_configure(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_configure(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN, TLK_GPIO_OUTPUT);

#if TLK_IS_ENABLED(CONFIG_TLK_SOFTWARE_TIMERS)
    tlk_task_sw_timer_start(&test_timer1);
#endif
}
```

83.3.2 Mode 1: Loop Mode (CONFIG_TLK_TASK_PLANNER_MODE_LOOP)

`user_loop()` is executed repeatedly by the platform dispatcher (`while (1) { tlk_loop(); }`) and toggles LED0 every second:

```
void user_loop(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
    tlk_api_time_delay(TLK_SEC_TO_US(1));
}

static void loop1(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_api_time_delay(TLK_SEC_TO_US(2));
}

TLK_REGISTER_LOOP(loop1)
```

`loop1()` is registered as an additional loop handler via `TLK_REGISTER_LOOP` and toggles LED1 every 2 seconds, resembling the Arduino-style `loop()` model.

83.3.3 Mode 2: Scheduler Mode (CONFIG_TLK_TASK_PLANNER_MODE_SCHEDULER)

`user_loop()` is called from the default task (its period set by `CONFIG_TLK_TASK_PLANNER_DEFAULT_TASK_LOOP_PERIOD_MS`) and just toggles LED0. `loop1()` is instead defined as a cooperatively-scheduled task via `TASK_SPAWN`, toggling LED1 and rescheduling itself every 2 seconds:

```
void user_loop(void)
{
    tlk_gpio_pin_toggle(PINMUX_LED_0_PORT, PINMUX_LED_0_PIN);
}

static void loop1(tTask* t)
{
    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    TASK_DELAY(t, TASK_MS(2000));
}

TASK_SPAWN(test_task1, loop1);
```

83.3.4 Optional: Software Timer

When `CONFIG_TLK_SOFTWARE_TIMERS` is enabled, a periodic software timer (`test_timer1`) is statically registered via `TLK_SW_TIMER_REGISTER` and toggles LED2 every second, independent of the loop/scheduler mode:

```
static void sw_timer_loop(void* arg)
{
    (void) arg;
    tlk_gpio_pin_toggle(PINMUX_LED_2_PORT, PINMUX_LED_2_PIN);
}
```

(continues on next page)

(continued from previous page)

```
TLK_SW_TIMER_REGISTER(test_timer1, sw_timer_loop, SW_TIMER_MODE_PERIODIC,␣
↪TASK_MS(1000));
```

A C++ variant of the same logic (`main.cpp`) is compiled instead of `main.c` when `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled.

Note

LED-to-mode mapping

LED	Controlled by	Mode dependency
LED0	`user_loop()`	Loop / Scheduler
LED1	`loop1()`	Loop / Scheduler
LED2	Software timer callback	`CONFIG_TLK_SOFTWARE_TIMERS`

83.4 Configuration Options

The sample's own `Kconfig` just enables the Task Planner subsystem; execution mode and timing are controlled by the Task Planner subsystem's own options.

Option	Description
`CONFIG_TLK_TASK_PLANNER_DEM`	Always enabled (`def_bool y`) when this sample is built; selects `TLK_TASK_PLANNER`.
`CONFIG_TLK_TASK_PLANNER_MOD`	(Task Planner subsystem) Enables cooperative loop-based execution.
`CONFIG_TLK_TASK_PLANNER_MOD`	(Task Planner subsystem) Enables task scheduler-based execution.
`CONFIG_TLK_TASK_PLANNER_DEF`	(Task Planner subsystem, scheduler mode only) Enables periodic execution of `user_loop()` from the default task.
`CONFIG_TLK_TASK_PLANNER_DEF`	(Task Planner subsystem) Execution period (ms) for `user_loop()` in scheduler mode; default `500`.
`CONFIG_TLK_SOFTWARE_TIMERS`	(Task Planner subsystem) Enables the periodic software timer that toggles LED2.

83.5 Build & Run

```
west tl-build samples/system_demo/task_planner_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/task_planner_demo.bin
```

Source code: `samples/system_demo/task_planner_demo/main.c`

CHAPTER

EIGHTYFOUR

AUDIO SAMPLE

84.1 Overview

The `audio_demo` sample demonstrates the Audio driver's record-and-playback pipeline: it records audio from a digital microphone (DMIC) into flash-backed storage using DMA linked-list buffers, then plays the recorded audio back out through the audio output path.

84.2 Features Demonstrated

- Audio input (recording) configuration via `tlk_audio_input_configure` / `tlk_audio_input_enable` / `tlk_audio_input_disable`
- Audio output (playback) configuration via `tlk_audio_output_configure` / `tlk_audio_output_enable` / `tlk_audio_output_disable`
- Double-buffered DMA linked-list transfers (`struct tlk_dma_ll_node`) for both RX and TX
- DMA interrupt callbacks (`tlk_dma_chn_add_callback`) that feed/drain the buffers to/from storage on every DMA transfer-complete interrupt
- Chip-specific audio pinmux configuration for AMIC bias, DMIC data/clock, and SDM (sigma-delta modulator) pins, selected per SoC (`CONFIG_TLK_CORE_TL321X`, `CONFIG_TLK_CORE_TL721X`, `CONFIG_TLK_CORE_B92`)
- Non-volatile storage read/write/erase (`tlk_storage_write`, `tlk_storage_read`, `tlk_storage_erase`) used as the recording buffer
- Benchmarking of erase/record/playback timing via `tlk_benchmark_begin` / `tlk_benchmark_end` / `tlk_benchmark_elapsed`

84.3 How It Works

84.3.1 Initialization

1. **Storage Setup** —A storage region is reserved at the top of ROM (`storage_addr`, sized `BUFFER_COUNT * BUFFER_SIZE` = 256 KiB) and located with `tlk_storage_find_device`.
2. **Pinmux Configuration** —Depending on the target core, AMIC bias, DMIC data/clock pins, and SDM P/N pins are set up (e.g. for `CONFIG_TLK_CORE_B92`, only DMIC pins are configured; the AMIC/SDM structs are left empty).

3. **DMA Linked Lists** —Two double-buffered linked lists are built: `tx_buffer0`/`tx_buffer1` for playback and `rx_buffer0`/`rx_buffer1` for recording, each node covering one 1024-byte `audio_buffer` slot.
4. **Audio Input/Output Configuration** —`tlk_audio_input_configure` and `tlk_audio_output_configure` are called with `TLK_AUDIO_48K` sample rate, 16-bit data width, and the left I2S channel, wired to DMA channels obtained from `tlk_dma_chn_request()`.
5. **DMA Callbacks** —`on_rx_buffer_fill` and `on_tx_buffer_sent` are registered as DMA transfer-complete callbacks; they call `save_buffer()` / `fill_buffer()` to move data between the DMA buffer and flash storage on each completed block.

84.3.2 Main Loop

```
while (1)
{
    audio_tx_buffer_index = 0;
    audio_rx_buffer_index = 0;

    is_rx_done = 0;
    is_tx_done = 0;

    tlk_benchmark_begin(&span);
    tlk_storage_erase(storage_addr, STORAGE_SIZE);
    tlk_benchmark_end(&span);

    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_benchmark_begin(&span);

    tlk_audio_input_enable();
    while (!is_rx_done) { }
    tlk_benchmark_end(&span);

    tlk_gpio_pin_toggle(PINMUX_LED_1_PORT, PINMUX_LED_1_PIN);
    tlk_benchmark_begin(&span);

    fill_buffer();
    tlk_audio_output_enable();
    while (!is_tx_done) { }
    tlk_benchmark_end(&span);
}
```

Each cycle: erase the storage region, record `BUFFER_COUNT` (256) blocks of `BUFFER_SIZE` (1024) bytes from the DMIC into storage (recording stops automatically once `audio_rx_buffer_index == BUFFER_COUNT`, disabling audio input), then play the recorded blocks back out (stopping once `audio_tx_buffer_index == BUFFER_COUNT`, disabling audio output). LED1 is toggled between the record and playback phases as a visual marker, and elapsed erase/record/playback time is logged via the benchmark span.

Note

Per-Core Pinmux The sample selects its AMIC/DMIC/SDM pin assignments at compile time

based on which core config (CONFIG_TLK_CORE_TL321X, CONFIG_TLK_CORE_TL721X, or CONFIG_TLK_CORE_B92) is active, since the audio pin layout differs between chips.

84.4 Configuration Options

Option	Description
CONFIG_TLK_AUDIO_I	Enables the sample; selects TLK_AUDIO, TLK_NV_STORAGE, and TLK_DMA_IRQ_HANDLER (default y, not user-facing)

Note

This sample has no user-selectable Kconfig choices beyond the auto-selected dependencies above —behavior is fixed at record-then-playback, with pin assignments chosen automatically based on the target core.

84.5 Build & Run

```
west tl-build samples/audio_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/audio_demo.bin
```

Source code: samples/audio_demo/main.c

CHAPTER

EIGHTYFIVE

MBEDTLS SAMPLE

85.1 Overview

The `mbedtls_demo` sample demonstrates using mbedTLS's PSA Crypto API on-device to generate random data and perform ECDSA (elliptic-curve) key generation, signing, and verification. The sample is built twice from the same logic: `main.c` (plain C) and `main.cpp` (a C++ variant that wraps the same headers in `extern "C"`), selected at build time by whether C++ wrappers are enabled.

85.2 Features Demonstrated

- PSA Crypto initialization (`psa_crypto_init`)
- Random number generation (`psa_generate_random`)
- EC key pair generation on the SECP256R1 curve (`psa_generate_key` with `PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1)`, 256-bit)
- Exporting private and public key material (`psa_export_key`, `psa_export_public_key`)
- ECDSA signing and verification over SHA-256 (`psa_sign_hash`, `psa_verify_hash` with `PSA_ALG_ECDSA(PSA_ALG_SHA_256)`)
- Key cleanup (`psa_destroy_key`)
- Hex-dump logging helper (`tlk_str_utils_convert_buf_to_hex` in `main.c`, `tlk_str_utils_buf_to_hex` in `main.cpp`)

85.3 How It Works

85.3.1 Initialization

1. `tlk_core_interrupt_enable()` is called first to enable interrupts.
2. The mbedTLS version string (`MBEDTLS_VERSION_STRING_FULL`) is logged.
3. `psa_crypto_init()` is called; if it fails, an error is logged and no tests run.

85.3.2 Test Sequence

Once PSA Crypto is initialized, two tests run in sequence, aborting early (via `break` out of a `do { } while(0)` block) if any step fails:

1. **Random Testing** —Generates 31 random bytes with `psa_generate_random` and logs them as hex.
2. **EC DSA Testing** —
   - Builds key attributes for an exportable SECP256R1 key pair usable for signing and verifying hashes (`PSA_KEY_USAGE_SIGN_HASH | PSA_KEY_USAGE_VERIFY_HASH | PSA_KEY_USAGE_EXPORT`)
   - Generates the key pair with `psa_generate_key`
   - Exports and logs both the private key (`psa_export_key`) and public key (`psa_export_public_key`)
   - Signs a zeroed 32-byte hash with `psa_sign_hash` and logs the signature
   - Verifies the signature with `psa_verify_hash`
   - Destroys the key with `psa_destroy_key`

```
psa_set_key_type(&attr, PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_SECP_R1));
psa_set_key_bits(&attr, 256);
psa_set_key_usage_flags(&attr,
                        PSA_KEY_USAGE_SIGN_HASH | PSA_KEY_USAGE_VERIFY_HASH |
                            PSA_KEY_USAGE_EXPORT);
psa_set_key_algorithm(&attr, PSA_ALG_ECDSA(PSA_ALG_SHA_256));
```

A final `"All tests success!"` or `"Some tests failed!"` message is logged, and the program then spins forever in an empty `for (;;) {}` loop.

Note

C vs C++ Build `CMakeLists.txt` globs all `*.c` files by default. If `CONFIG_TLK_ALLOW_CPP_WRAPPERS` is enabled, `main.c` is removed from the C sources and `main.cpp` (which wraps the same mbedTLS/PSA calls in `extern "C"` blocks) is built instead. Functionally the two files are equivalent.

85.4 Configuration Options

Option	Description
`CONFIG_TLK_MBEDTLS_DEM(`	Enables the sample; selects `TLK_MBEDTLS` and `TLK_API_PRINT` (default `y`, not user-facing)
`CONFIG_TLK_BLE_CONTROLl`	Explicitly disabled (`default n`) for this sample, since it does not use BLE

85.5 Build & Run

```
west tl-build samples/mbedtls_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/mbedtls_demo.bin
```

Source code: `samples/mbedtls_demo/main.c` (or `samples/mbedtls_demo/main.cpp` when built with `CONFIG_TLK_ALLOW_CPP_WRAPPERS`)

CHAPTER

EIGHTYSIX

MCUBOOT BOOTLOADER SAMPLE

86.1 Overview

The `mcuboot_bootloader_demo` sample builds MCUboot as the bootloader stage: it validates and jumps to the signed application in Slot 0, and can optionally expose a UART serial recovery (DFU) console for uploading new images without a debugger.

86.2 Features Demonstrated

- Bootloader startup and handoff to the application via `tlk_mcuboot_run()`
- Board-specific DFU-entry detection and recovery UART setup
- UART serial recovery transport configuration for `boot_serial`

86.3 How It Works

86.3.1 Initialization

1. **Global interrupts** are enabled via `tlk_core_interrupt_enable()`.
2. **Debug UART0** is configured if `CONFIG_TLK_DEBUG_LOG_INTERFACE_UART` is enabled, so bootloader log messages are visible.
3. `tlk_mcuboot_run()` is called, handing control to the bootloader core.

```
int main(void)
{
    tlk_core_interrupt_enable();

#if TLK_IS_ENABLED(CONFIG_TLK_DEBUG_LOG_INTERFACE_UART)
    configure_uart(TLK_UART0, CONFIG_TLK_UART0_BAUDRATE, TLK_UART_RX_TIMEOUT_
↪MUL_2);
#endif

    tlk_mcuboot_run();

    while (1)
    {
    }
}
```

86.3.2 Boot Sequence (tlk_mcuboot_run)

1. Logs startup and warns if the image is verified against the SDK's development signing key (see subsystem/mcuboot/port/keys.c).
2. Initializes the crypto backend used for signature verification.
3. Calls tlk_mcuboot_recovery_requested(). If it returns true, the bootloader enters serial recovery instead of booting:
   - tlk_mcuboot_prepare_recovery() configures the recovery UART and DFU indicator LED.
   - boot_serial_start() runs the boot_serial console until reset.
4. Otherwise, boot_go() locates and validates a bootable image in Slot 0/Slot 1 and the sample jumps to it.
5. If no valid image is found, the bootloader logs an error and halts.

86.3.3 DFU-Entry Detection

tlk_mcuboot_recovery_requested() is checked once, at boot, before boot_go() runs —the button must be held through power-up/reset, not pressed after the app has already booted. A 30 ms debounce confirms the press:

```
bool tlk_mcuboot_recovery_requested(void)
{
    tlk_gpio_configure(PINMUX_KEY_2_PORT, PINMUX_KEY_2_PIN, TLK_GPIO_OUTPUT);
    tlk_gpio_pin_write(PINMUX_KEY_2_PORT, PINMUX_KEY_2_PIN, 1);

    tlk_gpio_configure(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN, TLK_GPIO_INPUT_
↪PULL_DOWN);

    tlk_api_time_delay(TLK_MS_TO_US(1));

    if (!tlk_gpio_pin_read(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN))
    {
        return false;
    }

    tlk_api_time_delay(TLK_MS_TO_US(DFU_BUTTON_DEBOUNCE_MS));

    return tlk_gpio_pin_read(PINMUX_KEY_0_PORT, PINMUX_KEY_0_PIN) != 0;
}
```

When recovery is entered, tlk_mcuboot_prepare_recovery() turns on LED1 as a visual indicator and configures the recovery UART (RECOVERY_UART_TX_PORT/PIN = PC4, RECOVERY_UART_RX_PORT/PIN = PC5) at 115200 baud on the tlk_uart_module selected by CONFIG_TLK_MCUBOOT_SERIAL_UART.

Note

Bootloader vs. Application build This sample only builds the **bootloader** stage (TLK_MCUBOOT_MODE_BOOTLOADER). A separate application target built with CONFIG_TLK_MCUBOOT_MODE_APP runs under it, linked with space reserved for the

MCUboot header. That application also links `bootutil_public.c`, so it can call `boot_set_confirmed()` to mark itself confirmed and avoid a revert on the next reset.

86.4 Configuration Options

Option	Description
`CONFIG_TLK_MCUBOOT_SERIAL`	Compiles in `boot_serial` and enables the DFU-entry button check. Disabled by default —without it, the sample only boots the application.
`CONFIG_TLK_MCUBOOT_UART_MODULE`	Selects which `tlk_uart_module` (integer instance) carries the recovery console (default `0`). The corresponding `TLK_UARTx` must be enabled.
`CONFIG_TLK_MCUBOOT_KEY_WARNING`	Silences the development-key warning once the public key in `subsystem/mcuboot/port/keys.c` has been replaced with a real release key.
`CONFIG_TLK_MCUBOOT_BOOTLOADER_SIZE`	Flash size allocated for the bootloader binary (default `0x00090000`, 576KB).
`CONFIG_TLK_MCUBOOT_HEADER_SIZE`	Space reserved at the start of each application slot for the MCUboot image header (default `0x00000200`). Must exactly match the `--header-size` passed to `imgtool` when signing.
`CONFIG_TLK_MCUBOOT_SLOT_SIZE`	Size allocated for each application slot (default `0x000B0000`, 704KB).
`CONFIG_TLK_MCUBOOT_SCRATCH_SIZE`	Size allocated for swap operations (default `0x00004000`, 16KB).

Signing an Application Image

An application built with `CONFIG_TLK_MCUBOOT_MODE_APP` is linked with space reserved for the MCUboot header, but still needs to be signed with `imgtool` before the bootloader will accept it:

```
imgtool sign \
    --header-size   <CONFIG_TLK_MCUBOOT_HEADER_SIZE> \
    --pad-header \
    --slot-size     <CONFIG_TLK_MCUBOOT_SLOT_SIZE> \
    --version       1.0.0 \
    --key           <path-to-signing-key>.pem \
    app.bin app_signed.bin
```

Use the corresponding development signing key for local testing, or generate a real key pair and set `CONFIG_TLK_MCUBOOT_PRODUCTION_KEY`.

Testing Serial Recovery (DFU)

1. Build and flash this sample with `CONFIG_TLK_MCUBOOT_SERIAL_RECOVERY=y`.
2. Hold `KEY_0` down **through power-up or reset** —`LED1` turns on once recovery mode is entered.
3. Connect to the recovery UART pins at 115200 baud and use an `mcumgr`-compatible client to interact with `boot_serial`, e.g.:
   - `image list` —list images currently in the slots.

- `image upload` —upload a signed image. By default this targets the **primary** slot; pass the client's image-index flag (e.g. `-n 2`) to target the secondary slot for a normal test/confirm upgrade flow.
- `image test` / `reset` / `image confirm` —mark the uploaded image for one-time boot or permanent confirmation.

86.5 Build & Run

```
west tl-build samples/mcuboot_bootloader_demo --board TLSR9528A_EVK
west tl-bdt download --chip TLSR9528A -i build/mcuboot_bootloader_demo.bin
```

Source code: `samples/mcuboot_bootloader_demo/main.c`

CHAPTER

EIGHTYSEVEN

API REFERENCE

Complete API documentation for UniSDK, auto-generated from Doxygen comments in C header files via Doxygen + Breathe.

Tip

The API reference below is built from the C header files. Use the search box to find functions, macros, or types quickly.

87.1 Auto-Generation Notes

This API reference is auto-generated by **Doxygen + Breathe** from header files in the following directories:

- Public API: `api/include/`
- Core Driver API: `core/include/`, `core/common/include/`
- SoC Driver API: `soc/*/drivers/include/`

To regenerate:

```
python scripts/gen_api_rst.py
cd docs && sphinx-build -M html . _build
```

CHAPTER

EIGHTYEIGHT

KCONFIG REFERENCE

Note

Document Status: PLANNED —Automatic generation system under development

The Kconfig Reference will be automatically extracted from `Kconfig.build`, `Kconfig.chip`, and all `rsource` sub-files, providing a complete list of Kconfig options.

88.1 Planned Content

- Name, type, and default value of each configuration option
- Dependencies (`depends on`, `select`)
- Value ranges (`range`, `choice`)
- Help text
- Navigation organized by functional module

88.2 Implementation Plan

Based on extending the existing `kconfig_parser.py`, reference documentation in Markdown format will be automatically extracted from Kconfig source files.

The automatic generation system will be added later.

CHAPTER

EIGHTYNINE

YAML DESCRIPTION REFERENCE

Note

Document Status: PLANNED —Automatic generation system under development

The YAML Reference will be automatically extracted from YAML files in directories such as `core/*/properties/`, `core/*/registers/`, `soc/*/pinmux/`, providing a complete reference for device properties, register definitions, and pin multiplexing configurations.

89.1 Planned Content

89.1.1 Device Properties

- Configuration properties and capability descriptions for each peripheral (`core/*/properties/*.yaml`)

89.1.2 Register Definitions

- Register layout and bit field descriptions for each peripheral (`core/*/registers/*.yaml`)

89.1.3 Pin Multiplexing (Pinmux)

- Pin function assignments for each chip package (`soc/*/pinmux/*.yaml`)

89.2 Implementation Plan

Based on extending the existing `yaml_reader.py`, `properties_gen.py`, and `registers_gen.py`, structured reference documentation will be automatically generated.

The automatic generation system will be added later.

CHAPTER

NINETY

RELEASES

90.1 Version Naming Convention

UniSDK follows the **MAJOR.MINOR.PATCH** format (e.g., `1.2.3`):

- **MAJOR**: Incompatible API changes
- **MINOR**: Backward-compatible feature additions
- **PATCH**: Backward-compatible bug fixes

90.2 Current Version

Current version information is defined in `common/include/tlk_sdk_version.h`.

90.3 Version Compatibility

90.3.1 Chip Support Matrix

SDK Version	TL321X	TL721X	TLSR922x	TLSR952x	Notes
0.2.0	✓	✓	✓	✓	Initial version
0.3.0	✓	✓	✓	✓	Adds BLE Host v2, MCUboot, TL321X i2s
0.4.0	✓	✓	✓	✓	Modular SDK: TLKPKG packages, registry server, git fallback

90.3.2 Toolchain Versions

Component	Version Requirement
CMake	>= 3.20.0
Python	>= 3.10
Ninja	>= 1.10
West	>= 0.14.0
RISC-V GCC	V5.4.1+

90.4 Version History

90.4.1 v0.4.0

- *v0.4.0 Release Notes*

90.4.2 v0.3.0

- *v0.3.0 Release Notes*

90.4.3 v0.2.0

- *v0.2.0 Release Notes*

90.5 Upgrade Guide

Detailed upgrade guides and migration instructions will be provided with future version releases.

CHAPTER

NINETYONE

V0.2.0(FR)

91.1 Version

- SDK Version: unified_sdk V0.2.0
- Chip Version
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- Hardware EVK Version
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- Toolchain Version
  - TLSR952x/TL721x: TL32 ELF MCULIB V5F GCC14.2
  - TL321x: TL32 ELF MCULIB V5 GCC14.2

91.1.1 Features

- PMP driver - Physical Memory Protection driver for RISC-V privilege enforcement (`feat(pmp)`)
- User-mode framework - Infrastructure for running untrusted application code in RISC-V User Mode (`feat/user mode`)
- Cryptography —Hardware crypto acceleration: hash, PKE, SKE, TRNG (`feat(cryptography)`)
- RF fast-settle - Fast RF settling mode for improved RF wake-up latency (`feat(rf): Add the fast settle feature`)
- RF reset functions - New `tlk_rf_reset()` family of functions (`feat(rf): Add the rf reset functions`)
- RF mode-switching demo - Sample demonstrating dynamic RF mode changes (`feat(rf): Add the mode switching demo`)
- Legacy toolchain env var - `TELINK_BASE_SDK` environment variable alias for backward compatibility (`feat(toolchain)`)

- IRQ nesting - Updated interrupt controller code supporting interrupt preemption (`feat(IRQ code)`)
- USB Universal API - Comprehensive USB driver with unified access interface and TinyUSB integration layer
- I2C Universal API - Complete I2C driver with unified driver structure for all MCUs
- SPI Universal API - SPI driver with Master support and Unified APIs for all supported SOCs
- Power API - Power-rail control API (`tlk_power.h`)
- DMA API - Expanded DMA driver with unified dynamic configuration
- PLIC API - Unified Interrupt controller with common API for all supported SOCs
- ADC API - Unified ADC API for all supported SOCs
- mbedTLS module - PSA Crypto API integration with hardware TRNG acceleration
- Logger subsystem - Debug Log for simplifying the user experience with formatting and sending the debug messages
- Multi-level init system - Support for different system init levels
- Task planner and loops - Simple multi-task queue with multi-threading or serial execution
- Software timer - Mtimer based multi-instance software timer
- Storage system - Abstract storage device layer with SOC flash driver support
- Debug system - Assert handling and GPIO debug support
- Dynamic memory management - Configurable heap allocation with retention memory support
- Pinmux configuration tool - Interactive Python TUI for visual pin multiplexing configuration
- Toolchain auto-discovery - Automatic RISC-V GCC toolchain path detection and validation
- Zephyr module integration - UniSDK as a standard Zephyr RTOS module (`module.yml`) with CMake/Kconfig bridge
- Zephyr Kconfig bridge - SoC/Core/Board mapping from Zephyr to UniSDK (`CONFIG_SOC_SERIES_TELINK_*` → `CONFIG_TLK_CORE/SOC/BOARD`)
- Subsystem
  - BLE Controller - Complete BLE Link Layer with HCI, LL, ACL, Advertising, and Scanning support
- Doc
  - A full English documentation set was added in Unified SDK Documentation, covering:
    - **Getting Started** —Linux, Windows, macOS setup; first-build walkthrough; troubleshooting
    - **Introduction** —SDK positioning, architecture overview, chip family table
    - **Chips & Boards** —Per-chip specs (TL321X, TL721X, TLSR922X, TLSR952X) and EVK board docs

* **Peripheral Drivers** —GPIO, UART, PM, PLIC, Analog, I2C, SPI, ADC, DMA, WDT, Timer, USB
* **Samples** —gpio, uart, pm, adc, dma, i2c, spi, BLE advertising, BLE HCI controller, IRQ nesting, timer, watchdog, USB CDC, RF multi-mode, NV storage, task planner, mbedTLS, system log
* **Build System** —CMake flow, Kconfig system, West workspace, Zephyr module integration, Make support, extensions
* **Tools** —Menuconfig, Pinmux visual tool, BDT flash/debug utility
* **Development Guide** —Directory structure, public API design, core architecture, IDE integration, debugging
* **API Reference, Kconfig Reference, YAML Reference** —Scaffolding with index files
* **Connectivity** —BLE controller documentation
* **Terminology** —SDK-wide glossary

91.2 CodeSize

SOC	Sample	Config	Flash	RAM
TL321X	gpio	min	7234	4173
TL321X	gpio	suspend	11280	4300
TL321X	gpio	retention	12140	4816
TL321X	gpio	max	11432	4348
TL321X	gpio	max_all	19812	5112
TL321X	pm	suspend	6858	4173
TL321X	pm	retention	6858	4173
TL321X	pm	max	6858	4173
TL321X	pm	max_all	19320	5112
TL321X	uart	min	12546	4199
TL321X	uart	suspend	17204	4427
TL321X	uart	retention	18688	4943
TL321X	uart	max	21604	4607
TL321X	uart	max_all	23732	5135
TL721X	gpio	min	10118	4173
TL721X	gpio	suspend	14202	4284
TL721X	gpio	retention	15362	4804
TL721X	gpio	max	14334	4364
TL721X	gpio	max_all	23106	5132
TL721X	pm	suspend	9762	4173
TL721X	pm	retention	9762	4173
TL721X	pm	max	9762	4173
TL721X	pm	max_all	22618	5132
TL721X	uart	min	15426	4199
TL721X	uart	suspend	20118	4443
TL721X	uart	retention	21906	4963
TL721X	uart	max	24490	4623
TL721X	uart	max_all	26906	5155
TLSR952X	gpio	min	7530	4180

continues on next page

Table 1 – continued from previous page

SOC	Sample	Config	Flash	RAM
TLSR952X	gpio	suspend	12116	4296
TLSR952X	gpio	retention	13400	4812
TLSR952X	gpio	max	12280	4376
TLSR952X	gpio	max_all	19160	5080
TLSR952X	pm	suspend	7174	4180
TLSR952X	pm	retention	7174	4180
TLSR952X	pm	max	7174	4180
TLSR952X	pm	max_all	18720	5080
TLSR952X	uart	min	11854	4203
TLSR952X	uart	suspend	17044	4455
TLSR952X	uart	retention	18688	4971
TLSR952X	uart	max	19848	4575
TLSR952X	uart	max_all	22140	5103

CHAPTER

NINETYTWO

V0.3.0(FR)

92.1 Version

- SDK Version: unisdk V0.3.0
- Chip Version
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- Hardware EVK Version
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- Toolchain Version
  - TL321X: TL32 ELF MCULIB V5 GCC14.2 (andes-riscv32-v5-gcc14.2)
  - TL721X/TLSR952X/TLSR922X: TL32 ELF MCULIB V5F GCC14.2 (andes-riscv32-v5f-gcc14.2)
  - Zephyr build (all series): Zephyr RISC-V64 SDK v0.17.0

92.2 Features

- [BLE Host]
  - Added BLE Host v2 project, build configuration, and sample application (`ble_host_demo`), including the SAL (Service Abstraction Layer) timers implementation and an enabled ACL data path.
  - Moved the SAL layer into the BLE host and split the BLE host module out into its own repository, integrated via West.
- [MCUboot]
  - Introduced MCUboot support, including a bootloader sample and its README/API documentation.
- [Build System / Kconfig]

  - Improved the Kconfig configuration flow and fixed several follow-on issues in the chip/build configuration generation.
  - `chip.config` generation now honors `SOC/BOARD` values passed on the CMake command line (e.g. `make cmake BOARD=TL3218X_EVK`), so the board/SoC can be preselected non-interactively instead of only through the interactive `menuconfig` UI.
  - Propagated the toolchain type (Andes / Zephyr) into `build.config`, and the cached toolchain path is now invalidated when the selected SoC series changes.
- [Toolchain]
  - Updated the Zephyr toolchain to v0.17.0.
  - Added automatic winget-based toolchain installation with a fallback path.
- [Drivers]
  - Added missing RF driver functions and DCOC/LDO_PLL/HPMC calibration and compensation support.
  - Implemented the advanced random driver for TL721X.
  - Added DMA driver improvements.
  - Reworked the i2s demo and added i2s support for TL321X.
  - Added common driver source files.
- [Logging]
  - Added hexdump support to the logger.
- [Security]
  - Added Mbed TLS support.
- [Linker / Binary]
  - Linker script no longer forces the LMA address of FLASH sections.
  - Added binary identification values (SDK/app version signature) for the debugger.
- [Samples]
  - Updated existing samples to support C++.

92.3 Bug Fixes

- Critical Bugs:
  - [Build System]
    - Fixed the board selector so it works correctly when `SOC/BOARD` are passed on the command line —previously a first-time, non-interactive `chip.config` generation silently ignored these values and left the board unselected, causing configure-time Kconfig source failures.
      - Impact Scope: Any project first configuring a released/exported SDK tree (`DEV SDK mode`) with `-DBOARD=../-DSOC=...` on the command line.
      - Update Recommendation: Mandatory update for anyone relying on command-line board/SoC selection instead of interactive `menuconfig`.
  - [Audio]

        * Fixed a linker issue affecting audio builds.

- General Bugs:
  - [Power Management]
    * Reduced power consumption and added tick resolution for sleep duration.
  - [SPI]
    * Increased SPI driver speed.
  - [USB]
    * Fixed typos left over from a prior renaming pass.
  - [System]
    * Fixed `sys_init` priority ordering.
  - [Logger]
    * Fixed an unused-parameter build error that occurred when the logger is disabled.
  - [mtimer]
    * Guarded mtimer source with the correct `#if` conditional.
  - [CI]
    * Fixed the Andes CI job build.
  - [Samples / BLE Host v2]
    * Removed an audio demo file that was added by mistake.
    * Fixed several coding-style issues in the BLE Host v2 demo.
    * Renamed `ble_host_v2_demo` to `ble_host_demo`.
    * Removed building applications from within the `ble_host_v2` module.
  - [Misc]
    * Fixed a typo in `core/common/src/tlk_plic.c`.
    * Removed excessive blank lines in `nv_storage_demo` logs.

92.4 BREAKING CHANGES

- Change Description: The board-properties file path layout was changed (`refactor(properties): Change board properties path`).
  - Impact Scope: Any out-of-tree code or configuration referencing the old board-properties path.
  - Update Recommendation: Re-check board-properties references after upgrading; see the updated path layout in `boards/`.
- Change Description: `ble_host_v2_demo` was renamed to `ble_host_demo`, and the BLE host module now lives in a separate repository pulled in via West.
  - Impact Scope: Any project referencing the old `ble_host_v2_demo` sample name, or directly vendoring/pinning the BLE host sources in-tree.

- Update Recommendation: Update sample references to `ble_host_demo` and update West manifest pins as needed.

92.5 Note

- Development Note: The `SOC/BOARD` CMake cache variables only take effect the first time `chip.config` is generated. If `chip.config` already exists from a previous configure, delete it (or edit it directly) before re-running with new `SOC/BOARD` values.

92.6 Royalty fee for certain Audio Codec:

- This SDK may include options for multiple audio codecs, it should be noted that use of certain Codecs may incur Royalty fees. It is the end product manufacturer's responsibility to sign license agreement with the license onwers and pay royalty fees. Telink as an IC provider cannot cover these charges.
- Use of LC3+ codec: If you choose to use LC3+ codec, please contact Fraunhofer/Ericsson (Fraunhofer IIS: lc3-licensing@iis.fraunhofer.de and Ericsson: lc3.licensing@ericsson.com) for proper license agreement and royalty fee information. A flat fee is charged by these License owners for product incorporating LC3+ codec. The royalty fee is open and transparent and charged per device (e.g. Headset, TV ,Box, ⋯).
- Use of LC3 codec: LC3 usage is only free for product qualified as a Bluetooth product by Bluetooth SIG. If your product is not Bluetooth qualified and you choose to use LC3 codec, please contact Fraunhofer/Ericsson (Fraunhofer IIS: lc3-licensing@iis.fraunhofer.de and Ericsson: lc3.licensing@ericsson.com) for proper license agreement and royalty fee information. A flat fee is charged by these License owners for non-Bluetooth product incorporating LC3 codec. The royalty fee is open and transparent and charged per device (e.g. Headset, TV ,Box, ⋯).

92.7 CodeSize

`irq` is a new sample added in this release and has no v0.2.0 baseline to diff against (shown as "New"). `max_all` configs from v0.2.0 are no longer produced by the current sample set and are omitted below. Deltas are `(V0.3.0 - V0.2.0) / V0.2.0 * 100`.

SOC	Sample	Config	Flash	Flash Δ%	RAM	RAM Δ%
TL321X	gpio	min	8562	+18.4%	4173	+0.0%
TL321X	gpio	suspend	13108	+16.2%	4300	+0.0%
TL321X	gpio	retention	13952	+14.9%	4816	+0.0%
TL321X	gpio	max	13244	+15.9%	4348	+0.0%
TL321X	irq	min	17602	New	5356	New
TL321X	irq	suspend	21674	New	5508	New
TL321X	irq	retention	22518	New	6028	New
TL321X	irq	max	17602	New	5356	New
TL321X	pm	suspend	14912	+117.4%	4869	+16.7%
TL321X	pm	retention	14240	+107.6%	4869	+16.7%
TL321X	pm	max	14984	+118.5%	4869	+16.7%

continues on next page

Table 1 – continued from previous page

SOC	Sample	Config	Flash	Flash Δ%	RAM	RAM Δ%
TL321X	uart	min	13545	+8.0%	4355	+3.7%
TL321X	uart	suspend	17615	+2.4%	4507	+1.8%
TL321X	uart	retention	19131	+2.4%	5023	+1.6%
TL321X	uart	max	21115	-2.3%	4747	+3.0%
TL721X	gpio	min	11534	+14.0%	4173	+0.0%
TL721X	gpio	suspend	16032	+12.9%	4284	+0.0%
TL721X	gpio	retention	17240	+12.2%	4804	+0.0%
TL721X	gpio	max	16160	+12.7%	4364	+0.0%
TL721X	irq	min	20574	New	5368	New
TL721X	irq	suspend	24514	New	5524	New
TL721X	irq	retention	25722	New	6048	New
TL721X	irq	max	20574	New	5368	New
TL721X	pm	suspend	18136	+85.8%	4889	+17.2%
TL721X	pm	retention	17520	+79.5%	4889	+17.2%
TL721X	pm	max	18208	+86.5%	4889	+17.2%
TL721X	uart	min	16753	+8.6%	4367	+4.0%
TL721X	uart	suspend	20691	+2.8%	4523	+1.8%
TL721X	uart	retention	22515	+2.8%	5043	+1.6%
TL721X	uart	max	24347	-0.6%	4763	+3.0%
TLSR952X	gpio	min	8882	+18.0%	4180	+0.0%
TLSR952X	gpio	suspend	13932	+15.0%	4296	+0.0%
TLSR952X	gpio	retention	15224	+13.6%	4812	+0.0%
TLSR952X	gpio	max	14088	+14.7%	4376	+0.0%
TLSR952X	irq	min	17802	New	5372	New
TLSR952X	irq	suspend	22338	New	5536	New
TLSR952X	irq	retention	23630	New	6056	New
TLSR952X	irq	max	17802	New	5372	New
TLSR952X	pm	suspend	15920	+121.9%	4897	+17.2%
TLSR952X	pm	retention	15520	+116.3%	4897	+17.2%
TLSR952X	pm	max	15992	+122.9%	4897	+17.2%
TLSR952X	uart	min	14037	+18.4%	4371	+4.0%
TLSR952X	uart	suspend	18571	+9.0%	4535	+1.8%
TLSR952X	uart	retention	20263	+8.4%	5051	+1.6%
TLSR952X	uart	max	20839	+5.0%	4679	+2.3%

CHAPTER

NINETYTHREE

V0.4.0(FR)

93.1 Version

- SDK Version: unisdk V0.4.0
- Package Versions
  - MCU packages: B92 (TLSR922X/TLSR952X) 0.1.1, TL321X 0.1.1, TL721X 0.1.1
  - Modules: flashdb 0.1.1, mbedtls 0.1.1, tinyusb 0.1.1
  - All packages declare `sdk: "^0.4.0"`
- Chip Version
  - TLSR952x
  - TL721x A2
  - TL321x A0/A1
- Hardware EVK Version
  - TLSR952x: C1T266A20
  - TL721x: C1T314A20
  - TL321x: C1T331A20/C1T335A20
- Toolchain Version
  - TL321X: TL32 ELF MCULIB V5 GCC14.2 (andes-riscv32-v5-gcc14.2)
  - TL721X/TLSR952X/TLSR922X: TL32 ELF MCULIB V5F GCC14.2 (andes-riscv32-v5f-gcc14.2)
  - Zephyr build (all series): Zephyr RISC-V64 SDK v0.17.0

93.2 Features

- [Modular SDK / TLKPKG]
  - Introduced TLKPKG, the UniSDK package manager. The SDK is now split into a small core plus independently versioned packages: MCU packages (`packages/B92`, `packages/TL321X`, `packages/TL721X`) and modules (`modules/FlashDB`, `modules/mbedtls`, `modules/tinyUSB`).
  - MCU sources and the tinyUSB, Mbed TLS and FlashDB modules moved to their own repositories and are pulled in as submodules.

  - Added package versioning: the SDK version is defined in `version.yaml`, and each package declares its own version and the SDK range it is compatible with in `tlkpkg.yaml`.
  - Added package integrity checks on installation.
  - Added module autodiscovery: installed modules are found and added to the build automatically.
  - Module dependencies are now declared in each module's `dependency.yaml` instead of the West manifest.
  - Added per-package exporters (`export-all` command), packing of the Python scripts into the SDK, and the SDK/SRC split of exported trees.
- [Package Registry Server]
  - Added the package registry server with a sample registry, used to publish, list and download packages.
- [Git Publishing and Fallback]
  - Added publishing and fetching of packages through git: each package can be released as a tag in its own git repository.
  - Added a git fallback: when the TLKPKG server is unavailable, packages are installed directly from their git repositories. The VS Code extension uses the same fallback.
- [VS Code Extension]
  - Updated the UniSDK VS Code extension and prepared it for publishing: SDK management (several SDK installations, stored in a file instead of VS Code storage), separate SDK and SRC modes, package install and refresh from the sidebar, and a status refresh button.
  - Added the new project wizard: a new project is created outside the SDK from a sample, with its own Makefile and `.clangd` file.
- [Pinmux]
  - Added the pinmux GUI, including the board view and pin generation for a bare SoC when no board is selected.
- [Build System / Kconfig]
  - Added `prj.conf` support: each sample and project sets its Kconfig options in its own `prj.conf`.
  - External projects can now be built with `make`, outside the SDK tree.
  - Added the `GEN_ALL_BOARD_PROPERTIES` CMake option, which generates the properties of all boards of the selected SoC at build time.
  - Headers of disabled drivers are now checked: including a header whose driver is disabled in Kconfig reports an error.
  - On Windows, the Makefile now runs all configuration steps instead of CMake.
- [Toolchain]
  - Toolchain discovery now verifies the installed GCC version against the version required for the SoC series in `soc.yaml`.
- [MCUboot]

  - Added a recovery request through a RAM flag, so the application can ask the bootloader to enter recovery mode.
  - Updated the MCUboot bootloader sample and its README.
- [FlashDB]
  - Introduced the FlashDB module and the `flashdb_demo` sample.
- [Audio / USB]
  - Added a USB audio sample with microphone and speaker.
- [TLSR922X]
  - Updated the TLSR922X support in the B92 package.
- [CI]
  - The CI script now discovers all chips, samples and configs, and can be limited to specific SoC series or samples.
  - Added CI configs for most of the samples and refactored the memory report generator.

93.3 Bug Fixes

- Critical Bugs:
  - [Build System]
    - Fixed the configure flow (`kconfig.cmake`) and the SDK exporter.
      - Impact Scope: Projects configured from an exported SDK tree.
      - Update Recommendation: Mandatory update for anyone exporting or building from an exported SDK.
    - Fixed a race condition in CMake that made CI builds fail.
- General Bugs:
  - [Kconfig]
    - Fixed the board selection, and the case where an empty board is selected.
    - Added Kconfig files missing from the default core installation.
  - [Registers]
    - Fixed the generation of the enum arrays in the register generator.
  - [Drivers]
    - Global variables that were exposed by the drivers are no longer visible outside their source files.
  - [Pinmux]
    - Fixed the pin selector, the pin layout and the pinmux dependencies.
  - [TLKPKG]
    - Application modules are now installed exactly into the `modules` folder.
    - The `ci_config` folder is no longer included when packing a package.

    - Fixed the package installers, the publish code and the missing installation directive for MCU packages.
    - Fixed the display of installed modules in SDK and SRC trees.
  - [Export]
    - Fixed the packing of the sources and the crypto library, the MCU export and the module export flow.
  - [New Project]
    - Fixed the Makefile of new projects.
  - [VS Code Extension]
    - Fixed the MCU package path, the Kconfig UI flow and the message shown when no MCU package is installed.
    - The sidebar is now refreshed after a package install or an SDK switch.
  - [CI]
    - Fixed the CI scripts for the new submodules and the samples build, and removed a broken pre-commit cache.

93.4 BREAKING CHANGES

- Change Description: The SDK is now modular. MCU sources (`core/<CHIP>`) are no longer part of the main repository; they are installed as MCU packages under `packages/`. tinyUSB, Mbed TLS and FlashDB are installed as modules under `modules/`.
  - Impact Scope: Any project or script that references the old in-tree MCU or module paths, or that expects every chip to be present in the SDK.
  - Update Recommendation: Install the MCU packages and modules your project needs (VS Code extension or TLKPKG), and update path references to `packages/<CHIP>` and `modules/<MODULE>`.
- Change Description: Kconfig option names were standardized (`review(configs): Standardize config names`). Examples:
  - `TLK_UART0_ENABLED` → `TLK_UART0`, `TLK_GPIO_PORT_A_ENABLED` → `TLK_GPIO_PORTA`
  - `TLK_SYSTEM_TIMER_*` → `TLK_STIMER_*` (e.g. `TLK_SYSTEM_TIMER_IRQ_ENABLE` → `TLK_STIMER_IRQ`)
  - `TLK_CRITICAL_PRINTOUT` → `TLK_CRITICAL_PRINT`, `TLK_CRITICAL_UART*` → `TLK_CRITICAL_PRINT_UART*`
  - `TLK_DMA_IRQ_HANDLER` → `TLK_DMA_IRQ`, `TLK_CRYPTO_DMA_SUPPORT` → `TLK_CRYPTO_DMA`
  - `*_HAS_ENABLED_DEVICE` → `*_USE_DEVICE`, `*_AUTO_CONFIG` → `*_USE_AUTO_CONFIG`, `TLK_I2C_DMA` → `TLK_I2C_USE_DMA`
  - `*_NEST_SELECTED` → `*_USE_NEST`, `TLK_RANDOM_USED_PRNG/TRNG` → `TLK_RANDOM_USE_PRNG/TRNG`, `TLK_TRAP_ECALL_NESTING` → `TLK_TRAP_ECALL_NEST`
  - Impact Scope: Any `prj.conf`, defconfig, `build.config` or source code that uses the old option names.

  - Update Recommendation: Rename the options in your configuration files and in `#if CONFIG_...` checks.
- Change Description: Sample and project Kconfig options moved to `prj.conf`.
  - Impact Scope: Projects that set their options in other config files.
  - Update Recommendation: Move project options into the project's `prj.conf`.
- Change Description: The West manifest is no longer used for module dependencies; dependencies are declared in each module's `dependency.yaml`.
  - Impact Scope: Projects that pin module versions through the West manifest.
  - Update Recommendation: Use the module versions installed through TLKPKG.
- Change Description: Samples can no longer be built directly inside an installed SDK.
  - Impact Scope: Workflows that build samples in the SDK folder.
  - Update Recommendation: Create a project from the sample with the new project wizard and build it outside the SDK.

93.5 Note

- Development Note: A package can only be installed into an SDK whose version matches the `sdk` range in the package's `tlkpkg.yaml` (`^0.4.0` for all packages in this release).
- Development Note: When the TLKPKG server is not reachable, packages are fetched from their git repositories. Git access to the package repositories is then required.

93.6 CodeSize

Full CI memory report for all samples and configs built in this release (sizes in bytes): `memory_report.log`

```
SOC         Sample                                      Config
↪Flash         RAM        Text        Code      Rodata        Data
↪BSS         Dec         Hex
----------  ----------------------------------------  ------------  ----------
↪  ----------  ----------  ----------  ----------  ----------  ----------  --
↪--------  ----------
TL321X      adc_demo                                    def
↪19331        4625       19242       18118        1124          89
↪4536       23867        5d3b
TL321X      adc_demo                                    max
↪20291        4653       20198       19110        1088          93
↪4560       24851        6113
TL321X      adv_demo                                    def
↪18409       16937       18236       18072         164         173
↪16764       35173        8965
TL321X      audio_demo                                  def
↪26513       10913       26264       24876        1388         249
↪10664       37177        9139
TL321X      ble_controller_demo                         def
↪91428       25236       90536       90128         408         892
```

(continues on next page)

(continued from previous page)

```
↪24344      115772       1c43c
TL321X      ble_host_demo                                  def
↪160280       24180      158976      143180       15796        1304
↪22876      183156       2cb74
TL321X      dma_demo                                       def
↪18979        5173       18890       17646        1244          89
↪5084       24063        5dff
TL321X      dma_demo                                       max
↪24789        5413       24692       23300        1392          97
↪5316       30105        7599
TL321X      flashdb_demo                                   def
↪30356        9773       30140       27724        2416         216
↪9557       39913        9be9
TL321X      gpio_demo                                      def
↪7622         4181       7594        7430         164           28
↪4153       11775        2dff
TL321X      gpio_demo                                      max
↪9102         4265       9054        8890         164           48
↪4217       13319        3407
TL321X      i2c_demo/ds1307                                def
↪20538        4588       20422       19274        1148         116
↪4472       25010        61b2
TL321X      i2c_demo/ping_pong                             master
↪19574        4588       19466       18250        1216         108
↪4480       24054        5df6
TL321X      i2c_demo/ping_pong                             max
↪19858        4600       19750       18530        1220         108
↪4492       24350        5f1e
TL321X      i2c_demo/ping_pong                             slave
↪19034        4612       18926       17742        1184         108
↪4504       23538        5bf2
TL321X      i2s_demo                                       master
↪21798        8696       19626       18530        1096        2172
↪6524       28322        6ea2
TL321X      i2s_demo                                       slave
↪21770        8696       19598       18506        1092        2172
↪6524       28294        6e86
TL321X      irq_demo                                       def
↪17576        5320       17488       16504         984          88
↪5232       22808        5918
TL321X      irq_demo                                       max
↪18084        5328       17996       16928        1068          88
↪5240       23324        5b1c
TL321X      mbedtls_demo                                   def
↪195606        5460      195314      179582       15732         292
↪5168      200774       31046
TL321X      mcuboot_bootloader_demo                        def
↪233550       18248      232806      215374       17432         744
↪17504      251054       3d4ae
TL321X      mcuboot_bootloader_demo                        max
↪281772       20972      281004      260812       20192         768
↪20204      301976       49b98
```

(continues on next page)

(continued from previous page)

```
TL321X      nv_storage_demo                                  def                 ␣
 ↪21149        9737       21004       19548       1456          145          ␣
 ↪9592        30741       7815
TL321X      pm_demo                                          def                 ␣
 ↪14884        4877       14824       14360        464           60          ␣
 ↪4817        19701       4cf5
TL321X      pm_demo                                          max                 ␣
 ↪15100        4877       15040       14576        464           60          ␣
 ↪4817        19917       4dcd
TL321X      pmp_demo                                         def                 ␣
 ↪18142        4584       18030       16886       1144          112          ␣
 ↪4472        22614       5856
TL321X      pmp_demo                                         max                 ␣
 ↪18390        4596       18278       17098       1180          112          ␣
 ↪4484        22874       595a
TL321X      random_demo                                      def                 ␣
 ↪17867        4569       17770       16666       1104           97          ␣
 ↪4472        22339       5743
TL321X      random_demo                                      max                 ␣
 ↪18127        4581       18030       16922       1108           97          ␣
 ↪4484        22611       5853
TL321X      rf_demo                                          brx                 ␣
 ↪28828         17570      28602       27458       1144          226          ␣
 ↪17344         46172       b45c
TL321X      rf_demo                                          btx                 ␣
 ↪29272         17582      29046       27778       1268          226          ␣
 ↪17356         46628       b624
TL321X      rf_demo                                          mrx                 ␣
 ↪28900         17574      28674       27530       1144          226          ␣
 ↪17348         46248       b4a8
TL321X      rf_demo                                          mrx_fs              ␣
 ↪30892         17574      30662       29502       1160          230          ␣
 ↪17344         48236       bc6c
TL321X      rf_demo                                          mtx                 ␣
 ↪29340         17586      29114       27846       1268          226          ␣
 ↪17360         46700       b66c
TL321X      rf_demo                                          mtx_fs              ␣
 ↪31328         17586      31098       29814       1284          230          ␣
 ↪17356         48684       be2c
TL321X      rf_demo                                          prx                 ␣
 ↪29320         17570      29094       27938       1156          226          ␣
 ↪17344         46664       b648
TL321X      rf_demo                                          ptx                 ␣
 ↪29796         17582      29570       28290       1280          226          ␣
 ↪17356         47152       b830
TL321X      rf_demo                                          srt                 ␣
 ↪28836         17570      28610       27466       1144          226          ␣
 ↪17344         46180       b464
TL321X      rf_demo                                          srx                 ␣
 ↪28884         17570      28658       27514       1144          226          ␣
 ↪17344         46228       b494
TL321X      rf_demo                                          str                 ␣
```

(continues on next page)

(continued from previous page)

```
↪29272       17582       29046       27778       1268          226          ␣
↪17356       46628        b624
TL321X      rf_demo                                             stx          ␣
↪29312       17582       29086       27818       1268          226          ␣
↪17356       46668        b64c
TL321X      spi_demo                                            master       ␣
↪20526        4588       20426       19342       1084          100          ␣
↪4488        25014       61b6
TL321X      spi_demo                                            slave        ␣
↪19558        4588       19458       18406       1052          100          ␣
↪4488        24046       5dee
TL321X      stimer_demo                                         def          ␣
↪8878         4269       8826        8662        164           52           ␣
↪4217        13095       3327
TL321X      system_demo/log_demo                                def          ␣
↪12258        4440       12230       11150       1080           28          ␣
↪4412        16670       411e
TL321X      system_demo/log_demo                                loop         ␣
↪12338        4440       12310       11210       1100           28          ␣
↪4412        16750       416e
TL321X      system_demo/log_demo                                scheduler    ␣
↪12634        4460       12586       11534       1052           48          ␣
↪4412        17046       4296
TL321X      system_demo/task_planner_demo                       loop         ␣
↪17775        4561       17686       16626       1060           89          ␣
↪4472        22247       56e7
TL321X      system_demo/task_planner_demo                       scheduler    ␣
↪18206        4612       18070       17014       1056          136          ␣
↪4476        22682       589a
TL321X      system_demo/task_planner_demo                       timers       ␣
↪18462        4620       18318       17254       1064          144          ␣
↪4476        22938       599a
TL321X      tinyusb_demos/usb_device/cdc_demo                   def          ␣
↪31650        5440       31514       29814       1700          136          ␣
↪5304        36954       905a
TL321X      tinyusb_demos/usb_device/mic_demo                   def          ␣
↪35768        6000       35624       33784       1840          144          ␣
↪5856        41624       a298
TL321X      tinyusb_demos/usb_device/spk_demo                   def          ␣
↪34124        6060       33900       32060       1840          224          ␣
↪5836        39960       9c18
TL321X      uart_demo                                           def          ␣
↪13025        4315       12937       12700        237           88          ␣
↪4227        17252       4364
TL321X      uart_demo                                           max          ␣
↪18658        4451       18562       17934        628           96          ␣
↪4355        23013       59e5
TL321X      umode_demo                                          def          ␣
↪13418        4316       13330       13056        274           88          ␣
↪4228        17646       44ee
TL321X      umode_demo                                          max          ␣
↪13482        4316       13394       13092        302           88          ␣
```

(continues on next page)

(continued from previous page)

```
↪4228       17710       452e
TL321X      wdt_demo                                    def
↪18241        4569       18148       17020       1128          93
↪4476       22717       58bd
TL321X      wdt_demo                                    max
↪22549        4697       22452       21280       1172          97
↪4600       27149       6a0d
TL721X      adc_demo                                    def
↪22339        4637       22246       21114       1132          93
↪4544       26883       6903
TL721X      adc_demo                                    max
↪23323        4665       23226       22130       1096          97
↪4568       27891       6cf3
TL721X      adv_demo                                    def
↪20883        4621       20714       20542        172         169
↪4452       25335       62f7
TL721X      audio_demo                                  def
↪29601       10925       29348       27956       1392         253
↪10672       40273        9d51
TL721X      ble_controller_demo                         def
↪94078       12744       93190       92806        384         888
↪11856      105934       19dce
TL721X      ble_host_demo                               def
↪162948       11688      161648      145880      15768        1300
↪10388      173336       2a518
TL721X      dma_demo                                    def
↪22035        5185       21942       20690       1252          93
↪5092       27127       69f7
TL721X      dma_demo                                    max
↪27785        5429       27684       26284       1400         101
↪5328       33113       8159
TL721X      flashdb_demo                                def
↪33404        9785       33184       30760       2424         220
↪9565       42969       a7d9
TL721X      gpio_demo                                   def
↪10490        4181       10462       10290        172          28
↪4153       14643       3933
TL721X      gpio_demo                                   max
↪12066        4277       12014       11842        172          52
↪4225       16291       3fa3
TL721X      i2c_demo/ds1307                             def
↪23682        4600       23562       22406       1156         120
↪4480       28162       6e02
TL721X      i2c_demo/ping_pong                          master
↪22718        4600       22606       21382       1224         112
↪4488       27206       6a46
TL721X      i2c_demo/ping_pong                          max
↪23002        4612       22890       21662       1228         112
↪4500       27502       6b6e
TL721X      i2c_demo/ping_pong                          slave
↪22182        4624       22070       20878       1192         112
↪4512       26694       6846
```

(continues on next page)

(continued from previous page)

```
TL721X      i2s_demo                                    master
 ↪25234        8728       23038       21934       1104        2196
 ↪6532        31766       7c16
TL721X      i2s_demo                                    slave
 ↪25206        8728       23010       21910       1100        2196
 ↪6532        31738       7bfa
TL721X      irq_demo                                    def
 ↪20814        5332       20722       19730        992          92
 ↪5240        26054       65c6
TL721X      irq_demo                                    max
 ↪21322        5340       21230       20154       1076          92
 ↪5248        26570       67ca
TL721X      mbedtls_demo                                def
 ↪199030        5476      198734      182994      15740         296
 ↪5180      204210      31db2
TL721X      mcuboot_bootloader_demo                     def
 ↪237774       110296      237026      219574      17452         748
 ↪109548       347322       54cba
TL721X      mcuboot_bootloader_demo                     max
 ↪286024       113020      285252      265040      20212         772
 ↪112248       398272       613c0
TL721X      nv_storage_demo                             def
 ↪24201        9749       24052       22588       1464          149
 ↪9600        33801       8409
TL721X      pm_demo                                     def
 ↪18040        4897       17976       17504        472           64
 ↪4833        22873       5959
TL721X      pm_demo                                     max
 ↪18256        4897       18192       17720        472           64
 ↪4833        23089       5a31
TL721X      pmp_demo                                    def
 ↪21190        4592       21078       19926       1152          112
 ↪4480        25670       6446
TL721X      pmp_demo                                    max
 ↪21434        4604       21322       20134       1188          112
 ↪4492        25926       6546
TL721X      random_demo                                 def
 ↪20919        4581       20818       19706       1112          101
 ↪4480        25399       6337
TL721X      random_demo                                 max
 ↪21179        4593       21078       19962       1116          101
 ↪4492        25671       6447
TL721X      rf_demo                                     brx
 ↪31388        5254       31166       30014       1152          222
 ↪5032        36420       8e44
TL721X      rf_demo                                     btx
 ↪31828        5266       31606       30330       1276          222
 ↪5044        36872       9008
TL721X      rf_demo                                     mrx
 ↪31460        5258       31238       30086       1152          222
 ↪5036        36496       8e90
TL721X      rf_demo                                     mrx_fs
```

(continues on next page)

(continued from previous page)

```
↪33400        5258       33174       32006       1168          226          ␣
↪5032       38432        9620
TL721X      rf_demo                                              mtx          ␣
↪31900        5270       31678       30402       1276          222          ␣
↪5048       36948        9054
TL721X      rf_demo                                              mtx_fs       ␣
↪33836        5270       33610       32318       1292          226          ␣
↪5044       38880        97e0
TL721X      rf_demo                                              prx          ␣
↪31884        5254       31662       30498       1164          222          ␣
↪5032       36916        9034
TL721X      rf_demo                                              ptx          ␣
↪32344        5266       32122       30834       1288          222          ␣
↪5044       37388        920c
TL721X      rf_demo                                              srt          ␣
↪31396        5254       31174       30022       1152          222          ␣
↪5032       36428        8e4c
TL721X      rf_demo                                              srx          ␣
↪31448        5254       31226       30074       1152          222          ␣
↪5032       36480        8e80
TL721X      rf_demo                                              str          ␣
↪31828        5266       31606       30330       1276          222          ␣
↪5044       36872        9008
TL721X      rf_demo                                              stx          ␣
↪31876        5266       31654       30378       1276          222          ␣
↪5044       36920        9038
TL721X      spi_demo                                             master       ␣
↪24786        4608       24674       23582       1092          112          ␣
↪4496       29282        7262
TL721X      spi_demo                                             slave        ␣
↪23918        4608       23806       22746       1060          112          ␣
↪4496       28414        6efe
TL721X      stimer_demo                                          def          ␣
↪11938        4281       11882       11710        172           56          ␣
↪4225       16163        3f23
TL721X      system_demo/log_demo                                 def          ␣
↪15154        4440       15126       14038       1088           28          ␣
↪4412       19566        4c6e
TL721X      system_demo/log_demo                                 loop         ␣
↪15234        4440       15206       14098       1108           28          ␣
↪4412       19646        4cbe
TL721X      system_demo/log_demo                                 scheduler    ␣
↪15530        4460       15482       14422       1060           48          ␣
↪4412       19942        4de6
TL721X      system_demo/task_planner_demo                        loop         ␣
↪20827        4573       20734       19666       1068           93          ␣
↪4480       25307        62db
TL721X      system_demo/task_planner_demo                        scheduler    ␣
↪21254        4620       21118       20054       1064          136          ␣
↪4484       25738        648a
TL721X      system_demo/task_planner_demo                        timers       ␣
↪21510        4628       21366       20294       1072          144          ␣
```

(continues on next page)

(continued from previous page)

```
↪4484       25994       658a
TL721X      tinyusb_demos/usb_device/cdc_demo          def                 ␣
↪35282        5452       35142       33434       1708          140         ␣
↪5312       40594       9e92
TL721X      tinyusb_demos/usb_device/mic_demo          def                 ␣
↪39432        6012       39284       37436       1848          148         ␣
↪5864       45296       b0f0
TL721X      tinyusb_demos/usb_device/spk_demo          def                 ␣
↪37820        6072       37592       35744       1848          228         ␣
↪5844       43664       aa90
TL721X      uart_demo                                  def                 ␣
↪16077        4327       15985       15740        245           92         ␣
↪4235       20312       4f58
TL721X      uart_demo                                  max                 ␣
↪21582        4467       21482       20846        636          100         ␣
↪4367       25949       655d
TL721X      umode_demo                                 def                 ␣
↪16466        4328       16374       16092        282           92         ␣
↪4236       20702       50de
TL721X      umode_demo                                 max                 ␣
↪16530        4328       16438       16128        310           92         ␣
↪4236       20766       511e
TL721X      wdt_demo                                   def                 ␣
↪21293        4581       21196       20060       1136           97         ␣
↪4484       25777       64b1
TL721X      wdt_demo                                   max                 ␣
↪25473        4713       25372       24192       1180          101         ␣
↪4612       30085       7585
TLSR952X    adc_demo                                   def                 ␣
↪19159        4637       19058       17930       1128          101         ␣
↪4536       23695       5c8f
TLSR952X    adc_demo                                   max                 ␣
↪20207        4665       20102       19010       1092          105         ␣
↪4560       24767       60bf
TLSR952X    adv_demo                                   def                 ␣
↪16263        4555       16134       15966        168          129         ␣
↪4426       20689       50d1
TLSR952X    audio_demo                                 def                 ␣
↪26805       10841       26632       25280       1352          173        ␣
↪10668       37473        9261
TLSR952X    ble_controller_demo                        def                 ␣
↪86646       12868       85802       85438        364          844        ␣
↪12024       98670       1816e
TLSR952X    ble_host_demo                              def                ␣
↪155488       11812      154232      138480      15752         1256        ␣
↪10556      166044       2889c
TLSR952X    dma_demo                                   def                 ␣
↪19127        5189       19026       17778       1248          101         ␣
↪5088       24215       5e97
TLSR952X    dma_demo                                   max                 ␣
↪25453        5441       25340       23944       1396          113         ␣
↪5328       30781       783d
```

(continues on next page)

(continued from previous page)

```
TLSR952X    flashdb_demo                                    def
 ↪30352        9789       30124       27704        2420         228
 ↪9561        39913        9be9
TLSR952X    gpio_demo                                       def
 ↪7974         4188        7938        7770         168          36
 ↪4152        12126        2f5e
TLSR952X    gpio_demo                                       max
 ↪9566         4284        9506        9338         168          60
 ↪4224        13790        35de
TLSR952X    i2c_demo/ds1307                                 def
 ↪20726        4604       20598       19446        1152         128
 ↪4476        25202        6272
TLSR952X    i2c_demo/ping_pong                              master
 ↪19750        4604       19630       18410        1220         120
 ↪4484        24234        5eaa
TLSR952X    i2c_demo/ping_pong                              max
 ↪20034        4616       19914       18690        1224         120
 ↪4496        24530        5fd2
TLSR952X    i2c_demo/ping_pong                              slave
 ↪19218        4628       19098       17910        1188         120
 ↪4508        23726        5cae
TLSR952X    i2s_demo                                        master
 ↪21826        8724       19630       18530        1100        2196
 ↪6528        28354        6ec2
TLSR952X    i2s_demo                                        slave
 ↪21798        8724       19602       18506        1096        2196
 ↪6528        28326        6ea6
TLSR952X    irq_demo                                        def
 ↪17846        5336       17746       16758         988         100
 ↪5236        23082        5a2a
TLSR952X    irq_demo                                        max
 ↪18354        5344       18254       17182        1072         100
 ↪5244        23598        5c2e
TLSR952X    mbedtls_demo                                    def
 ↪195958        5768      195654      179918       15736         304
 ↪5464      201422      312ce
TLSR952X    mcuboot_bootloader_demo                         def
 ↪235070      110588      234314      216870       17444         756
 ↪109832      344902       54346
TLSR952X    mcuboot_bootloader_demo                         max
 ↪282976      113256      282196      261992       20204         780
 ↪112476      395452       608bc
TLSR952X    nv_storage_demo                                 def
 ↪21149        9753       20992       19532        1460         157
 ↪9596        30745        7819
TLSR952X    pm_demo                                         def
 ↪15892        4905       15816       15348         468          76
 ↪4829        20721        50f1
TLSR952X    pm_demo                                         max
 ↪16108        4905       16032       15564         468          76
 ↪4829        20937        51c9
TLSR952X    pmp_demo                                        def
```

(continues on next page)

(continued from previous page)

```
↪18222        4596        18102        16954        1148          120          ␣
↪4476        22698        58aa
TLSR952X    pmp_demo                                                            ␣
↪18466        4608        18346        17162        1184          120          ␣
↪4488        22954        59aa
TLSR952X    random_demo                                                         ␣
↪17951        4585        17842        16734        1108          109          ␣
↪4476        22427        579b
TLSR952X    random_demo                                                         ␣
↪18211        4597        18102        16990        1112          109          ␣
↪4488        22699        58ab
TLSR952X    rf_demo                                                             ␣
↪26336        5190        26154        25006        1148          182          ␣
↪5008        31344        7a70
TLSR952X    rf_demo                                                             ␣
↪26780        5202        26598        25326        1272          182          ␣
↪5020        31800        7c38
TLSR952X    rf_demo                                                             ␣
↪26476        5194        26294        25146        1148          182          ␣
↪5012        31488        7b00
TLSR952X    rf_demo                                                             ␣
↪27464        5194        27278        26130        1148          186          ␣
↪5008        32472        7ed8
TLSR952X    rf_demo                                                             ␣
↪26920        5206        26738        25466        1272          182          ␣
↪5024        31944        7cc8
TLSR952X    rf_demo                                                             ␣
↪27904        5206        27718        26446        1272          186          ␣
↪5020        32924        809c
TLSR952X    rf_demo                                                             ␣
↪26804        5190        26622        25462        1160          182          ␣
↪5008        31812        7c44
TLSR952X    rf_demo                                                             ␣
↪27288        5202        27106        25822        1284          182          ␣
↪5020        32308        7e34
TLSR952X    rf_demo                                                             ␣
↪26344        5190        26162        25014        1148          182          ␣
↪5008        31352        7a78
TLSR952X    rf_demo                                                             ␣
↪26396        5190        26214        25066        1148          182          ␣
↪5008        31404        7aac
TLSR952X    rf_demo                                                             ␣
↪26780        5202        26598        25326        1272          182          ␣
↪5020        31800        7c38
TLSR952X    rf_demo                                                             ␣
↪26828        5202        26646        25374        1272          182          ␣
↪5020        31848        7c68
TLSR952X    spi_demo                                                            ␣
↪21886        4612        21766        20678        1088          120          ␣
↪4492        26378        670a
TLSR952X    spi_demo                                                            ␣
↪20826        4612        20706        19650        1056          120          ␣
```

(continues on next page)

(continued from previous page)

```
↪4492       25318       62e6
TLSR952X    stimer_demo                                 def
↪9422         4288        9358        9190        168          64
↪4224       13646       354e
TLSR952X    system_demo/log_demo                        def
↪12618        4444       12582       11498       1084          36
↪4408       17026       4282
TLSR952X    system_demo/log_demo                        loop
↪12698        4444       12662       11558       1104          36
↪4408       17106       42d2
TLSR952X    system_demo/log_demo                        scheduler
↪12994        4464       12938       11882       1056          56
↪4408       17402       43fa
TLSR952X    system_demo/task_planner_demo               loop
↪17863        4577       17762       16698       1064         101
↪4476       22339       5743
TLSR952X    system_demo/task_planner_demo               scheduler
↪18290        4624       18146       17086       1060         144
↪4480       22770       58f2
TLSR952X    system_demo/task_planner_demo               timers
↪18550        4632       18398       17330       1068         152
↪4480       23030       59f6
TLSR952X    tinyusb_demos/usb_device/cdc_demo           def
↪31858        5400       31710       30094       1616         148
↪5252       37110       90f6
TLSR952X    tinyusb_demos/usb_device/mic_demo           def
↪36504        5956       36352       34600       1752         152
↪5804       42308       a544
TLSR952X    tinyusb_demos/usb_device/spk_demo           def
↪34296        5936       34144       32408       1736         152
↪5784       40080       9c90
TLSR952X    uart_demo                                   def
↪13125        4331       13025       12784        241         100
↪4231       17356       43cc
TLSR952X    uart_demo                                   max
↪19182        4479       19070       18438        632         112
↪4367       23549       5bfd
TLSR952X    umode_demo                                  def
↪13498        4332       13398       13120        278         100
↪4232       17730       4542
TLSR952X    umode_demo                                  max
↪13562        4332       13462       13156        306         100
↪4232       17794       4582
TLSR952X    wdt_demo                                    def
↪18325        4585       18220       17088       1132         105
↪4480       22805       5915
TLSR952X    wdt_demo                                    max
↪23029        4725       22916       21740       1176         113
↪4612       27641       6bf9
```

CHAPTER

NINETYFOUR

CONTRIBUTION GUIDE

Community developers and contributors are welcome to contribute code, documentation, examples, and issue reports to the UniSDK project.

> **Note on SDK code contributions**: This guide focuses on the **unisdk-doc** documentation repository. For SDK code contributions (unisdk), please refer to the unisdk repository.

94.1 Documentation Repository: unisdk-doc

The documentation repository (unisdk-doc) follows the same branch model as the SDK code repository.

94.1.1 Branch Model

```
main                                          <- Stable (latest release)
  ^ merge
release-v*.*.*                                <- Release branches
  ^ merge
develop                                       <- Active development
  ^ merge
doc/<topic>  fix/<topic>  feat/<topic>        <- Feature / fix branches
```

Branch	Purpose	CI/CD Deployment
`main`	Stable branch. Only updated when a release is finalized.	Pushes deploy to `/en/latest/`
`develop`	Active development branch. All documentation changes land here first.	Pushes deploy to `/en/dev/`
`release-v*.*.*`	Release preparation branch. Created from `develop` before tagging.	Tag pushes deploy to `/en/v*.*.*/`
`doc/*`	Feature branches for documentation changes.	No deployment
`fix/*`	Feature branches for documentation fixes.	No deployment

94.1.2 Lifecycle

1. **Development**: Create a feature branch from `develop` -> make changes -> open PR -> merge into `develop`
2. **Release**: When ready, create `release-v*.*.*` from `develop` -> finalize -> tag -> CI builds the tag
3. **Stable**: Merge the release branch into `main`, updating the stable documentation

---

94.2 Document Version vs SDK Version

Document versions are **aligned with SDK versions without extra tags** for doc-only fixes:

Scenario	Doc Tag	Description
SDK release	`v<SDK version>` (e.g. `v0.2.0`)	One-to-one mapping, doc and SDK released together
Doc-only fix	**No tag**	Merge directly to `main`, `latest` auto-updates
Dev preview	No tag (develop -> `dev`)	CI deploys to `/en/dev/`
Stable placeholder	No tag (main -> `latest`)	CI deploys to `/en/latest/`

Core principles:

1. Doc tags are one-to-one with SDK tags: at most one doc tag per SDK release
2. Doc-only fixes do not create new tags; merge to `main` updates `latest`
3. Published historical versions (e.g. `v0.2.0`) are not retroactively updated

---

94.3 Release Process

```
graph LR
    A[develop] -->|PR reviewed| B[release-v*.*.*]
    B -->|QA testing| C{Pass?}
    C -->|Yes| D[Tag & release]
    C -->|No| B
    D --> E[Merge to main]
    D --> F[Deploy to /en/v*.*.*/]
    E --> G[Deploy to /en/latest/]
```

1. **Create a release branch** from `develop`:

   ```
   git checkout develop
   git checkout -b release-v0.x.x
   ```

2. **Finalize**: Make last-minute fixes on the release branch, run QA.
3. **Tag the release**:

```
git tag v0.x.x
git push origin v0.x.x
```

CI builds and deploys the tag to `/en/v0.x.x/`.

4. **Merge to main**:

```
git checkout main
git merge release-v0.x.x
git push origin main
```

CI builds and deploys to `/en/latest/`.

94.4 Multi-Version Maintenance

```
graph TB
    A[develop] -->|create| B[release-v0.3.0]
    B -->|fix| B
    B -->|tag| C[v0.3.0]
    B -->|merge to| D[main]
    C -->|deploy| E[/en/v0.3.0/]

    A -->|cherry-pick critical fixes| F[v0.2.x maintenance]
    F -->|tag| G[v0.2.1]
    G -->|deploy| H[/en/v0.2.1/]
```

- Only the **latest minor version** receives regular maintenance
- The **previous minor version** accepts only security fixes and critical bug fixes (via cherry-pick)
- **Older versions** are not maintained; mark as `DEPRECATED` in docs
- Each minor version gets a `release-v*.*.x` maintenance branch

94.5 CI/CD Behavior

Trigger	Source Checked Out	Deploy Path
Push to `develop`	`develop` branch	`/en/dev/`
Push to `main`	`develop` branch (snapshot)	`/en/latest/`
Push tag `v*.*.*`	Tagged commit	`/en/v*.*.*/`

All versions are built and deployed to the `gh-pages` branch, which serves the documentation via GitHub Pages.

94.6 Document Structure Standards

94.6.1 Directory Naming

Rule	Description	Example
Snake case	Lowercase letters + underscores	`getting_started/`, `chips_boards/`
Singular form	Directory names use singular	`peripherals/` (already singular)
Index entry	Every directory must have `index.md` or `index.rst`	—
Images	Place images in `_images/` subdirectory	`peripherals/_images/gpio_1.png`

Images currently in `pics/` should be migrated to `_images/` over time.

94.6.2 Frontmatter Metadata

Every documentation page must include a frontmatter block:

```
---
title: Document Title        # Required: display title
status: DRAFT                # Required: STABLE/DRAFT/DEPRECATED/PLANNED
since: v0.1.0                # Optional: version first introduced
updated: v0.2.0              # Optional: version last updated
description: Short summary   # Optional: for search and indexing
tags: [gpio, peripheral]     # Optional: categorization tags
---
```

Status values:

Status	Meaning
`STABLE`	Content is complete, reviewed, and ready for production use
`DRAFT`	Content is under development, may be incomplete or unreviewed
`DEPRECATED`	Content is outdated and should not be used for new designs
`PLANNED`	Content is planned but not yet written

94.6.3 Version Difference Markers

Use frontmatter fields to track version history:

```
---
title: GPIO Driver Guide
status: STABLE
since: v0.1.0
updated: v0.2.0
---
```

In document body, use Sphinx directives for per-element version tracking:

```
.. versionadded:: v0.2.0
   Added DMA transfer mode support

.. deprecated:: v0.1.0
   ``legacy_api()`` is deprecated, use ``new_api()`` instead
```

94.6.4 File Format Guidelines

Scenario	Recommended Format	Reason
TOC files	`.rst`	Needs toctree directive
Content pages	`.md` (MyST)	Easier to read and write
API reference	`.rst` (auto-generated)	Produced by `gen_api_rst.py`
Table-heavy pages	`.rst`	RST table syntax is more powerful

Markdown conventions:

- Heading levels: # -> ## -> ### -> #### (max 4 levels)
- Code blocks: annotate language (```` ```c ````, ```` ```bash ````)
- Images: use `![alt](path)` with optional `:width:` directive
- Admonitions: use MyST directives like ```` {note}` ````, ```` ```{tip}` ````
- Links: use MyST `{ref}` and `{doc}` for cross-references

RST conventions:

- Use `.. code-block:: c` instead of `::` + indentation
- Use `:ref:` and `:doc:` for cross-references
- Use `.. versionadded::` / `.. deprecated::` for version tracking

94.6.5 Cross-Reference Conventions

Reference Type	Syntax	Example
Document link	`{doc}` / `:doc:`	{doc}`getting_started/index`
Section link	`{ref}` / `:ref:`	{ref}`my-section-label`
External link	Markdown link	[text](url)
API function	`{c:func}` / `:c:func:`	{c:func}`tlk_gpio_init`
API type	`{c:type}` / `:c:type:`	{c:type}`tlk_gpio_config_t`
Term reference	`{term}` / `:term:`	{term}`GPIO`

Label naming convention:

```
<module>-<section>    Examples: gpio-introduction, uart-dma-mode
```

94.7 Submitting Issues

- Use GitHub Issues to report documentation bugs or request improvements
- Provide the page URL and describe what is incorrect or missing
- Attach screenshots or error logs if applicable

94.8 Code Standards (SDK Contributions)

For contributions to the SDK codebase (unisdk):

- C code should follow the format defined in `.clang-format`
- Run `pre-commit` checks before submitting
- See the unisdk repository for detailed guidelines

CHAPTER

NINETYFIVE

TECHNICAL SUPPORT

95.1 Contact Us

- **Sales Inquiry**: telinksales@telink-semi.com
- **Technical Forum**: https://forum.telink-semi.cn/
- **Official Website**: https://www.telink-semi.cn/
- **Documentation Center**: https://doc.telink-semi.cn/

95.2 Submitting Issues

When submitting technical issues, please provide the following information:

1. UniSDK version
2. Target chip model and board
3. Operating system version
4. Toolchain version
5. Reproduction steps
6. Error logs or screenshots

95.3 Community Resources

- **GitHub Issues**: https://github.com/telink-semi/unisdk-doc/issues
- **Gitee Repository**: https://gitee.com/telink-semi
- **Product Selector**: https://products.telink-semi.cn/
- **Development Tools**: https://www.telink-semi.cn/development-tools

95.4 FAQ

See *Troubleshooting*.

95.5 Resource Links

- *Getting Started*
- *Development Guide*
- *Samples and Demos*
- *About Telink*

CHAPTER

NINETYSIX

UNISDK GLOSSARY

This document defines the core terminology used in UniSDK to ensure consistency across documentation and community communication.

96.1 Build System

Term	Description
Build System	CMake + Ninja based compilation framework
Configuration System	Kconfig-based compile-time configuration management
Target	CMake build target, can be an executable or library
Generator	CMake build backend; UniSDK uses Ninja
Pre-build Check	Validation scripts that run automatically before compilation

96.2 West Tool

Term	Description
West	Zephyr project management tool, the unified build entry point in UniSDK
Workspace	The project directory managed by West, containing `.west` configuration
tl-build	UniSDK custom West command for building projects
tl-config	UniSDK custom West command for configuring projects
tl-bdt	UniSDK custom West command for flashing and debugging

96.3 Hardware Abstraction

Term	Description
Chip / SoC	Telink System-on-Chip
Board / EVK	Telink official Evaluation Kit
Pinmux	Programmable chip pin function configuration
Core Layer	Chip core driver code (`core/`)
SoC Layer	Chip-specific functionality code (`soc/`)

96.4 Kconfig Configuration

Term	Description
Kconfig	Linux kernel configuration system
chip.config	Chip-specific configuration
build.config	Build option configuration
.config	Final merged configuration
autoconf.h	C language configuration macro definition header file
capabilities.h	Chip capability declaration header file
Menuconfig	Terminal-based interactive configuration interface

96.5 Peripherals & Drivers

Term	Description
GPIO	General Purpose Input/Output
UART	Universal Asynchronous Receiver/Transmitter
I2C	Inter-Integrated Circuit
SPI	Serial Peripheral Interface
ADC	Analog-to-Digital Converter
DMA	Direct Memory Access
PM	Power Management
PLIC	Platform-Level Interrupt Controller

96.6 Document Status Markers

Marker	Description
STABLE	Content is complete and ready to use
DRAFT	Content is being written, may be incomplete
DEPRECATED	Will be deprecated soon; migration to new solution is recommended
PLANNED	Planned but not yet started

CHAPTER

NINETYSEVEN

ABOUT TELINK SEMICONDUCTOR

Telink Semiconductor is a leading fabless semiconductor company dedicated to the design and development of low-power, high-performance wireless connectivity System-on-Chip (SoC) solutions. Founded in 2010 and headquartered in Shanghai, China, Telink has grown to become a global player in the IoT connectivity space.

97.1 Our Mission

To provide best-in-class wireless connectivity solutions that power the next generation of IoT devices —enabling smarter, more connected, and more energy-efficient products across consumer, industrial, and automotive markets.

97.2 Technology Leadership

Telink's SoCs integrate a rich set of wireless protocols, offering true multi-protocol support on a single chip:

Protocol	Certification	Key Applications
Bluetooth LE	Bluetooth SIG Board Member	IoT, wearables, beacons
Bluetooth Mesh	Bluetooth SIG	Smart lighting, building automation
Zigbee	CSA Member	Smart home, industrial control
Thread	Thread Group Contributor	Matter over Thread
Matter	CSA Member	Smart home interoperability
Wi-Fi	-	High-bandwidth IoT
2.4GHz Proprietary	-	Gaming, HID devices

97.3 Global Presence

- **Headquarters:** Shanghai, China (Zhangjiang Hi-Tech Park)
- **R&D Centers:** Shanghai, Taipei, Ningbo, Shenzhen
- **Sales & Support:** China, Taiwan, North America, Europe, Ukraine

- **Employees:** 500+ worldwide

97.4 Why Telink?

- **Proven Technology:** Billions of devices powered by Telink SoCs worldwide
- **Comprehensive SDK:** UniSDK provides a unified development experience across all chip series
- **Low Power Leadership:** Industry-leading power consumption for battery-constrained devices
- **Rich Ecosystem:** Complete toolchain including IDE, debugger, and visualization tools
- **Global Support:** FAE teams across the globe for timely technical assistance

---

97.5 Learn More

- Official Website
- Product Selector
- Technical Forum
- Documentation Center
- GitHub